

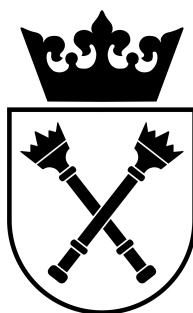
UNIWERYSTET JAGIELLOŃSKI

PYTANIA DO EGZAMINU LICENCJACKIEGO

NA KIERUNKU INFORMATYKA

TEORIA

Karol CHRZĄSTEK
Małgorzata DYMEK
Michał PIOTROWSKI
Dominik WOŁEK
Dagmara ZDYBAŁ



Rok akademicki 2019/2020

Spis treści

1	Zasada indukcji matematycznej.	15
2	Porządki częściowe i liniowe. Elementy największe, najmniejsze, maksymalne i minimalne.	16
3	Relacja równoważności i zbiór ilorazowy.	18
4	Metody dowodzenia twierdzeń: wprost, nie wprost, przez kontrapozycję.	19
5	Metody numeryczne rozwiązywania równań nieliniowych: bisekcji, siecznych, Newtona.	20
5.1	Metoda połowienia (bisekcji)	20
5.2	Metoda siecznych	20
5.3	Metoda Newtona (stycznych)	21
6	Rozwiązywanie układów równań liniowych: metoda eliminacji Gaussa, metody iteracyjne Jacobiego i Gaussa-Seidla.	23
6.1	Metoda eliminacji Gaussa	23
6.2	Metoda iteracyjna Jacobiego	24
6.2.1	Algebraicznie	24
6.2.2	Macierzowo	24
6.3	Metoda iteracyjna Gaussa-Seidla	24
6.3.1	Algebraicznie	24
6.3.2	Macierzowo	24
7	Wartości i wektory własne macierzy: numeryczne algorytmy ich wyznaczania.	25
8	Interpolacja wielomianowa: metody Lagrange’a i Hermite’a. Efekt Rungego.	27
8.1	Interpolacja wielomianowa	27
8.2	Wzór interpolacyjny Lagrange’a	27
8.3	Interpolacja Hermite’a	28
8.3.1	Algorytm	28
8.4	Efekt Rungego	29
9	Zmienne losowe dyskretne. Definicje i najważniejsze rozkłady.	31

10	Zmienne losowe ciągłe. Definicje i najważniejsze rozkłady.	32
11	Łącuchy Markowa. Rozkład stacjonarny.	34
12	Testy statystyczne: test z, test t-Studenta, test chi-kwadrat.	35
13	Wzór Bayesa i jego interpretacja.	37
14	Istnienie elementów odwrotnych względem mnożenia w strukturze $(Zm, +, *)$ w zależności od liczby naturalnej m. Rozszerzony algorytm Euklidesa.	38
14.1	Algorytm Euklidesa	38
14.2	Rozszerzony algorytm Euklidesa	39
15	Ortogonalność wektorów w przestrzeni $\mathbb{R}^n$; związki z liniową niezależnością. Metoda ortonormalizacji Grama-Schmidta	40
15.1	Ortonormalizacja Grama-Schmidta	42
16	Liczby Stirlinga I i II rodzaju i ich interpretacja.	43
17	Twierdzenia Eulera i Fermata; funkcja Eulera.	44
18	Konfiguracje i t-konfiguracje kombinatoryczne.	46
19	Cykl Hamiltona, obwód Eulera, liczba chromatyczna - definicje i twierdzenia.	48
19.1	Cykl Hamiltona	48
19.2	Obwód Eulera	48
19.3	Liczba chromatyczna	49
20	Algorytm Forda-Fulkersona wyznaczania maksymalnego przepływu.	50
21	Rozwiązywanie równań rekurencyjnych przy użyciu funkcji tworzących (generujących) oraz przy użyciu równania charakterystycznego.	52
21.1	Funkcje tworzące.	52
21.2	Równanie charakterystyczne.	55

22 Ciąg i granica ciągu liczbowego, granica funkcji.	56
22.1 Ciągi.	56
22.2 Funkcje.	58
23 Ciągłość i pochodna funkcji. Definicja i podstawowe twierdzenia.	61
23.1 Ciągłość.	61
23.2 Pochodna.	63
24 Ekstrema funkcji jednej zmiennej. Definicje i twierdzenia.	66
25 Całka Riemanna funkcji jednej zmiennej.	68
26 Pochodne cząstkowe funkcji wielu zmiennych; różniczkowalność i różniczka funkcji.	69
27 Ekstrema funkcji wielu zmiennych. Definicje i twierdzenia.	71
28 Twierdzenie o zmianie zmiennych w rachunku całkowym; współrzędne walcowe i sferyczne.	73
29 Metody dowodzenia poprawności pętli.	75
30 Odwrotna Notacja Polska: definicja, własności, zalety i wady, algorytmy.	77
30.1 Algorytm obliczenia wartości wyrażenia ONP	77
30.2 Algorytm konwersji z notacji infiksowej do ONP	77
31 Modele obliczeń: maszyna Turinga.	79
32 Modele obliczeń: automat skończony, automat ze stosem.	81
32.1 Automat skończony deterministyczny	81
32.2 Automat skończony niedeterministyczny	82
32.3 Automat ze stosem	82
33 Złożoność obliczeniowa - definicja notacji: O, Ω, Θ.	83
34 Złożoność obliczeniowa - pesymistyczna i średnia.	84
35 Metoda "dziel i zwyciężaj": zalety i wady.	86
35.1 Zalety	86
35.2 Wady	86

35.3 Przykłady	87
36 Lista: ujęcie abstrakcyjne, możliwe implementacje i ich złożoności.	88
36.1 Implementacja za pomocą tablic (array)	88
36.2 Implementacja za pomocą Listy Wiązanej (Linked List)	89
37 Kolejka i kolejka priorytetowa: ujęcie abstrakcyjne, możliwe implementacje i ich złożoności.	92
37.1 Kolejka	92
37.2 Sposoby implementacji kolejki	93
37.3 Kolejka Priorytetowa	93
37.4 Sposoby implementacji kolejki priorytetowej	94
38 Algorytmy sortowania QuickSort i MergeSort: metody wyboru pivota w QS; złożoności.	95
38.1 QuickSort.	95
38.1.1 MergeSort.	96
39 Algorytm sortowania bez porównań (sortowanie przez zliczanie, sortowanie kubełkowe oraz sortowanie pozycyjne).	97
39.1 CountSort.	97
39.2 BucketSort.	97
39.3 RadixSort.	98
40 Reprezentacja drzewa binarnego za pomocą porządków (preorder, inorder, postorder).	99
41 Algorytmy wyszukiwania następnika i poprzednika w drzewach BST; usuwanie węzła.	100
41.1 Wyszukiwanie następnika i poprzednika w BST.	100
41.2 Usuwanie węzła z BST.	101
42 B-drzewa: operacje i ich złożoność.	102
42.1 Wyszukiwanie	103
42.2 Wstawianie	103
42.3 Usuwanie	103
43 Drzewa AVL: rotacje, operacje z wykorzystaniem rotacji i ich złożoność.	104
43.1 Rotacje	104

44 Algorytmy przeszukiwania wszerz i w głąb w grafach.	106
44.1 Przeszukiwanie grafu wszerz (<i>breadth first search</i> , BFS)	106
44.1.1 Algorytm	106
44.1.2 Cechy	106
44.1.3 Wariacje algorytmu	106
44.1.4 Zastosowania	107
44.2 Przeszukiwanie grafu włąb (<i>depth first search</i> , DFS)	107
44.2.1 Algorytm	107
44.2.2 Wersja z zapamiętywaniem rodzica	107
44.2.3 Cechy	108
44.2.4 Wariacje algorytmu	108
44.2.5 Zastosowania	108
45 Algorytmy wyszukiwania najkrótszej ścieżki (Dijkstry oraz Bellmana-Forda).	109
45.1 Algorytm Dijkstry	109
45.1.1 Algorytm	109
45.1.2 Złożoność	110
45.1.3 Cechy	110
45.2 Algorytm Bellmana-Forda	110
45.2.1 Algorytm	110
45.2.2 Złożoność	111
45.2.3 Cechy	111
45.3 Algorytm Floyd-Warshalla	111
45.3.1 Algorytm	111
45.3.2 Złożoność	112
45.3.3 Cechy	112
46 Programowanie dynamiczne: podział na podproblemy, porównanie z metodą "dziel i zwyciężaj".	113
47 Algorytm zachłanny: przykład optymalnego i nieoptymalnego wykorzystania.	115
48 Kolorowania wierzchołkowe (grafów planarnych) i krawędziowe grafów, algorytmy i ich złożoności.	116
48.1 Kolorowanie krawędziowe grafów	116
48.2 Kolorowanie wierzchołkowe grafów planarnych	116

49 Algorytmy wyszukiwania minimalnego drzewa rozpinającego: Boruvki, Prima i Kruskala.	117
49.1 Algorytm Boruvki	117
49.2 Algorytm Prima	117
49.3 Algorytm Kruskala	117
50 Najważniejsze algorytmy wyznaczania otoczki wypukłej zbioru punktów w układzie współrzędnych (Grahama, Jarvisa, algorytm przyrostowy (quickhull)).	118
51 Problemy P, NP, NP-zupełne i zależności między nimi. Hipoteza P vs. NP.	121
52 Automat minimalny, wybrany algorytm minimalizacji.	122
52.1 Automat minimalny	122
52.2 Minimalizacja automatu	123
53 Lemat o pompowaniu dla języków regularnych.	124
54 Warunki równoważne definicji języka regularnego: automat, prawa kongruencja syntaktyczna, wyrażenia regularne.	125
54.1 Wyrażenia regularne	125
54.2 Automat skończony	126
54.3 Prawa kongruencja syntaktyczna	127
54.4 Warunki równoważne definicji języka regularnego	127
54.5 Konstrukcja automatu skończenie stanowego rozpoznającego wyrażenie regularne.	129
55 Automaty niedeterministyczne i deterministyczne (w tym ze stosem); determinizacja.	132
55.1 Determinizacja automatów skończenie stanowych	132
55.2 Determinizm automatu ze stosem	132
56 Problemy rozstrzygalne i nierozstrzygalne w teorii języków.	134
57 Klasy języków w hierarchii Chomsky’ego oraz ich zamkniętość ze względu na operacje boolowskie, homomorfizmy, itp.	136
58 Reprezentacja liczb całkowitych; arytmetyka.	139
58.1 Kod znak-moduł prosty	139

58.1.1	Reguły:	139
58.1.2	Przykłady:	139
58.1.3	Arytmetyka	139
58.2	Kod odwrotnościowy	139
58.2.1	Reguły	139
58.2.2	Przykłady	140
58.2.3	Arytmetyka	140
58.2.4	Przykłady	140
58.3	Kod uzupełnieniowy	141
58.3.1	Kodowanie	141
58.3.2	Dekodowanie	141
58.3.3	Przykłady kodowania	142
58.3.4	Arytmetyka	142
58.4	Kod nadmiarowy	143
58.4.1	Kodowanie	143
58.4.2	Deokodowanie	143
58.4.3	Arytmetyka	143
59	Reprezentacja liczb rzeczywistych; arytmetyka zmiennopozycyjna.	146
59.1	Reprezentacja cecha-mantysa	146
59.2	Kodowanie zmiennopozycyjne	146
59.3	Normalizacja	147
59.4	Algorytmy kodowania i dekodowania	147
59.4.1	Algorytm kodowania	147
59.4.2	Algorytm dekodowania	148
59.5	Arytmetyka	149
59.5.1	Dodawanie kodów zmiennopozycyjnych	149
59.5.2	Mnożenie kodów zmiennopozycyjnych	150
60	Różnice w wywołaniu funkcji statycznych, niestatycznych i wirtualnych w C++.	152
60.1	Funkcje statyczne.	152
60.2	Funkcje niestatyczne.	153
60.3	Funkcje wirtualne.	153
61	Sposoby przekazywania parametrów do funkcji (przez wartość, przez referencję). Zalety i wady.	156
61.1	Przekazywanie przez wartość.	156
61.2	Przekazywanie przez referencję.	157

62	Wskaźniki, arytmetyka wskaźników, różnica między wskaźnikiem a referencją w C++.	158
63	Podstawowe założenia paradygmatu obiektowego: dziedziczenie, abstrakcja, enkapsulacja, polimorfizm.	161
64	Funkcje zaprzyjaźnione i ich związek z przeładowaniem operatorów w C++.	166
64.1	Przeładowania operatorów a funkcje zaprzyjaźnione	167
65	Programowanie generyczne na podstawie szablonów w języku C++.	168
65.1	Domyślne argumenty szablonów	169
65.2	Specjalizacja szablonów	170
66	Podstawowe kontenery w STL z szerszym omówieniem jednego z nich.	171
66.1	array - kontener sekwencyjny	171
67	Obsługa sytuacji wyjątkowych w C++.	174
67.1	Szkielet obsługi wątków	174
67.2	Rzucanie wyjątku	175
68	Obsługa plików w języku C.	176
69	Model wodospadu a model spiralny wytwarzania oprogramowania.	178
69.1	Model kaskadowy	178
69.1.1	Zasady	178
69.1.2	Fazy modelu kaskadowego	178
69.1.3	Cechy	179
69.2	Model spiralny	180
69.2.1	Fazy modelu spiralnego	180
69.2.2	Cechy	180
70	Diagram sekwencji i diagram przypadków użycia w języku UML.	181
70.1	Diagram sekwencji.	181
70.2	Diagram przypadków użycia.	182
71	Klasyfikacja testów.	184
71.1	Poziomy testów.	184
71.1.1	Testy jednostkowe.	184

71.1.2	Testy integracyjne.	185
71.1.3	Testy systemowe.	186
71.1.4	Testy akceptacyjne.	186
71.2	Typy testów.	187
72	Model Scrum: struktura zespołu, proces wytwarzania oprogramowania, korzyści modelu.	188
72.1	Struktura zespołu	189
72.2	Zdarzenia	191
72.3	Artefakty	193
73	Wymagania w projekcie informatycznym: klasyfikacja, źródła, specyfikacja, analiza.	194
73.1	Klasyfikacja	194
73.2	Źródła	195
73.3	Analiza	197
74	Analiza obiektowa: modele obiektowe i dynamiczne, obiekty encyjne, brzegowe i sterujące.	198
74.1	Diagram klas	198
74.1.1	Klasy	198
74.1.2	Widoczność pól	199
74.1.3	Związki	199
74.2	Modele dynamiczne	201
74.2.1	Diagram stanów	201
74.2.2	Diagram sekwencji	203
74.3	Encja-Brzeg-Sterowanie	205
75	Wzorce architektury systemów.	207
75.1	Architektura warstwowa	207
75.1.1	Architektura otwarta	207
75.1.2	Architektura zamknięta	208
75.1.3	Architektura trójwarstwowa	208
75.2	Model View Controller	209
75.3	Pipeline (filtry i potoki)	210
75.4	Client - Server	211
75.5	Peer-to-Peer	212

76 Relacyjny model danych, normalizacja relacji (w szczególności algorytm doprowadzenia relacji do postaci Boyce’a-Codda), przykłady.	213
76.1 Relacyjny model danych	213
76.2 Relacja bazodanowa	213
76.3 Typy relacji bazodanowej	214
76.4 Własności relacji bazodanowej	214
76.5 Postaci normalne	214
76.6 Normalizacja	215
77 Indeksowanie w bazach danych: drzewa B+, tablice o organizacji indeksowej, indeksy haszowe, mapy binarne.	218
77.1 Typy indeksów (dla plików sekwencyjnych)	219
77.2 Indeksy dla standardowych typów danych	219
77.3 Drzewa B+	220
78 Podstawowe cechy transakcji (ACID). Metody sterowania współbieżnością transakcji, poziomy izolacji transakcji, przykłady.	221
78.1 Transakcja	221
78.2 Przykład transakcji:	222
78.3 ACID	222
78.3.1 ACID dla przykładowej transakcji	223
78.4 Stany transakcji:	223
78.5 Techniki sterowania współbieżnością transakcji	224
78.5.1 Typy blokad	224
78.5.2 Blokady na poziomie tabeli	225
78.5.3 Sterowanie współbieżnością transakcji w systemie Microsoft SQL Server.	225
78.5.4 Implementacja poziomów izolacji transakcji:	226
78.5.5 Poziom izolacji Cursor Stability	226
78.5.6 Poziom izolacji Snapshot	227
79 Złączenia, grupowanie, podzapytania w języku SQL.	228
79.1 JOIN	228
79.2 GROUP BY	229
79.3 Podzapytania	230
79.3.1 Podzapytania niepowiązane	230
79.3.2 Podzapytania powiązane	231
80 Szeregowalność harmonogramów w bazach danych.	232

81 Definicja cyfrowego układu kombinacyjnego - przykłady układów kombinacyjnych i ich implementacje.	234
81.1 Układy arytmetyczne.	236
81.1.1 Półsumator (Half Adder).	236
81.1.2 Sumator (Full Adder).	236
81.1.3 Dekoder.	238
81.1.4 Multiplexer.	239
82 Definicja cyfrowego układu sekwencyjnego - przykłady układów sekwencyjnych i ich implementacje.	241
83 Minimalizacja funkcji logicznych.	245
83.1 Minimalizacja z wykorzystaniem algebry Boole'a	245
83.2 Minimalizacja z wykorzystaniem tablic Karnaugh'a	247
84 Programowalne układy logiczne PLD (ROM, PAL, PLA).	248
84.1 ROM	248
84.2 PAL	249
84.3 PLA	250
85 Schemat blokowy komputera (maszyna von Neumanna).	251
86 Zarządzanie procesami: stany procesu, algorytmy szeregowania z wywłaszczaniem.	253
86.1 Proces	253
86.2 Proces a program	253
86.3 Zarządzanie procesami	253
86.4 Stany procesu	254
86.5 Cykl zmiany stanów procesu	254
86.6 Szeregowanie procesów	255
86.6.1 Funkcja priorytetu	255
86.6.2 Tryb decyzji	256
86.6.3 Reguła arbitrażu	256
86.7 Algorytmy planowania przydziału procesora	257
86.7.1 FCFS	257
86.7.2 LCFS	257
86.7.3 SJF	257

87 Muteks, semafor, monitor jako narzędzia synchronizacji procesów.	259
87.1 Sekcja krytyczna	259
87.2 Semafor	259
87.2.1 Implementacja semafora ogólnego	260
87.3 Muteks (Zamek)	260
87.4 Monitor (Zmienna warunkowa)	261
87.4.1 Ogólny schemat definicji monitora	261
88 Pamięć wirtualna i mechanizm stronicowania.	262
88.1 Stronicowanie	262
88.2 Mechanizm stronicowania na żądanie	263
88.2.1 Problemy zastępowania stron	263
88.2.2 Problem wyboru ofiary	263
88.2.3 Klasyfikacja algorytmów wymiany z względu na okoliczności sprowadzania i usuwania stron	263
88.2.4 Klasyfikacja algorytmów wymiany ze względu na sposób za- stępowania stron	264
88.2.5 Klasyfikacja algorytmów wymiany ze względu na przydział ra- mek dla procesów	264
88.2.6 Dobór liczby ramek	264
88.2.7 Algorytmy wymiany na żądanie	265
88.2.8 Przykład działania algorytmów wymiany na żądanie	265
89 Systemy plikowe - organizacja fizyczna i logiczna (na przykładzie wybranego systemu uniksopodobnego).	267
89.1 Organizacja fizyczna	267
89.2 organizacja logiczna	268
89.2.1 Zadania systemu operacyjnego	268
89.2.2 Atrybuty pliku	268
89.2.3 Struktura pliku	268
89.3 UNIX	269
89.3.1 Informacje ogólne	269
89.3.2 Format partycji	269
89.3.3 Fizyczna struktura pliku	270
89.3.4 Struktura wpisu katalogowego	270
90 Model ISO OSI. Przykłady protokołów w poszczególnych warstwach.	271

91 Adresowanie w protokołach IPv4 i IPv6.	273
91.1 Adresowanie w IPv4.	273
91.1.1 Adresowanie oparte na klasach	273
91.1.2 Adresowanie bezklasowe	274
91.2 Adresowanie w IPv6.	275
91.2.1 Typy adresów	275
91.2.2 Klasyfikacja adresów.	275
91.2.3 Skąd wziąć adres MAC?	276
92 Najważniejsze procesy zachodzące w sieci komputerowej od momen- tu wpisania adresu strony WWW do wyświetlenia strony w prze- glądarce (komunikat HTTP, segment TCP, system DNS, pakiet IP, ARP, ramka).	277
93 Działanie przełączników Ethernet, sieci VLAN, protokół STP.	282
93.1 Działanie przełączników Ethernet	282
93.2 Sieci VLAN	283
93.3 Protokół STP	288
94 Rola routerów i podstawowe protokoły routingu (RIP, OSPF).	290
94.1 Protokoły wektora odległości	291
94.1.1 RIP	291
94.1.2 RIP2	292
94.2 Protokoły stanu łącza	293
94.3 OSPF	293
95 Szyfrowanie z kluczem publicznym, podpis cyfrowy, certyfikaty.	295
95.1 Szyfrowanie z kluczem publicznym	295
95.2 Podpis cyfrowy	295
95.2.1 Skrót(Hash)	295
95.2.2 Uzyskanie podpisu cyfrowego	295
95.2.3 Sprawdzenie podpisu cyfrowego	296
95.3 Certyfikaty	296
96 Wirtualne sieci prywatne, protokół IPsec.	297
96.1 Zaufany VPN (z ang. Trusted VPN)	297
96.2 Bezpieczny VPN (Secure VPN)	298
96.3 Hybrydowy VPN	298
96.4 Zalety VPN	298

96.5	Protokół IPsec	299
96.5.1	Usługi zapewniane przez IPsec	299
96.5.2	Składowe standardu IPsec	299
96.5.3	Security Association	300

Matematyczne podstawy informatyki

1 Zasada indukcji matematycznej.

Twierdzenie 1.1 Zasada indukcji matematycznej. Niech $T(n)$ - funkcja/forma zdaniowa zmiennej $n \in \mathbb{N}$. Jeżeli:

1. zachodzi $T(0)$
2. $\forall n \in \mathbb{N} \quad T(n) \Rightarrow T(n+1)$

to wtedy $T(n)$ jest prawdziwa dla każdego $n \in \mathbb{N}$.

Schemat dowodu:

Niech $M = \{n \in \mathbb{N} : T(n) \text{ zachodzi}\}$, $M \subset \mathbb{N}$. Wtedy wg twierdzenia:

1. $\Rightarrow 0 \in M$
2. $\Rightarrow n \in M \Rightarrow n+1 \in M$

Zatem z **aksjomatu 5** wnioskujemy $M = \mathbb{N}$.

Twierdzenie 1.2 Aksjomat 5 liczb naturalnych (Peano). Niech będzie dany zbiór, którego elementami są liczby naturalne, o następujących właściwościach:

1. J jest elementem tego zbioru.
2. Wraz z liczbą naturalną należącą do tego zbioru, należy do niego również jej następnik.

Wtedy zbiór ten zawiera wszystkie liczby naturalne. $(Z \subset \mathbb{N}) \wedge (J \in Z) \wedge (\forall k \in \mathbb{N} k^* \in Z) \Rightarrow Z = \mathbb{N}$.

2 Porządki częściowe i liniowe. Elementy największe, najmniejsze, maksymalne i minimalne.

Definicja 2.1 *Częściowy porządek.* Niech X zbiór, $R \subset X \times X$ relacja. Wtedy R nazywamy **relacją częściowego porządku** w $X \Leftrightarrow$

1. R **zwrotna** ($\forall x \in X \ xRx$),
2. R **przechodnia** ($\forall x, y, z \in X \ xRy \wedge yRz \Rightarrow xRz$),
3. R **antysymetryczna** ($\forall x, y \in X \ xRy \wedge yRx \Rightarrow x = y$).

Piszemy: $\leq, \preceq, \prec$ gdy $\neq$. Przykład: $(\mathbb{R}, \leq)$, gdzie $x \preceq y \Leftrightarrow x \leq y \ \forall x, y \in \mathbb{R}$. Wtedy $(\mathbb{R}, \preceq)$ jest częściowym porządkiem.

Jeżeli $(X, \mathbb{R})$ jest częściowym porządkiem, to elementy $x, y \in X$ nazywamy **porównywalnymi** $\Leftrightarrow xRy \vee yRx$.

Diagram Hassego - graf skierowany przedstawiający częściowy porządek w zbiorze, w odpowiedni sposób przedstawiony graficznie.

Definicja 2.2 *Liniowy porządek.* Niech X zbiór, $R \subset X \times X$ relacja. Wtedy R nazywamy **relacją liniowego porządku** w $X \Leftrightarrow$

1. R **zwrotna** ($\forall x \in X \ xRx$),
2. R **przechodnia** ($\forall x, y, z \in X \ xRy \wedge yRz \Rightarrow xRz$),
3. R **antysymetryczna** ($\forall x, y \in X \ xRy \wedge yRx \Rightarrow x = y$),
4. R **spójna** ($\forall x, y \in X \ xRy \vee yRx \vee x = y$).

Definicja 2.3 Niech $\preceq$ jest relacją częściowego porządku wówczas element m jest to:

1. **Element maksymalny**, jeśli $\forall a \in A \quad m \preceq a \Rightarrow a = m$,
2. **Element minimalny**, jeśli $\forall a \in A \quad a \preceq m \Rightarrow a = m$,
3. **Element największy**, jeśli $\forall a \in A \quad a \preceq m$,
4. **Element najmniejszy**, jeśli $\forall a \in A \quad m \preceq a$.

3 Relacja równoważności i zbiór ilorazowy.

Definicja 3.1 Relację $R \subset X \times X$ nazywamy **relacją równoważności** $\Leftrightarrow$ relacja R jest:

1. **zwrotna** $(\forall x \in X \ xRx)$,
2. **symetryczna** $(\forall x, y \in X \ xRy \Rightarrow yRx)$,
3. **przechodnia** $(\forall x, y, z \in X \ xRy \wedge yRz \Rightarrow xRz)$.

Definicja 3.2 Niech $R \subset X \times X$ będzie relacją równoważności, $X \neq \emptyset$, $x \in X$. **Klasą abstrakcji** elementu x (względem relacji R) nazywamy:

$$[x]_R = \{y \in X : xRy\}$$

Element x nazywamy reprezentantem klasy abstrakcji $[x]_R$.

Definicja 3.3 **Zbiorem ilorazowym** zbioru X przez relację R nazywamy zbiór wszystkich klas abstrakcji.

$$X/R = \{[y]_R : y \in X\} \subset X$$

4 Metody dowodzenia twierdzeń: wprost, nie wprost, przez kontrapozycję.

Dla prawdziwości zdania:

$$p \Rightarrow q$$

Definicja 4.1 Dowód wprost. Metoda dowodu wprost polega na założeniu, że p jest prawdą i pokazaniu, że wówczas q jest prawdą.

Definicja 4.2 Dowód nie wprost. Metoda dowodu nie wprost opiera się na następującej tautologii rachunku zdań, zwanej prawem kontrapozycji:

$$(p \Rightarrow q) \Leftrightarrow (\neg q \Rightarrow \neg p).$$

Zatem stosując tę metodę zakładamy, że q jest zdaniem fałszywym i pokazujemy, że p jest również zdaniem fałszywym.

Definicja 4.3 Dowód przez zaprzeczenie Metoda dowodu przez zaprzeczenie opiera się na następującej tautologii rachunku zdań:

$$(p \Rightarrow q) \Leftrightarrow (\neg p \vee q) \Leftrightarrow \neg(p \wedge \neg q)$$

Stosując to podejście zakładamy, że p jest prawdą a q fałszem i pokazujemy, że prowadzi to do sprzeczności, to znaczy, pokazujemy że $(p \wedge \neg q)$ jest fałszem.

5 Metody numeryczne rozwiązywania równań nieliniowych: bisekcji, siecznych, Newtona.

5.1 Metoda połowienia (bisekcji)

Założenia:

- f jest funkcją ciągłą w przedziale $[a, b]$,
- $f(a)f(b) < 0$.

Z własności Darboux funkcji ciągłych, funkcja f ma miejsce zerowe w przedziale $[a, b]$.

Definicja 5.1 *Algorytm bisekcji polega na obliczeniu $f(c_k)$, gdzie $c_k = \frac{a_k+b_k}{2}$ i zastąpieniu przez c_k tej z liczb a_k, b_k dla której funkcja f ma taki sam znak.*

$$\begin{aligned}(a_{k+1}, b_{k+1}) &= (c_k, b_k) \text{ jeżeli } f(a_k)f(c_k) > 0 \\ (a_{k+1}, b_{k+1}) &= (a_k, c_k) \text{ jeżeli } f(b_k)f(c_k) > 0\end{aligned}$$

Jeżeli $f(c_k) = 0$ to kończymy obliczenia.

- Metoda bisekcji jest **niezawodna**, ale **wolno zbieżna**.
- W każdym kroku szerokość przedziału jest dzielona przez dwa.
- Kryteria zakończenia obliczeń:
 - osiągnięto dokładność $\delta : |e_n| < \delta$
 - wartość funkcji jest bliska 0: $f(c_n) < \epsilon$
 - wykonano M iteracji
- Algorytm bisekcji w k -tej iteracji przybliża rozwiązanie α z **dokładnością**:
 $|x_k - \alpha| \leq \frac{|b-a|}{2^k}$.

5.2 Metoda siecznych

Założenia:

- f jest funkcją ciągłą w przedziale $[a, b]$,

- $f(a)f(b) < 0$.

$$x_{k+1} = x_k - \frac{f(x_k)(x_k - x_{k-1})}{f(x_k) - f(x_{k-1})}$$

gdzie $x_0 = a$, $x_1 = b$.

- W metodzie siecznych rezygnujemy z założenia, że funkcja na końcach przedziału ma różne znaki.
- Należy kontrolować zachowanie otrzymanego ciągu. **Może się zdarzyć, że metoda wyprodukuje ciąg rozbieżny!**
- korzyści płynące ze stosowania tej metody to zdecydowanie **szybsza zbieżność** ciągu iteracji, jeśli x_n, x_{n+1} już są dobrymi **przybliżeniami pierwiastka**.

5.3 Metoda Newtona (stycznych)

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Zmodyfikowana metoda Newtona:

$$x_{n+1} = x_n - \frac{f'(x_n) \pm \sqrt{(f'(x_n))^2 - 2f(x_n)f''(x_n)}}{f''(x_n)}$$

- Metoda Newtona w wielu sytuacjach daje **bardzo szybką zbieżność**.
- Podstawową wadą tej metody jest konieczność obliczenia pochodnej funkcji.
- Zmodyfikowana metoda Newtona daje bardzo szybką zbieżność, jeśli x_n jest już dobrym przybliżeniem pierwiastka.
- Koszt wyznaczenia kolejnego przybliżenia w zmodyfikowanej metodzie może przerastać korzyści płynące z szybszej zbieżności.

Wielowymiarowa metoda Newtona.

Jeśli $f = (f_1, f_2, \dots, f_n) : \mathbb{R}^n \rightarrow \mathbb{R}^n$ jest różniczkowalną funkcją wielu zmiennych możemy przybliżyć ją lokalnie:

$$f(x) \approx f(x_0) + Df(x_0)(x - x_0)$$

gdzie

$$Df(x_0) = \begin{bmatrix} \frac{\delta f_1}{\delta x_1}(x_0) & \frac{\delta f_1}{\delta x_2}(x_0) & \dots & \frac{\delta f_1}{\delta x_n}(x_0) \\ \frac{\delta f_2}{\delta x_1}(x_0) & \frac{\delta f_2}{\delta x_2}(x_0) & \dots & \frac{\delta f_2}{\delta x_n}(x_0) \\ \dots & \dots & \dots & \dots \\ \frac{\delta f_n}{\delta x_1}(x_0) & \frac{\delta f_n}{\delta x_2}(x_0) & \dots & \frac{\delta f_n}{\delta x_n}(x_0) \end{bmatrix}$$

Rozwiązując równanie $f(x_0) + Df(x_0)(x - x_0) = 0$ otrzymujemy:

$$x^{(i+1)} = x^{(i)} - [Df(x^{(i)})]^{-1} f(x^{(i)})$$

$$[Df(x^{(i)})](x^{(i+1)} - x^{(i)}) = -f(x^{(i)})$$

6 Rozwiązywanie układów równań liniowych: metoda eliminacji Gaussa, metody iteracyjne Jacobiego i Gaussa-Seidla.

6.1 Metoda eliminacji Gaussa

Obliczając rząd macierzy metodą Gaussa należy za pomocą operacji elementarnych na wierszach sprowadzić macierz do macierzy schodkowej. Wtedy wszystkie niezerowe wiersze są liniowo niezależne i można łatwo odczytać rząd macierzy.

$$\begin{bmatrix} 1 & -1 & 2 & 2 \\ 2 & -2 & 1 & 0 \\ -1 & 2 & 1 & -2 \\ 2 & -1 & 4 & 0 \end{bmatrix} \xrightarrow{w_2-2w_1, w_3+w_1, w_4-2w_1} \begin{bmatrix} 1 & -1 & 2 & 2 \\ 0 & 0 & -3 & -4 \\ 0 & 1 & 3 & 0 \\ 0 & 1 & 0 & -4 \end{bmatrix} \xrightarrow{w_2 \leftrightarrow w_3} \begin{bmatrix} 1 & -1 & 2 & 2 \\ 0 & 1 & 3 & 0 \\ 0 & 0 & -3 & -4 \\ 0 & 1 & 0 & -4 \end{bmatrix} \sim$$

$$\xrightarrow{w_4-w_2} \begin{bmatrix} 1 & -1 & 2 & 2 \\ 0 & 1 & 3 & 0 \\ 0 & 0 & -3 & -4 \\ 0 & 0 & -3 & -4 \end{bmatrix} \xrightarrow{w_4-w_3} \begin{bmatrix} 1 & -1 & 2 & 2 \\ 0 & 1 & 3 & 0 \\ 0 & 0 & -3 & -4 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Metody iteracyjne

Ogólna postać metody iteracyjnej:

$$Ax = b$$

$$Qx^{n+1} = (Q - A)x^n + b = \tilde{b}$$

$$x^0 = (0, 0, 0)$$

$$\begin{bmatrix} 5 & -2 & 3 \\ 2 & 4 & 2 \\ 2 & -1 & -4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 10 \\ 0 \\ 0 \end{bmatrix}$$

$$\begin{cases} 5x_1 + (-2)x_2 + 3x_3 = 10 \\ 2x_1 + 4x_2 + 2x_3 = 0 \\ 2x_1 + (-1)x_2 + (-4)x_3 = 0 \end{cases}$$

6.2 Metoda iteracyjna Jacobiego

6.2.1 Algebraicznie

$$\begin{cases} x_1^{N+1} = \frac{1}{5}(10 + 2x_2^N - 3x_3^N) \\ x_2^{N+1} = \frac{1}{4}(-2x_1^N - 2x_3^N) \\ x_3^{N+1} = -\frac{1}{4}(x_2^N - 2x_1^N) \end{cases}$$

6.2.2 Macierzowo

$$Q = D \quad (\text{diagonalna})$$

6.3 Metoda iteracyjna Gaussa-Seidla

6.3.1 Algebraicznie

$$\begin{cases} x_1^{N+1} = \frac{1}{5}(10 + 2x_2^N - 3x_3^N) \\ x_2^{N+1} = \frac{1}{4}(-2x_1^N - 2x_3^{N+1}) \\ x_3^{N+1} = -\frac{1}{4}(x_2^{N+1} - 2x_1^{N+1}) \end{cases}$$

6.3.2 Macierzowo

$$Q = L + D \quad (\text{diagonalna i dolnotrójkątna})$$

7 Wartości i wektory własne macierzy: numeryczne algorytmy ich wyznaczania.

$$A \in \mathbb{C}^{n \times n} \quad \text{szukamy} \quad \lambda \in \mathbb{C}$$

Definicja 7.1 *Jeżeli*

$$Ax = \lambda x$$

*dla $x \neq 0$ to λ jest **wartością własną** A , a x - **wektorem własnym** A odpowiadającym λ .*

Definicja 7.2 *Metoda potęgowa służy do wyznaczania **największej wartości własnej** macierzy. Korzysta z założeń:*

$$\begin{aligned} Ax &= \lambda x \\ x^{N+1} &= A^{N+1}x^0 \end{aligned}$$

Algorytm:

1. Dla danego x_0 oblicz $y_0 = \frac{x_0}{\|x_0\|_2}$
2. W i -tej iteracji: $x_i = Ay_i$, $y_i = \frac{x_i}{\|x_i\|_2}$
3. i -te przybliżenie największej wartości własnej: $\lambda_i = \frac{\langle x_i, y_{i-1} \rangle}{\langle y_{i-1}, y_{i-1} \rangle}$

Definicja 7.3 *Odwrotna metoda potęgowa służy do wyznaczenia **najmniejszej wartości własnej** macierzy.*

Algorytm:

1. $Ax_1 = x_0$, $Ax_i = x_{i-1}$
2. i -te przybliżenie najmniejszej wartości własnej: $\frac{1}{\lambda_i} = \frac{\langle x_i, x_{i-1} \rangle}{\langle x_{i-1}, x_{i-1} \rangle}$

Definicja 7.4 Metoda QR pozwala na przybliżone wyznaczenie wszystkich wartości własnych macierzy. Dla:

$$1. A_i = Q_i R_i$$

$$2. A_{i+1} = R_i Q_i$$

A_i dąży do macierzy przekątniowej z wartościami własnymi na przekątnej.

Q, R to macierze z rozkładu QR :

$A = QR$, Q – ortonormalna ($QQ^T = I$), R – górnotrójkątna

$$A = [a_1 \dots a_n], Q = [q_1 \dots q_n], q_1 = \frac{a_1}{\|a_1\|_2}, q_i = \frac{\tilde{q}_i}{\|\tilde{q}_i\|_2}$$

$$\tilde{q}_i = a_i - \sum_{j=1}^{i-1} r_{ji} q_j, \quad r_{ji} = \frac{\langle a_i, q_j \rangle}{\langle q_j, q_j \rangle}, r_{jj} = \|\tilde{q}_j\|_2$$

8 Interpolacja wielomianowa: metody Lagrange’a i Hermite’a. Efekt Rungego.

8.1 Interpolacja wielomianowa

Interpolacja wielomianowa jest metodą numeryczną przybliżania funkcji. Wielomian p o możliwie najniższym stopniu interpoluje wartości y_k w węzłach x_k gdy dla danych $n + 1$ punktów (x_i, y_i)

$$p(x_i) = y_i \quad (0 \leq i \leq n)$$

Jeśli są to wartości pewnej funkcji f to mówimy też że p interpoluje f .

8.2 Wzór interpolacyjny Lagrange’a

Wyrażamy wielomian p jako sumę

$$p(x) = \sum_{k=0}^n y_k l_k(x)$$

w której l_k są wielomianami zależnymi od węzłów $x_0, x_1, \dots, x_n$ ale nie od wartości $y_0, y_1, \dots, y_n$. Gdyby jedna z nich, mianowicie y_i , była równa 1, a pozostałe by zniknęły, to mielibyśmy równość

$$\delta_{ij} = p_n(x_j) = \sum_{k=0}^n y_k l_k(x_j) = \sum_{k=0}^n \delta_k l_k(x_j) = l_i(x_j) \quad (0 \leq j \leq n)$$

(δ_{ij} jest deltą Kroneckera). Łatwo znaleźć wielomian l_i o tej własności. Jego zerami są wszystkie węzły oprócz x_i , czyli dla pewnej stałej c jest

$$l_i(x) = c(x - x_0) \dots (x - x_{i-1})(x - x_{i+1}) \dots (x - x_n)$$

a c wynika z warunku $l_i(x_i) = 1$:

$$l_i(x) = \prod_{j=0 \wedge j \neq i}^n \frac{x - x_j}{x_i - x_j}. \quad (0 \leq i \leq n).$$

Zatem wzór interpolacyjny Lagrange’a ma postać:

$$w(x) = \sum_{i=0}^n y_i \prod_{j=0 \wedge j \neq i}^n \frac{x - x_j}{x_i - x_j}.$$

8.3 Interpolacja Hermite’a

Interpolacja umożliwia znalezienie wielomianu przybliżającego według wartości

$$y_1 = f(x_1), y_2 = f(x_2), \dots, y_n = f(x_n)$$

na n zadanych węzłach $x_1, x_2, \dots, x_n$ oraz na wartościach pochodnych na wybranych węzłach

$$f'(x_1), f''(x_1), \dots, f^{(k_1)}(x_1), \dots, f'(x_n), \dots, f^{(k_n)}(x_n).$$

Węzeł zadany bez pochodnej jest węzłem pojedynczym, a węzeł z zadanymi pochodnymi $1, 2, \dots, k$ jest węzłem $k + 1$ -krotnym.

8.3.1 Algorytm

Algorytm jest podobny jak przy interpolacji Newtona. Kolumnę wypełnia się wszystkimi wartościami węzłów (jeżeli węzeł jest k -krotny, to umieszczamy go w tabeli k razy)

x_i	$f(x_i)$
x_0	$f(x_0)$
x_1	$f(x_1)$
x_2	$f(x_2)$
$\vdots$	$\ddots$
x_n	$f(x_n)$

Następnie dopisuje się do każdej kolumny kolejne różnice dzielone, z tym wyjątkiem, że przy węzłach k -krotnych, $k > 1$, gdzie, de facto, nie można obliczyć różnicy dzielonej, podstawia się wartości kolejnych pochodnych na węzłach podzielone przez silnię ze stopnia pochodnej. (W tabeli przedstawiony jest 3-krotny węzeł x_1).

x_i	$f(x_i)$	$f[x_{i-1}]$	$f[x_{i-2}, x_{i-1}, x_i]$
x_0	$f(x_0)$		
x_1	$f(x_1)$	$f[x_0, x_1]$	
x_1	$f(x_1)$	$f'(x_1)$	$f[x_0, x_1, x_1]$
x_1	$f(x_1)$	$f'(x_1)$	$\frac{f''(x_1)}{2!}$
x_2	$f(x_2)$	$f[x_1, x_2]$	$f[x_1, x_1, x_2]$
$\vdots$	$\vdots$	$\vdots$	
x_n	$f(x_n)$	$f[x_{n-1}, x_n]$	$f[x_{n-2}, x_{n-1}, x_n]$

Tabele uzupełnia się do końca jak przy interpolacji Newtona, uznając ciągłe pochodne na węzłach wielokrotnych jako różnice dzielone rzędu drugiego.

x_i	$f(x_i)$	$f[x_{i-1}]$	$f[x_{i-2}, x_{i-1}, x_i]$	$f[x_{i-3}, x_{i-2}, x_{i-1}, x_i]$	$\dots$	$f[x_{i-n}, \dots, x_i]$
x_0	$f(x_0)$					
x_1	$f(x_1)$	$f[x_0, x_1]$				
x_1	$f(x_1)$	$f'(x_1)$	$f[x_0, x_1, x_1]$			
x_1	$f(x_1)$	$f'(x_1)$	$\frac{f''(x_1)}{2!}$	$f[x_0, x_1, x_1, x_1]$		
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	
x_n	$f(x_n)$	$f[x_{n-1}, x_n]$	$f[x_{n-2}, x_{n-1}, x_n]$	$f[x_{n-3}, x_{n-2}, x_{n-1}, x_n]$	$\dots$	$f[x_0, \dots, x_n]$

Definiując a_i jako wartości na przekątnej, $i = 1, 2, 3, \dots, m$, gdzie m to suma krotności węzłów, otrzymuje się wielomian:

$$w(x) = \sum_{i=0}^m a_i \prod_{j=0}^{i-1} (x - \bar{x}_j),$$

gdzie $\bar{x}_i = x_1, x_1, \dots, x_1, x_2, x_2, \dots, x_2, \dots, x_n$, przy czym każdy k -krotny węzeł występuje k razy.

8.4 Efekt Rungego

Efekt Rungego - pogorszenie jakości interpolacji wielomianowej, mimo zwiększenia liczby jej węzłów. Początkowo ze wzrostem liczby węzłów n przybliżenie poprawia się, jednak po dalszym wzroście n , zaczyna się pogarszać, co jest szczególnie widoczne na końcach przedziałów.

Takie zachowanie się wielomianu interpolującego jest zjawiskiem typowym dla interpolacji za pomocą wielomianów wysokich stopni przy stałych odległościach węzłów. Występuje ono również, jeśli interpolowana funkcja jest nieciągła albo odbiega znacząco od funkcji gładkiej.

Aby uniknąć tego efektu, stosuje się interpolację z węzłami coraz gęściej upakowanymi na krańcach przedziału interpolacji. Np. węzłami interpolacji n -punktowej wielomianowej powinny być miejsca zerowe wielomianu Czebyszewa n -tego stopnia.

9 Zmienne losowe dyskretne. Definicje i najważniejsze rozkłady.

Definicja 9.1 *Zmienne dyskretne.*

$$P(x) = P(X = x)$$

Obliczanie prawdopodobieństwa: $P(X \in A) = \sum_{x \in A} P(x)$.

Skumulowana funkcja rozkładu: $F(x) = P(X \leq x) = \sum_{y \leq x} P(y)$

Całkowite prawdopodobieństwo: $\sum_x P(x) = 1$.

Wartość oczekiwana: $EX = \sum_x xP(x)$.

Wariancja: $VarX = \sigma^2 = E[(X - \mu)^2]$.

Rozkład	$P(x)$	EX	VarX	
Bernoulli(p)	$P(x) = \begin{cases} p, & \text{for } x = 1 \\ q = (1 - p), & \text{for } x = 0 \end{cases}$	p	pq	próba
Binomial(n, p)	$P(x) = \binom{n}{x} p^x (1 - p)^{n-x} \text{ for } x = 0, 1, \dots$	np	npq	liczba sukcesów z n prób
Geometric(p)	$P(x) = (1 - p)^{x-1} p \text{ for } x = 1, 2, \dots$ $P(X > k) = (1 - p)^k$	$\frac{1}{p}$	$\frac{1-p}{p^2}$	liczba prób do sukcesu
Poiss(λ)	$P(x) = e^{-\lambda} \frac{\lambda^x}{x!} \text{ for } x = 0, 1, \dots$	λ	λ	rozkład zdarzeń rzadkich

10 Zmienne losowe ciągłe. Definicje i najważniejsze rozkłady.

Definicja 10.1 *Zmienne ciągłe.*

$$f(x) = F'(x)$$

Obliczanie prawdopodobieństwa: $P(X \in A) = \int_A f(x)dx$.

Skumulowana funkcja rozkładu: $F(x) = P(X \leq x) = \int_{-\infty}^x f(y)dy$.

Całkowite prawdopodobieństwo: $\int_{-\infty}^{\infty} f(x)dx = 1$.

Wartość oczekiwana: $EX = \int x f(x)dx$.

Wariancja: $VarX = \int_{-\infty}^{\infty} (x - \mu)^2 f(x)dx$.

Rozkład	f(x), F(x)	EX	VarX	
Unif(a,b)	$f(x) = \frac{1}{b-a}$ for $a \leq x \leq b$ $F(x) = \begin{cases} 0, & \text{for } x < a \\ \frac{x-a}{b-a}, & \text{for } a \leq x < b \\ 1, & \text{for } x \geq b \end{cases}$	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$	
Exp(λ)	$f(x) = \lambda e^{-\lambda x}$ for $x \geq 0$ $F(x) = 1 - e^{-\lambda x}$	$\frac{1}{\lambda}$	$\frac{1}{\lambda^2}$	modelowanie czasu, brak pamięci
Gamma(α, λ)	$f(x) = \frac{\lambda^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\lambda x}$ $F(x) = \frac{\lambda^\alpha}{\Gamma(\alpha)} \int_0^x t^{\alpha-1} e^{-\lambda t} dt$	$\frac{\alpha}{\lambda}$	$\frac{\alpha}{\lambda^2}$	łączny czas α niezależnych zdarzeń $\sim Exp(\lambda)$
N(μ, σ)	$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$ $F(x) = \Phi(x)$ dla N(0,1)	μ	σ^2	

$$Bin(n, p) \approx Poiss(\lambda) \quad (1)$$

$$P(T \leq t) = P(X \geq \alpha) \quad (2)$$

$$T \sim Gamma(\alpha, \lambda), X \sim Poiss(\lambda t)$$

$$Binomial(n, p) = N(np, \sqrt{np(1-p)}) \tag{3}$$

$$X_i \sim Bernoulli(p), S_n = \sum_{i=1}^n X_i, 0.05 \leq p \leq 0.95$$

11 Łancuchy Markowa. Rozkład stacjonarny.

$$P = \begin{bmatrix} p_{11} & p_{12} & \dots & p_{1n} \\ p_{21} & p_{22} & \dots & p_{2n} \\ \dots & \dots & \dots & \dots \\ p_{n1} & p_{n2} & \dots & p_{nn} \end{bmatrix}$$

Rozkład w czasie h : $P_h = P_0 * P^h$

Rozkład stacjonarny: $\pi P = \pi, \sum \pi_i = 1$

12 Testy statystyczne: test z, test t-Studenta, test chi-kwadrat.

Rozkład t-studenta

$t = \frac{\hat{\theta} - \theta}{s(\hat{\theta})} \leftarrow$ zastępujemy $Std(\hat{\theta})$ przez $s(\hat{\theta})$, $n-1$ stopni swobody

$$\bar{X} \pm t_{\frac{\alpha}{2}}^{(n-1)} \frac{s(\hat{\theta})}{\sqrt{n}}$$

Z-testy

Hipoteza ze- rowa	Parametr, estymator	jeśli H_0 jest prawdziwa:		Statystyka
H_0	$\theta, \hat{\theta}$	$E(\hat{\theta})$	$Var(\hat{\theta})$	$Z = \frac{\hat{\theta} - \theta_0}{\sqrt{Var(\hat{\theta})}}$
$\mu = \mu_0$	$\mu, \bar{X}$	μ_0	$\frac{\sigma^2}{n}$	$Z = \frac{\bar{X} - \mu_0}{\frac{\sigma}{\sqrt{n}}}$
$p = p_0$	$p, \hat{p}$	p_0	$\frac{p_0(1-p_0)}{n}$	$Z = \frac{\hat{p} - p_0}{\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}}$
$\mu_X - \mu_Y = D$	$\mu_X - \mu_Y,$ $\bar{X} - \bar{Y}$	D	$\frac{\sigma_X^2}{n} + \frac{\sigma_Y^2}{n}$	$Z = \frac{\bar{X} - \bar{Y} - D}{\sqrt{\frac{\sigma_X^2}{n} + \frac{\sigma_Y^2}{n}}}$
$p_1 - p_2 = D$	$p_1 - p_2,$ $\hat{p}_1 - \hat{p}_2$	D	$\frac{p_1(1-p_1)}{n} + \frac{p_2(1-p_2)}{m}$	$Z = \frac{\hat{p}_1 - \hat{p}_2 - D}{\sqrt{\frac{\hat{p}_1(1-\hat{p}_1)}{n} + \frac{\hat{p}_2(1-\hat{p}_2)}{m}}}$
$p_1 = p_2$	$p_1 - p_2,$ $\hat{p}_1 - \hat{p}_2$	0	$p(1-p) \left(\frac{1}{n} + \frac{1}{m} \right)$ gdzie $p = p_1 = p_2$	$Z = \frac{\hat{p}_1 - \hat{p}_2}{\sqrt{\hat{p}(1-\hat{p}) \left(\frac{1}{n} + \frac{1}{m} \right)}}$ gdzie $\hat{p} = \frac{n\hat{p}_1 + m\hat{p}_2}{n+m}$

T-testy

Hipoteza zerowa	Warunki	Statystyka	Stopnie swobody
$\mu = \mu_0$	Rozmiar próby n ; nieznana σ	$t = \frac{\bar{X} - \mu_0}{\frac{s}{\sqrt{n}}}$	$n - 1$
$\mu_X - \mu_Y = D$	Rozmiary prób n, m nie- znane, równe $\sigma_X = \sigma_Y$	$t = \frac{\bar{X} - \bar{Y} - D}{s_p \sqrt{\frac{1}{n} + \frac{1}{m}}}$	$n + m - 2$
$\mu_X - \mu_Y = D$	Rozmiary prób n, m ; nieznane, różne $\sigma_X \neq \sigma_Y$	$t = \frac{\bar{X} - \bar{Y} - D}{\sqrt{\frac{s_X^2}{n} + \frac{s_Y^2}{m}}}$	aproxymacja Satterthwaite

Rozkład obserwacji o rozkładzie normalnym i wspólnej wariancji σ^2

$$\frac{(n-1)s^2}{\sigma^2} = \sum_{i=1}^n \left(\frac{X_i - \bar{X}}{\sigma} \right)^2 \sim \text{Chi-square}(n-1) \sim \text{Gamma}\left(\frac{n-1}{2}, \frac{1}{2}\right)$$

Zatem przedział ufności:

$$\left[\frac{(n-1)s^2}{\chi_{\frac{\alpha}{2}}^2}, \frac{(n-1)s^2}{\chi_{1-\frac{\alpha}{2}}^2} \right]$$

Testy Chi kwadrat

H_0	H_A	Test statistic	Rejection region	P-value
$\sigma^2 = \sigma_0^2$	$\sigma^2 > \sigma_0^2$ $\sigma^2 < \sigma_0^2$ $\sigma^2 \neq \sigma_0^2$	$\frac{(n-1)s^2}{\sigma_0^2}$	$\chi_{obs}^2 > \chi_{\alpha}^2$ $\chi_{obs}^2 < \chi_{\alpha}^2$ $\chi_{obs}^2 \geq \chi_{\frac{\alpha}{2}}^2$ or $\chi_{obs}^2 \leq \chi_{\frac{\alpha}{2}}^2$	$P\chi^2 \geq \chi_{obs}^2$ $P\chi^2 \leq \chi_{obs}^2$ $2 \min(P\chi^2 \geq \chi_{obs}^2, P\chi^2 \leq \chi_{obs}^2)$

Statystyka Chi-kwadrat

$$\chi^2 = \sum_{k=1}^N \frac{(Obs(k) - Exp(k))^2}{Exp(k)}, R = [\chi_{\alpha}^2, +\infty], P = P\chi^2 \geq \chi_{obs}^2$$

Rule of thumb: $Exp(k) \geq 5$ for all $k = 1, \dots, N$.

Test Chi-kwadrat niezależności A i B

$$\chi_{obs}^2 = \sum_{i=1}^k \sum_{j=1}^m \frac{(Obs(i, j) - \hat{Exp}(i, j))^2}{\hat{Exp}(i, j)}, \hat{Exp}(i, j) = \frac{(n_{i.})(n_{.j})}{n}$$

13 Wzór Bayesa i jego interpretacja.

Prawdopodobieństwo warunkowe

$$P(E|F) = \frac{P(E \cap F)}{P(F)} \quad (4)$$

$$P(E_1 \cap \dots \cap E_n) = \prod_{i=1}^n P(X_i | X_1, \dots, X_{i-1}) \quad (5)$$

$$P(E) = \sum_{i=1}^n P(E|F_i)P(F_i) \text{ dla } \bigcap_{i=1}^n F_i = \Omega \quad (6)$$

Wzór Bayesa:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} \quad \text{przy } P(B) > 0$$

dowód:

$$P(B|A) = \frac{P(A \cap B)}{P(A)} \rightarrow P(A \cap B) = P(B|A) * P(A)$$

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \rightarrow P(A|B) * P(B) = P(A \cap B) = P(B|A) * P(A) \quad / : P(B)$$

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

14 Istnienie elementów odwrotnych względem mnożenia w strukturze $(Z_m, +, *)$ w zależności od liczby naturalnej m . Rozszerzony algorytm Euklidesa.

Twierdzenie 14.1 *Tożsamość Bézouta*

Dla niezerowych liczb całkowitych a oraz b o największym wspólnym dzielniku d , istnieją liczby całkowite x oraz y , nazywane współczynnikami Bézouta, które spełniają równanie

$$ax + by = d$$

Definicja 14.2 *Elementy odwrotne modulo*

*W strukturze $(Z_m, +, *)$ element odwrotny względem mnożenia dla x to taki x^{-1} , że:*

$$x \cdot x^{-1} = 1 \bmod m$$

Element odwrotny istnieje, jeśli $NWD(x, m) = 1$

Aby obliczyć element odwrotny do x w Z_m należy użyć rozszerzonego algorytmu Euklidesa.

14.1 Algorytm Euklidesa

Oblicza największy wspólny dzielnik a, b

```
def euclid(a, b):  
    while b != 0:  
        r = a % b  
        a, b = b, r  
    return a
```

Wersja z odejmowaniem:

```
def euclid(a, b):  
    while a != b:  
        if a > b:
```

```

a = a - b
else:
b = b - a
return a

```

14.2 Rozszerzony algorytm Euklidesa

Oprócz obliczenia największego wspólnego dzielnika, algorytm znajduje także współczynniki Bézouta.

Poprzedni algorytm obliczał ciąg r_n będący resztami z dzielenia, tzn.:

$$r_0 = a, r_1 = b \quad r_{i+1} = r_{i-1} - q_i r_i$$

gdzie q_i oznacza wynik całkowitego dzielenia r_{i-1} przez r_i .

Wprowadźmy dwa dodatkowe ciągi: x_i oraz y_i :

$$\begin{aligned} x_0 = 1, x_1 = 0 & \quad x_{i+1} = x_{i-1} - q_i x_i \\ y_0 = 0, y_1 = 1 & \quad y_{i+1} = y_{i-1} - q_i y_i \end{aligned}$$

Możemy zauważyć, że:

$$r_i = x_i * a + y_i * b$$

```

def euclid(a, b):
x, x' = 0, 1
y, y' = 1, 0
r, r' = b, a
while r != 0:
q = r' // r
r', r = r, (r' - q * r)
x', x = x, (x' - q * x)
y', y = y, (y' - q * x)
return r', x', y'

```

Ponadto ten algorytm pozwala obliczyć takie x', y' , że: $ax' + by' = 0$ (wystarczy zwrócić x i y , zamiast x', y').

15 Ortogonalność wektorów w przestrzeni $\mathbb{R}^n$; związki z liniową niezależnością. Metoda ortonormalizacji Grama-Schmidta

Definicja 15.1 *Ortogonalność.*

Dla wektorów x, y w przestrzeni $\mathbb{R}^n$ z zadany iloczynem skalarnym $\cdot$

$$x \perp y \Leftrightarrow x \cdot y = 0$$

Definicja 15.2 *Liniowa niezależność*

Wektory $x_1, x_2, \dots, x_n$ w przestrzeni $\mathbb{R}^n$ nazywamy liniowo niezależnymi jeśli

$$a_1 * x_1 + a_2 * x_2 + \dots + a_n * x_n = 0 \Rightarrow a_1 = a_2 = \dots = a_n = 0$$

tzn., nie istnieje taki układ skalarów $a_1, a_2, \dots, a_n$ dla których suma $\sum_{i=1}^n a_i * x_i = 0$ poza i przynajmniej jeden skalar nie równa się zeru.

Zbiór wektorów, który nie jest liniowo niezależny, jest zbiorem liniowo zależnym. Oznacza to, że przynajmniej jeden jego element jest kombinacją liniową pozostałych wektorów.

W przestrzeni $\mathbb{R}^n$ zbiór liniowo niezależnych nie może zawierać więcej niż n wektorów.

Twierdzenie 15.3 *Każdy układ ortogonalny wektorów w przestrzeni $\mathbb{R}^n$ jest liniowo niezależny.*

Definicja 15.4 *Baza przestrzeni $\mathbb{R}^n$*

Zbiór wektorów $B \subset \mathbb{R}^n$ nazywamy bazą przestrzeni $\mathbb{R}^n$ jeśli:

1. Jest liniowo niezależny
2. Generuje przestrzeń $\mathbb{R}^n$ tzn. każdy wektor $\in \mathbb{R}^n$ można zapisać jako kombinację liniową wektorów z B

Twierdzenie 15.5 *Każdy ortogonalny układ n wektorów w $\mathbb{R}^n$ jest bazą.*

Definicja 15.6 Ortonormalność

Zbiór wektorów A przestrzeni $\mathbb{R}^n$ z zadany iloczynem skalarnym $\cdot$ jest ortonormalny, jeśli

$$\forall x, y \in A \quad x \cdot y = \begin{cases} 0, & x \neq y \\ 1, & x = y \end{cases}$$

Zatem wektory należące do A są wersorami.

15.1 Ortonormalizacja Grama-Schmidta

Ortonormalizacja to proces przekształcania układu niezależnych liniowo wektorów w układ wektorów ortonormalnych.

Projekcję wektora $\mathbf{v}$ na wektor $\mathbf{u}$ definiujemy jako:

$$\text{proj}_{\mathbf{u}} \mathbf{v} = \frac{\mathbf{v} \cdot \mathbf{u}}{\mathbf{u} \cdot \mathbf{u}} \mathbf{u}$$

Algorytm:

1. Dla zbioru wektorów $A = \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ oblicz $\{\mathbf{u}_1, \dots, \mathbf{u}_n\}$ gdzie

$$\mathbf{u}_i = \mathbf{v}_i - \sum_{j=1}^{i-1} \text{proj}_{\mathbf{u}_j} \mathbf{v}_i$$

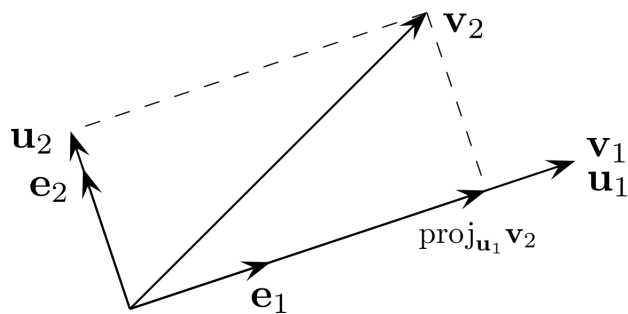
tzn:

$$\mathbf{u}_1 = \mathbf{v}_1$$

$$\mathbf{u}_2 = \mathbf{v}_2 - \text{proj}_{\mathbf{u}_1} \mathbf{v}_2$$

$$\mathbf{u}_3 = \mathbf{v}_3 - \text{proj}_{\mathbf{u}_1} \mathbf{v}_3 - \text{proj}_{\mathbf{u}_2} \mathbf{v}_3$$

2. Wektory $\{\mathbf{u}_1, \dots, \mathbf{u}_n\}$ podziel przez ich normy, uzyskując $\{\mathbf{e}_1, \dots, \mathbf{e}_n\}$



16 Liczby Stirlinga I i II rodzaju i ich interpretacja.

Definicja 16.1 *Liczby Stirlinga I rodzaju.*

Dla dowolnego $n \geq 1$ mamy:

1. $c(0, 0) = 1$
2. $c(n, 0) = c(0, n) = 0$
3. $c(n, k) = c(n-1, k-1) + (n-1) * c(n-1, k)$

W szczególności:

1. $c(n, n) = 1, c(n, 1) = (n-1)!$
2. $\sum_{k=0}^n c(n, k) = n!$

Interpretacja: Przez $c(n, k)$ ($[n]_k$) oznaczamy liczbę permutacji zbioru n -elementowego, które mają rozkład na dokładnie k cykli rozłącznych.

Definicja 16.2 *Liczby Stirlinga II rodzaju.*

Dla dowolnego $n \geq 1$ mamy:

1. $S(0, 0) = 1,$
2. $S(n, 0) = S(0, n) = 0,$
3. $S(n, k) = S(n-1, k-1) + k \times S(n-1, k).$

w szczególności $S(n, n) = S(n, 1) = 1.$

Interpretacja: Przez $S(n, k)$ ($\{n\}_k$) oznaczamy liczbę rozmieszczeń n rozróżnialnych kul na k nierozróżnialnych stosach w taki sposób, aby żaden stos nie był pusty.

17 Twierdzenia Eulera i Fermata; funkcja Eulera.

Definicja 17.1 *Funkcja Eulera (Tocjent)*

Funkcja przypisująca każdej liczbie naturalnej liczbę liczb względnie pierwszych z nią i nie większych od niej.

$$\sum_{m|n} \varphi(m) = n$$

Własności

1. $\varphi(n) \leq n - 1$ dla każdego $n > 1$
2. $\varphi(p) \leq p - 1$ dla każdego p będącego liczbą pierwszą
3. $\varphi(mn) = \varphi(m)\varphi(n)$ jeśli $NWD(m, n) = 1$ (m i n względnie pierwsze)
4. $\varphi(p^k) = p^{k-1} \cdot (p - 1)$ jeśli p jest liczbą pierwszą
5. $\varphi(n) = n \left(1 - \frac{1}{p_1}\right) \left(1 - \frac{1}{p_2}\right) \cdots \left(1 - \frac{1}{p_n}\right)$ gdzie $p_1, p_2, \dots, p_n$ są czynnikami pierwszymi liczby n
6. jeżeli $n = \prod_{i=1}^k p_i^{k_i}$ jest rozkładem liczby n na czynniki pierwsze $\varphi(n) = \prod_{i=1}^k \varphi(p_i^{k_i})$

Definicja 17.2 *Małe Twierdzenie Fermata*

Jeżeli p jest liczbą pierwszą, to dla dowolnej liczby całkowitej a , liczba $a^p - a$ jest podzielna przez p .

$$1. a^{p-a} \equiv 0 \pmod{p}$$

Jeśli p jest liczbą pierwszą, a a jest taką liczbą całkowitą, że liczby a i p są względnie pierwsze to $a^{p-1} - 1$ dzieli się przez p .

$$1. a^{p-1} - 1 \equiv 0 \pmod{p}$$

$$2. a^{p-1} \equiv 1 \pmod{p}$$

Definicja 17.3 Twierdzenie Eulera

Jeżeli $m \in \mathbb{Z}_+$ oraz $a \in \mathbb{Z}$ są liczbami względnie pierwszymi to m dzieli liczbę $a^{\varphi(m)} - 1$, gdzie $\varphi(m)$ oznacza wartość funkcji Eulera.

1. $a^{\varphi(m)} \equiv 1 \pmod{m}$

18 Konfiguracje i t-konfiguracje kombinatoryczne.

Definicja 18.1 *Konfiguracją kombinatoryczną B o parametrach (n, k, r) nazywamy rodzinę k -elementowych podzbiorów w X , jeżeli każdy element $x \in X$ występuje w dokładnie r podbiorach. Zachodzą warunki:*

1. $n = |X|$
2. $|B_i| = k \quad \forall i = 1, \dots, b$
3. $n \cdot r = b \cdot k$ gdzie b to liczba k -elementowych podzbiorów zbioru n -elementowego

Twierdzenie 18.2 *Istnieje konfiguracja kombinatoryczna B o parametrach (n, k, r) wtedy i tylko wtedy, gdy:*

1. $k | n \cdot r$
2. $\frac{n \cdot r}{k} \leq \binom{n}{k}$

Definicja 18.3 *Niech X dowolny zbiór, $|X| = n$. B rodzina k -podzbiorów X jest **t-konfiguracją kombinatoryczną** o parametrach (n, k, r_t) wtw gdy dla każdego t -podzbioru $T \subset X$ liczba bloków z B zawierających T jest równa r_t .*

Twierdzenie 18.4 *Jeśli B jest t -konfiguracją kombinatoryczną, to B jest s -konfiguracją kombinatoryczną dla $s = 1, 2, \dots, t-1$. Uwagi:*

$$1. \ r_{t-1} = r_t \cdot \frac{n-t+1}{k-t+1}$$

2. *Jeżeli B jest t -konfiguracją o parametrach (n, k, r_t) , to dla $1 \leq s \leq (t-1)$*

$$r_s = r_t \cdot \frac{(n-s)(n-s-1) \dots (n-t+1)}{(k-s)(k-s-1) \dots (k-t+1)}$$

3. *Jeśli B jest t -konfiguracją o parametrach (n, k, r_t) , to*

$$(k-s)(k-s-1) \dots (k-t+1) | r_t (n-s)(n-s-1) \dots (n-t+1) \\ \text{dla } 0 \leq s \leq (t-1)$$

warunek 3. dla $t \geq 2$ jest konieczny dla istnienia t -konfiguracji, ale nie wystarczający

19 Cykl Hamiltona, obwód Eulera, liczba chromatyczna - definicje i twierdzenia.

19.1 Cykl Hamiltona

Definicja 19.1 Ścieżka Hamiltona. Ścieżką Hamiltona w grafie G nazywamy ścieżkę, która przechodzi przez wszystkie wierzchołki G .

Definicja 19.2 Cykl Hamiltona. Powiemy, że graf G ma cykl Hamiltona, jeśli istnieje w nim cykl przechodzący przez wszystkie wierzchołki. Taki graf nazywamy hamiltonowskim.

Twierdzenie 19.3 Niech $G = (V, E)$ będzie grafem o $n \geq 3$ wierzchołkach. Jeśli dla dowolnych różnych, niesąsiednich wierzchołków $u, v \in V$ zachodzi warunek $d(u) + d(v) \geq n$, to G jest hamiltonowski.

Twierdzenie 19.4 Jeśli G jest grafem o $n \geq 3$ wierzchołkach w którym minimalny stopień wierzchołka wynosi co najmniej $\frac{n}{2}$, to G jest hamiltonowski.

Twierdzenie 19.5 Niech $G = (V, E)$ będzie grafem o $n \geq 2$ wierzchołkach i takim, że $d(u) + d(v) \geq n - 1$ dla dwóch dowolnych różnych, niesąsiednich wierzchołków $u, v \in V$. Wtedy G ma ścieżkę hamiltona.

19.2 Obwód Eulera

Definicja 19.6 Droga Eulera. Drogą Eulera w grafie G nazywamy drogę $v_1, v_2, \dots, v_m$, w której każda krawędź grafu G użyta jest dokładnie raz.

Definicja 19.7 Obwód Eulera. Jeśli w grafie G istnieje droga Eulera, w której pierwszy i ostatni wierzchołek są identyczne, to nazywamy ją obwodem Eulera. Graf ten nazywamy wtedy eulerowskim.

Twierdzenie 19.8 *Niech G będzie grafem spójnym. Wówczas następujące warunki są równoważne:*

1. G jest grafem eulerskim,
2. stopień każdego wierzchołka w G jest parzysty.

Twierdzenie 19.9 *Niech G będzie grafem spójnym. Wówczas G ma drogę Eulera $\Leftrightarrow$ w G są dokładnie zero lub dwa wierzchołki stopnia nieparzystego.*

19.3 Liczba chromatyczna

Definicja 19.10 Kolorowanie wierzchołkowe. *Kolorowaniem wierzchołkowym grafu $G = (V, E)$ przy użyciu (co najwyżej) k kolorów nazywamy funkcję $c : V \rightarrow \{1, \dots, k\}$ spełniającą warunek $c(u) \neq c(v) \quad \forall u, v \in V : uv \in E$.*

Definicja 19.11 Liczba chromatyczna. *Liczbą chromatyczną grafu G nazywamy najmniejszą liczbę $k \in \mathbb{N}$, dla której istnieje kolorowanie wierzchołkowe G przy użyciu k kolorów. Oznaczamy $\chi(G)$.*

1. $\chi(G) = 1 \Leftrightarrow |E(G)| = 0$
2. $\chi(T) = 2$ dla każdego drzewa T o przynajmniej dwóch wierzchołkach
3. $\chi(C_{2k}) = 2, \chi(C_{2k+1}) = 3$
4. $\chi(K_n) = n$

Twierdzenie 19.12 *Niech G będzie dowolnym grafem. Wtedy $\chi(G) \leq d_{\max}(G) + 1$.*

Twierdzenie 19.13 *Niech G będzie grafem spójnym. Wówczas $\chi(G) \leq d_{\max}(G)$, o ile G nie jest grafem pełnym ani cyklem o nieparzystej liczbie wierzchołków.*

20 Algorytm Forda-Fulkersona wyznaczania maksymalnego przepływu.

Definicja 20.1 *Siecią przepływową* nazywamy zestaw $(V, \vec{E}, s, t, c)$, gdzie $(V, \vec{E})$ jest grafem skierowanym wraz z wyróżnionymi wierzchołkami s - **źródłem** i t - **ujściem** oraz **funkcją pojemności krawędzi** $c : \vec{E} \rightarrow \mathbb{R}$.

Definicja 20.2 *Przepływem* w sieci $(V, \vec{E}, s, t, c)$ nazywamy funkcję $f : \vec{E} \rightarrow \mathbb{R}$ spełniającą następujące warunki:

1. $f(\vec{uv}) \leq c(\vec{uv})$ dla każdej krawędzi $\vec{uv} \in \vec{E}$,
2. dla każdego ustalonego $v \in V \setminus \{s, t\}$:

$$\sum_{\vec{uv} \in \vec{E}} f(\vec{uv}) = \sum_{\vec{vw} \in \vec{E}} f(\vec{vw}).$$

Liczbę $|f| = \sum_{\vec{su} \in \vec{E}} f(\vec{su}) = \sum_{\vec{wt} \in \vec{E}} f(\vec{wt})$ nazywamy **wartością przepływu** f .

Przepływ f nazywamy **maksymalnym**, jeśli jego wartość jest największa spośród wszystkich przepływów w danej sieci.

Definicja 20.3 Niech $(V, \vec{E}, s, t, c)$ będzie siecią przepływową oraz f pewnym przepływem tej sieci. **Siecią rezydualną** nazywamy sieć $G_f = (V, E_f, s, t, c_f)$, gdzie $E_f = \vec{E} \cup \{\vec{vu} : \vec{uv} \in \vec{E}\}$ z funkcją pojemności określoną wzorem:

$$c_f(\vec{uv}) = c(\vec{uv}) - f(\vec{uv}) \quad \text{dla } \vec{uv} \in \vec{E} \text{ t. że } c(\vec{uv}) > 0$$

$$c_f(\vec{vu}) = f(\vec{uv}) \quad \text{dla } \vec{uv} \notin \vec{E} \text{ t. że } \vec{vu} \in \vec{E} \text{ i } f(\vec{uv}) > 0$$

oraz $c_f(\vec{uv}) = 0$ w pozostałych przypadkach.

Definicja 20.4 *Ścieżką rozszerzającą dla przepływu f nazywamy dowolną ścieżkę (skierowaną) $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k$ w sieci rezydualnej G_f łączącą wierzchołki $v_0 = s$ i $v_k = t$, w której $c_f(v_i \vec{v}_{i+1}) > 0$ dla każdego $i = 0, \dots, k-1$.*

Twierdzenie 20.5 *Algorytm Forda-Fulkersona. W celu znalezienia maksymalnego przepływu f w sieci przepływowej $(V, \vec{E}, s, t, c)$:*

1. przyjmujemy początkowo dowolny (np. zerowy) przepływ f ,
2. budujemy sieć rezydualną G_f ,
3. dopóki w G_f istnieje ścieżka rozszerzająca $P = v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k$ określamy $c_p = \min_{i=0, \dots, k-1} c_f(v_i \vec{v}_{i+1})$, a następnie zwiększamy o c_p wartość przepływu f na krawędziach $v_i \vec{v}_{i+1}$ tej ścieżki i uaktualniamy wartości funkcji c_f sieci rezydualnej.

Definicja 20.6 *Przekrojem w sieci przepływowej $(V, \vec{E}, s, t, c)$ nazywamy parę zbiorów (S, T) spełniającą warunki:*

1. $S \cup T = V$,
2. $S \cap T = \emptyset$,
3. $s \in S, t \in T$.

Pojemnością przekroju (S, T) nazywamy liczbę

$$c(S, T) = \sum_{(u,v) \in \vec{E} \cap (S \times T)} c(u\vec{v}).$$

Twierdzenie 20.7 *Niech $(V, \vec{E}, s, t, c)$ będzie siecią przepływową. Wówczas maksymalna wartość przepływu jest równa minimalnej pojemności przekroju w tej sieci.*

21 Rozwiązywanie równań rekurencyjnych przy użyciu funkcji tworzących (generujących) oraz przy użyciu równania charakterystycznego.

Definicja 21.1 *Liniowym równaniem rekurencyjnym rzędu r (o stałych współczynnikach) nazywamy równanie postaci:*

$$x_{n+r} = c_1 x_{n+r-1} + \cdots + c_r x_n + f(n)$$

*w którym $c_i \in \mathbb{R}, c_r \neq 0$ oraz $f : \mathbb{N} \rightarrow \mathbb{R}$ jest dowolną funkcją. Jeśli funkcja f jest stale równa zero, to powyższe równanie nazywamy **jednorodnym**.*

Definicja 21.2 *Rozwiązaniem szczególnym równania nazywamy dowolny ciąg (x_n) spełniający zależność rekurencyjną. Dla każdego równania jednorodnego rozwiązanie szczególnym jest ciąg stały równy zero.*

Rozwiązaniem ogólnym równania nazywamy wzór ogólny (zależny od pewnych parametrów) pozwalający wyznaczyć wszystkie rozwiązania szczególne.

21.1 Funkcje tworzące.

Definicja 21.3 *Funkcją tworzącą (generującą) ciągu (a_n) nazywamy szereg*

$$\sum_{n=0}^{\infty} a_n x^n.$$

Jest to tylko inny zapis ciągu (wyraz a_n to współczynnik przy x^n) - nie interesują nas kwestie zbieżności, podstawiania wartości za x , ciągłości/różniczkowalności funkcji określonej takim szeregiem itd.

Jeżeli jednak funkcja tworząca pewnego ciągu "wygląda" jak zbieżny szereg określający funkcję znaną z analizy to możemy używać skróconego zapisu, np. $\sum_{n=0}^{\infty} \frac{1}{n!} x^n = e^x$.

Twierdzenie 21.4 Operacje na funkcjach tworzących.

1. Funkcje tworzące można **dodawać, odejmować i mnożyć przez liczbę** - odpowiadają temu operacje dodawania, odejmowania i mnożenia przez liczbę wyrazów ciągu.
2. **Mnożeniu dwóch funkcji tworzących** odpowiada operacja "splotu", tzn. jeśli $A(x) = \sum_{n=0}^{\infty} a_n x^n$ i $B(x) = \sum_{n=0}^{\infty} b_n x^n$, to $A(x) * B(x) = \sum_{n=0}^{\infty} c_n x^n$, gdzie $c_n = \sum_{k=0}^n a_k b_{n-k}$.
3. Jeśli funkcja tworząca $A(x)$ ma "element odwrotny", tzn. taką funkcję tworzącą $B(x)$ dla której $A(x) * B(x) = 1$, to mówimy że $A(x)$ jest **odwracalna** i piszemy $\frac{1}{A(x)} = B(x)$.
4. **Mnożeniu funkcji** tworzącej **przez x** odpowiada przesunięcie ciągu (dopisanie na początek wyrazu zerowego). Operacją odwrotną jest odjęcie wyrazu zerowego i podzielenie przez x .
5. Funkcje tworzące można **różniczkować** według wzoru:

$$\left(\sum_{n=0}^{\infty} a_n x^n\right)' = \sum_{n=1}^{\infty} n a_n x^{n-1} = \sum_{n=0}^{\infty} (n+1) a_{n+1} x^n$$

6. Zachodzą wzory na **pochodne** znane z analizy, np. $(A(x) * B(x))' = A'(x)B(x) + A(x)B'(x)$.
7. Operacja **"całkowania"** (odwrotna do różniczkowania):

$$\int \left(\sum_{n=0}^{\infty} a_n x^n\right) = \sum_{n=0}^{\infty} \frac{a_n}{n+1} x^{n+1} = \sum_{n=0}^{\infty} \frac{a_n - 1}{n} x^n$$

Wyraz a_n można "odzyskać" z funkcji tworzącej $A(x)$ wykonując jej n -krotne różniczkowanie.

Twierdzenie 21.5 Dla dowolnej liczby naturalnej $m \geq 1$ zachodzi wzór:

$$\frac{1}{(1-x)^m} = \sum_{n=0}^{\infty} \binom{n+m-1}{n} x^n.$$

Dla dowolnego $\alpha \in \mathbb{R}$ oraz $|x| < 1$ zachodzi wzór:

$$(1+x)^\alpha = \sum_{n=0}^{\infty} \binom{\alpha}{n} x^n$$

21.2 Równanie charakterystyczne.

Definicja 21.6 *Równanie charakterystyczne* równania rekurencyjnego. Weźmy równanie rekurencyjne jednorodne

$$a_n = Aa_{n-1} + Ba_{n-2}$$

gdzie dane są współczynniki A, B . Załóżmy, że ma ono rozwiązanie postaci $a_n = t^n$. Podstawiając otrzymujemy:

$$t^n = At^{n-1} + Bt^{n-2}.$$

Dzielimy obie strony przez t^{n-2} :

$$t^2 = At^1 + B$$

$$t^2 - At^1 - B = 0.$$

Równanie to nazywamy równaniem charakterystycznym równania rekurencyjnego. W tym przypadku jest to równanie kwadratowe.

Jeżeli nie ma ono pierwiastków podwójnych, wówczas:

$$a_n = Cr_1^n + Dr_2^n.$$

Jeżeli ma pierwiastki podwójne, to:

$$a_n = (C + Dn)r_1^n.$$

C i D są dowolnymi stałymi a r_1 i r_2 są pierwiastkami równania charakterystycznego. Jeżeli dane jest a_1 i a_2 wówczas można łatwo ułożyć układ równań i otrzymać ich wartość.

22 Ciąg i granica ciągu liczbowego, granica funkcji.

22.1 Ciągi.

Definicja 22.1 Ciąg liczbowy. Ciągiem liczbowym nazywamy funkcję $\mathbb{N} \rightarrow \mathbb{R}$. Wartość tej funkcji dla liczby naturalnej n nazywamy n -tym wyrazem ciągu i oznaczamy przez a_n, b_n itp. Ciągi o takich wyrazach oznaczamy odpowiednio przez $(a_n), (b_n)$ itp. Zbiór wyrazów ciągu (a_n) , tj. $\{a_n : n \in \mathbb{N}\}$ oznaczamy krótko przez $\{a_n\}$.

Definicja 22.2 Granica właściwa ciągu. Ciąg (a_n) jest zbieżny do granicy właściwej $a \in \mathbb{R}$, co zapisujemy:

$$\lim_{n \rightarrow \infty} a_n = a$$

wtedy i tylko wtedy, gdy

$$\forall \varepsilon > 0 \quad \exists n_0 \in \mathbb{N} \quad \forall n \in \mathbb{N} \quad [(n > n_0) \Rightarrow (|a_n - a| < \varepsilon)]$$

Definicja 22.3 Granice niewłaściwe ciągu.

Ciąg (a_n) jest zbieżny do granicy niewłaściwej ∞ , co zapisujemy:

$$\lim_{n \rightarrow \infty} a_n = \infty$$

wtedy i tylko wtedy, gdy:

$$\forall \varepsilon > 0 \quad \exists n_0 \in \mathbb{N} \quad \forall n \in \mathbb{N} \quad [(n > n_0) \Rightarrow (a_n > \varepsilon)]$$

Ciąg (a_n) jest zbieżny do granicy niewłaściwej $-\infty$, co zapisujemy:

$$\lim_{n \rightarrow \infty} a_n = -\infty$$

wtedy i tylko wtedy, gdy:

$$\forall \varepsilon < 0 \quad \exists n_0 \in \mathbb{N} \quad \forall n \in \mathbb{N} \quad [(n > n_0) \Rightarrow (a_n < \varepsilon)]$$

Twierdzenie 22.4 *O ograniczoności ciągu zbieżnego.* Jeśli ciąg jest zbieżny do granicy właściwej, to jest ograniczony.

Twierdzenie 22.5 *O równoważności granic.*

$$\lim_{n \rightarrow \infty} a_n = 0 \Leftrightarrow \lim_{n \rightarrow \infty} |a_n| = 0.$$

Twierdzenie 22.6 *O dwóch ciągach.* Jeśli ciągi (a_n) , (b_n) spełniają warunki:

1. $a_n \leq b_n \quad \forall n \geq n_0$

2. $\lim_{n \rightarrow \infty} a_n = \infty$

to

$$\lim_{n \rightarrow \infty} b_n = \infty.$$

Prawdziwe jest także analogiczne twierdzenie dla ciągów zbieżnych do granicy niewłaściwej $-\infty$.

Twierdzenie 22.7 *O trzech ciągach.* Jeśli ciągi (a_n) , (b_n) , (c_n) spełniają warunki:

1. $a_n \leq b_n \leq c_n \quad \forall n \geq n_0$

2. $\lim_{n \rightarrow \infty} a_n = \lim_{n \rightarrow \infty} c_n = b$

to

$$\lim_{n \rightarrow \infty} b_n = b.$$

Twierdzenie 22.8 *O ciągu monotonicznym i ograniczonym.* Jeżeli ciąg (a_n) jest niemalejący dla $n \geq n_0$ oraz ograniczony z góry, to jest zbieżny do granicy właściwej $\sup\{a_n : n \geq n_0\}$.

Prawdziwe jest także analogiczne twierdzenie dla ciągu nierosnącego i ograniczonego z dołu.

22.2 Funkcje.

Definicja 22.9 *Heinego granicy właściwej funkcji w punkcie.* Niech $x_0 \in \mathbb{R}$ oraz niech funkcja f będzie określona przynajmniej na sąsiedztwie $S(x_0)$. Liczba g jest granicą właściwą funkcji f w punkcie x_0 , co zapisujemy

$$\lim_{x \rightarrow x_0} f(x) = g$$

wtedy i tylko wtedy, gdy

$$\forall_{(x_n): \{x_n\} \subset S(x_0)} [(\lim_{n \rightarrow \infty} x_n = x_0) \Rightarrow (\lim_{n \rightarrow \infty} f(x_n) = g)].$$

Definicja 22.10 *Cauchy'ego granicy właściwej funkcji w punkcie.* Niech $x_0 \in \mathbb{R}$ oraz niech funkcja f będzie określona przynajmniej na sąsiedztwie $S(x_0)$. Liczba g jest granicą właściwą funkcji f w punkcie x_0 , co zapisujemy

$$\lim_{x \rightarrow x_0} f(x) = g$$

wtedy i tylko wtedy, gdy

$$\forall \varepsilon > 0 \quad \exists \delta > 0 \quad \forall x \in S(x_0) \quad [(|x - x_0| < \delta) \Rightarrow (|f(x) - g| < \varepsilon)].$$

Funkcja f ma granicę niewłaściwą ∞ w punkcie x_0 wtedy i tylko wtedy, gdy

$$\forall \varepsilon > 0 \quad \exists \delta > 0 \quad \forall x \in S(x_0) \quad [(|x - x_0| < \delta) \Rightarrow (f(x) > \varepsilon)].$$

Twierdzenie 22.11 *O nieistnieniu granicy funkcji w punkcie.* Jeśli istnieją ciągi (x'_n) , (x''_n) spełniające warunki:

1. $\lim_{n \rightarrow \infty} x'_n = x_0$, przy czym $x'_n \neq x_0 \quad \forall n \in \mathbb{N}$ oraz $\lim_{n \rightarrow \infty} f(x'_n) = g'$,
2. $\lim_{n \rightarrow \infty} x''_n = x_0$, przy czym $x''_n \neq x_0 \quad \forall n \in \mathbb{N}$ oraz $\lim_{n \rightarrow \infty} f(x''_n) = g''$,
3. $g' \neq g''$,

to granica $\lim_{x \rightarrow x_0} f(x)$ nie istnieje (właściwa ani niewłaściwa).

Twierdzenie 22.12 *Warunek konieczny i wystarczający istnienia granicy.* Funkcja f ma w punkcie x_0 granicę właściwą (niewłaściwą) wtedy i tylko wtedy, gdy

$$\lim_{x \rightarrow x_0^-} f(x) = \lim_{x \rightarrow x_0^+} f(x)$$

Wspólna wartość granic jednostronnych jest wtedy granicą funkcji.

Twierdzenie 22.13 *O niesitnieniu granicy funkcji w nieskończoności.* Jeżeli istnieją ciągi (x'_n) , (x''_n) spełniające warunki:

1. $\lim_{n \rightarrow \infty} x'_n = \infty$ oraz $\lim_{n \rightarrow \infty} f(x'_n) = g'$,
2. $\lim_{n \rightarrow \infty} x''_n = \infty$ oraz $\lim_{n \rightarrow \infty} f(x''_n) = g''$,
3. $g' \neq g''$,

to nie istnieje granica $\lim_{x \rightarrow x_0} f(x)$ (właściwa ani niewłaściwa).

Twierdzenie 22.14 *O dwóch funkcjach.* Jeśli funkcje f i g spełniają warunki:

1. $f(x) \leq g(x) \quad \forall x \in S(x_0)$,
2. $\lim_{x \rightarrow x_0} f(x) = \infty$,

to

$$\lim_{x \rightarrow x_0} g(x) = \infty.$$

Twierdzenie 22.15 Granice specjalne

$$\lim_{x \rightarrow +\infty} a^x x^\alpha \stackrel{[0, \infty]}{=} 0 \text{ dla } a \in (0, 1), \alpha \geq 0$$

$$\lim_{x \rightarrow 0} \frac{\sin x}{x} \stackrel{[\frac{0}{0}]}{=} 1 \text{ oraz } \lim_{x \rightarrow 0} \frac{\arcsin x}{x} \stackrel{[\frac{0}{0}]}{=} 1$$

$$\lim_{x \rightarrow 0} (1+x)^{\frac{1}{x}} \stackrel{1^\infty}{=} e \text{ oraz } \lim_{x \rightarrow \infty} (1+x)^{\frac{1}{x}} \stackrel{\infty^0}{=} 1$$

$$\lim_{x \rightarrow 0} \frac{a^x - 1}{x} \stackrel{[\frac{0}{0}]}{=} \ln a \text{ dla } a > 0, \text{ w szczególności } \lim_{x \rightarrow 0} \frac{e^x - 1}{x} \stackrel{[\frac{0}{0}]}{=} 1$$

$$\lim_{x \rightarrow 0} \frac{\log_a(1+x)}{x} \stackrel{[\frac{0}{0}]}{=} \frac{1}{\ln a}, \text{ w szczególności } \lim_{x \rightarrow 0} \frac{\ln(1+x)}{x} \stackrel{[\frac{0}{0}]}{=} 1$$

$$\lim_{x \rightarrow \pm\infty} \left(1 + \frac{a}{x}\right)^x \stackrel{[1^\infty]}{=} e^a, \text{ dla } a \in \mathbb{R}$$

$$\lim_{x \rightarrow 0} \frac{(1+x)^a - 1}{x} \stackrel{[\frac{0}{0}]}{=} a, \text{ dla } a \in \mathbb{R}$$

Twierdzenie 22.16 Reguła de L’Hospiala. Jeżeli funkcja f i g spełniają warunki:

1. $\lim_{x \rightarrow x_0} f(x) = \lim_{x \rightarrow x_0} g(x) = 0$ (∞), przy czym $g(x) \neq 0$ dla $x \in S(x_0)$
2. istnieje granica $\lim_{x \rightarrow x_0} \frac{f'(x)}{g'(x)}$ (właściwa lub niewłaściwa),

to

$$\lim_{x \rightarrow x_0} \frac{f(x)}{g(x)} = \lim_{x \rightarrow x_0} \frac{f'(x)}{g'(x)}$$

23 Ciągłość i pochodna funkcji. Definicja i podstawowe twierdzenia.

23.1 Ciągłość.

Definicja 23.1 *Funkcja ciągła w punkcie.* Niech $x_0 \in \mathbb{R}$ oraz niech funkcja f będzie określona przynajmniej na otoczeniu $O(x_0)$. Funkcja f jest ciągła w punkcie x_0 wtedy i tylko wtedy, gdy

$$\lim_{x \rightarrow x_0} f(x) = f(x_0).$$

Funkcja jest ciągła na zbiorze, jeżeli jest ciągła w każdym punkcie tego zbioru.

Twierdzenie 23.2 *Warunek konieczny i wystarczający ciągłości funkcji.* Funkcja jest ciągła w punkcie wtedy i tylko wtedy, gdy jest w tym punkcie ciągła lewostronnie i prawostronnie.

Definicja 23.3 Nieciągłość funkcji. Niech $x_0 \in \mathbb{R}$ oraz niech funkcja f będzie określona przynajmniej na otoczeniu $O(x_0)$. Funkcja f jest nieciągła w punkcie x_0 wtedy i tylko wtedy, gdy nie istnieje granica $\lim_{x \rightarrow x_0} f(x)$ albo gdy $\lim_{x \rightarrow x_0} f(x) \neq f(x_0)$.

Nieciągłość pierwszego rodzaju. Jeżeli istnieją granice skończone $\lim_{x \rightarrow x_0^-} f(x)$, $\lim_{x \rightarrow x_0^+} f(x)$ oraz

$$\lim_{x \rightarrow x_0^-} f(x) \neq f(x_0) \quad \text{lub} \quad \lim_{x \rightarrow x_0^+} f(x) \neq f(x_0).$$

Mówimy, że funkcja f ma w punkcie x_0 nieciągłość pierwszego rodzaju typu "skok", jeżeli spełnia warunek

$$\lim_{x \rightarrow x_0^-} f(x) \neq \lim_{x \rightarrow x_0^+} f(x).$$

Mówi, że funkcja f ma w punkcie x_0 nieciągłość pierwszego rodzaju typu "luka", jeżeli spełnia warunek

$$\lim_{x \rightarrow x_0^-} f(x) = \lim_{x \rightarrow x_0^+} f(x) \neq f(x_0).$$

Nieciągłość drugiego rodzaju. Jeżeli co najmniej jedna z granic

$$\lim_{x \rightarrow x_0^-} f(x), \quad \lim_{x \rightarrow x_0^+} f(x)$$

nie istnieje lub jest niewłaściwa.

Twierdzenie 23.4 Weierstrassa o ograniczoności funkcji ciągłej. Jeżeli funkcja jest ciągła na przedziale domkniętym i ograniczonym, to jest na nim ograniczona.

Twierdzenie 23.5 Weierstrassa o osiągnięciu kresów. Jeżeli funkcja f jest ciągła na przedziale domkniętym $[a, b]$, to

$$\exists c \in [a, b] \quad f(c) = \inf_{x \in [a, b]} f(x) \quad \text{oraz} \quad \exists d \in [a, b] \quad f(d) = \sup_{x \in [a, b]} f(x)$$

Twierdzenie 23.6 Darboux o przyjmowaniu wartości pośrednich. Jeżeli funkcja f jest ciągła na przedziale $[a, b]$ oraz spełnia warunek $f(a) < f(b)$, to

$$\forall w \in (f(a), f(b)) \exists c \in (a, b) \quad f(c) = w.$$

23.2 Pochodna.

Definicja 23.7 Iloraz różnicowy. Niech $x_0 \in \mathbb{R}$ oraz niech funkcja f będzie określona przynajmniej na otoczeniu $O(x_0, r)$, gdzie $r > 0$. Ilorazem różnicowym funkcji f w punkcie x_0 odpowiadającym przyrostowi Δx , gdzie $0 < |\Delta x| < r$, zmiennej niezależnej nazywamy liczbę

$$\frac{\Delta f}{\Delta x} \stackrel{\text{def}}{=} \frac{f(x_0 + \Delta x) - f(x_0)}{\Delta x}$$

Definicja 23.8 Pochodna właściwa funkcji. Niech $x_0 \in \mathbb{R}$ oraz niech funkcja f będzie określona przynajmniej na otoczeniu $O(x_0)$. Pochodną właściwą funkcji f w punkcie x_0 nazywamy granicę właściwą

$$f'(x_0) \stackrel{\text{def}}{=} \lim_{x \rightarrow x_0} \frac{f(x) - f(x_0)}{x - x_0}$$

Inaczej mówiąc pochodna funkcji f jest granicą ilorazu różnicowego gdy $\Delta x \rightarrow 0$. Mamy zatem

$$f'(x_0) \stackrel{\text{def}}{=} \lim_{\Delta x \rightarrow 0} \frac{f(x_0 + \Delta x) - f(x_0)}{\Delta x}$$

Twierdzenie 23.9 Warunek konieczny istnienia pochodnej właściwej funkcji. Jeżeli funkcja ma pochodną właściwą w punkcie, to jest ciągła w tym punkcie. Implikacja odwrotna nie jest prawdziwa.

Definicja 23.10 *Pochodne jednostronne właściwe funkcji.* Niech $x_0 \in \mathbb{R}$ oraz niech funkcja f będzie określona przynajmniej na otoczeniu $O(x_0^-)$. Pochodną lewostronną właściwą funkcji f w punkcie x_0 nazywamy granicę właściwą

$$f'_-(x_0) \stackrel{\text{def}}{=} \lim_{x \rightarrow x_0^-} \frac{f(x) - f(x_0)}{x - x_0}$$

Analogicznie definiujemy $f'_+(x_0)$.

Jeżeli funkcja ma w punkcie pochodną lewostronną (prawostronną) właściwą, to jest w nim ciągła lewostronnie (prawostronnie).

Definicja 23.11 *Pochodna funkcji na zbiorze.* Funkcja ma pochodną właściwą na zbiorze wtedy i tylko wtedy, gdy ma pochodną właściwą w każdym punkcie tego zbioru.

Definicja 23.12 *Pochodna niewłaściwa funkcji.* Niech f będzie funkcją ciągłą w punkcie $x_0 \in \mathbb{R}$. Funkcja f ma w punkcie x_0 pochodną niewłaściwą wtedy i tylko wtedy, gdy

$$\lim_{x \rightarrow x_0} \frac{f(x) - f(x_0)}{x - x_0} = \infty \quad \text{albo} \quad \lim_{x \rightarrow x_0} \frac{f(x) - f(x_0)}{x - x_0} = -\infty$$

Podobnie definiujemy pochodne niewłaściwe jednostronne.

Twierdzenie 23.13 Zastosowanie różniczki do obliczeń przybliżonych. Jeżeli funkcja f ma pochodną właściwą w punkcie x_0 , to

$$f(x_0 + \Delta x) \approx f(x_0) + f'(x_0)\Delta x$$

Przy czym błąd, jaki popełniamy zastępując przyrost funkcji

$$\Delta f = f(x_0 + \Delta x) - f(x_0)$$

jej różniczką $df = f'(x_0)\Delta x$, dąży szybciej do zera niż przyrost zmiennej niezależnej Δx , tzn.

$$\lim_{\Delta x \rightarrow 0} \frac{\Delta f - df}{\Delta x} = 0.$$

Twierdzenie 23.14 Rolle’a. Jeśli funkcja f spełnia warunki:

1. jest ciągła na $[a, b]$
2. ma pochodną właściwą lub niewłaściwą na (a, b) ,
3. $f(a) = f(b)$,

to istnieje punkt $c \in (a, b)$ taki, że:

$$f'(c) = 0.$$

Twierdzenie 23.15 Lagrange’a. Jeżeli funkcja f spełnia warunki:

1. jest ciągła na $[a, b]$,
2. ma pochodną właściwą lub niewłaściwą na (a, b) ,

to istnieje punkt $c \in (a, b)$ taki, że

$$f'(c) = \frac{f(b) - f(a)}{b - a}$$

24 Ekstrema funkcji jednej zmiennej. Definicje i twierdzenia.

Definicja 24.1 *Minimum lokalne funkcji.* Funkcja f ma w punkcie $x_0 \in \mathbb{R}$ minimum lokalne, jeżeli:

$$\exists \delta > 0 \quad \forall x \in S(x_0, \delta) \quad f(x) \geq f(x_0).$$

Analogicznie definiujemy **maksimum lokalne**.

Minimum lokalne jest właściwe, jeżeli:

$$\exists \delta > 0 \quad \forall x \in S(x_0, \delta) \quad f(x) > f(x_0).$$

Analogicznie definiujemy **maksimum lokalne właściwe**.

Twierdzenie 24.2 *Fermata, warunek konieczny istnienia ekstremum.* Jeżeli funkcja f ma:

1. ekstremum lokalne w punkcie x_0 ,
2. pochodną $f'(x_0)$,

to

$$f'(x_0) = 0.$$

Twierdzenie 24.3 *I warunek wystarczający istnienia ekstremum.* Jeżeli funkcja f spełnia warunki:

1. $f'(x_0) = 0$,
2. $\exists \delta > 0 \begin{cases} f'(x) > 0 & \forall x \in S(x_0^-, \delta), \\ f'(x) < 0 & \forall x \in S(x_0^+, \delta), \end{cases}$

to w punkcie x_0 ma maksimum lokalne właściwe. Analogicznie dla minimum.

Twierdzenie 24.4 II warunek wystarczający istnienia ekstremum.

Jeżeli funkcja f spełnia warunki:

1. $f'(x_0) = f''(x_0) = \dots = f^{(n-1)}(x_0) = 0$,
2. $f^{(n)}(x_0) < 0$,
3. n jest liczbą parzystą, gdzie $n \geq 2$,

to w punkcie x_0 ma maksimum lokalne właściwe. Analogicznie dla minimum,

25 Całka Riemanna funkcji jednej zmiennej.

Definicja 25.1 *Całka oznaczona Riemanna.* Niech funkcja f będzie ograniczona na przedziale $[a, b]$. Całkę oznaczoną Riemanna z funkcji f na przedziale $[a, b]$ definiujemy wzorem

$$\int_a^b f(x) dx \stackrel{\text{def}}{=} \lim_{\delta(P) \rightarrow 0} \sum_{k=1}^n f(x_k^*) \Delta x_k,$$

o ile po prawej stronie znaku równości granica jest właściwa oraz nie zależy od sposobu podziałów P przedziału $[a, b]$ ani od sposobów wyboru punktów pośrednich x_k^* , gdzie $1 \leq k \leq n$. Ponadto przyjmujemy

$$\int_a^a f(x) dx \stackrel{\text{def}}{=} 0 \quad \text{oraz} \quad \int_b^a f(x) dx \stackrel{\text{def}}{=} - \int_a^b f(x) dx \quad \text{dla } a < b$$

Funkcję, dla której istnieje całka Riemanna, nazywamy całkowaną.

Twierdzenie 25.2 *Warunek wystarczający całkowania funkcji.* Jeżeli funkcja f jest ograniczona na przedziale $[a, b]$ i ma na tym przedziale skończoną liczbę punktów nieciągłości I rodzaju, to jest na nim całkowana.

Twierdzenie 25.3 *Obliczanie całek przy pomocy sumy całkowej podziału równomiernego.* Jeżeli funkcja f jest całkowana na przedziale $[a, b]$, to

$$\int_a^b f(x) dx = \lim_{n \rightarrow \infty} \left[\frac{b-a}{n} \sum_{k=1}^n f\left(a + k \frac{b-a}{n}\right) \right]$$

Twierdzenie 25.4 *Newtona - Leibniza, główne twierdzenie rachunku całkowego.* Jeżeli funkcja f jest ciągła na przedziale $[a, b]$, to

$$\int_a^b f(x) dx = F(b) - F(a) = [F(x)]_a^b,$$

gdzie F oznacza dowolną funkcję pierwotną funkcji f na tym przedziale.

26 Pochodne cząstkowe funkcji wielu zmiennych; różniczkowalność i różniczka funkcji.

Definicja 26.1 Pochodne cząstkowe. Niech funkcja f będzie określona przynajmniej na otoczeniu punktu (x_0, y_0) . Pochodną cząstkową pierwszego rzędu funkcji f względem x w punkcie (x_0, y_0) określamy wzorem:

$$\frac{\partial f}{\partial x}(x_0, y_0) \stackrel{\text{def}}{=} \lim_{\Delta x \rightarrow 0} \frac{f(x_0 + \Delta x, y_0) - f(x_0, y_0)}{\Delta x}$$

Pochodną tą oznacza się także symbolami: $f_x(x_0, y_0)$, $D_1 f(x_0, y_0)$.

Jeżeli granica określająca pochodną cząstkową jest właściwa (niewłaściwa), to mówimy że pochodna ta jest właściwa (niewłaściwa). Jeżeli granica nie istnieje to to samo mówimy o pochodnej cząstkowej.

Definicja 26.2 Funkcja różniczkowalna w punkcie. Niech istnieją pochodne cząstkowe $\frac{\partial f}{\partial x}(x_0, y_0)$ $\frac{\partial f}{\partial y}(x_0, y_0)$. Funkcja f jest różniczkowalna w punkcie (x_0, y_0) wtedy i tylko wtedy, gdy spełniony jest warunek:

$$\lim_{(h,k) \rightarrow (0,0)} \frac{f(x_0 + h, y_0 + k) - f(x_0, y_0) - \frac{\partial f}{\partial x}(x_0, y_0)h - \frac{\partial f}{\partial y}(x_0, y_0)k}{\sqrt{h^2 + k^2}} = 0$$

Definicja 26.3 Różniczka funkcji. Niech funkcja f ma pochodne cząstkowe pierwszego rzędu w punkcie (x_0, y_0) . Różniczką funkcji f w punkcie (x_0, y_0) nazywamy funkcję $df(x_0, y_0)$ zmiennych $\Delta x, \Delta y$ określoną wzorem:

$$df(x_0, y_0)(\Delta x, \Delta y) \stackrel{\text{def}}{=} \frac{\partial f}{\partial x}(x_0, y_0)\Delta x + \frac{\partial f}{\partial y}(x_0, y_0)\Delta y$$

Twierdzenie 26.4 Zastosowanie różniczki funkcji do obliczeń przybliżonych. Niech funkcja f ma ciągłe pochodne cząstkowe pierwszego rzędu w punkcie (x_0, y_0) . Wtedy

$$f(x_0 + \Delta x, y_0 + \Delta y) \approx f(x_0, y_0) + df(x_0, y_0)(\Delta x, \Delta y)$$

Przy czym błąd $\delta(\Delta x, \Delta y)$ powyższego przybliżenia, tj. różnica $\Delta f - df$, dąży szybciej do 0 niż wyrażenie $\sqrt{(\Delta x)^2 + (\Delta y)^2}$. Oznacza to, że spełnia równość:

$$\lim_{(\Delta x, \Delta y) \rightarrow (0,0)} \frac{\delta(\Delta x, \Delta y)}{\sqrt{(\Delta x)^2 + (\Delta y)^2}} = 0$$

$$f(x_0 + \Delta x, y_0 + \Delta y) \approx f(x_0, y_0) + \frac{\partial f}{\partial x}(x_0, y_0)\Delta x + \frac{\partial f}{\partial y}(x_0, y_0)\Delta y$$

27 Ekstrema funkcji wielu zmiennych. Definicje i twierdzenia.

Definicja 27.1 *Minimum lokalne funkcji dwóch zmiennych.*

1. Funkcja f ma w punkcie (x_0, y_0) minimum lokalne, jeżeli istnieje otoczenie tego punktu takie, że dla dowolnego (x, y) z tego otoczenia zachodzi nierówność

$$f(x, y) \geq f(x_0, y_0)$$

Przy ostrej nierówności mówimy o minimum lokalnym **właściwym**.

Definicja 27.2 *Maksimum lokalne funkcji dwóch zmiennych.*

1. Funkcja f ma w punkcie (x_0, y_0) maksimum lokalne, jeżeli istnieje otoczenie tego punktu takie, że dla dowolnego (x, y) z tego otoczenia zachodzi nierówność

$$f(x, y) \leq f(x_0, y_0)$$

Przy ostrej nierówności mówimy o maksimum lokalnym **właściwym**.

Twierdzenie 27.3 *Warunek konieczny istnienia ekstremum.* Jeżeli funkcja f spełnia warunki:

1. ma ekstremum lokalne w punkcie (x_0, y_0)
2. istnieją pochodne cząstkowe $\frac{\partial f}{\partial x}(x_0, y_0)$, $\frac{\partial f}{\partial y}(x_0, y_0)$

to

$$\frac{\partial f}{\partial x}(x_0, y_0) = 0, \quad \frac{\partial f}{\partial y}(x_0, y_0) = 0$$

Funkcja może mieć ekstrema tylko w punktach, w których wszystkie jej pochodne cząstkowe pierwszego rzędu się zerują albo w punktach, w których choć jedna z nich nie istnieje.

Twierdzenie 27.4 Warunek wystarczający istnienia ekstremum.
Niech funkcja f ma ciągle pochodne cząstkowe rzędu drugiego na otoczeniu punktu (x_0, y_0) oraz niech

1. $\frac{\partial f}{\partial x}(x_0, y_0) = 0, \frac{\partial f}{\partial y}(x_0, y_0) = 0$

2. $\det \begin{bmatrix} \frac{\partial^2 f}{\partial^2 x}(x_0, y_0) & \frac{\partial^2 f}{\partial x \partial y}(x_0, y_0) \\ \frac{\partial^2 f}{\partial x \partial y}(x_0, y_0) & \frac{\partial^2 f}{\partial^2 y}(x_0, y_0) \end{bmatrix} > 0$

Wtedy w punkcie (x_0, y_0) funkcja f ma ekstremum lokalne i jest to:

1. *minimum, gdy $\frac{\partial^2 f}{\partial^2 x}(x_0, y_0) > 0$,*

2. *maksimum, gdy $\frac{\partial^2 f}{\partial^2 x}(x_0, y_0) < 0$.*

28 Twierdzenie o zmianie zmiennych w rachunku całkowym; współrzędne walcowe i sferyczne.

Definicja 28.1 *Twierdzenie o zmianie zmiennych w rachunku całkowym.* Niech

1. odwzorowanie $T : \begin{cases} x = \phi(u, v, w) \\ y = \psi(u, v, w) \\ z = \chi(u, v, w) \end{cases}$ przekształca różnowartościowo wnętrze obszaru regularnego Δ na wnętrze obszaru regularnego V ,
2. funkcje ϕ, ψ, χ mają ciągle pochodne cząstkowe rzędu pierwszego na pewnym zbiorze otwartym zawierającym obszar Δ ,
3. funkcja f jest ciągła na obszarze V ,
4. jacobian J_T jest różny od zera wewnątrz obszaru Ω .

Wtedy

$$\iiint_V f(x, y, z) dx dy dz = \iiint_\Omega f(\phi(u, v, w), \psi(u, v, w), \chi(u, v, w))$$

$$|J_T(u, v, w)| du dv dw$$

gdzie

$$J_T(u, v, w) \stackrel{\text{def}}{=} \det \begin{bmatrix} \frac{\partial \phi}{\partial u}(u, v, w) & \frac{\partial \phi}{\partial v}(u, v, w) & \frac{\partial \phi}{\partial w}(u, v, w) \\ \frac{\partial \psi}{\partial u}(u, v, w) & \frac{\partial \psi}{\partial v}(u, v, w) & \frac{\partial \psi}{\partial w}(u, v, w) \\ \frac{\partial \chi}{\partial u}(u, v, w) & \frac{\partial \chi}{\partial v}(u, v, w) & \frac{\partial \chi}{\partial w}(u, v, w) \end{bmatrix}$$

Definicja 28.2 Współrzędne walcowe. Położenie punktu P w przestrzeni można opisać trójką liczb (φ, ϱ, h) , gdzie:

φ - oznacza miarę kąta między rzutem promienia wodzącego punktu P na płaszczyznę xOy , a dodatnią częścią osi Ox , $0 \leq \varphi \leq 2\pi$ albo $-\pi < \varphi \leq \pi$

ϱ - oznacza odległość rzutu punktu P na płaszczyznę xOy od początku układu współrzędnych, $0 \leq \varrho < \infty$

h - oznacza odległość (dodatnią dla $z > 0$ i ujemną dla $z < 0$) punktu P od płaszczyzny xOy , $-\infty < h < \infty$

Zależność między współrzędnymi walcowymi i kartezajńskimi.

$$W : \begin{cases} x = \varrho \cos \varphi \\ y = \varrho \sin \varphi \\ z = h \end{cases}$$

Współrzędne walcowe w całce potrójnej. Niech:

1. obszar Ω we współrzędnych walcowych będzie obszarem normalnym
2. funkcja f będzie ciągła na obszarze U , które jest obrazem obszaru Ω przy przekształceniu walcowym; $U = W(\Omega)$.

Wtedy

$$\iiint_{\Omega} f(x, y, z) dx dy dz = \iiint_{\Omega} f(\varrho \cos \varphi, \varrho \sin \varphi, h) \varrho dh d\varrho d\varphi$$

Definicja 28.3 Współrzędne sferyczne. Położenie punktu P przestrzeni można opisać trójką liczb (φ, ψ, ϱ) , gdzie

φ - oznacza miarę kąta między rzutem promienia wodzącego punktu P na płaszczyznę xOy , a dodatnią częścią osi Ox , $0 \leq \varphi \leq 2\pi$ albo $-\pi < \varphi \leq \pi$

ψ - oznacza miarę kąta między promieniem wodzącym punktu P , a płaszczyznę xOy , $-\frac{\pi}{2} \leq \psi \leq \frac{\pi}{2}$

ϱ - oznacza odległość punktu P od początku układu współrzędnych, $0 \leq \varrho < \infty$

Zależność między współrzędnymi sferycznymi i kartezjańskimi.

$$S : \begin{cases} x = \varrho \cos \varphi \cos \psi \\ y = \varrho \sin \varphi \cos \psi \\ z = \varrho \sin \psi \end{cases}$$

Współrzędne sferyczne w całce potrójnej. Niech:

1. obszar Ω we współrzędnych sferycznych będzie obszarem normalnym
2. funkcja f będzie ciągła na obszarze U , które jest obrazem obszaru Ω przy przekształceniu walcowym; $U = S(\Omega)$.

Wtedy

$$\iiint_U f(x, y, z) dx dy dz = \iiint_{\Omega} f(\varrho \cos \varphi \cos \psi, \varrho \sin \varphi \cos \psi, \varrho \sin \psi) \varrho^3 d\varrho d\psi d\varphi$$

Teoretyczne podstawy informatyki

29 Metody dowodzenia poprawności pętli.

- **Asercja** - warunek logiczny wyrażający zależności między zmiennymi algorytmu w kontekście ich aktualnego stanu.
- **Asercja początkowa** - asercja określająca warunek wejściowy (warunek danych wejściowych) algorytmu.

- **Asercja końcowa** - asercja określająca wyniki algorytmu (warunek danych wyjściowych).
- Asercje początkowa i końcowa są znane w momencie definiowania problemu i poszukiwania algorytmu, stanowiąc **specyfikację algorytmu**
- Algorytm A ma własność **określoności obliczeń** względem warunku α , jeśli dla każdego danych spełniających warunek α działanie algorytmu A nie zostanie zerwane (oznaczenie $obl(\alpha, A)$).
- Algorytm A ma **własność stopu** względem warunku α , jeśli dla każdego danych spełniających warunek α działanie algorytmu A nie ciągnie się w nieskończoność (oznaczenie $stop(\alpha, A)$).
- Algorytm A jest **częściowo poprawny** względem warunku początkowego α oraz warunku końcowego β , gdy dla każdego danych spełniających warunek początkowy α , jeżeli działanie algorytmu A dochodzi do końca, wyniki spełniają warunek końcowy β (oznaczenie $cp(\alpha, A, \beta)$).

$$cp(\alpha, z \leftarrow w, \beta) \iff (\alpha \Rightarrow \beta|_{z \leftarrow w})$$
 Przy asercji początkowej α i asercji końcowej β podstawienie $z \leftarrow w$ jest częściowo poprawne wtedy i tylko wtedy, gdy poprawna jest implikacja od asercji α do asercji β , z zastąpieniem każdego wystąpienia zmiennej z podstawianym wyrażeniem w .
- Algorytm A jest **całkowicie poprawny** (semantycznie poprawny) względem warunku początkowego α oraz warunku końcowego β , jeśli ma własność określoności obliczeń względem warunku α , ma własność stopu względem warunku α oraz jest częściowo poprawny względem warunku α i warunku β (oznaczenie $sp(\alpha, A, \beta)$).
- Formalnie: $sp(\alpha, A, \beta) \iff obl(\alpha, A) \wedge stop(\alpha, A) \wedge cp(\alpha, A, \beta)$

30 Odwrotna Notacja Polska: definicja, własności, zalety i wady, algorytmy.

- notacja postfiksowa,
- jednoznacznie wyznacza kolejność wykonywania działań,
- pozwala na całkowitą rezygnację z nawiasów.
- Zalety:
 - ułatwione obliczenia na komputerze - wykorzystuje jedynie stos
 - prosty algorytm konwersji między standardowym zapisem infiksowym a ONP wykorzystywany w wielu algorytmach wykorzystujących jako wejście zapis działań brak konieczności stosowania nawiasów
- Wady:
 - mniej czytelny dla człowieka (wymaga przyzwyczajenia)
 - podczas zapisu na kartce “12 34” + może wyglądać jak “123 4” + (xD)

30.1 Algorytm obliczenia wartości wyrażenia ONP

Dla wszystkich symboli z wyrażenia ONP wykonuj:
jeśli i-ty symbol jest liczbą, to odłóż go na stos,
jeśli i-ty symbol jest operatorem (funkcją):
zdejmij ze stosu oczekiwaną liczbę elementów (parametrów)
odłóż na stos wynik działania operatora (wynik funkcji)
Zdejmij ze stosu wynik.

30.2 Algorytm konwersji z notacji infiksowej do ONP

```
do {  
    weź_kolejny_element_z_wejścia;  
    if ( element_jest_operandem )  
        dopisz_element_na_wyjście;  
    else // element jest operatorem, przecinkiem lub nawiasem  
        if ( element_jest_operatorem o1) {
```

```

while ( (lewa łączność(o1) and
prior_(operatora_na_stosie o2) >= prior_(elementu o1))
or (prawa łączność(o1) and
prior_(operatora_na_stosie o2) > prior_(elementu o1)) ){
przepisz_operator_(o2) ze_stosu_na_wyjście ;
}
wstaw_operator_(o1) _na_wyjście ;
}
else
if ( element == '(' )
wstaw_element_na_stos;
else
if ( element jest ',' )
zignoruj go i wczytaj kolejny symbol z wejścia
else
if( element == ')' ) {
while( na_stosie_jest element różny od '(' )
przepisz_go_na_wyjście;
Usuń_ze_stosu_( ;
}
} while(!koniec_danych);
while( niepusty_stos )
przepisz_element_ze_stosu_na_wyjście;

```

31 Modele obliczeń: maszyna Turinga.

Definicja 31.1 *Formalna definicja maszyny Turinga.* Maszynę Turinga opisujemy poprzez krotkę:

$$MT = \langle Q, \Sigma, \delta, \tau, q_0, B, F \rangle$$

gdzie:

- Q - skończony zbiór stanów,
- $q_0 \in Q$ - stan początkowy,
- $F \subset Q$ - zbiór stanów końcowych,
- τ - skończony zbiór dopuszczalnych symboli,
- $b \in \tau$ - symbol pusty,
- $\Sigma \subset \tau \setminus \{b\}$ - zbiór symboli wejściowych,
- δ - funkcja sterująca.
 - W wersji niedeterministycznej postaci maszyny Turinga:

$$\delta \subset (Q \times \tau) \times (\tau \times \{\leftarrow, \rightarrow, \downarrow\} \times Q)$$

- W wersji deterministycznej:

$$\delta : Q \times \tau \mapsto \tau \times \{\leftarrow, \rightarrow, \downarrow\} \times Q$$

Definicja 31.2 *Tablica sterująca.* Dwuwymiarowa tablica indeksowana:

- Stanami w jednym wymiarze,
- Symbolami taśmy w drugim wymiarze.

Elementy tablicy

- Odpowiadają wszystkim parom stanu i czytanego symbolu stanowiąc dziedzinę funkcji sterowania δ ,
- Zawierają trójki działania maszyny, czyli wartości funkcji sterowania δ .

32 Modele obliczeń: automat skończony, automat ze stosem.

32.1 Automat skończony deterministyczny

Definicja 32.1 *Automat skończony deterministyczny oznaczamy piątką parametrów: $\mathcal{A} = (S, A, f, s_0, T)$ gdzie:*

S - skończony zbiór stanów

A - skończony alfabet wejściowy

f - funkcja przejścia $S \times A \rightarrow S$

s_0 - stan początkowy $\in S$

T - zbiór terminali (stanów akceptujących) $\in S$

Definicja 32.2 *Automaty $\mathcal{A}_1$ i $\mathcal{A}_2$ są równoważne, jeżeli rozpoznają ten sam język, czyli:*

$$L(\mathcal{A}_1) = L(\mathcal{A}_2)$$

Definicja 32.3 *Każdy automat $\mathcal{A} = (S, A, f, s_0, T)$ wyznacza w wolnym monoidzie A^* prawą kongruencję automatową określoną w następujący sposób:*

$$\forall u, v \in A^*$$

$$u \sim_{\mathcal{A}} v \Leftrightarrow f(s_0, u) = f(s_0, v)$$

Definicja 32.4 *Niech $L \subset A^*$ będzie dowolnym językiem, a $u \in A^*$ dowolnym słowem. Pochodną Brzozowskiego (residuum) z języka L względem słowa u nazywamy język*

$$u^{-1}L = \{w \in A^* : uw \in L\}$$

Definicja 32.5 *Automat ilorazowy to automat, w którym stanami są klasy równoważności*

Definicja 32.6 *Monoidem przejść automatu $\mathcal{A}$ nazywamy monoid*

$$\mathcal{M}(\mathcal{A}) = \tau_{\mathcal{A}}(A^*) \subset S^S$$

Definicja 32.7 *Każdy język skończony jest akceptowany przez pewien deterministyczny automat skończony*

32.2 Automat skończony niedeterministyczny

Definicja 32.8 *Automat skończony niedeterministyczny oznaczamy piątką parametrów: $\mathcal{A}_{ND} = (S, A, f, S_0, T)$ gdzie:*

S - skończony zbiór stanów

A - skończony alfabet wejściowy

f - funkcja przejścia $S \times A \rightarrow \mathcal{P}(S)$

S_0 - zbiór stanów początkowych $\subset S$

T - zbiór terminali (stanów akceptujących) $\in S$

Słowo x jest akceptowane gdy $f^(S_0, x) \cap T \neq \emptyset$*

32.3 Automat ze stosem

Definicja 32.9 *Automat ze stosem to $\mathcal{AS} = (A, A_S, Q, f, s_0, z_0, Q_F)$ gdzie:*

A - jest alfabetem

A_S - jest dowolnym, skończonym i niepustym zbiorem zwanym alfabetem stosu

Q - jest dowolnym, skończonym i niepustym zbiorem zwanym zbiorem stanów

$f: A_S \times Q \times (A \cup \{1\}) \rightarrow \mathcal{P}_{sk}(A_S^ \times Q)$ jest funkcją przejść*

$q_0 \in Q$ jest stanem początkowym

$z_0 \in A_S$ jest symbolem początkowym stosu

$Q_F \subset Q$ zbiorem stanów końcowych

33 Złożoność obliczeniowa - definicja notacji: O , Ω , Θ .

Definicja 33.1 Niech $f, g, h : \mathbb{N} \rightarrow \mathbb{R}_+ \cup \{0\}$, wtedy:

- $f(n) = O(g(n))$ - f jest **co najwyżej rzędu** g , gdy istnieje $c > 0$ i $n_0 \in \mathbb{N}$, takie że $f(n) \leq cg(n)$ dla każdego $n \geq n_0$.
- $f(n) = \Omega(g(n))$ - f jest **co najmniej rzędu** g , gdy $g(n) = O(f(n))$
- $f(n) = \Theta(g(n))$ - f jest **dokładnie rzędu** g , gdy $f(n) = O(g(n))$ i $f(n) = \Omega(g(n))$.

34 Złożoność obliczeniowa - pesymistyczna i średnia.

Definicja 34.1 *Niech:*

- D_n - zbiór danych rozmiaru n ,
- $t(d)$ - liczba operacji dominujących,
- X_n - zmienna losowa dla $t(d) \in D_n$,
- p_{kn} - rozkład prawdopodobieństwa zmiennej X_n .

Optymistyczna złożoność czasowa:

$$Opt(n) = \inf\{t(d) : d \in D_n\}$$

Średnia złożoność czasowa:

$$A(n) = ave(X_n) = \sum_{k \geq 0} kp_{nk}$$

Pesymistyczna złożoność czasowa:

$$W(n) = \sup\{t(d) : d \in D_n\}$$

Definicja 34.2 Koszt amortyzowany

Koszt amortyzowany operacji jest średnim czasem wykonania przypadającym na jedną operację w pesymistycznym ciągu operacji. Koszt amortyzowany różni się od kosztu średniego tym, że bierze pod uwagę pesymistyczny ciąg operacji i nie jest metodą probabilistyczną.

Przykład:

Tablica dynamiczna (np. vector w C++) podwaja swoją długość w przypadku, gdy jest pełna i dodajemy do niej nowy element. Alokuje wówczas dwukrotnie większą pamięć i kopiuje wszystkie elementy. Koszt takiego rozszerzenia jest równy $\theta(n)$.

Obliczmy koszt amortyzowany wstawienia elementu do tablicy przy wstawieniu n elementów ($n = 2^k$, $k \in \mathbb{N}$), zaczynając od pustej tablicy o długości 1.

$$T(n) = \frac{1}{n} \left(1 + 2 + 4 + \dots + \frac{n}{2} \right) = \frac{1}{n} \sum_{i=0}^{\log_2(n)-1} 2^i = \frac{n-1}{n} = 1 - \frac{1}{n} = \theta(1)$$

Zatem sumaryczny koszt wstawienia do takiej tablicy n elementów jest równy $\theta(n)$

35 Metoda "dziel i zwyciężaj": zalety i wady.

Algorytmy "dziel i zwyciężaj" podlegają poniższemu schematowi:

1. Dziel

Dany problem dzielony jest rekurencyjnie na podproblemy, aż do uzyskania przypadków bazowych.

2. Zwyciężaj

Rozwiązujemy przypadki bazowe (najczęściej jest to możliwe w czasie stałym).

3. Łącz

Wyniki otrzymane z podproblemów są łączone aż do uzyskania wyniku danego problemu

35.1 Zalety

- Algorytm podzielony jest na trzy ortogonalne części: dziel (*divide*), zwyciężaj (*conquer*) i łącz (*merge / combine*), co ułatwia jego zrozumienie jak i analizę.
- Możliwość łatwego przekształcenia algorytmu w algorytm równoległy (przy pomocy wzorca redukcji / dekompozycji asynchronicznej).
- Optymalne użycie pamięci podręcznej (*cache*): problemy dostatecznie małe (w szczególności bazowe) rozwiązywane są w pamięci nieprzekraczającej wielkości pamięci podręcznej.
- Znacząco lepsza złożoność obliczeniowa od bardziej prymitywnych podejść (jak na przykład *brute-force*).

35.2 Wady

- W przypadku podziału na dwa i więcej podproblemy uzyskujemy rekurencję nieliniową. Oznacza to, że przekształcenie algorytmu rekurencyjnego w algorytm iteracyjny jest nietrywialne (w szczególności nie zostanie to wykonane przez kompilator).
- Potencjalnie wysoka złożoność pamięciowa algorytmu rekurencyjnego / iteracyjnego ze stosem. Ponadto rozmiar stosu może przekroczyć pamięć komputera.

- Możliwość wielokrotnego rozwiązywania identycznych problemów.
Problem ten rozwiązuje się stosując praktyki należące do programowania dynamicznego (np. *memoization*).

35.3 Przykłady

- Binsearch
- Merge sort
- Quicksort
- Szybkie potęgowanie
- Algorytm Karacuby (szybkie mnożenie liczb całkowitych)
- Szybka transformata Fouriera
- Algorytm znajdowania otoczki wypukłej

36 Lista: ujęcie abstrakcyjne, możliwe implementacje i ich złożoności.

Definicja 36.1 *Lista jest abstrakcyjną strukturą danych (ADT), która posiada następujące właściwości*

- Przechowuje elementy **tego samego typu**
- Wykorzystuje pamięć w sposób **dynamiczny lub statyczny**
- Dostęp do danych jest **sekwencyjny**
- Elementy w liście są **liniowo uporządkowane** zgodnie z ich pozycją na liście
 - element a_i znajduje się na pozycji 'i'
 - element a_i poprzedza a_{i+1} dla $i=1, 2, \dots, n-1$
 - element a_i następuje po a_{i-1} dla $i=2, \dots, n$
- Wyróżnia się dwa podstawowe sposoby implementacji listy jako ADT
 - Tablica
 - Lista wiązana

Definicja 36.2 *W sensie matematycznym lista jest skończonym ciągiem elementów ustalonego typu Node: $a_1, a_2, \dots, a_n, n \geq 0$ przy czym Node, zwany typem bazowym, może być np. typem **klasy, struktury, int, string** itp.*

36.1 Implementacja za pomocą tablic (array)

Tablica wykorzystuje pamięć w sposób **statyczny**. Oznacza to, że ma z góry określony rozmiar oraz pewna jej część może być niewykorzystana. Tablica posiada zmienne wskazujące na jej maksymalny rozmiar, ilość zajętych miejsc. Indeksowanie elementów zazwyczaj zaczyna się od 0 i kończy na maxsize-1.

36.2 Implementacja za pomocą Listy Wiązanej (Linked List)

Rozróżnia się dwa podstawowe rodzaje:

1. Lista wiązana **jednokierunkowa**
Z każdego elementu możliwe jest przejście do jego następnika NEXT
2. Lista wiązana **dwukierunkowa**
Z każdego elementu możliwe jest przejście do jego następnika NEXT i poprzednika PREV

W liście wiązanej wyróżnia się dwie podstawowe zmienne typu Node potrzebne do jej obsługi

1. HEAD
2. TAIL

Dodatkowo, lista wiązana jest **cykliczna** jeżeli

1. $next(TAIL) \rightarrow HEAD$
2. $prev(HEAD) \rightarrow TAIL$

Pojedynczy element listy jest klasą lub strukturą, zwyczajowo nazywa się Node (węzeł). Przechowuje pole data o pożądanym typie danych oraz wskaźnik (wskaźniki) na element następny (i poprzedni).

Przykład implementacji struktury Node w C++

```
struct Node {  
    int data;  
    struct Node* next;  
    struct Node* prev;  
};
```

Przykład implementacji klasy Node w Java

```
class Node<E> {  
    E data;  
    Node<E> next;  
    Node<E> prev;  
};
```

Przykład implementacji klasy LinkedList w Java

```
class LinkedList<E> {  
    Node<E> head; // head of the list  
    Node<E> tail; // tail of the list  
    int counter = 0;  
    ...  
}
```

Operacje i złożoności

1. `CREATE(l: List)`

Tworzy nową listę
 $O(1)$ - czas stały

2. `APPEND(l: List; d: Data)`

Dodaje na końcu listy l rekord d
 $O(1)$ - czas stały

3. `INSERT(l: List; d: Data; i: Integer)`

Dodaje rekord d do określonego miejsce i w liście l
 $O(n)$ - czas liniowy

4. `FIND(l: List; i: Integer)`

Znajduje i-ty w kolejności rekord z listy
 $O(\min(i, n))$ - czas liniowy

5. `DELETE(l: List; i: Integer)`

Usuwa -ity w kolejności rekord z listy
 $O(\min(i, n))$ - czas liniowy

6. `MAKENULL(l: List)`

Czyści listę, usuwając wszystkie rekordy
 $O(1)$ - czas stały

37 Kolejka i kolejka priorytetowa: ujęcie abstrakcyjne, możliwe implementacje i ich złożoności.

37.1 Kolejka

Definicja 37.1 *Kolejka FIFO (First In First Out).*

Jest to lista, w której

- *Wstawianie nowego elementu odbywa się na końcu kolejki **rear***
- *Usuwanie elementu odbywa się na początku kolejki **front***
- *Wyróżnia się dwa podstawowe sposoby implementacji kolejki*
 - *Tablica cykliczna*
 - *Lista wiązana*

Operacje i złożoności

1. `Create()`

Tworzy pustą kolejkę

2. `isEmpty()`

Sprawdza czy kolejka pusta

3. `Enqueue(x)`

Wstawia nowy element x

4. `Front()`

Odczytuje pierwszy element w kolejce

5. `Dequeue()`

Usuwa pierwszy element w kolejce i go zwraca

37.2 Sposoby implementacji kolejki

Na początku zauważmy, że **nieefektywną** realizacją kolejki w tablicy jest przyjęcie, że początek kolejki **front** będzie zawsze równy 0, a **rear** będzie indeksem do pierwszego wolnego elementu tablicy.

Najbardziej **efektywnym** rozwiązaniem jest reprezentacja kolejki w **tablicy cyklicznej**. Operacje wstawiania i usuwania zwiększają pozycje końca i początku kolejki o 1 modulo rozmiar tablicy, wykorzystując metodę:

```
private int Add (int i){  
    return (i+1) % maxSize; // reszta z dzielenia  
}
```

37.3 Kolejka Priorytetowa

Definicja 37.2 *Kolejka Priorytetowa*

Jest to lista, w której

- *Usuwany jest zawsze element o największej (najmniejszej) wartości*

Operacje

1. **Insert(x, q)**

Wstawienie elementu 'x' do kolejki q

2. **Max(Q)**

Odczyt największego elementu w kolejce (element ten jest zwracany jako wynik operacji, kolejka się nie zmienia)

3. **Min(Q)**

Odczyt największego elementu w kolejce (element ten jest zwracany jako wynik operacji, kolejka się nie zmienia)

4. `Delete_Max(Q)`

Usunięcie największego elementu z kolejki q

5. `Delete_Min(Q)`

Usunięcie największego elementu z kolejki q

37.4 Sposoby implementacji kolejki priorytetowej

- Lista - Tablica Nieuporządkowana NIEEFEKTYWNA!
- Lista - Tablica Uporządkowana Rosnąco (Malejąco) NIEEFEKTYWNA!
- Kopiec Max (Min)

38 Algorytmy sortowania QuickSort i MergeSort: metody wyboru pivotu w QS; złożoności.

38.1 QuickSort.

```
def quickSort(arr, start, end):
    if (start < end)
        pivot = partition(arr, start, end)

        quickSort(arr, start, pivot - 1)
        quickSort(arr, pivot + 1, end)

def partition (arr, start, end):
    pivot = arr[end]
    i = (start - 1)

    for j in range [start, end- 1]:
        if (arr[j] < pivot):
            i++;
            swap arr[i] and arr[j]

    swap arr[i + 1] and arr[end])
    return (i + 1)
```

Złożoność: pesymistyczna - $O(n^2)$, średnia i optymistyczna - $O(n \log_2 n)$.

Sposoby wybrania pivotu

1. Pierwszy element
2. Ostatni element
3. Mediana z pierwszego, środkowego i ostatniego
4. Losowy element

38.1.1 MergeSort.

```
def mergeSort (arr, start, end):  
    if end > start:  
        middle = (start+end)/2  
  
        mergeSort (arr, start, middle)  
        mergeSort (arr, middle+1, end)  
  
        merge (arr, start, middle, end)
```

Złożoność: pesymistyczna, średnia, optymistyczna - $O(n \log n)$.

39 Algorytm sortowania bez porównań (sortowanie przez zliczanie, sortowanie kubełkowe oraz sortowanie pozycyjne).

39.1 CountSort.

```
def countSort (arr):  
    count = []  
    for a in arr:  
        count[a] += 1  
  
    i = 0;  
    for j in range [0, arr.len]:  
        while (count[i] == 0):  
            i++  
        arr[j] = i  
        count[i]--
```

Złożoność: $O(n + k)$.

39.2 BucketSort.

```
bucketSort (arr):  
    n = arr.len  
    buckets = [{} for i in range [1,n]]  
  
    for a in arr:  
        buckets[n*arr[i]].add(arr[i])  
  
    for b in buckets:  
        sort(b)  
  
    i = 0  
    for b in buckets:  
        for a in b:  
            arr[i++] = a
```

Złożoność: pesymistyczna - $O(n^2)$, średnia - $O\left(n + \frac{n^2}{k} + k\right)$.

39.3 RadixSort.

```
def radixSort (arr):  
    for all digits ascending:  
        countSort(arr, digit)
```

Złożoność: $O(d(n + k))$, gdzie d jest liczbą cyfr.

40 Reprezentacja drzewa binarnego za pomocą porządków (preorder, inorder, postorder).

```
def preorder (v):  
    func(v)  
    preorder(v.left)  
    preorder(v.right)  
  
def inorder (v):  
    inorder(v.left)  
    func(v)  
    inorder(v.right)  
  
def postorder (v):  
    postorder(v.left)  
    postorder(v.right)  
    func(v)
```

Możemy odtworzyć wyjściowe drzewo, jeśli mamy inorder i post- lub pre-order.

41 Algorytmy wyszukiwania następnika i poprzednika w drzewach BST; usuwanie węzła.

Definicja 41.1 *Binarne drzewo poszukiwań (BST)* - dynamiczna struktura danych będąca drzewem binarnym, w którym lewe poddrzewo każdego węzła zawiera wyłącznie elementy o kluczach mniejszych niż klucz węzła a prawe poddrzewo zawiera wyłącznie elementy o kluczach nie mniejszych niż klucz węzła.

```
struct BSTNode {
    int key;
    struct BSTNode* parent;
    struct BSTNode* left;
    struct BSTNode* right;
};
```

41.1 Wyszukiwanie następnika i poprzednika w BST.

Definicja 41.2 *Następnik* danego węzła to węzeł, który jest odwiedzany jako następny w przypadku przechodzenia drzewa metodą in-order. Sposób wyznaczenia nie wymaga porównywania żadnych kluczy, jest to w praktyce:

- skrajnie lewy element prawego poddrzewa węzła, jeśli posiada on prawe poddrzewo; wpp
- pierwszy napotkany przodek węzła, dla którego węzeł jest w lewym poddrzewie.

```
define BST_FIND_SUCCESSOR(Node):
    if (Node->right != NULL)
        return BST_SEARCH_MIN_KEY(Node->right)
    Node_tmp = Node->parent
    while (Node_tmp != NULL and Node_tmp->left != Node)
        Node = Node_tmp
    Node_tmp = Node_tmp->parent
    return Node_tmp
```

Definicja 41.3 *Poprzednik* danego węzła to węzeł, który jest odwiedzany jako poprzedni w przypadku przechodzenia drzewa metodą in-order. Sposób wyznaczenia nie wymaga porównywania żadnych kluczy, jest to w praktyce:

- skrajnie prawy element lewego poddrzewa węzła, jeśli posiada on lewe poddrzewo; wpp
- pierwszy napotkany przodek węzła, dla którego węzeł jest w prawym poddrzewie.

```
define BST_FIND_PREDECESSOR(Node):  
    if (Node->left != NULL)  
        return BST_SEARCH_MAX_KEY(Node->left)  
    Node_tmp = Node->parent  
    while (Node_tmp != NULL and Node_tmp->right != Node)  
        Node = Node_tmp  
        Node_tmp = Node_tmp->parent  
    return Node_tmp
```

41.2 Usuwanie węzła z BST.

Twierdzenie 41.4 *Usuwanie węzła z BST wymaga rozważenia trzech przypadków:*

1. w przypadku, gdy usuwany węzeł jest liściem wskaźnik do węzła w jego ojcu zastępowany jest wskaźnikiem do węzła pustego,
2. w przypadku, gdy usuwany węzeł ma jednego syna to dany węzeł usuwamy a jego syna podstawiamy w miejsce usuniętego węzła,
3. w przypadku, gdy usuwany węzeł ma dwóch synów to po jego usunięciu wstawiamy w jego miejsce węzeł, który jest jego następnikiem.

42 B-drzewa: operacje i ich złożoność.

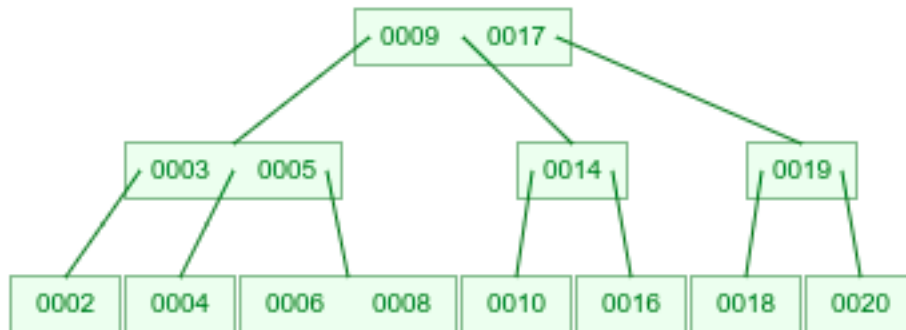
B-drzewa to rozszerzone drzewa BST. Jest ono typowo zoptymalizowane pod kątem operacji wejścia/wyjścia (np. odczytu z dysku twardego) - operacje długotrwałe takie jak odczyt węzła z dysku są wykonywane o wiele rzadziej niż dla typowych drzew BST, natomiast operacje takie jak wyszukanie odpowiedniej wartości w węźle załadowanym już do pamięci ram są praktycznie pomijane ze względu na nieporównywalnie większą szybkość ich wykonywania. B-drzewa mają ustalony rząd.

B-drzewo rzędu K musi spełniać poniższe warunki:

- Korzeń jest liściem lub ma od 2 do K synów
- Wszystkie liście są na tym samym poziomie
- Każdy węzeł wewnętrzny (oprócz korzenia) ma od $\lceil K/2 \rceil$ do K synów. Jeżeli jakiś węzeł ma s synów, to musi mieć $s-1$ kluczy (wartości)
- Każdy liść zawiera od $\lceil K/2 \rceil - 1$ do $K - 1$ kluczy (wartości)

Warunek drugi gwarantuje niewielką wysokość drzewa (dla drzewa o n kluczach: w najgorszym wypadku $\log_{K/2} n / (K/2)$; w najlepszym wypadku $\log_K n / K$)

Generalnie złożoność wszystkich operacji na B-drzewach wynosi $O(K \log_K n)$, gdzie n to ilość kluczy (wartości) w drzewie, a K to rząd drzewa.



Odmianą B-drzew są drzewa B+, które różnią się tym, że wszystkie wartości trzymane są w liściach oraz mają powiązania pomiędzy liśćmi (tak aby można było po kolei przejść wszystkie wartości w liściach).

42.1 Wyszukiwanie

Wyszukiwanie przebiega bardzo podobnie jak w drzewie BST, z tą różnicą, że nie mamy tylko lewego i prawego poddrzewa, a możemy mieć ich więcej, więc musimy zdecydować, do którego pójść. Oczywiście szukana wartość może również znajdować się na danym poziomie. Wtedy kończymy wyszukiwanie. Ponieważ rząd drzewa zazwyczaj jest wysoki (np. 1024), aby usprawnić wyszukiwanie zadanej wartości w danym węźle, najczęściej używamy wyszukiwania binarnego.

42.2 Wstawianie

Aby wstawić wartość szukujemy liść, do którego powinna trafić, a następnie wstawiamy ją w odpowiednie miejsce w tym liściu.

Jeżeli jednak okazałoby się, że liść ma już za dużo kluczy (wartości) ($\geq K$), wtedy musimy go podzielić - jedną z wartości z liścia przenosimy w odpowiednie miejsce do rodzica, a jako prawego i lewego syna tej przeniesionej wartości wstawiamy odpowiednio liście powstałe po podzieleniu tego oryginalnego liścia.

Jeżeli okazałoby się, że po przeniesieniu wartości z liścia do rodzica, rodzic ma za dużo kluczy (wartości), całą operację dzielenia powtarzamy rekurencyjnie na rodzicu - w razie potrzeby aż do korzenia.

42.3 Usuwanie

Usuwanie rozpoczynamy od znalezienia wartości, którą chcemy usunąć

Następnie bierzemy największą wartość z lewego poddrzewa lub najmniejszą wartość z prawego poddrzewa i wstawiamy w miejsce usuwanego elementu

Jeżeli w drzewie zostały naruszone warunki dotyczące ilości dozwolonych elementów w węźle, musimy to drzewo zrównoważyć. Proces ten jest generalnie dość skomplikowany i ciężki do opisanego algorytmem ze względu na mnogość możliwych przypadków, dlatego najlepiej spojrzeć na część praktyczną i przykłady.

43 Drzewa AVL: rotacje, operacje z wykorzystaniem rotacji i ich złożoność.

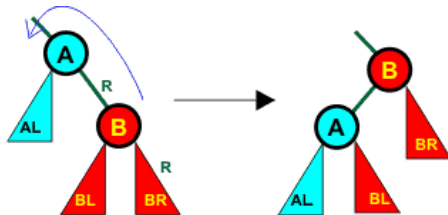
Drzewa AVL są odmianą drzew BST, w której dla każdego węzła mamy dodatkowy warunek: różnica między wysokością prawego i lewego poddrzewa danego węzła nie może być większa niż 1. Spełnienie tego warunku zapewnia nam dobre zrównoważenie drzewa, a co za tym idzie utrzymanie szybkości operacji wywszukiwania w tym drzewie.

Operacja wyszukiwania w drzewie AVL jest identyczna, jak dla zwykłego drzewa BST, natomiast w przypadku operacji wstawiania i usuwania, przebiegają one na początku tak samo jak w zwykłych BST, natomiast potem w razie potrzeby należy drzewo zrównoważyć poprzez rotacje.

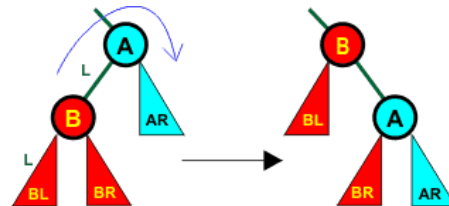
Operacje wyszukiwania, wstawiania oraz usuwania mają złożoność $O(\log_2 n)$, gdzie n to ilość elementów w drzewie

43.1 Rotacje

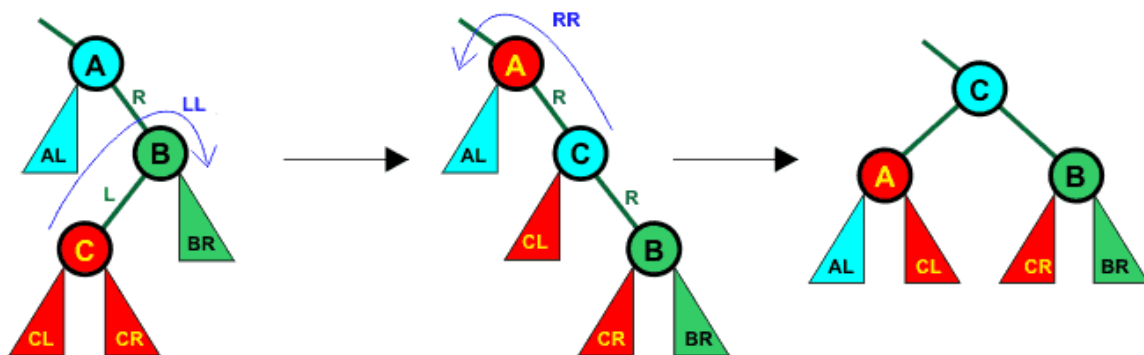
Rotacja RR



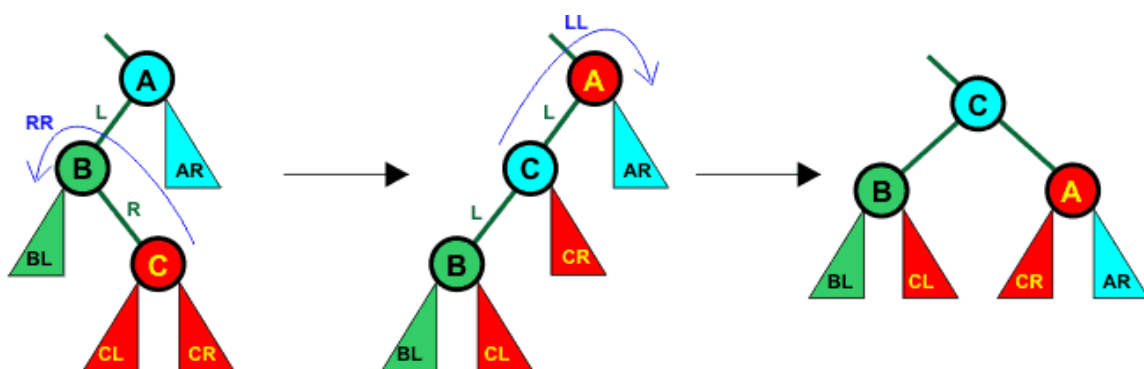
Rotacja LL



Rotacja RL



Rotacja LR



Gdy nierównowaga drzewa idzie dwa razy na prawo (tzn. gdy dla danego węzła prawe poddrzewo jest głębsze niż lewe i potem dla pierwszego węzła prawego poddrzewa jego prawe poddrzewo jest głębsze) do zbalansowania używamy rotacji RR, natomiast, gdy nierównowaga idzie najpierw na prawo potem na lewo, używamy rotacji RL, itd.

44 Algorytmy przeszukiwania wszerz i w głąb w grafach.

Algorytmy przeszukiwania grafu polegają na przejściu przez wszystkie osiągalne z wierzchołka startowego s wierzchołki grafu G .

44.1 Przeszukiwanie grafu wszerz (*breadth first search*, BFS)

44.1.1 Algorytm

Do przeszukiwania użyjemy kolejki $Q = \{s\}$. Ponadto używamy koloru czarnego do oznaczania odwiedzonych już wierzchołków.

Dopóki kolejka nie jest pusta:

1. Zdejm x z kolejki
2. Pomaluj x na czarno
3. Dla każdego niepomalowanego na czarno v , sąsiadującego z x wrzuć v do kolejki

44.1.2 Cechy

1. Złożoność obliczeniowa: $\theta(e + v)$
2. Złożoność pamięciowa: $\theta(v)$
3. Kompletność: BFS dla nieskończonego grafu zwróci wszystkie poprawne odpowiedzi, w przeciwieństwie do DFS.

44.1.3 Wariacje algorytmu

- Faktyczne "przeszukiwanie": zwracamy wierzchołek, jeśli jego wartość jest równa poszukiwanej.
- Przypisywanie wierzchołkom odległości od s .
- Przypisanie wierzchołkom rodzica, w celu znalezienia drogi do s

44.1.4 Zastosowania

- Garbage collection
- Najkrótsza droga do s (licząc liczbę krawędzi)
- Wykrywanie cykli grafu nieskierowanego
- Sprawdzanie dwudzielności grafu
- Social networking
- Algorytmy sieci przepływowych (np. wyznaczanie maksymalnego przepływu)
- Crawlery

44.2 Przeszukiwanie grafu wgłąb (*depth first search*, DFS)

44.2.1 Algorytm

Używamy koloru czarnego do oznaczania odwiedzonych już wierzchołków.

Dla danego wierzchołka x (zaczynając od s):

1. Pomaluj x na czarno
2. Wywołaj się rekurencyjnie dla każdego niepomalowanego na czarno v , sąsiadującego z x

44.2.2 Wersja z zapamiętywaniem rodzica

Słownik $P = \{s : null\}$ przypisuje każdemu wierzchołkowi (poza s) rodzica. Używamy koloru czarnego do oznaczania odwiedzonych już wierzchołków, oraz szarego do wierzchołków właśnie odwiedzanych.

Dla danego wierzchołka x (zaczynając od s):

1. Pomaluj x na szaro
2. Dla każdego białego v , sąsiadującego z x :

- (a) $P[v] = x$
 - (b) Wywołaj się rekurencyjnie na v
3. Pomaluj x na czarno

44.2.3 Cechy

1. Złożoność obliczeniowa: $\theta(e + v)$
2. Złożoność pamięciowa: $\theta(v)$
3. Niekompletność dla grafów nieskończonych

44.2.4 Wariacje algorytmu

- Faktyczne "przeszukiwanie": zwracamy wierzchołek, jeśli jego wartość jest równa poszukiwanej.
- Przypisywanie wierzchołkom odległości od s .
- Przypisanie wierzchołkom rodzica, w celu znalezienia drogi do s

44.2.5 Zastosowania

- Garbage collection
- Minimalne drzewo rozpinające (dla grafów nieważonych)
- Wykrywanie cykli w grafie skierowanym
- Sortowanie topologiczne
- Wykrywanie cykli grafu nieskierowanego
- Algorytmy sieci przepływowych (np. wyznaczanie maksymalnego przepływu)
- Crawlery
- Labirynty

45 Algorytmy wyszukiwania najkrótszej ścieżki (Dijkstry oraz Bellmana-Forda).

W algorytmach wyszukiwania najkrótszej ścieżki używamy procedury $relax(u, v, w)$, sprawdzającej, czy ścieżka z u do w przez v jest krótsza od aktualnie najkrótszej ścieżki.

$relax(u, v, w)$:

Jeśli $D[u, w] = D[u, v] + D[v, w]$ to:

$$D[u, w] = D[u, v] + D[v, w]$$

$$P[u, w] = v$$

45.1 Algorytm Dijkstry

Algorytm Dijkstry pozwala na wyliczenie w grafie $G = \{V, E\}$ najkrótszej ścieżki od wierzchołka startowego s do wszystkich pozostałych wierzchołków. $|V| = v, |E| = e$

45.1.1 Algorytm

Posługujemy się słownikiem $P = \{\}$ przypisującym każdemu wierzchołkowi rodzica (w celu znalezienia ścieżki), słownikiem D oznaczającym odległość wierzchołków od s oraz kolejką priorytetową $S = \{v \in V : D[v]\}$ nieodwiedzonych wierzchołków. $D[s] = 0; D[\forall v \in V \setminus \{s\}] = \infty$.

Używamy uproszczonej procedury $relax$ - wierzchołkiem startowym (u) jest zawsze s

$relax(v, w)$:

Jeśli $D[w] = D[v] + e_{v \rightarrow w}$ to:

$$D[w] = D[v] + e_{v \rightarrow w}$$

$$P[w] = v$$

Dopóki $S \neq \emptyset$:

1. Usunąć wierzchołek v o najmniejszej wartości z S
2. Dla każdego wierzchołka w sąsiadującego z v wywołać $relax(v, w)$

45.1.2 Złożoność

- Inicjalizacja - $\theta(v)$
- $\theta(v)$ usunięcie z kolejki
- $O(e)$ pomniejszeń wartości w D
- Przy pomocy kopca binarnego możemy uzyskać $O(\log(v))$ dla pomniejszenia wartości, $O(\log(v))$ dla usunięcia z kolejki. Złożoność: $\theta((e + v) \log(v))$
- Przy pomocy kopca Fibonacciego możemy uzyskać $\theta(1)$ dla pomniejszenia wartości, $\theta(\log(v))$ dla usunięcia z kolejki. Złożoność: $O(v \cdot \log(v) + e)$

45.1.3 Cechy

- Nie działa poprawnie dla grafów z wagami ujemnymi
- Algorytm zachłanny

45.2 Algorytm Bellmana-Forda

Algorytm Bellmana-Forda pozwala na wyliczenie w grafie G najkrótszej ścieżki z wierzchołka startowego s do wszystkich pozostałych wierzchołków.

45.2.1 Algorytm

Posługujemy się słownikiem $P = \{\}$, przypisującym każdemu wierzchołkowi rodzica (w celu znalezienia ścieżki), słownikiem D oznaczającym odległość wierzchołków od s . $D[s] = 0$; $D[v] = \infty \forall v \in V \setminus \{s\}$.

Ponownie używamy uproszczonej procedury $relax$.

Dla każdego wierzchołka w $V \setminus \{s\}$:

Wywołaj $relax(u, v)$ dla każdej krawędzi $(u, v) \in E$

Dodatkowo można wykonać sprawdzenie, czy w grafie nie ma cykli o wadze ujemnej:

Dla każdej krawędzi $(u, v) \in E$:

Jeśli $D[v] > D[u] + e_{u \rightarrow v}$, to istnieje cykl o wadze ujemnej.

45.2.2 Złożoność

$O(v \cdot e)$

45.2.3 Cechy

- Działa poprawnie dla grafów z wagami ujemnymi (bez cykli o wagach ujemnych)
- Pozwala decydować, czy graf ma cykle ujemne
- Używany w protokole RIP
- Algorytm można zoptymalizować: jeśli w danej iteracji nie dokonał zmiany, należy przerwać jego działanie (ścieżki już są najkrótsze)

45.3 Algorytm Floyda-Warshalla

Algorytm Floyda-Warshalla pozwala wyliczyć najkrótsze ścieżki pomiędzy wszystkimi wierzchołkami w grafie G (nieposiadającym ujemnych cykli).

45.3.1 Algorytm

Posługujemy się słownikiem $P = \{\}$, przypisującym parze (u, w) rodzica dla u , na ścieżce od u do w , słownikiem D przypisującym parze (u, w) długość najkrótszej (aktualnie) ścieżki od u do w .

$$D[(u, w) : u, w \in V] = \begin{cases} 0, & \text{jeśli } u = w \\ e_{u \rightarrow w}, & \text{jeśli } (u, w) \in E \\ \infty, & \text{jeśli } (u, w) \notin E \end{cases}$$

Wywołaj $relax(u, v, w)$ dla każdej trójki $(u, v, w) : u, w, v \in V$ wywołaj

45.3.2 Złożoność

$$\theta(v^3)$$

45.3.3 Cechy

- Prostota
- Pozwala obliczyć za jednym razem wszystkie ścieżki pomiędzy wierzchołkami
- Działa dla grafów z krawędziami ujemnymi (ale nie z cyklami ujemnymi)
- Złożoność jest niezależna od gęstości grafu

46 Programowanie dynamiczne: podział na podproblemy, porównanie z metodą "dziel i zwyciężaj".

Definicja 46.1 *Programowanie dynamiczne*

Technika projektowania algorytmów polegająca na **podziale problemu na mniejsze podproblemy** względem kilku parametrów (podobnie jak w metodzie dziel i zwyciężaj).

Wyniki rozwiązywania podproblemów są **zapisywane w tabeli**, dzięki czemu w przypadku natrafienia na ten sam podproblem nie trzeba go ponownie rozwiązywać.

Cechy

- Problem posiada powtarzające się podproblemy
- Podproblemy musi cechować **własność optymalnej podstruktury**
- Zastosowanie brute force prowadzi do **ponadwielomianowej** liczby rozwiązań podproblemów, podczas gdy sama liczba różnych podproblemów jest wielomianowa

Niejednokrotnie stosowanie techniki programowania dynamicznego daje w rezultacie algorytm pseudowielomianowy.

Definicja 46.2 *Własność optymalnej podstruktury*

Oznacza, że optymalne rozwiązanie problemu jest **funkcją optymalnych rozwiązań podproblemów**, czyli znając optymalne rozwiązania podproblemów można efektywnie wyznaczyć rozwiązanie problemu.

Definicja 46.3 *Funkcja celowa*

W zadaniach programowania liniowego liniowa funkcja, dla której szukane jest optymalne rozwiązanie minimum lub maksimum. Dla zdefiniowanego zadania programowania liniowego

Optymalizacja – problem polegający na znalezieniu ekstremum zadanej funkcji celu.

Metody Programowania Dynamicznego

- Metoda zstępująca z zapamiętywaniem polega na rekurencyjnym wywoływaniu funkcji z zapamiętywaniem wyników. Metoda ta jest podobna do metody dziel i zwyciężaj – różni się od niej tym, że jeśli rozwiązanie danego problemu jest już w tabeli z wynikami, to należy je po prostu stamtąd odczytać.
- Metoda wstępująca polega na rozwiązywaniu wszystkich możliwych podproblemów, zaczynając od tych o najmniejszym rozmiarze. Wówczas w momencie rozwiązywania podproblemu na pewno są już dostępne rozwiązania jego podproblemów. W tym podejściu nie zużywa się pamięci na rekurencyjne wywołania funkcji. Może się jednak okazać, że część podproblemów została rozwiązana nadmiarowo (nie były one potrzebne do rozwiązania głównego problemu).

Klucz do zaprojektowania algorytmu tą techniką leży w znalezieniu **równania rekurencyjnego** opisującego optymalną wartość **funkcji celu** dla danego problemu jako funkcji optymalnych wartości funkcji celu dla podproblemów o mniejszych rozmiarach. Programowanie dynamiczne znajduje optymalną wartość funkcji celu dla całego zagadnienia, rozwiązując podproblemy od najmniejszego do największego i zapisując optymalne wartości w tablicy. Pozwala to zastąpić wywołania rekurencyjne odwołaniami do odpowiednich komórek wspomnianej tablicy i gwarantuje, że każdy podproblem jest rozwiązywany tylko raz. Rozwiązanie ostatniego z rozpatrywanych podproblemów jest na ogół wartością rozwiązania zadanego zagadnienia.

Przykład - Ciąg Fibonacciego

```
# Programowanie dynamiczne wstępujące, wersja z listą.
def fibonacci(n):
    F = [0] * n # trzymamy wszystkie wartości
    for i in range(2, n+1):
        F[i] = F[i-1] + F[i-2]
    return F[n]

# Programowanie dynamiczne zstępujące.
FIBONACCI = {0:0, 1:1} # globalny słownik

def fibonacci(n):
    global FIBONACCI
    if n not in FIBONACCI:
        FIBONACCI[n] = fibonacci(n-1) + fibonacci(n-2)
    return FIBONACCI[n]
```

47 Algorytm zachłanny: przykład optymalnego i nieoptymalnego wykorzystania.

Definicja 47.1 Algorytmy zachłanne (*ang. greedy algorithms*)

Algorytmy podejmujące w każdym kroku taką decyzję, która w danej chwili jest najkorzystniejsza (**lokalnie optymalna**) licząc, że doprowadzi to do znalezienia rozwiązania **globalnie optymalnego**.

Cechy

- Nie zawsze znajdują rozwiązanie optymalne
- Są podzbiorem algorytmów **heurystycznych**
- Są to algorytmy **deterministyczne** – nie ma w nich losowości

Przykład - Pakowanie plecaka

```
posortuj nierosnąco przedmioty według wartości  $c[j]/w[j]$   
aktualna_waga:=0
```

```
for i:=1 to n do  
  if  $w[i] + aktualna\_waga \leq W$  then  
    dodaj i-ty przedmiot do plecaka  
    aktualna_waga := aktualna_waga +  $w[i]$ 
```

Dane wejściowe, dla których algorytm będzie działał

- Optymalnie

Pojemność plecaka: 8

Dane wejściowe: 5, 3, 2, 1, 1, 1, 1, 1, 1, 1

Rozwiązanie: $5 + 3 + 2$

- Nieoptymalnie

Pojemność plecaka: 7

Dane wejściowe: 3, 3, 2, 2

Rozwiązanie: $3 + 3 = 6$:(

Rozwiązanie optymalne: $3 + 2 + 2 = 7$

48 Kolorowania wierzchołkowe (grafów planarnych) i krawędziowe grafów, algorytmy i ich złożoności.

48.1 Kolorowanie krawędziowe grafów

Problem kolorowania krawędzi, podobnie jak klasycznego kolorowania wierzchołków, jest NP-trudny – nie istnieją wielomianowe algorytmy kolorujące grafy zawsze w sposób optymalny.

Przykład nieoptymalnego algorytmu kolorowania krawędziowego:

1. Użyj BFS (Breadth First Search - Przeszukiwanie wszerz) do poruszania się po grafie.
2. Wybierz wierzchołek i nadaj różne kolory jego krawędziom.
3. Przejdź do kolejnego wierzchołka.
4. Powtarzaj aż wszystkie krawędzie zostaną pokolorowane.

48.2 Kolorowanie wierzchołkowe grafów planarnych

Definicja 48.1 *Graf planarny - graf, który można narysować na płaszczyźnie tak, by krzywe obrazujące krawędzie grafu nie przecinały się ze sobą.*

Twierdzenie 48.2 *Liczba chromatyczna grafu planarnego jest równa lub mniejsza niż 4*

Każdy graf planarny można pokolorować 5 kolorami w czasie liniowym.

49 Algorytmy wyszukiwania minimalnego drzewa rozpinającego: Boruvki, Prima i Kruskala.

49.1 Algorytm Boruvki

1. Weź wszystkie wierzchołki, ale bez krawędzi - każdy wierzchołek jest osobnym drzewem.
2. Dla każdego drzewa znajdź krawędź o najmniejszej wadze w oryginalnym grafie, która łączy je z innym drzewem.
3. Dodaj tę krawędź do MST.
4. Powtarzaj, dopóki drzewa nie połączyły się w jedno.

Złożoność: $O(E \log V)$.

49.2 Algorytm Prima

1. Zainicjalizuj MST jednym wierzchołkiem, wybranym losowo z grafu.
2. Spośród krawędzi łączących MST z wierzchołkami jeszcze w nim niebędącymi wybierz tę, która ma najmniejszą wagę, i dodaj ją do MST.
3. Powtarzaj krok 2 aż wszystkie wierzchołki znajdą się w MST.

Złożoność: Zależy od użytych struktur danych. $O(V^2)$ lub $O(E \log V)$, jeśli użyta została lista sąsiedztwa.

49.3 Algorytm Kruskala

1. Posortuj krawędzie niemalejąco pod względem wag.
2. Wybierz krawędź o najmniejszej wadze. Sprawdź czy tworzy cykl z MST utworzonym do tej pory. Jeśli nie, uwzględnij tę krawędź w MST. W przeciwnym wypadku, odrzuć.
3. Powtarzaj krok 2 aż otrzymasz $V - 1$ krawędzi w MST

Złożoność: Sortowanie krawędzi zajmuje $O(E \log E)$. Po sortowaniu, iterujemy po krawędziach i używamy algorytmu find-union który może zająć $O(\log V)$. Zatem ogólna złożoność to $O(E \log E + E \log V)$. Wartość E jest mniejsza niż V^2 , więc $O(\log V) = O(\log E)$. Złożoność: $O(E \log E)$ lub $O(E \log V)$.

50 Najważniejsze algorytmy wyznaczania otoczki wypukłej zbioru punktów w układzie współrzędnych (Grahama, Jarvisa, algorytm przyrostowy (quickhull)).

Definicja 50.1 *Algorytm Grahama* - złożoność $O(n \log n)$.

1. Wybierz punkt (ozn. O) o najmniejszym y (jeśli jest kilka *ex aequo*, to ten z nich o najmniejszym x).
2. Posortuj pozostałe punkty leksykograficznie względem:
 - kąta pomiędzy wektorem OP_i a osią X ,
 - długości wektora OP_i .
3. Przeglądaj listę posortowanych punktów:
 - Od bieżącej pozycji weź trzy kolejne punkty (ozn. A, B, C).
 - Jeśli punkt B leży na zewnątrz trójkąta AOC , to może należeć do otoczki wypukłej. Przejdź do następnej trójki punktów.
 - Jeśli punkt B leży wewnątrz trójkąta AOC , to znaczy, że nie należy do otoczki. Usuń punkt B z listy i cofnij się o jedną pozycję (o ile bieżąca pozycja jest różna od początkowej).
4. Pozostałe na liście punkty stanowią otoczkę wypukłą.

Definicja 50.2 Algorytm Jarvisa - średnia złożoność $O(kn)$, gdzie n - liczba punktów, k - liczba punktów należących do otoczki; pesymistyczna - $O(n^2)$.

1. P_1 - punkt na otoczce wypukłej o najmniejszej współrzędnej y (jeśli jest więcej niż jeden, wybierany jest ten o najmniejszej współrzędnej x),
2. $P_0 := [-\infty, y(P_1)]$,
3. $i := 1$,
4. powtarzaj:
 - P_{i+1} - punkt N , dla którego kąt $P_{i-1}P_iN$ jest największy,
 - jeśli $N = P_1$, koniec iterowania,
 - $i := i + 1$,
5. ostatecznie otoczkę tworzą punkty $P_1 \dots P_i$.

Implementację można usprawnić, odrzucając w każdej iteracji punkty znajdujące się po prawej stronie wektora P_1P_i , ponieważ na pewno nie będą należały do otoczki. Zabieg ten nie wpływa jednak na asymptotyczną złożoność obliczeniową algorytmu.

Definicja 50.3 Quickhull - średnia złożoność $O(n \log n)$, pesymistyczna złożoność - $O(n^2)$.

QuickHull(P):

1. Znajdź w P punkty o minimalnym i maksymalnym x (A oraz B).
2. Podziel P na dwa podzbiory S_1 i S_2 znajdujące się nad i pod prostą AB .
3. Wywołaj rekurencyjnie $QuickHull(A, B, S_1)$ i $QuickHull(B, A, S_2)$.

QuickHull(A, B, P) - funkcja pomocnicza:

1. Jeśli P jest pusty – koniec.
2. Jeśli P ma jeden element, ten punkt należy do otoczki – koniec.
3. W przeciwnym razie:
 - Znajdź w P punkt C najbardziej oddalony od prostej AB . Ten punkt należy do otoczki wypukłej.
 - Odrzuć wszystkie punkty z wnętrza trójkąta ABC , nie mogą należeć do otoczki.
 - Znajdź zbiór S_1 punktów znajdujących się po prawej stronie prostej AC oraz analogiczny zbiór S_2 dla prostej BC . (Stronę określa znak równania ogólnego prostej).
 - Wywołaj rekurencyjnie $QuickHull(A, C, S_1)$ i $QuickHull(B, C, S_2)$.

51 Problemy P, NP, NP-zupełne i zależności między nimi. Hipoteza P vs. NP.

Definicja 51.1 Problem P (ang. *deterministic polynomial, deterministycznie wielomianowy*) – problem decyzyjny, dla którego rozwiązanie można **znaleźć** w czasie wielomianowym.

Definicja 51.2 Problem NP (ang. *nondeterministic polynomial, niedeterministycznie wielomianowy*) – problem decyzyjny, dla którego rozwiązanie można **zweryfikować** w czasie wielomianowym.

Twierdzenie 51.3 $P \stackrel{?}{=} NP$. Każdy problem P jest NP, jednak nie wiadomo czy istnieje problem NP niebędący P.

Definicja 51.4 Problem NP-zupełny (ang. *NP-Complete*) – taki problem NP, do którego można w czasie wielomianowym zredukować dowolny inny problem NP. Czasami zamiast redukcji w czasie wielomianowym używa się redukcji w pamięci logarytmicznej.

Stąd wynika, że jeśli potrafimy rozwiązać jakikolwiek problem NP-zupełny w czasie wielomianowym, to potrafimy rozwiązać w takim czasie wszystkie problemy NP.

52 Automat minimalny, wybrany algorytm minimalizacji.

52.1 Automat minimalny

Definicja 52.1 *Automat $\mathcal{A} = (S, A, f, s_0, T)$ rozpoznający język L jest minimalny, jeśli posiada najmniejszą liczbę stanów spośród wszystkich automatów rozpoznających język L .*

Definicja 52.2 *Dla dowolnego języka $L \in \mathcal{REC}(A^*)$ automat*

$$\mathcal{A}_{P_L^r} = (A^*/P_L^r, A, f^*, [1]_{P_L^r}, T),$$

gdzie $T = \{[w]_{P_L^r} : w \in L\}$ jest automatem minimalnym rozpoznającym język L

Definicja 52.3 *Konstrukcja automatu minimalnego z wykorzystaniem prawej kongruencji automatowej*

$L \subset A^*$ - dowolny język

$\Theta_L \subset A^* \times A^*$ - relacja równoważności dzieląca A^* na dwie klasy:

$$L \text{ oraz } A^* \setminus L$$

ρ_i dla $i \in \mathbb{N}$ - zstępujący ciąg relacji:

$$\rho_1 = \Theta_L,$$

$$\rho_i = \{(u, w) \in A^* \times A^* : \forall a \in A \cup \{\mathbf{1}\} : (ua, wa) \in \rho_{i-1}\} \text{ dla } i = 2, \dots$$

Wtedy

$$\bigcap_{i \in \mathbb{N}} \rho_i = P_L^r$$

52.2 Minimalizacja automatu

Definicja 52.4 Algorytm “Minimalizuj 1”

http://wazniak.mimuw.edu.pl/index.php?title=J%C4%99zyki%2C_automaty_i_obliczenia/Wyk%C5%82ad_5:_Algorytmy_konstrukcji_automatu_minimalnego

Wejście L - język

Wyjście: automat minimalny $\mathcal{A} = (S_L, A, f_F, s_L, T_L)$ taki, że $L\mathcal{A} = L$

```
1       $S_L \leftarrow \{L\};$ 
2       $Put(\mathcal{L}, L);$ 
3      while  $\mathcal{L} \neq \emptyset$  do
4       $M \leftarrow \text{\texttt{\textit{zdejmij}}}(\mathcal{L});$ 
5      foreach  $a \in A$  do
6       $N \leftarrow a^{-1}M;$ 
7      if  $N \cap S_L = \emptyset$  then
8       $S_L \leftarrow S_L \cup \{N\};$ 
9       $Put(\mathcal{L}, N);$ 
10     end if
11     end for
12     end while
13     foreach  $M \in S_L$  do
14     foreach  $a \in A$  do
15      $f_L(M, a) \leftarrow a^{-1}M;$ 
16     end for
17     end for
18      $s_L \leftarrow L;$ 
19      $T_L \leftarrow \{u^{-1}L : u \in L\};$ 
20     return  $\mathcal{A}';$ 
```

53 Lemat o pompowaniu dla języków regularnych.

Definicja 53.1 Niech $L \subset A^*$ będzie językiem rozpoznawalnym. Istnieje liczba naturalna $N \geq 1$ taka, że dowolne słowo $w \in L$ o długości $|w| \geq N$ można rozłożyć na katenację:

$$w = v_1uv_2$$

gdzie $v_1, v_2 \in A^*, u \in A^+, |v_1u| < N$ oraz

$$v_1u^*v_2 \subset L$$

Definicja 53.2 Jeśli rozpoznawalny język $L \subset A^*$ jest nieskończony, to istnieją $v_1, v_2 \in A^*, u \in A^+$, takie, że

$$v_1u^*v_2 \subset L$$

54 Warunki równoważne definicji języka regularnego: automat, prawa kongruencja syntaktyczna, wyrażenia regularne.

54.1 Wyrażenia regularne

Definicja 54.1 Rodzina regularnych podzbiorów

Niech A będzie dowolnym zbiorem. Rodzina $REG(A^*)$ regularnych podzbiorów A^* to najmniejsza rodzina R podzbiorów A^* taka, że:

1. $\emptyset \in R, \forall a \in A \ a \in R$
2. jeśli $X, Y \in R$, to: $X \cup Y, X \cdot Y \in R$
3. jeśli $X \in R$, to: $X^* = \bigcup_{n=0}^{\infty} X^n \in R$

Definicja 54.2 Wyrażenie regularne

Niech A będzie dowolnym alfabetem, a zbiór $\{+, \star, \emptyset, (,)\}$ alfabetem takim, że $A \cap \{+, \star, \emptyset, (,)\} = \emptyset$.

Słowo $\alpha \in (A \cup \{+, \star, \emptyset, (,)\})^*$ jest wyrażeniem regularnym nad alfabetem A wtedy i tylko wtedy, gdy:

1. $\alpha = \emptyset$
2. $\alpha = a \in A$
3. α jest w postaci $(\beta + \gamma), (\beta\gamma), \gamma^*$, gdzie β, γ są wyrażeniami regularnymi nad alfabetem A

Rodzinę wyrażeń regularnych nad A nazywamy WR .

54.2 Automat skończony

Definicja 54.3 *Automat*

Automatem nad alfabetem A nazywamy system $\mathcal{A} = (S, f)$ w którym:

- S jest dowolnym zbiorem, zwanym zbiorem stanów
- $f : S \times A \rightarrow S$ jest funkcją przejść

Automat $\mathcal{A}$ nazywamy skończonym, jeśli zbiór S jest skończony

Automat $\mathcal{A}$ ze stanem $s \in S$ taki, że

$$f(a, A^*) = Ss$$

nazywamy automatem z generatorem s .

Definicja 54.4 *Język rozpoznawalny*

Język $L \subset A^*$ jest rozpoznawalny wtedy i tylko wtedy, gdy istnieją:

- Automat skończony $\mathcal{A} = (S, f)$
- stan $s_0 \in S$
- zbiór $T \subset S$

takie, że

$$L = \{w \in A^* : f(s_0, w) \in T\}$$

s_0 - stan początkowy

T - zbiór stanów końcowych automatu

$$\mathcal{A} = (S, A, f, s_0, T)$$

$L = L(\mathcal{A})$ - język rozpoznawany przez automat $\mathcal{A}$

$REC(A^*)$ - rodzina wszystkich języków rozpoznawalnych nad alfabetem A

54.3 Prawa kongruencja syntaktyczna

Definicja 54.5 Prawa kongruencja

Każdy automat $\mathcal{A} = (S, f)$ z generatorem s_0 wyznacza na wolnym monoidzie A^* prawą kongruencję $\sim_{\mathcal{A}}$:

$$\forall u, v \in A^*$$

$$u \sim_{\mathcal{A}} v \Leftrightarrow f(s_0, u) = f(s_0, v)$$

Relacja $\sim_{\mathcal{A}}$ ma skończony indeks wtedy i tylko wtedy, gdy automat $\mathcal{A}$ jest skończony.

Twierdzenie 54.6 Każda prawa kongruencja $p \subset (A^*)^2$ wyznacza automat, zwany ilorazowym

$$\mathcal{A}_p = (A^*/_p, f^*), \text{ gdzie } f^*([w]_p, u) = [wu]_p$$

- $\mathcal{A}_p$ jest automatem z generatorem $[1]_p$
- $\mathcal{A}_p$ jest automatem skończonym wtedy i tylko gdy relacja p ma skończony indeks

54.4 Warunki równoważne definicji języka regularnego

Twierdzenie 54.7 Twierdzenie Kleenego

Dla każdego skończonego alfabetu A

$$REC(A^*) = REG(^*)$$

Twierdzenie 54.8 Dla dowolnego języka regularnego $L \in A^*$ następujące warunki są równoważne:

- $L = L(G)$ dla gramatyki G typu 3 (regularnej, lewoliniowej)
- $L = L(G)$ dla gramatyki G prawoliniowej
- $L = L(\mathcal{A})$ dla automatu deterministycznego $\mathcal{A}$
- $L = L(\mathcal{A}_{ND})$ dla automatu niedeterministycznego $\mathcal{A}_{ND}$
- $L = L(\mathcal{A}_{ND})$ dla automatu niedeterministycznego $\mathcal{A}_{ND}^p$ z pustymi przejściami
- $L = \|\alpha\|$ dla wyrażenia regularnego α nad alfabetem A
- $L \in REG(A^*)$
- monoid syntaktyczna $M(L)$ jest skończony
- liczba różnych pochodnych Brzozowskiego dla języka L jest skończona
- L jest sumą wybranych klas równoważności pewnej prawej kongruencji na A^* o skończonym indeksie oraz $L = \bigcup_{w \in L} [w]_p$
- istnieje skończony monoid M i homomorfizm

$$\varphi : A^* \rightarrow M : L = \varphi^{-1}(\phi(L))$$

54.5 Konstrukcja automatu skończenie stanowego rozpoznającego wyrażenie regularne.

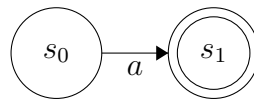
- automat rozpoznający język pusty



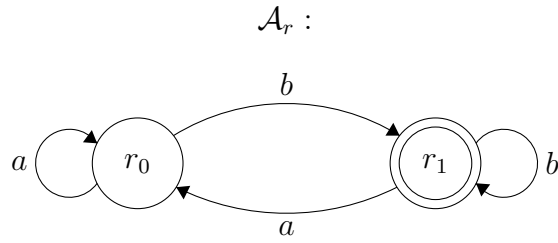
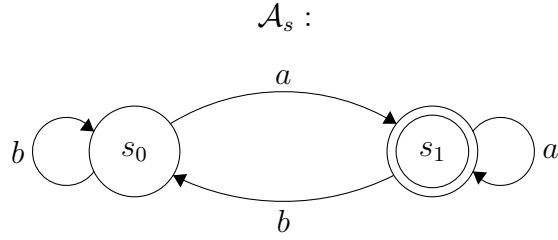
- automat rozpoznający język $\{1\}$



- automat rozpoznający język $\{a\} \forall a \in A$



- dla automatu $\mathcal{A}_r$ rozpoznającego wyrażenie r i automatu $\mathcal{A}_s$ rozpoznającego wyrażenie s : $L(\mathcal{A}_x) = L(r + s)$
 $\mathcal{A}_r = (S_r, A, f_r, s_{0r}, T_r)$
 $\mathcal{A}_s = (S_s, A, f_s, s_{0s}, T_s)$
 $\mathcal{A}_x = (S_x, A, f_x, s_{0x}, T_x)$



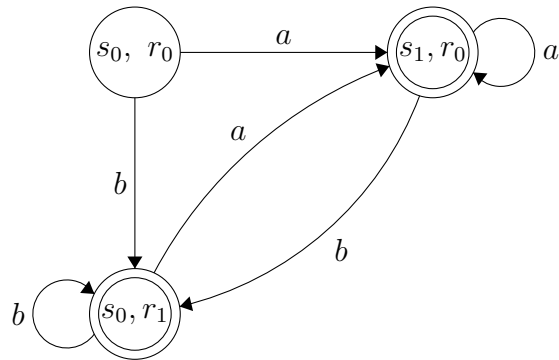
- automat rozpoznający język $r + s$:

$$s_{0x} = \{s_{0s}, s_{0r}\}$$

$$S_x = S_s \times S_r$$

$$f_x(\{s, r\}, a) = \{f_s(s, a), f_r(r, a)\}$$

$$T_x \ni t_x = \{s, r\} \Leftrightarrow s \in T_s \vee r \in T_R$$



- automat rozpoznający język sr :

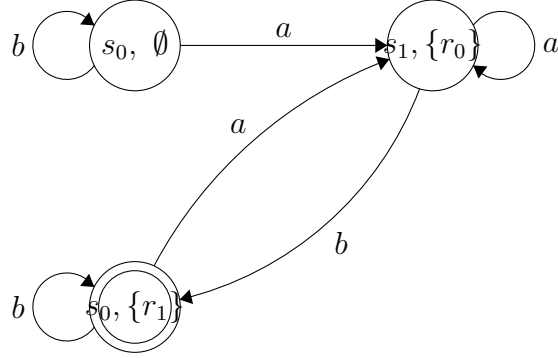
$$s_{0x} = \{s_{0s}, \emptyset\}$$

$$S_x = (S_s \times \mathcal{P}(S_r))$$

$$f_x(\{s, rs\}, a) = \{s_t, \bigcup_{r \in rs} f_r(r)\}$$

$$f_s(s, a) = s_t \in T_s \rightarrow f_x(\{s, rs\}) = \{f_s(s, a), \{r_0\} \cup \bigcup_{r \in rs} f_r(r, a)\}$$

$$\{s, rs\} = t_x \in T_x \Leftrightarrow rs \cap T_r \neq \emptyset$$



- automat rozpoznający język r^* : $s_{0x} = s_{0r}$

$$f_x(r, a) = f_r(r)$$

$$f_x(r_t \in R_t, 1) = r_{0x}$$

$$T_x = r_{0x}$$

55 Automaty niedeterministyczne i deterministyczne (w tym ze stosem); determinizacja.

55.1 Determinizacja automatów skończenie stanowych

Twierdzenie 55.1 *Język $L \in A^*$ jest rozpoznawany przez automat deterministyczny wtedy i tylko wtedy, gdy jest rozpoznawany przez automat niedeterministyczny.*

$\mathcal{A}_{ND} = (S, f, I, F)$ - automat niedeterministyczny

$\mathcal{A}_D = (P(S), \bar{f}, I, T)$ - równoważny automat deterministyczny, gdzie:

$$\begin{aligned}\bar{f} &: P(S) \times A^* \rightarrow P(S) \\ \forall Q \in P(S), \forall w \in A^* \quad \bar{f}(Q, w) &= \bigcup_{s \in Q} f(s, w) \\ T &= \{Q \in P(S) \mid Q \cap F \neq \emptyset\}\end{aligned}$$

55.2 Determinizm automatu ze stosem

Definicja 55.2 ***Deterministyczny automat ze stosem***

Automatem $\mathcal{AS} = (A, A_S, Q, q_0, z_0, f, Q_F)$ nazywamy **deterministycznym**, jeśli dla każdej konfiguracji $(z, q, a) \in A_S^* \times Q \times (A \cup \{1\})$

$$\begin{aligned}\#f(z, q, a) &\leq 1 \text{ oraz} \\ f(z, q, 1) &\neq \emptyset \Rightarrow f(z, q, a) = \emptyset \quad \forall a \in A\end{aligned}$$

Język akceptowany przez automat deterministyczny nazywamy językiem deterministycznym.

Twierdzenie 55.3 *Każdy język deterministyczny jest jednoznaczny (rozpoznawany przez gramatykę jednoznaczną), ale nie na odwrót.*

Twierdzenie 55.4 *Każdy język deterministyczny jest kontekstowy (rozpoznawany przez niedeterministyczny automat ze stosem), ale nie na odwrót.*

56 Problemy rozstrzygalne i nierozstrzygalne w teorii języków.

Problemy rozpatrywane w kategorii języków

- Należenie $\in$ - Problem przynależności języka L do jednego z typów języków L_i gdzie $i \in 0, 1, 2, 3$ generowanych przez odpowiednie gramatyki G_i
- Inkluzja $\subset$ - Problem zawierania się języka L_i w języku L_k
- Równoważność $\equiv$ - Problem równoważności dwóch języków $L_1 \equiv L_2$
- Pustość $\emptyset$ - Problem języka pustego czyli nie generującego żadnego słowa
- Nieskończoność ∞ - Problem języka nieskończonego czyli generującego nieskończenie wiele słów
- Jednoznaczność

Definicja 56.1 Wyprowadzenie lewostronne (prawostronne)

Niech $G = (V_N, V_T, v_0, P)$ będzie gramatyką bezkontekstową.

Lewostronnym (prawostronnym) wyprowadzeniem słowa $w \in V_T^*$ w gramatyce G nazywamy wyprowadzenie $v_0 \mapsto w_1 \mapsto \dots \mapsto w_n = w$ takie, że dla każdego $i = 0, \dots, n-1$, w_{i+1} jest generowane bezpośrednio z w_i przez zastąpienie pierwszego z lewej (prawej) symbolu nieterminalnego występującego w słowie w_i . Jeśli chcemy zaznaczyć, że wyprowadzenie jest lewostronne lub prawostronne, to posługujemy się zapisem

$v_0 \mapsto_L^* w$, $v_0 \mapsto_P^* w$.

Definicja 56.2 Jednoznaczność -

Gramatyka bezkontekstowa G jest **jednoznaczna** $\Leftrightarrow$ każde słowo generowane przez tę gramatykę ma **dokładnie jedno wyprowadzenie lewostronne (prawostronne)**.

Język bezkontekstowy L nazywamy **jednoznaczny**, jeśli istnieje jednoznaczna gramatyka bezkontekstowa generująca ten język.

Jednoznaczność gramatyki oznacza istnienie dokładnie jednego drzewa wyvodu dla każdego generowanego słowa.

—	3	2	1	0
Należenie $\in$	T	T	T	N
Inkluzja $\subset$	T	N	N	N
Równoważność $\equiv$	T	N	N	N
Pustość $\emptyset$	T	T	N	N
Nieskończoność ∞	T	T	N	N
Jednoznaczność	T	N	-	-

Oznaczenia

T - rozstrzygalny

N - nierozstrzygalny

57 Klasy języków w hierarchii Chomsky'ego oraz ich zamkniętość ze względu na operacje bołowskie, homomorfizmy, itp.

Definicja 57.1 Gramatyka

Jest to system $G = (V_N, V_T, P, v_0)$, w którym

- $V_N \neq \emptyset$ - skończony zbiór symboli nieterminalnych (alfabet nieterminalny)
- $V_T \neq \emptyset$ - skończony zbiór symboli terminalnych (alfabet terminalny)
- $P \subseteq (V_N \cup V_T)^+ \times (V_N \cup V_T)^*$ - skończona relacja, zbiór produkcji (praw)
- $v_0 \in V_N$ - symbol początkowy (startowy)

Ponadto zakładamy, że $V_N \cap V_T = \emptyset$ i dla każdego $(u, v) \in P$; $\#_{V_N} u \geq 1$.
Zatem w gramatyce alfabet terminalny i nieterminalny są rozłącznymi zbiorami, a słowo u występujące po lewej stronie produkcji zawiera co najmniej jeden symbol nieterminalny. Fakt, że para $(u, v) \in P$, zapisujemy:

$u \rightarrow v \in P$ lub $u \rightarrow_G v$.

Wykorzystujemy też zdefiniowane dla systemów przepisujących pojęcia generowania bezpośredniego " $\mapsto$ " i generowania " $\mapsto^*$ ".

Definicja 57.2 Gramatyka $G = (V_N, V_T, P, v_0)$ jest typu (i) dla $i = 0, 1, 2, 3$ wtedy i tylko wtedy, gdy spełnia następujące warunki:

- *Gramatyka*

Każdy system spełniający definicję 57.1

- *Kontekstowa*

Gramatyka, w której każde prawo ze zbioru P ma postać $u_1vu_2 \rightarrow u_1xu_2$ gdzie $u_1, u_2 \in (V_N \cup V_T)^$, $v \in V_N$, $x \in (V_N \cup V_T)^+$ lub $v_0 \rightarrow 1$, przy czym, jeśli $v_0 \rightarrow 1 \in P$, to v_0 nie występuje po prawej stronie w żadnym prawie z P*

- *Bezkontekstowa*

Gramatyka, w której każde prawo ze zbioru P ma postać $v \rightarrow x$ gdzie $v \in V_N$, $x \in (V_N \cup V_T)^$*

- *Regularna*

Gramatyka, w której każde prawo ze zbioru P ma postać $v \rightarrow v'x$ lub $v \rightarrow x$

gdzie $v, v' \in V_N$; $x \in V_T^$*

W oparciu o wprowadzone typy gramatyk określamy odpowiadające im rodziny (klasy) języków, oznaczając przez

$\mathcal{L}_0$ rodzinę wszystkich języków typu 0

$\mathcal{L}_1$ rodzinę wszystkich języków typu 1 - kontekstowych

$\mathcal{L}_2$ rodzinę wszystkich języków typu 2 - bezkontekstowych

$\mathcal{L}_3$ rodzinę wszystkich języków typu 3 - regularnych

Pomiędzy wprowadzonymi rodzinami języków zachodzą następujące zależności:

$$\mathcal{L}_3 \subsetneq \mathcal{L}_2 \subsetneq \mathcal{L}_1 \subsetneq \mathcal{L}_0$$

Zamkniętości języków na operacje

—	3	2	1	0
$\cup$	T	T	T	T
$\cdot$	T	T	T	T
$\star$	T	T	T	T
$\setminus$	T	N	T	N
$\cap$	T	N	T	T

Wytwarzanie oprogramowania

58 Reprezentacja liczb całkowitych; arytmetyka.

58.1 Kod znak-moduł prosty

58.1.1 Reguły:

1. Sygnalizacja znaku poprzez najwyższą pozycję rejestru (najczęściej pierwszą pozycję od lewej).
2. Cyfra 0 na pozycji znaku oznacza dodatniość.
3. Niezerowa cyfra na pozycji znaku oznacza ujemność.
4. Cyfry określające wartość liczby (pozostałe cyfry) bez zmian (bez żadnego kodowania).

58.1.2 Przykłady:

- Przykład w systemie dziesiętnym, $q = 10$, $c = 5$, $u = 2$, $n = 8$.
 $[+216.7]_{zmp} = 0|00216.70$
 $[-216.7]_{zmp} = 1|00216.70$
- Przykład w systemie binarnym, $q = 2$, $c = 10$, $u = 5$, $n = 16$.
 $[+1001100.101]_{zmp} = 0|0000100110.10100$
 $[-1001100.101]_{zmp} = 1|0000100110.10100$

58.1.3 Arytmetyka

Kodowanie znak-moduł prosty przez swoją skomplikowaną i czasochłonną arytmetykę nie nadaje się do praktycznego stosowania (konieczność niestandardowego obsługiwanego przepełnień)

58.2 Kod odwrotnościowy

58.2.1 Reguły

- Najbardziej znacząca cyfra (najczęściej lewa) jest cyfrą znaku.
- Zerowa cyfra znaku oznacza dodatniość.

- Najwyższa (ostatnia) cyfra znaku oznacza ujemność.
- Cyfry wartości liczby dodatniej pozostają bez zmian
- Cyfry wartości dla liczby ujemnej:
 - W systemie dwójkowym są odwróceniem bitów.
 - W systemie dziesiętnym są uzupełnieniem każdej cyfry do liczby 9.
 - Ogólnie są uzupełnieniem każdej cyfry do najwyższej cyfry systemu, czyli do wartości $q-1$.

58.2.2 Przykłady

- System dziesiętny: $q = 10$, $c = 7$, $u = 5$

$$[-182073.6954]_{odw} = [-0|0182073.69540]_{odw} = 9|9917926.30459$$

- - System binarny: $q = 2$, $c = 7$, $u = 5$

$$[-101110.011]_{odw} = [-0|0101110.01100]_{odw} = 1|0010001.10011$$

58.2.3 Arytmetyka

Algorytm sumowania:

- Dodaj kody, a w przypadku przepełnienia zwiększ wynik o najmniejszą możliwą liczbę dodatnią.

58.2.4 Przykłady

- W systemie dziesiętnym, $q=10$, $c=7$, $u=5$

+0 0000675.61900	liczby
-0 0000491.27800	
+0 0000184.34100	suma liczb
0 0000184.34100	kod sumy

0 0000675.61900	kody
9 9999508.72199	
10 0000184.34099	suma kodów
0 0000184.34099	pominięcie przepętlenia
+0 0000000.00001	korekta
0 0000184.34100	ostateczna suma kodów

- w systemie binarnym, $q=2$, $c=7$, $u=5$

+0 0000101.10100	liczby
-0 0000010.01100	
+0 0000100.01000	suma liczb
0 0000100.01000	kod sumy

0 0000101.10100	kody
1 1111101.10011	
10 0000100.00111	suma kodów
0 0000100.00111	pominięcie przepętlenia
+0 0000000.00001	korekta
0 0000100.01000	ostateczna suma kodów

58.3 Kod uzupełnieniowy

58.3.1 Kodowanie

Kodowanie uzupełnieniowe liczby ujemnej polega na uzupełnieniu każdej cyfry do najwyższej cyfry systemu (odwróceniu bitów) i dodaniu do otrzymanej wartości liczby 1 na najmniej znaczącym miejscu.

58.3.2 Dekodowanie

Dekodowanie liczby ujemnej zakodowanej uzupełnieniowo polega na odjęciu od kodu liczby 1 na najmniej znaczącym miejscu z uzupełnieniem w otrzymanym kodzie każdej cyfry do najwyższej cyfry systemu (odwróceniem bitów).

58.3.3 Przykłady kodowania

- w systemie dziesiętnym ($q = 10$, $c = 7$, $u = 5$)

-0 0000 491.27800	liczba
9 9999508.72199	kod odwrotnościowy
+0 0000000.00001	korekta $+10^{-5}$
9 9999508.72200	kod uzupełnieniowy

- w systemie binarnym $q = 10$, $c = 7$, $u = 5$.

-0 000 1010.01100	liczba
1 1110101.10011	kod odwrotnościowy
+0 0000000.00001	korekta $+2^{-5}$
1 1110101.10100	kod uzupełnieniowy

58.3.4 Arytmetyka

Suma kodów uzupełnieniowych jest zawsze kodem uzupełnieniowym sumy.

- w systemie dziesiętnym ($q = 10$, $c = 7$, $u = 5$)

-0 00 13490.27800	liczby
-0 0000 673.60900	
9 9986509.72199	kody odwrotnościowe
9 9999326.39099	
9 9986509.72200	kody po korektach (uzupełnieniowe)
9 9999326.39100	
19 9985836.11300	suma kodów uzupełnieniowych
9 9985836.11300	pominięcie przepełnienia

- w systemie binarnym $q = 10$, $c = 7$, $u = 5$

-0 00 10010.01100	liczby
-0 0000 110.10100	
1 1101101.10011	kody odwrotnościowe
1 1111001.01011	
1 1101101.10100	kody po korektach (uzupełnieniowe)
1 1111001.01100	
11 1100111.00000	suma kodów uzupełnieniowych
1 1100111.00000	pominięcie przepełnienia

58.4 Kod nadmiarowy

58.4.1 Kodowanie

1. Kodowanie nadmiarowe liczby dodatniej polega na ustawieniu cyfry znaku na wartość jednostkową.

```
-0 0340659.71280  liczba
 9|9659340.28719  kod odwrotnościowy
+0 0000000.00001  korekta +10-5
 9|9659340.28720  kod uzupełnieniowy
-9 0000000.00000  odjęcie  $-(10^8 - 10^7) = -9 \cdot 10^7$ 
 0|9659340.28720  kod nadmiarowy
```

2. Kod nadmiarowy liczby ujemnej to kod uzupełnieniowy z odjęciem najwyższej cyfry systemu na pozycji znaku.

```
-0 0101100.11010  liczba
 1|1010011.00101  kod odwrotnościowy
+0 0000000.00001  korekta +2-5
 1|1010011.00110  kod uzupełnieniowy
-1 0000000.00000  odjęcie  $-(2^8 - 2^7) = -1 \cdot 2^7$ 
 0|1010011.00110  kod nadmiarowy
```

58.4.2 Dekodowanie

- Dekodowanie nadmiarowe kodu liczby dodatniej (rozpoznawanego po niezerowej cyfrze znaku) polega na wyzerowaniu cyfry znaku.
- Dekodowanie nadmiarowe kodu liczby ujemnej (rozpoznawanego po zerowej cyfrze znaku) wymaga ustawienia najwyższej cyfry systemu w miejscu znaku z następującym dekodowaniem uzupełnieniowym.

58.4.3 Arytmetyka

Kod nadmiarowy sumy liczb to suma kodów nadmiarowych składników z odjętą wartością 1 na pozycji znaku.

1. w systemie dziesiętnym

+0 0239156.07840	
+0 0049710.25368	liczby
1 0239156.07840	kody nadmiarowe
1 0049710.25368	po przesunięciach
2 0288866.33208	suma kodów nadmiarowych
-1 0000000.00000	korekta wyniku
1 0288866.33208	kod nadmiarowy sumy

-0 0239156.07840	
-0 0049710.25368	liczby
9 9760843.92159	
9 9950289.74631	kody odwrotnościowe
9 9760843.92160	
9 9950289.74632	kody uzupełnieniowe
0 9760843.92160	korekty znaków: $-9 \cdot 10^7$
0 9950289.74632	kody nadmiarowe liczb
1 9711133.66792	suma kodów nadmiarowych
-1	korekta wyniku
0 9711133.66792	kod nadmiarowy sumy

2. w systemie binarnym

+0 0110011.01011	
+0 0011110.01100	liczby
1 0110011.01011	
1 0011110.01100	kody nadmiarowe po przesunięciach
10 1010001.10111	suma kodów nadmiarowych
-1 0000000.00000	korekta wyniku
1 1010001.10111	kod nadmiarowy sumy

-0 0110011.00011	liczby
-0 0111110.01100	
1 1001100.11100	kody odwrotnościowe
1 1000001.10011	
1 1001100.11101	kody uzupełnieniowe
1 1000001.10100	
0 1001100.11101	korekty znaków: $-2 \cdot 2^7$
0 1000001.10100	kody nadmiarowe liczb
1 0001110.10001	suma kodów nadmiarowych
-1 0000000.00000	korekta wyniku
0 0001110.10001	kod nadmiarowy sumy

59 Reprezentacja liczb rzeczywistych; arytmetyka zmiennopozycyjna.

59.1 Reprezentacja cecha-mantysa

Liczby postaci

$$m \times q^c$$

gdzie m - mantysa, q - podstawa systemu liczbowego, c - cecha

- Mantysa - ciąg najbardziej znaczących cyfr liczby.
- Cecha - wykładnik potęgi podstawy systemu w iloczynie z mantysą równy reprezentowanej liczbie.
- W rejestrze komputera pamiętane są wyłącznie mantysa oraz cecha, zaś podstawa systemu jest ustalona, zatem nie musi być reprezentowana i nie jest konieczna do działań.
- Różnorodne sposoby zapamiętania w komputerze liczb w postaci cecha-mantysą noszą nazwę **kodowań zmiennopozycyjnych**.

59.2 Kodowanie zmiennopozycyjne

- Kodowanie zmiennopozycyjne jest powszechnie stosowane w komputerach dla reprezentacji liczb rzeczywistych.
- W systemie dziesiętnym dla oddzielenia mantysy i cechy powszechne użycie litery E (lub e). Liczba 28.05

$$\begin{aligned} 28.05e0 &= 2805e-2 = +2805000e-5 = \\ &= 0.0002805e+5 = 2.805e1 = 0.2805e2 \end{aligned}$$

- W systemie binarnym dla oddzielenia mantysy i cechy powszechne użycie litery D (lub d). Liczba $(9.625)_{10} = (1001.101)_2$

$$\begin{aligned} 0.00001001101d1000 &= 1001101000d-110 = \\ &= 1.001101d11 = 0.1001101d100 \end{aligned}$$

- Różnorodność reprezentacji zmiennopozycyjnej wymusza konieczność ujednoliczenia zwaną **normalizacją**.

	normalizacja całkowita	normalizacja do jedynki	normalizacja do ułamka
28.05	2805e-2	2.805e1	.2805e2
-35000000	-35e6	-3.5e7	-.35e8
0.0000576	576e-6	5.76e-4	.576e-3
$(1001.101)_2$	1001101d-11	1.001101d11	.1001101d100
$(1010000)_2$	101d100	1.01d110	.101d111
$(-0.00001011)_2$	-1011d1001	-1.011d111	-.1011d1000

59.3 Normalizacja

59.4 Algorytmy kodowania i dekodowania

Przyjmujemy oznaczenia:

- w_m - liczba cyfr reprezentacji mantysy (bez cyfry znaku)
- w_c - liczba cyfr reprezentacji cechy (bez cyfry znaku)
- n - liczba cyfr rejestru

Zatem:

$$n = 1 + w_m + 1 + w_c$$

59.4.1 Algorytm kodowania

Algorytm kodowania zmiennopozycyjnego obejmuje poniższe kroki:

1. Przedstaw liczbę w systemie o podstawie q .
2. Zaokrąglaj wartość liczby do w_m cyfr.
3. Zaokrąglony wynik sprowadź do znormalizowanej postaci potęgowej $m \cdot q^c$
4. Jeżeli cecha jest zbyt duża (zbyt dodatnia) by zmieścić się w przewidzianym zakresie w_c to zasygnalizuj błąd przepełnienia i zakończ.
5. Jeżeli cecha jest zbyt mała (zbyt ujemna) by zmieścić się w zakresie w_c to zakoduj zerową mantysę oraz zerową wartość cechy i zakończ.
6. Zakoduj stałopozycyjnie mantysę na $w_m + 1$ pozycjach.
7. Zakoduj stałopozycyjnie cechę na $w_c + 1$ pozycjach.

Przykłady dziesiętne:

przykład	<i>pierwszy</i>	<i>drugi</i>	<i>trzeci</i>
liczba	28.5	−12345	−0.0005678
zaokrąglenie	28.5	−12300	−0.00568
znormalizowana postać potęgowa	2.85e1	−1.23e4	−5.68e−4
kod mantysy (odwrotnościowy)	0 285	9 876	9 431
kod cechy (zna-moduł prosty)	0 1	0 4	1 4
zawartość rejstru	0 285 0 1	9 876 0 4	9 431 1 4

Przykłady binarne:

liczba dziesiętna	23.5	−59	0.1875	−0.078125
liczba dwójkowa	10111.1	−11011	0.0011	−0.000101
znormalizowana postać potęgowa	0.10111d101	−0.111011d110	0.11d−10	0.101d−11
kod mantysy (uzupełnieniowy)	0 101111	1 000101	0 11000	1 101000
kod cechy (nadmiarowy)	1 0101	1 0110	0 1110	0 1101
zawartość rejstru	0 101111 1 0101	1 000101 1 0110	0 11000 0 1110	1 101000 0 1101

59.4.2 Algorytm dekodowania

System dziesiętny:

1. Dekoduj mantysę i cechę.
2. Opcjonalnie eliminuj postać potęgową.

System binarny:

1. Dekoduj mantysę i cechę.
2. Sprowadź cechę do postaci dziesiętnej.

3. Zdenormalizuj mantysę.
4. Zapisz mantysę w postaci dziesiętnej.

Przykłady binarne:

rejestr	0 101010101 1 00111	1 001100111 0 11011	1 001111111 1 10101
kod cechy (nadmiarowy)	1 00111	0 11011	1 10101
cecha binarnie	111	−101	10101
cecha dziesiętnie	7	−5	21
kod mantysy (odwrotnościowy)	0 101010101	1 001100111	1 001111111
mantysa zdekodowana	0.101010101	−0.110011	−0.11
mantysa zdenormalizowana	1010101.01	−0.00000110011	−11 0 000 000 000 000 000 000
dekodowana liczba	85.25	−0.02490234375	−3145728

59.5 Arytmetyka

- Brak ogólnego algorytmu działania na kodach zmiennopozycyjnych.
- Jedyna możliwa droga obejmuje oczywiste kroki:
 1. Dekodowanie argumentów.
 2. Wykonanie działania.
 3. Zakodowanie wyniku.

59.5.1 Dodawanie kodów zmiennopozycyjnych

Algorytm obejmuje kroki:

1. Zmodyfikuj mantysy dla zrównania cech (z możliwością denormalizacji).
2. Wyznacz mantysę sumy w postaci sumy mantys (możliwe wobec równych cech).
3. Zaokrąglij sumę mantys do liczby cyfr rejestru.
4. Znormalizuj wynik sumy (skoryguj cechę wyniku).

Przykład:

argumenty	33.5	26.25
postaci binarne	100001.1	11010.01
zaokrąglenie mantys	100010.0	11010.10
znormalizowane postaci potęgowe	0.1000100×2^6	0.110101×2^5
notacja naukowa	0.1000100d110	0.110101d101
zrównanie cech	0.1000100d110	0.0110101d110
suma mantys	$ \begin{array}{r} 0.1000100 \\ +0.0110101 \\ \hline 0.1111001 \quad d110 \end{array} $	
zaokrąglenie mantysy	$ \begin{array}{r} 0.111101 \quad d110 \end{array} $	
postać tradycyjna	111101	
wynik dziesiętnie	61.0	

59.5.2 Mnożenie kodów zmiennopozycyjnych

Algorytm może obejmować kroki:

1. Mantysa iloczynu to iloczyn mantys, dlatego pomnóż mantysy.
2. Cecha iloczynu to suma cech, zatem dodaj cechy.
3. Zaokrąglij mantysę wyniku, iloczynu mantys.
4. Znormalizuj mantysę modyfikując cechę.

Przykład:

argumenty	33.5	26.25
postaci naukowe	0.100010d110	0.110101d101
	<i>według poprzedniego podpunktu</i>	
iloczyn mantys	$ \begin{array}{r} 0.100010 \\ \times 0.110101 \\ \hline 100010 \\ 100010 \\ 100010 \\ +100010 \\ \hline 11100001010 \\ \text{czyli } 0.011100001010 \end{array} $	
mantysa zaokrąglona	0.0111000	
suma cech	$ \begin{array}{r} 110 \\ +101 \\ \hline 1011 \end{array} $	
postać naukowa wyniku	0.0111d1011	
normalizacja	0.1100d1010	
tradycyjna postać binarna	1110000000	
postać dziesiętna	896.0	

60 Różnice w wywołaniu funkcji statycznych, nie-statycznych i wirtualnych w C++.

60.1 Funkcje statyczne.

- **Globalne funkcje statyczne** to funkcje o zakresie widoczności ograniczonym do pliku źródłowego, który je zawiera (a dokładniej do ich *jednostce translacji*, czyli pliku który jest kompilowany - zatem pliku źródłowego wraz z dołączonymi includami).
- **Metody statyczne klas** to metody których wywołanie jest niezależne od instancjonowania klasy - wszystkie instancje klasy współdzielą kopię statycznych metod i pól. Instancja klasy nie jest nam potrzebna do wywołania statycznej metody (aczkolwiek można jej użyć).

```
#include <iostream>
using namespace std;

// statyczna funkcja globalna dostępna tylko z naszego pliku
static void sayHello(){
    cout << "hello" << endl;
}

class ExampleClass {
public:
    static void sayBye(){
        cout << "bye" << endl;
    }
};

int main(){
    sayHello();

    ExampleClass::sayBye(); // wywołanie bez instancjonowania klasy

    ExampleClass e();
    e.sayBye(); // wywołanie dla instancji klasy
}
```

60.2 Funkcje niestatyczne.

- **Niestatyczne funkcje globalne** są dostępne ze wszystkich plików, które include'ują zawierające je plik źródłowy - trzeba zatem uważać na konflikty nazw.
- **Niestatyczne metody klas** wymagają do wywołania instancji klasy - mogą zachowywać się różnie dla różnych instancji; mogą być oznaczone `final` i `override`.

```
#include <iostream>
using namespace std;

// niestatyczna funkcja globalna dostępna dla wszystkich plików
void sayHello(){
    cout << "hello" << endl;
}

class ExampleClass {
public:
    void sayBye(){
        cout << "bye" << endl;
    }
};

int main(){
    sayHello();

    // ExampleClass::sayBye() - wywołanie niepoprawne

    ExampleClass e();
    e.sayBye(); // wywołanie dla instancji klasy
}
```

60.3 Funkcje wirtualne.

- **Wirtualne metody klas** są przydatne w polimorfizmie.
- W przypadku zwykłej metody, wywołanie jej dla wskaźnika typu klasy podstawowej zawsze wywoła instancję z klasy podstawowej, nawet jeżeli wskaźnik będzie tak naprawdę wskazywał na klasę pochodną.

- Słowo kluczowe `virtual` wymusza dedukcję której metody użyć na podstawie zawartości wskaźnika, a nie jego typu. Dedukcja następuje w czasie wykonania programu.

Bez `virtual`:

```
#include <iostream>
using namespace std;

struct Base {
    void f() {
        cout << "base" << endl;
    }
};

struct Derived : Base {
    void f() {
        cout << "derived" << endl;
    }
};

int main(){
    Base b;
    Derived d;

    // call through reference
    Base& br = b; // the type of br is Base&
    Base& dr = d; // the type of dr is Base& as well
    br.f(); // "base"
    dr.f(); // "base"

    // cal through pointer
    Base* bp = &b; // the type of bp is Base*
    Base* dp = &d; // the type of dp is Base* as well
    bp->f(); // "base"
    dp->f(); // "base"

    return 0;
}
```

Przy użyciu virtual:

```
#include <iostream>
using namespace std;

struct Base {
    virtual void f() {
        cout << "base" << endl;
    }
};

struct Derived : Base {
    void f() override { // 'override' is optional
        cout << "derived" << endl;
    }
};

int main(){
    Base b;
    Derived d;

    // call through reference
    Base& br = b; // the type of br is Base&
    Base& dr = d; // the type of dr is Base& as well
    br.f(); // "base"
    dr.f(); // "derived"

    // call through pointer
    Base* bp = &b; // the type of bp is Base*
    Base* dp = &d; // the type of dp is Base* as well
    bp->f(); // "base"
    dp->f(); // "derived"

    return 0;
}
```

61 Sposoby przekazywania parametrów do funkcji (przez wartość, przez referencję). Zalety i wady.

61.1 Przekazywanie przez wartość.

Przekazywanie argumentu przez wartość oznacza, że argumentem może być **rvalue** (np. wyrażenie arytmetyczne), a przekazanie lvalue (zmiennej) powoduje utworzenie kopii.

- Za zaletę można uznać fakt możliwości użycia wyrażenia w wywołaniu (zakładając że nie używamy wyniku tego wyrażenia nigdzie indziej - nie potrzebujemy tworzyć zbędnej zmiennej).
- Kwestia tworzenia kopii może być zaletą lub wadą - umożliwia nam to swobodne modyfikowanie wartości bez modyfikowania oryginalnej zmiennej (co może być chcianą funkcjonalnością), ale może stanowić niepotrzebne obciążenie w przypadku gdy dana zmienna nie jest lub może być modyfikowana (wtedy tworzenie kopii jest niepotrzebne).

```
#include <iostream>
using namespace std;

void add2(int x){
    x += 2;
    cout << x << endl; // 7
}

int main(){
    int a = 5;
    cout << a << endl; // 5

    add2(a);
    cout << a << endl; // 5

    return 0;
}
```

61.2 Przekazywanie przez referencję.

Funkcja otrzymuje jako argument adres zmiennej, a nie jej wartość. Mając adres może odwołać się do pamięci tej zmiennej i zmienić jej zawartość. Wszelka modyfikacja takiego argumentu wewnątrz funkcji powoduje zmianę skojarzonej z tym argumentem zmiennej.

- Zaletą jest nietworzenie niepotrzebnych kopii, możliwość zmiany zmiennej spoza zakresu funkcji (zamiast np. zwracania i przypisywania nowej wartości).
- Nieprzemyślane użycie może spowodować nieprzewidzianą przez programistę zmianę zawartości zmiennej.
- Brak możliwości użycia wyrażenia jako argumentu funkcji - musimy zapisać jego wynik w zmiennej, by móc ją przekazać.

```
#include <iostream>
using namespace std;

void add2(int & x){
    x += 2;
    cout << x << endl; // 7
}

int main(){
    int a = 5;
    cout << a << endl; // 5

    add2(a);
    cout << a << endl; // 7

    return 0;
}
```

62 Wskaźniki, arytmetyka wskaźników, różnica między wskaźnikiem a referencją w C++.

Definicja 62.1 Wskaźnik – typ zmiennej odpowiedzialnej za przechowywanie adresu do innej zmiennej (innego miejsca w pamięci) w obrębie naszej aplikacji. Wskaźnik może wskazywać na jakąś zmienną, strukturę, tablicę czy nawet funkcję.

Definicja 62.2 Operatory związane ze wskaźnikami w C++

```
int a = 5;

// Creation of pointer to int variable
int *intPtr = nullptr; // In older versions NULL

// Using & (ampersand) to get address of variable
// then assign to pointer
intPtr = &a;

// Using * as dereference operator
// to get value of pointed variable
cout << *intPtr << endl;
```

Definicja 62.3 Arytmetyka wskaźników

Mając wskaźnik `int *p` wskazujący na hipotetyczny adres 10, wykonując na nim operację `p+2` dostajemy wskaźnik wskazujący na adres $10 + 2 * \text{sizeof}(\text{int})$, czyli zakładając, że typ `int` zajmuje w naszym kompilatorze 4 bajty dostajemy $10 + 2 * 4 = 18$.

Jest to przydatne szczególnie przy operowaniu na strukturach, których pola są ułożone po kolei w pamięci, np. tablice:

```
int *a = new int[10] {0, 10, 20, 30, 40, 50, 60, 70, 80, 90};  
// Now a pointing to first element in array  
cout << *a << endl; // 0  
  
// Using dereference operator with pointer arithmetic  
cout << *(a+3) << endl; // 30
```

Jak więc widać $*(a+i) \Leftrightarrow a[i]$.

Definicja 62.4 Wskaźnik a referencja

Referencję możemy traktować jako dodatkową nazwę na już istniejący obiekt. Jest to coś w rodzaju stałego wskaźnika. Składnia referencji zabrania jednak wykonywania operacji znanych dla wskaźników:

- Dla referencji nie można zmienić referowanego obiektu - raz ustawiony (a musi być ustawiony prawie zawsze podczas jej deklaracji) nie zmienia się przez cały czas jej życia
- Nie jest dostępna arytmetyka referencji (w przeciwieństwie do arytmetyki wskaźników)

Po kompilacji (w kodzie maszynowym) referencje i wskaźniki niczym się nie różnią. Korzystając z referencji świadomie rezygnujemy z pewnych możliwości oferowanych nam przez wskaźniki. W zamian dostajemy:

- Łatwiejszą notację - nie musimy używać operatora wyłuskania (*) do odwoływania się do obiektu
- Większe bezpieczeństwo - brak arytmetyki dla referencji znacznie utrudnia popełnienie błędów związanych z niepoprawnym odwołaniem do pamięci

Przykład użycia referencji

```
int n = 6;
```

```
// Deklaracja referencji na typ int i przypisanie referowanego obiektu  
// Referencję deklarujemy dodając & (ampersand) między nazwą typu a zmienne  
// (nie mylić z & jako operatorem pobrania adresu omawianym przy wskaźnikach)
```

```
int &r = n;
```

```
cout << r << endl; // 6
```

```
// Zmiana wartości odwołując się do referncji  
// zmienia tak na prawdę wartość oryginalnego obiektu
```

```
r = 7;
```

```
cout << n << endl; //7
```

63 Podstawowe założenia paradygmatu obiektowego: dziedziczenie, abstrakcja, enkapsulacja, polimorfizm.

Definicja 63.1 *Dziedziczenie (kompozycja)*

Umożliwia stworzenie hierarchii obiektów w programie. Polega na przejęciu właściwości i funkcjonalności obiektów innej klasy i ewentualnej modyfikacji tych właściwości i funkcjonalności w taki sposób, by były one bardziej wyspecjalizowane.

```
class Mammal {
public:
void feed() {
cout << "sucking" << endl;
}

virtual void giveVoice() = 0; // Pure virtual method
};

class Cat : public Mammal {
public:
void giveVoice() override {
cout << "meow" << endl;
}
};

int main() {
Cat cat = new Cat();
cat.feed();
cat.giveVoice();
/*
sucking
meow
*/
}
```

Definicja 63.2 *Abstrakcja*

Każdy obiekt w systemie służy jako model abstrakcyjnego "wykonawcy", który może wykonywać pracę, opisywać i zmieniać swój stan, oraz komunikować się z innymi obiektami w systemie, bez ujawniania, w jaki sposób zaimplementowano dane cechy.

```
class DiscountComputator {
private:
int factor;
...

public:
// Setter
void setFactor(int factor) {
this.factor = factor;
}

double compute(double price) {
...
}
};

int main() {
DiscountComputator dc = new DiscountComputator();

dc.setFactor(5);

// We are not interested how it is calculated
double result = dc.compute(123.45);
}
```

Definicja 63.3 *Enkapsulacja*

Enkapsulacja zapewnia, że obiekt nie może zmieniać stanu innych obiektów w nieokreślony sposób. Każdy typ obiektu dostarcza interfejsu, który określa sposób współpracy z innymi obiektami. Jedynie za pomocą określonych metod mamy możliwość zmienić stan obiektu, bezpośredni dostęp do zmiennych jest zabroniony.

```
class MyClass {
private:
    int field;

public:
    // Setter
    void setField(int field) {
        this.field = field;
    }

    // Getter
    int getField() {
        return field;
    }
};

int main() {
    MyClass clazz = new MyClass();

    clazz.setField(5);
    int field = clazz.getField();

    /*
    // The `field` is private, so the following
    // will cause an error during compilation
    clazz.field = 5;
    int field = clazz.field;
    */
}
```

Definicja 63.4 Polimorfizm

Referencje i wskaźniki obiektów mogą dotyczyć obiektów różnego typu, a wywołanie metody dla referencji spowoduje zachowanie odpowiednie dla pełnego typu obiektu wywoływanego. Zazwyczaj można wyróżnić dwa rodzaje polimorfizmu: dynamiczny - wykonywany podczas działania programu, a także statyczny - na etapie kompilacji.

```
class Point {
protected:
int x, y;
public:
// Constructor with initializer list
Point(int x, int y) : x(x), y(y) {
}

virtual void printPoint() {
cout << "(" << x << ", " << y << ")" << endl;
}

void printClassName() {
cout << "Point" << endl;
}
};

class ColoredPoint : public Point {
protected:
int color;
public:
// Constructor with initializer list
ColoredPoint(int x, int y, int color)
: Point(x, y), color(color) {
}

void printPoint() {
cout << "(" << x << ", " << y
<< ", color = " << color << ")" << endl;
}

void printClassName() {
cout << "ColoredPoint" << endl;
};
};
```

Definicja 63.5

```
int main() {
    Point *p = new ColoredPoint(1, 2, 3);
    p->printPoint();
    p->printClassName();
    delete p;

    ColoredPoint *cp = new ColoredPoint(1, 2, 3);
    cp->printPoint();
    cp->printClassName();
    delete cp;

    return 0;
}

/* Output:
(1, 2, color = 3)
Point
(1, 2, color = 3)
ColoredPoint
*/
```

Jak widzimy dla obu wskaźników instancjonujemy klasę *ColoredPoint*. W przypadku wskaźnika *p* i wywołania metody *printClassName* została wywołana metoda z klasy *Point*, mimo że wskaźnik wskazuje na obiekt typu *ColoredPoint*. Dzieje się tak, ponieważ kompilator domyślnie bierze pod uwagę typ wskaźnika, a nie wskazywanego obiektu. Aby temu zaradzić musimy oznaczyć wołaną metodę jako *virtual*. Spowoduje to, że kompilator przy wywołaniu tej metody weźmie pod uwagę typ wskazywanego obiektu, a nie typ wskaźnika, czego przykładem jest metoda *printPoint* z powyższego kodu.

64 Funkcje zaprzyjaźnione i ich związek z przeładowaniem operatorów w C++.

Definicja 64.1 *Funkcja zaprzyjaźniona dla danej klasy to funkcja oznaczona słowem kluczowym friend. Funkcja ta ma dostęp do niepublicznych (private i protected) składowych klasy.*

- Funkcje zaprzyjaźnione mogą się odnosić wyłącznie do obiektów globalnych lub danych przez argumenty
- Mechanizm zaprzyjaźnienia jest specyficzny dla języka C++
- Dowolna ilość klas może deklarować zaprzyjaźnienie jednej funkcji
- Funkcje zaprzyjaźnione mogą być przeładowane, zarówno w definiowanej klasie, jak i na zewnątrz
- Funkcje zaprzyjaźnione mogą być definiowane wewnątrz klasy, ale nie stanowią metody klasy definiowanej i są szczególną definicją zewnętrznej funkcji mimo zapisu w ciele definicji klasy.

64.1 Przeładowania operatorów a funkcje zaprzyjaźnione

Przy przeładowaniu operatorów często stosuje się *friend* w celu dostępu do pól prywatnych przez funkcje zewnętrzne.

```
1      class Foo {
2      public:
3      Foo(float val) : data(val) {}
4      private:
5      float data;
6
7      friend std::ostream& operator<<(std::ostream& os, const Foo& obj)
8      };
9
10     std::ostream& operator<<(std::ostream& os, const Foo& obj)
11     {
12     return os << obj.data;
13     }
14
15     int main()
16     {
17     Foo obj(1.23);
18     std::cout << obj << '\n';
19     }
```


65 Programowanie generyczne na podstawie szablonów w języku C++.

Programowanie generyczne to paradygmat programowania w którym algorytmy (funkcje, klasy) są zapisywane bez specyfikacji typów, na których działają. W C++ szablony *template* pozwalają na programowanie generyczne.

Zalety szablonów:

- Zmniejszają redundancję
- Wydajny, elastyczny, wielozadaniowy kod (np. kontenery i algorytmy w STL)
- Metaprogramowanie
- Polimorfizm rozwiązywany w czasie kompilacji

W C++ istnieją szablony:

- klas
- struktur
- funkcji (również metod)
- zmiennych
- koncepcji
- rodzin typów

```

1      #include <iostream>
2
3      template <typename T>
4      class ExampleTemplate {
5      public:
6          T data[5];
7
8          T getMax() {
9              T max = data[0];
10             for (auto i : data) {
11                 if (i > max) {
12                     max = i;
13                 }
14             }
15             return max;
16         }
17     };
18
19     int main() {
20         ExampleTemplate <double> doubleObj = {0.5, 165.43, 1.76, 98, 23};
21         std::cout << doubleObj.getMax() << std::endl;
22         //165.43
23
24         ExampleTemplate <bool> boolObj = {true, false, false, true, true};
25         std::cout << boolObj.getMax() << std::endl;
26         //1
27     }

```

65.1 Domyślne argumenty szablonów

Argumenty końcowe szablonu mogą mieć domyślne wartości.

```

1      template <typename T, typename S = double>
2      class X{
3      //...
4      };
5
6      X<int> x1;
7      X<int, double> x2;

```

65.2 Specjalizacja szablonów

Możliwe jest dostarczenie niestandardowego kodu dla danego zestawu argumentów szablonu.

```
1      template<typename T>
2      struct is_void : std::false_type {};
3
4      // specjalizacja szablonu dla T = void
5      template<>
6      struct is_void<void> : std::true_type {};
```

Przykładem takiej specjalizacji jest vector booli - wartości są przechowywane w sposób optymalny, tzn. przy pomocy operacji binarnych, na pojedynczych bitach, nie bajtach.

66 Podstawowe kontenery w STL z szerszym omówieniem jednego z nich.

Definicja 66.1 *Standard Template Library*

STL – biblioteka C++ zawierająca algorytmy, kontenery, iteratory oraz inne konstrukcje w formie szablonów, gotowe do użycia w programach.

Obecnie występują trzy rodzaje kontenerów

- Kontenery Sekwencyjne

Implementują struktury danych, które zapewniają sekwencyjny dostęp do ich elementów.

- Kontenery Asocjacyjne

Implementują posortowane struktury danych, które da się szybko przeszukiwać (złożoność $O(\log n)$)

- Nieuporządkowane Kontenery Asocjacyjne (unordered)

Implementują nieposortowane (haszowane) struktury danych, które mogą być szybko przeszukiwane (zamortyzowana złożoność $O(1)$, w najgorszym przypadku $O(n)$)

array	statyczna, ciągła tablica
vector	dynamiczna, ciągła tablica
deque	dwustronnie zakończona kolejka
forward_list	lista jednokierunkowa
list	lista dwukierunkowa

Kontenery zarządzają pamięcią alokowaną do przechowywania ich elementów, i zapewniają metody pozwalające na dostęp do nich, bezpośrednio lub przez iteratory (obiekty o właściwościach podobnych do wskaźników).

66.1 array - kontener sekwencyjny

`std::array` (tablica) jest kontenerem enkapsulującym tablice o stałej, ustalonej w trakcie kompilacji liczbie elementów.

set	kolekcja unikalnych kluczy
unordered_set	posortowana po kluczach
map	słownik - kolekcja par klucz-wartość
unordered_map	posortowana po kluczach, klucze są unikalne
multiset	kolekcja kluczy
unordered_multiset	posortowana po kluczach
multimap	słownik - kolekcja par klucz-wartość
unordered_multimap	posortowana po kluczach

Jako typ agregowany, może być zainicjalizowana korzystając z zasad aggregate-initialization, poprzez nie więcej niż N inicjalizatorów konwertowalnych do

$T : std :: array < int, 3 > a = 1, 2, 3;$

Ta struktura łączy w sobie wydajność i dostępność tablic w stylu języka C, z wszystkimi korzyściami standardowego kontenera, takimi jak

- znajomość własnego rozmiaru
- wspieranie przypisywania
- iteratory dostępu bezpośredniego

Tablica długości 0 jest specjalnym przypadkiem ($N == 0$)

Wtedy `array.begin() == array.end()`. Efekt wywołania `front()` lub `back()` na tablicy długości 0 jest niezdefiniowany.

Tablica taka może zostać wykorzystana jako krotka N elementów tego samego typu.

Metody

at	Dostęp do wskazanego elementu ze sprawdzeniem zakresów
operator[]	Dostęp do wskazanego elementu
front	Dostęp do pierwszego elementu
back	Dostęp do ostatniego elementu
data	Bezpośredni dostęp do tablicy opakowywanej przez kontener

begin	Zwraca iterator na początek kontenera
end	Zwraca iterator za koniec kontenera
rbegin	Zwraca odwrócony iterator na początek
rend	Zwraca odwrócony iterator za koniec kontenera

empty	Sprawdza czy kontener jest pusty
size	Zwraca liczbę elementów
max_size	Zwraca maksymalną możliwą liczbę elementów
fill	Wypełnia kontener wskazaną wartością
swap	Zmienia wartości

67 Obsługa sytuacji wyjątkowych w C++.

Wyjątki pozwalają zareagować na sytuacje, w których istnieje ryzyko niewykonania określonego zadania.

Jeżeli w jakimś miejscu programu zajdzie nieoczekiwana sytuacja, programista piszący ten kod powinien zasignalizować o tym. Dawniej polegało to na zwróceniu specyficznej wartości, co nie było zbyt szczęśliwym rozwiązaniem, bo sygnał musiał być taki jak wartość zwracana przez funkcję. W przypadku obsługi sytuacji wyjątkowej mówi się o obiekcie sytuacji wyjątkowej, co często zastępowane jest słowem "wyjątek". W C++ wyjątki się "rzuca", służy do tego instrukcja `throw`.

67.1 Szkielet obsługi wątków

Tam gdzie spodziewamy się wyjątku umieszczamy blok `try`, w którym umieszczamy "podejrzane" instrukcje. Za tym blokiem muszą (tzn. musi przynajmniej jedna) pojawić się bloki `catch`.

```
//jakaś zwykła funkcja, lub funkcja main
try // w instrukcjach poniżej może coś się nie udać
{
    fun();
    fun2(); //podejrzane funkcje
}
catch(std::string obj)
{
    //tu coś robimy, na przykład piszemy o błędzie
}
```

W instrukcji `catch` umieszczamy typ jakim będzie wyjątek. Rzucić możemy wyjątek typu `int`, `char`, `std::string` i inne, dlatego tu określamy co nas interesuje. Nazwa tego obiektu nie jest konieczna, ale jeżeli chcemy znać wartość musimy ten obiekt nazwać. Bloków `catch` może być więcej, najczęściej tyle ile możliwych typów do złapania. Co ważne jeżeli rzucimy wyjątek konkretnego typu to "wpadnie" on do pierwszego dobrego `catch` nawet jeżeli inne nadają się lepiej (podobnie jak z instrukcjami `if else`). Dotyczy to zwłaszcza klas dziedziczonych.

Zawsze dobrze jest się zabezpieczyć blokiem `catch(...)`

67.2 Rzucanie wyjątku

Pisząc funkcję możemy stwierdzić że coś poszło nie tak i chcemy zasygnalizować wyjątek.

```
double Dziel(double a, double b) //funkcja zwraca iloraz a / b
{
    if(b == 0)    {
        std::string wyjatek = "dzielenie przez zero!"; //przez zero się nie dzieli
        throw wyjatek; //rzucamy wyjątek
    }
    return a / b;
}
```

Po instrukcji `throw` umieszczamy obiekt który chcemy rzucić (u nas jest to `std::string`). W tym miejscu działanie funkcji jest natychmiast przerywane i nasz łańcuch znaków wędruje do bloków `catch`.

68 Obsługa plików w języku C.

Standardowa biblioteka wejścia/wyjścia języka C udostępnia funkcje do operowania na plikach, tj. odczytu i zapisu danych z plików tekstowych i binarnych.

Sposób pracy z plikami w języku C jest następujący:

1. Otwarcie pliku o określonej nazwie w określonym trybie (odczyt, zapis, dopisywanie) Funkcja `fopen`, prototyp:

```
FILE *fopen(const char *filename, const char *mode);
```

Funkcja `fopen` otwiera plik, którego nazwa podana jest w pierwszym argumencie. Drugim jest łańcuch znaków zawierający litery oznaczające sposób otwarcia pliku:

- "r" - otwiera plik do czytania
- "r+" - otwiera plik do czytania i nadpisywania (aktualizacja)
- "w" - otwiera plik do nadpisywania (zamazuje starą treść)
- "w+" - otwiera plik do nadpisywania i czytania
- "a" - otwiera plik do dopisywania (jeśli plik nie istnieje, to jest tworzony)
- "a+" - otwiera plik do dopisywania i odczytu (jeśli plik nie istnieje, to jest tworzony)
- "t" - otwiera plik w trybie tekstowym
- "b" - otwiera plik w trybie binarnym

Litery można ze sobą łączyć, np. "rwb" albo "wt". Funkcja zwraca wskaźnik do pliku (`FILE *`) lub `NULL`, gdy pliku nie udało się otworzyć (nie istnieje, jest już otwarty w innym programie itp. . Przykład wywołania:

```
FILE *f = fopen("dane.txt", "rt");
```

2. Wykonywanie operacji odczytu/zapisu.
Funkcje do formatowanego odczytu/zapisu.

```
int fscanf(FILE *stream, const char *format, ...);  
int fprintf(FILE *stream, const char *format, ...);
```

Funkcje działają tak jak scanf i printf, przy czym jako pierwszy argument przyjmują wskaźnik do pliku uzyskany w punkcie 1 (ogólnie - wskaźnik do strumienia danych).

Przykłady:

```
//odczyt liczby całkowitej z pliku
int i;
char lan[] = { "Operacje plikowe" };
FILE *f = fopen("dane.txt", "wt");
fscanf(f, "%d", &i);
//zapis łańcucha znaków do pliku
fprintf(f, "%s", lan);
//zamknięcie pliku
fclose(f);
```

69 Model wodospadu a model spiralny wytwarzania oprogramowania.

69.1 Model kaskadowy

Nazwa model kaskadowy (ang. waterfall model) wprowadzona została przez Winstona W. Royce w roku 1970, w artykule „Managing the Development of Large Software Systems”

69.1.1 Zasady

- Tworzenie systemu podzielone na wyizolowane etapy; każdy z nich musi być zakończony, zanim rozpoczną się prace w następnym etapie;
- Wyjścia z jednego etapu są wykorzystywane jako wejścia do etapu następnego;
- Każdy etap podzielony jest na dwie części: części twórczej oraz weryfikacji i/lub zatwierdzenia;
- Ponowna praca, jeśli jest konieczna, jest wykonywana w kolejnych etapach bez powrotu na pierwotny etap, na którym dany produkt został utworzony, np. gdy pojawi się nowe wymaganie;

69.1.2 Fazy modelu kaskadowego

1. Planowanie

- Wstępne założenia dotyczące budowy systemu
- cele biznesowe
- podstawowe wymagania
- podstawowe założenia jakościowe i parametry techniczne, standardy, organizacja i działanie systemu;

2. Analiza

- Zdefiniowane zostaje przeznaczenie systemu.
- Wynikiem analizy powinien być poprawny, kompletny, spójny i jednoznaczny model systemu.

3. Projekt

- Projekt systemu
 - (a) Dekompozycja systemu na mniejsze podsystemy.
 - (b) Wybór strategii budowania systemu.
- Projekt obiektów
 - (a) Definicja obiektów dziedziny realizacyjnej.
 - (b) Wybór odpowiednich gotowych komponentów.
 - (c) Precyzyjne opisywanie interfejsów dla nowo tworzonych obiektów.

4. Implementacja

- Tworzenie kodu źródłowego aplikacji.
- Mapowanie np. modeli UML na kod.

5. Testowanie

- Znajdowanie różnic między rzeczywistymi fragmentami systemu a ich modelami.
- Celem jest wykrycie jak największej ilości błędów.

6. Pielęgnacja

69.1.3 Cechy

- kolejnej fazy nie powinno się rozpoczynać, jeśli poprzednia się nie zakończy.
- kolejność wykonywania prac musi być ściśle przestrzegana;
- koszty opracowania i akceptacji dokumentów są wysokie i dlatego iteracje są również kosztowne oraz wymagają powtarzania wielu prac.
- marginalizacja roli klienta w procesie wytwarzania oprogramowania.
- model kaskadowy powinien być używany jedynie wówczas, gdy wymagania są jasne i zrozumiałe.
- uzyskanie produktu zgodnego z wymaganiami klienta silnie zależy od ich stabilności;
- próba adaptacji do zmieniających się wymagań jest b. kosztowna;
- bardzo wysoki koszt błędów popełnionych we wstępnych etapach;

69.2 Model spiralny

Proces tworzenia ma postać spirali, której każda pętla reprezentuje jedną fazę procesu. Najbardziej wewnętrzna pętla przedstawia początkowe etapy projektowania, np. studium wykonalności, kolejna definicji wymagań systemowych itd.

69.2.1 Fazy modelu spiralnego

Każda pętla spirali podzielona jest na cztery sektory

1. Ustalanie celów – definiowanie konkretnych celów wymaganych w tej fazie przedsięwzięcia. Identyfikacja ograniczeń i zagrożeń. Ustalanie planów realizacji.
2. Rozpoznanie i redukcja zagrożeń – przeprowadzenie szczegółowej analizy rozpoznanych zagrożeń, ich źródeł i sposobów zapobiegania. Podejmuje się odpowiednie kroki zapobiegawcze.
3. Tworzenie i zatwierdzanie – tworzenie oprogramowania w oparciu o najbardziej odpowiedni model, wybrany na podstawie oceny zagrożeń.
4. Ocena i planowanie – recenzja postępu prac i planowanie kolejnej fazy przedsięwzięcia bądź zakończenie procesu produkcyjnego.

69.2.2 Cechy

- ciągłe monitorowanie i pomiar zmian;
- zmiany poddawane są review użytkownika;
- próba minimalizacji ryzyka niepowodzenia;

Model kaskadowy	Model spiralny
Działa w sposób sekwencyjny	Działa w sposób ewolucyjny
Błędy są wykrywane i naprawiane po zakończeniu etapów	Błędy są wykrywane wcześniej
Odpowiedni dla mniejszych projektów	Odpowiedni dla większych projektów
Wymagania określone na początku	Możliwość dodania wymagań po kilku iteracjach
Ograniczona możliwość zmian w trakcie trwania projektu	Większa możliwość dostosowania projektu w trakcie trwania
Mały koszt	Wysoki koszt

70 Diagram sekwencji i diagram przypadków użycia w języku UML.

Definicja 70.1 *UML - Unified Modelling Language.*

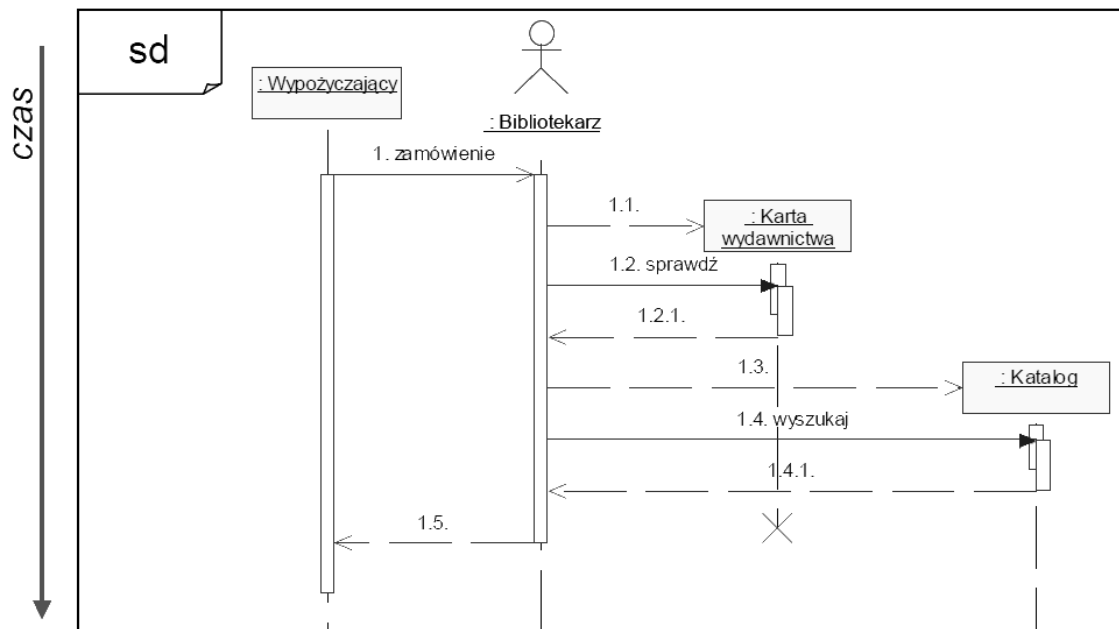
- rodzina notacji graficznych;
- służy do opisywania i projektowania systemów informatycznych;
- narodził się z unifikacji wielu obiektowych języków modelowania graficznego;
- nadzorowany przez organizację OMG.

70.1 Diagram sekwencji.

Definicja 70.2 *Diagram sekwencji* - opisuje interakcje pomiędzy częściami systemu w postaci sekwencji komunikatów wymienianych między nimi.

Diagramy sekwencji dobrze ukazują dynamiczne aspekty realizacji scenariuszy, w których dochodzi do złożonych oddziaływań pomiędzy obiektami. Podstawowymi oddziaływaniami są wymiany komunikatów oznaczane następującymi symbolami:

- przesłanie komunikatu asynchronicznego – strzałka zwykła,
- wywołanie funkcji – strzałka z wypełnionym grotem,
- powrót z funkcji – strzałka rysowana linią przerywaną.



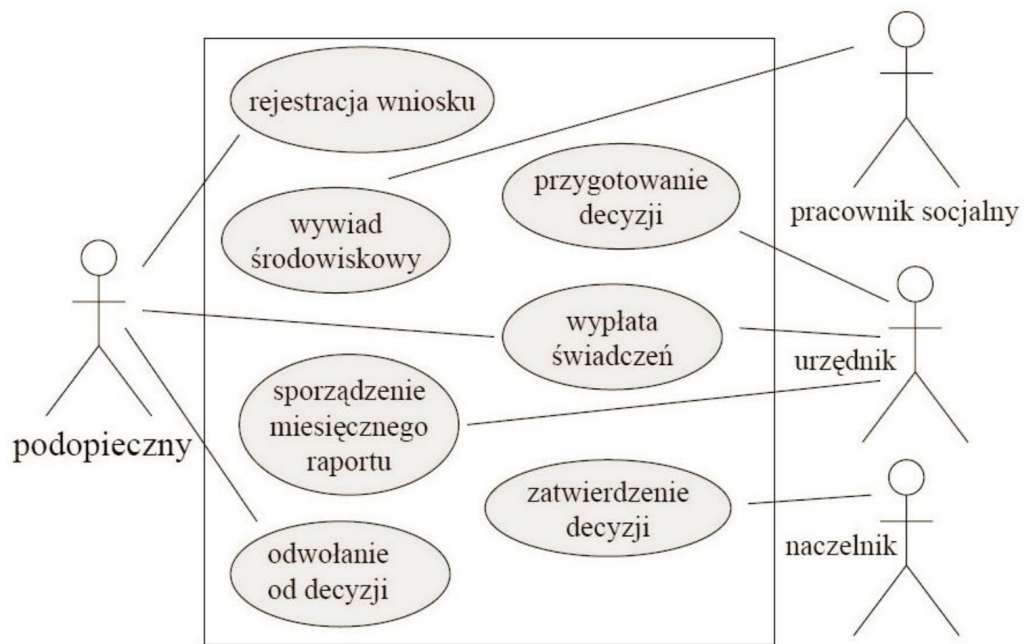
70.2 Diagram przypadków użycia.

Definicja 70.3 *Diagram przypadków użycia służy do przedstawiania:*

- *interakcji aktora z systemem,*
- *zadań, jakie wykonuje system,*
- *wymagań funkcjonalnych.*

Aktorzy mogą być:

- *Ludźmi wchodzącymi w interakcję,*
- *Systemami zewnętrznymi*
- *Częściami systemu, które mają wpływ na funkcjonowanie systemu, ale same przez ten system nie mogą być zmieniane (jak np. zegar systemowy).*



71 Klasyfikacja testów.

71.1 Poziomy testów.

Definicja 71.1 *Poziom testów określa sposób testowania ze względu na postać testowanego obiektu w kontekście cyklu życia (co testujemy?).*

71.1.1 Testy jednostkowe.

TESTY JEDNOSTKOWE	
Podstawa testów	wymagania na moduły, projekt szczegółowy, kod
Typowe obiekty	moduły, programy, funkcje, klasy, procedury

- testowanie modułowe, unit testing
- defekty szukane w izolacji od reszty systemu
- wykorzystanie zaślepek i sterowników
- przeprowadzane przez deweloperów – autorów testowanego kodu
- usuwanie defektów bezpośrednio po znalezieniu
- brak formalnego procesu zarządzania defektami

TTD - Test Driven Development.

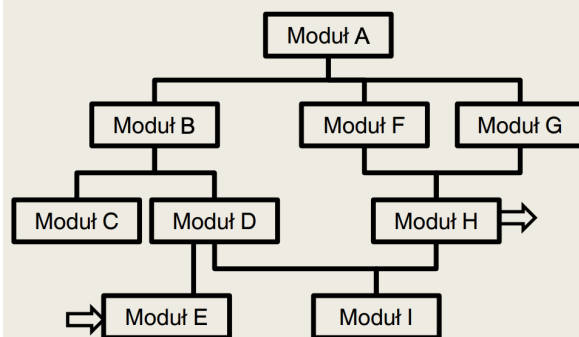
- **Napisanie testu** - sprawdza, że programista rozumie zachowanie nowego kodu.
- **Uruchomienie testu**, który nie przechodzi (testuje uprząż testową i sam test; pokazuje, że wystąpi awaria, gdy kod będzie błędny).
- **Napisanie kodu** w minimalnej ilości, wystarczającej do zdania testu; jeśli konieczne, dokonanie refaktoryzacji.

71.1.2 Testy integracyjne.

TESTY INTEGRACYJNE	
Podstawa testów	projekt systemu, architektura, przypadki użycia
Typowe obiekty	interfejsy, podsystemy, konfiguracje systemów

- testuje interfejsy i interakcje
- im większy zakres integracji, tym trudniej izolować defekty
- rozumienie architektury testowanych modułów/systemów
- może odbywać się na wielu poziomach (np. integracja systemów)

Strategie testów integracyjnych



H – zwraca wynik
E – pobiera wejście
B+C+D+E – jedna funkcjonalność
główna transakcja: wejście-wyjście

*połączenie oznacza, że moduł
położony wyżej wywołuje niższy*

top-down: A-BFG > B-CD > F-H, G-H > D-EI, H-I

bottom-up: D-EI > H-I > B-CD > F-H, G-H > A-BFG

funkcjonalne: B-CD, D-E > ...

sekwencja przeprowadzania transakcji: E-D-B-A-G-H > ...

big-bang: wszystko integrowane i testowane naraz (nie polecane)

71.1.3 Testy systemowe.

TESTY SYSTEMOWE	
Podstawa testów	wymagania na system, przypadki użycia, specyfikacja funkcjonalna, raporty analizy ryzyka
Typowe obiekty	system, podręczniki użytkownika i operatora, konfiguracja systemu

- sprawdza zachowanie systemu jako całości
- zakres testu określony w planie testów
- środowisko testowe podobne do produkcyjnego – minimalizacja ryzyka awarii zależnych od środowiska
- często konieczność testowania z niekompletnymi wymaganiami
- zwykle przeprowadzane przez niezależny zespół testerski

71.1.4 Testy akceptacyjne.

TESTY AKCEPTACYJNE	
Podstawa testów	wymagania użytkownika, wymagania na system, przypadki użycia, procesy biznesowe, raporty analizy ryzyka
Typowe obiekty	Procesy biznesowe w pełni zintegrowanego systemu, procesy operacyjne i utrzymania systemu, procedury, raporty, dane konfiguracyjne

- cel: zyskanie zaufania do systemu
- znajdowanie defektów nie jest głównym celem
- często przeprowadzane przez klienta lub użytkownika
- ocenia gotowość systemu, ale niekoniecznie ostatni etap testów

Typowe formy testów akceptacyjnych:

- **testy akceptacyjne użytkownika (UAT)** – sprawdzenie gotowości do użycia
- **testy operacyjne (OAT)** – akceptacja przez administratora systemu (testy backupu, przywracania systemu, zarządzania użytkownikami, utrzymania, migracji danych, bezpieczeństwa itp.)
- **testy akceptacyjne wymagane kontraktem/regulacjami**
- **testy alfa, beta (polowe)**
 - alfa: przeprowadzane u producenta, ale nie przez zespół deweloperski
 - beta: przeprowadzane u klienta przez klienta/potencjalnego użytkownika

71.2 Typy testów.

Definicja 71.2 *Typ testów to zbiór czynności testowych właściwych dla weryfikacji systemu w oparciu o konkretny powód lub cel testów (**jak testujemy?**).*

1. Testowanie funkcjonalne

Testowanie funkcji wykonywanej przez oprogramowanie - **co system robi**.

2. Testowanie niefunkcjonalne

Testowanie niefunkcjonalnej charakterystyki jakościowej (np. niezawodność, wydajność, użyteczność) - **jak system działa**. Wyrażalne ilościowo (np. czas odpowiedzi w testach wydajności).

3. Testowanie strukturalne

Oparte na strukturze (np. kod, graf przepływu sterowania, struktura menu, model procesu biznesowego). Zwykle wykonywane po testach czarnoskrzynkowych, aby sprawdzić stopień przetestowania i wyrazić go ilościowo (**pokrycie**).

4. Retesty i testy regresji

Związane ze zmianą, tzn. potwierdzenie usunięcia defektów (retesty - testy zmienionych fragmentów kodu) oraz poszukiwanie niezamierzonych zmian (regresja - pogarszanie się jakości systemu przy wprowadzaniu zmian; testujemy niezmienione fragmenty programu- często automatyzowane).

72 Model Scrum: struktura zespołu, proces wytwarzania oprogramowania, korzyści modelu.

Definicja 72.1 *SCRUM* jest:

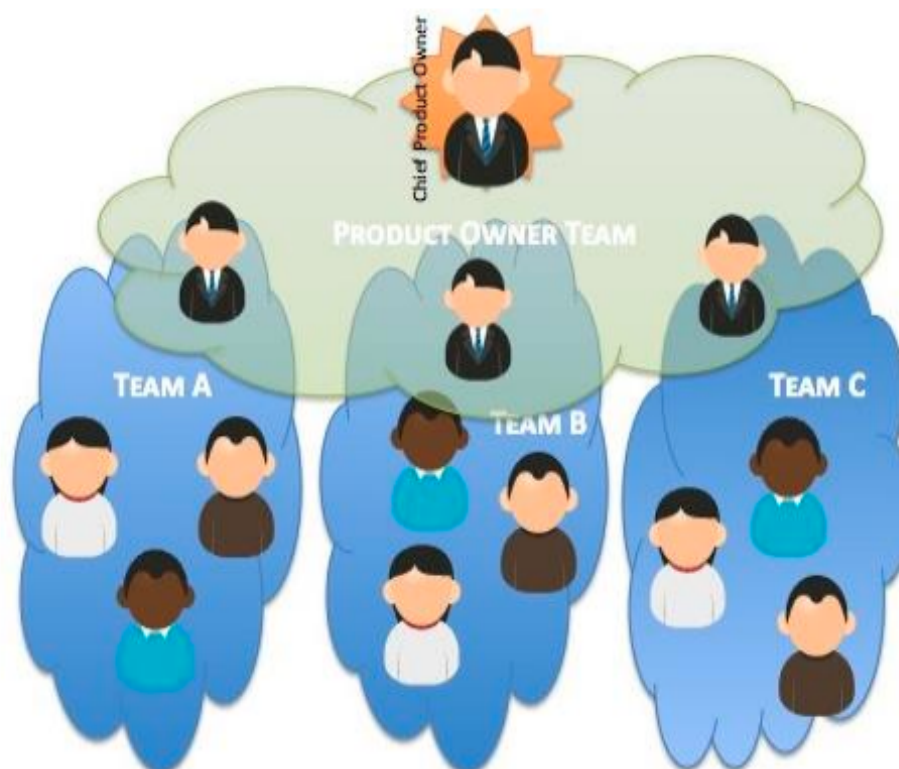
- *lekki*
- *łatwy do zrozumienia*
- *bardzo trudny do opanowania*

Teoria SCRUMa opiera się na trzech filarach:

- **Przejrzystość** - wszystkie istotne aspekty procesu muszą być widoczne dla osób odpowiedzialnych za osiągane rezultaty
- **Adaptacja** - powinna być ciągła. Korekta musi być wykonana jak najszybciej, by ograniczyć dalsze następstwa problemów
- **Inspekcja** - poddawane regularnej inspekcji są zarówno scrumowe artefakty jak i postępy prac

72.1 Struktura zespołu

Definicja 72.2 *Struktura zespołu*



Definicja 72.3 *Product owner* - jest odpowiedzialny za maksymalizację wartości produktu i pracy Zespołu Deweloperskiego. jest jedyną osobą zarządzającą Rejestrem Produktu (*Product Backlog*) co rozumiemy przez:

- Jasne artykułowanie elementów Rejestru Produktu i określanie ich kolejności w sposób zapewniający osiągnięcie założonych celów i misji
- Zapewnianie, że Rejestr Produktu jest dostępny, przejrzysty oraz jasny dla wszystkich i, że dobrze opisuje to, czym Zespół Scrumowy będzie się zajmował

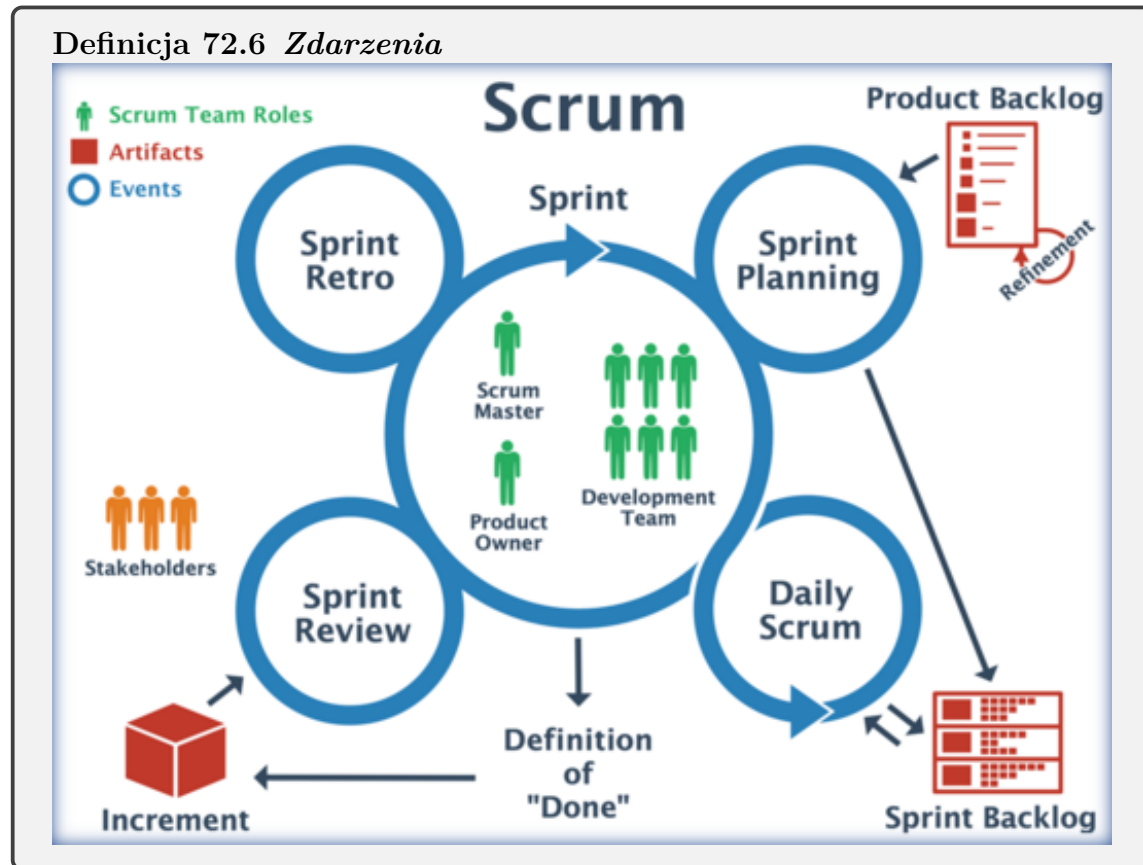
Definicja 72.4 *Zespół Deweloperski*:

- Jest złożony z profesjonalistów
- Ma za zadanie dostarczenie (na zakończenie każdego Sprintu), gotowego do wydania Przyrostu produktu
- Zalecana liczebność: 3-9 osób
- Są samoorganizujące się
- Jest wielofunkcyjny, w swoim składzie posiadają wszystkie umiejętności niezbędne do wytworzenia Przyrostu
- Scrum nie przewiduje tytułów innych niż “Deweloper” dla członków Zespołu Deweloperskiego
- Odpowiedzialność za wykonywaną pracę ponosi cały Zespół Deweloperski; nie składają się z podzespołów

Definicja 72.5 *Scrum Master* - jest odpowiedzialny za to, by Scrum był rozumiany i stosowany. Scrum Masterzy dokonują tego poprzez upewnianie się, że Zespół Scrumowy stosuje się do założeń teorii Scruma, jego praktyk i reguł postępowania. Scrum Master wspiera zarówno Właściciela Produktu jak i zespół deweloperski.

72.2 Zdarzenia

Definicja 72.6 *Zdarzenia*



Definicja 72.7 *Planowanie sprintu* - planowanie co zostanie wykonane podczas najbliższego sprintu:

- Jest ograniczone czasowo (8h dla miesięcznego sprintu i proporcjonalnie)
- Planowanie Sprintu składa się z dwóch części:
 - Co zostanie dostarczone
 - Jaka praca zostanie wykonana

Definicja 72.8 *Sprint* - serce SCUMa:

- *Ograniczony czasowo - trwa miesiąc lub krócej*
- *Podczas Sprintu wytwarzany jest Przyrost „Ukończonej”, używalnej i potencjalnie gotowej do wydania funkcjonalności*
- *Ma stałą długość przez cały okres trwania prac*
- *Niedozwolone są zmiany, które wpłyną na cel Sprintu*
- *Skład Zespołu Deweloperskiego i jego cel jakościowy pozostają niezmiennie*
- *Przerwanie sprintu:*
 - *Tylko product owner ma prawo przerwać sprint*
 - *Sprint może zostać przerwany, jeśli cel Sprintu się zdezaktualizuje*
 - *Powinien zostać przerwany, jeśli kontynuowanie prac nie ma sensu w zaistniałych okolicznościach*
 - *Przerwanie Sprintów zużywają zasoby, ponieważ wszyscy muszą się przegrupować podczas kolejnego Planowania Sprintu, aby móc rozpocząć nowy*

Definicja 72.9 *Daily scrum* - jest spotkaniem ograniczonym czasowo do 15 minut, w którym każdy z członków zespołu wyjaśnia:

- *Co zostało wykonane od ostatniego spotkania*
- *Co zostanie wykonane przed kolejnym spotkaniem*
- *Jakie przeszkody stoją na drodze*

Definicja 72.10 *Przegląd sprintu* - organizowany jest na zakończenie Sprintu w celu przeprowadzenia inspekcji Przyrostu i dostosowaniu, jeśli zajdzie taka potrzeba, Rejestru Produktu.

Podczas Przeglądu Sprintu Zespół Scrumowy i interesariusze wspólnie omawiają to, co zostało ukończone w Sprincie.

Może trwać maksymalnie 4h dla miesięcznego Sprintu.

Definicja 72.11 Retrospektywa Sprintu - jest okazją dla zespołu Scrumowego do przeprowadzenia inspekcji swoich działań i opracowania planu usprawnień. Ma na celu:

- Sprawdzenie, co działo się w ostatnim Sprincie, biorąc pod uwagę ludzi, zależności, procesy i narzędzia
- Zidentyfikowanie i uporządkowanie istotnych elementów, które sprawdziły się w działaniu oraz tych, które kwalifikują się do poprawy
- Stworzenie planu wprowadzania w życie usprawnień sposobu wykonywania pracy przez Zespół Scrumowy

72.3 Artefakty

Definicja 72.12 Rejestr produktu - uporządkowana lista tego co może być potrzebne w produkcie. Jest to jedyne źródło wymaganych zmian, które mają zostać wprowadzone. Odpowiedzialny z rejestr produktu jest Product owner.

Definicja 72.13 Rejestr sprintu - podzbiór elementów Rejestru produktu. Jest to lista rzeczy do wykonania w danym sprincie rozszerzona dodatkowo o plan ich wykonania. Rejestr Sprintu definiuje pracę, jaką Zespół Deweloperski wykona by przekształcić elementy Rejestru Produktu w „Ukończony” Przyrost. Rejestr Sprintu należy tylko i wyłącznie do Zespołu Deweloperskiego.

Definicja 72.14 Przyrost - suma wszystkich elementów Rejestru Produktu zakończonych podczas Sprintu i wszystkich Sprintów poprzednich. Na koniec Sprintu nowy Przyrost musi być „Ukończony”.

73 Wymagania w projekcie informatycznym: klasyfikacja, źródła, specyfikacja, analiza.

Definicja 73.1 ***Wymagania** - opis funkcji (usług), które mają być realizowane przez system i opis ograniczeń dla systemu.
Wymagania nie opisują jak system ma działać a co ma wykonywać.*

73.1 Klasyfikacja

Definicja 73.2 *Wymagania dzielimy na:*

- **Funkcjonalne** - opisują jakie funkcje powinien mieć system, np. “wprowadzanie nowej faktury” lub “generowanie raportu miesięcznego”
- **Pozafunkcjonalne** - opisują najczęściej pewne własności danych funkcjonalności, np. “minimum 20 faktur na godzinę”, “średni czas między awariami 2000000h”, itp.

Definicja 73.3 ***Klasyfikacja atrybutów jakości oprogramowania***

- **Functionality** - funkcjonalność
- **Usability** – użyteczność
- **Reliability** – niezawodność
- **Performance** – wydajność
- **Security** - bezpieczeństwo

w skrócie **FURPS**

73.2 Źródła

Definicja 73.4 *System powinien...*

Jest to sposób spisywania wymagań w stylu: “System powinien umożliwić wystawianie faktur”, “System powinien generować zestawienie miesięczne faktur”, “Faktura powinna zawierać co najmniej jedną pozycję”.

Zalety:

- *Łatwość spisywania*

Wady:

- *Słaba czytelność*
- *Trudne sprawdzanie kompletności, spójności*

Obecnie nie używany

Definicja 73.5 *Funkcje systemu*

Polega na opisywaniu poszczególnych funkcji systemu. Analogicznie do funkcji matematycznych, każda funkcja systemu informatycznego ma swoje wejście, wyjście, efekty uboczne.

Przykładowo, rozpatrując funkcję wystawiania faktury, wejściami mogą być pozycje faktury, wyjściem wydruk faktury (lub wysłanie jej faksem), natomiast efektem ubocznym zapisanie tej faktury w rejestrze faktur.

Wady:

- *Słaba czytelność*
- *Trudne do zrozumienia*

Obecnie nie używany

Definicja 73.6 Przypadki użycia

Przykład:

UC1: Wystawianie faktury

Atrybuty:

Główny aktor: Użytkownik

Priorytet: Wysoki

Źródło: Łukasz Olek

Główny scenariusz:

1. Sprzedawca pragnie wystawić fakturę.
2. Sprzedawca wpisuje pozycje faktury.
3. System podlicza fakturę, nadaje jej nowy numer i zapisuje w rejestrze.
4. Sprzedawca drukuje fakturę.

Rozszerzenia:

3.A. Sprzedawca nie dodał żadnej pozycji

3.A.1. System prosi o ponowne wprowadzenie pozycji (powrót do 2.)

Zalety:

- Łatwość spisywania
- Czytelność
- Łatwość zrozumienia i wyobrażenia sobie przyszłego systemu

Jako mapę łączącą pojedyncze przypadki użycia można potraktować diagram przypadków użycia (UML).

Definicja 73.7 *Historyjki użytkownika*

Who? What? Why?

Wzór:

jako - osoba, przypisana rola

chcę - cacha, funkcjonalność, czynność

ponieważ - uzasadnienie, rezultat, korzyść

73.3 Analiza

Definicja 73.8 *Celem analizy wymagań jest stworzenie modelu systemu, zwanego modelem analitycznym. Wysilek uczestników projektu skupia się na strukturalizowaniu i formalizowaniu zabranych wcześniej wymagań.*

Inaczej mówiąc po przeanalizowaniu przez nas zebranych wymagań musimy to przełożyć na diagramy UML.

Definicja 73.9 *Model analityczny* - reprezentuje tworzony system z perspektywy użytkownika; opis co system powinien robić

Definicja 73.10 *Analityczny model obiektowy* (diagramy klas) - odzwierciedla indywidualne koncepcje korzystania z systemu, ich właściwości i relacje między nimi

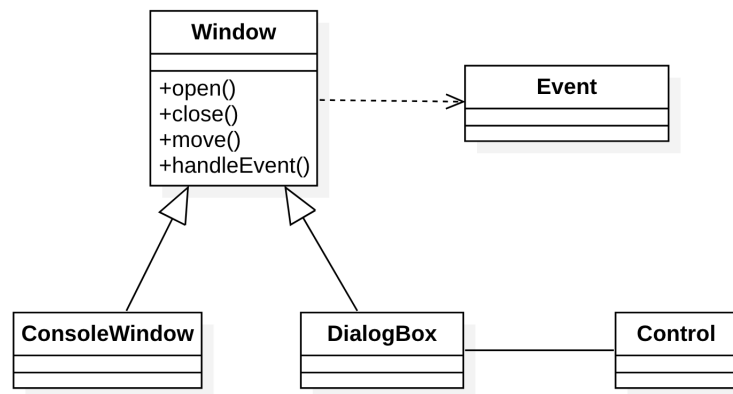
Definicja 73.11 *Model dynamiczny* (diagramy sekwencji i stanów) - koncentruje się na zachowaniu systemu

74 Analiza obiektowa: modele obiektowe i dynamiczne, obiekty encjowe, brzegowe i sterujące.

74.1 Diagram klas

Diagram klas ma za zadanie:

- przedstawić strukturę systemu w modelach obiektowych poprzez ilustrację struktury klas i zależności między nimi
- przedstawić podział odpowiedzialności pomiędzy klasy systemu i rodzaj wymienianych pomiędzy nimi komunikatów

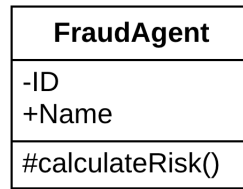


74.1.1 Klasy

Podstawowa część diagramu klas. Służą m.in. do opanowania słownictwa tworzonego systemu.

Klasa składa się z:

1. Nazwy
2. Atrybutów - czyli nazwanych właściwości klasy
3. Operacji - implementacji pewnej usługi, której można żądać od obiektów klasy
4. Odpowiedzialności (opcjonalnie)



74.1.2 Widoczność pól

- + public
- - private
- # protected
- ~ package

74.1.3 Związki

- Asocjacja



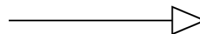
- Zależność

Informuje nas, że klasa używa innej klasy. Przykład: EventHandler i Event.



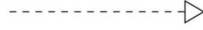
- Generalizacja

Czyli dziedziczenie lub uogólnienie. Określa zależność między klasą pochodną a klasą bazową.



- Realizacja

Określa zależność między implementacją a interfejsem. Przykład: ArrayList i List.



- Agregacja

Określa zależność między częścią a całością. Przykład: cegła i mur.



- Kompozycja

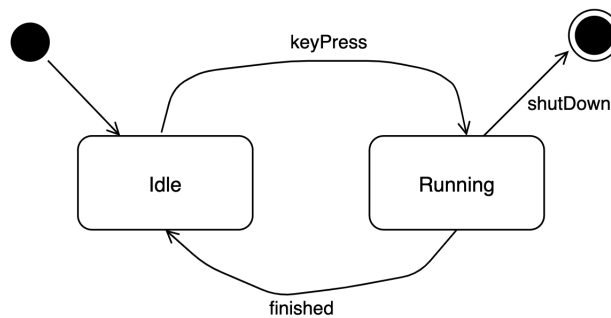
Zwana również silną agregacją. Tak jak agregacja, określa zależność między częścią a całością, jednak część nie może istnieć bez całości. Przykład: wydział i uczelnia.



74.2 Modele dynamiczne

74.2.1 Diagram stanów

W UML dynamiczne aspekty systemu modeluje się za pomocą maszyny stanów. Diagram stanów zapewnia graficzną reprezentację stanów, przejść, zdarzeń i akcji. Notacja ta pozwala zwizualizować zachowanie obiektu w sposób, który pozwala podkreślić ważne elementy życia tego obiektu.



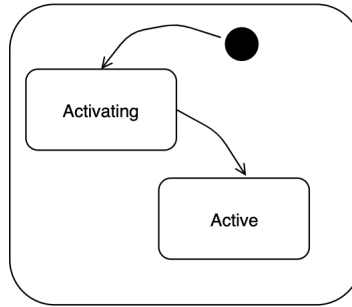
- **Stan**

Stan na diagramie reprezentuje sytuację lub stan obiektu, w którym spełnia on pewien warunek, wykonuje jakąś czynność lub czeka na jakieś zdarzenie. Obiekt pozostaje w stanie przez określony czas. Graficznie stan nerenodeorwany jest jako prostokąt z zaokrąglonymi narożnikami.



- Podstany

Możliwe jest podzielenie stanu na wewnętrzne podstany, z wewnętrznymi przejściami, akcjami, itp.



- Stan początkowy

Wskazuje domyślne miejsce początkowe dla danego stanu lub automatu. Stan początkowy jest reprezentowany jako wypełnione czarne kółko.



- Stan końcowy

Wskazuje, że wykonanie automatu stanów lub danego stanu zostało zakończone. Stan końcowy jest reprezentowany jako wypełnione czarne koło otoczone niewypełnionym okręgiem.

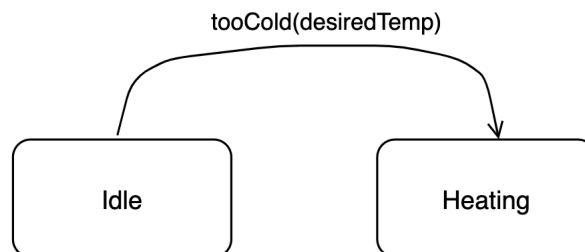


• Przejścia

Przejście to związek między dwoma stanami wskazujący, że obiekt w pierwszym stanie wykona określone działania i przejdzie w drugi stan, gdy wystąpi określone zdarzenie i spełnione zostaną określone warunki.

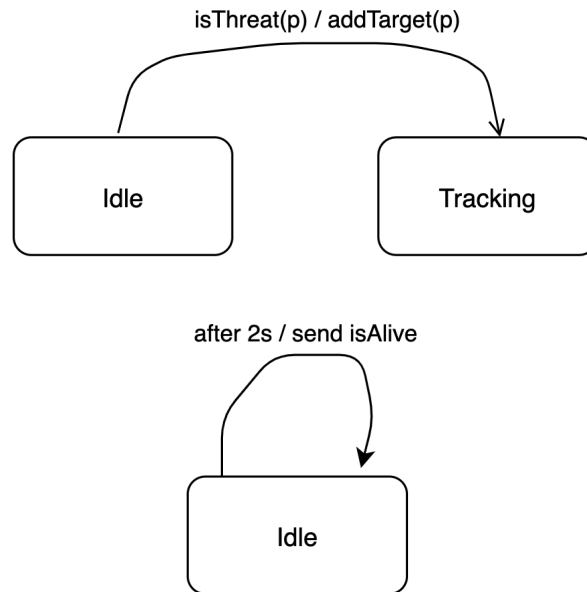
- Zdarzenia

Wystąpienie bodźca, który może wywołać zmianę stanu.



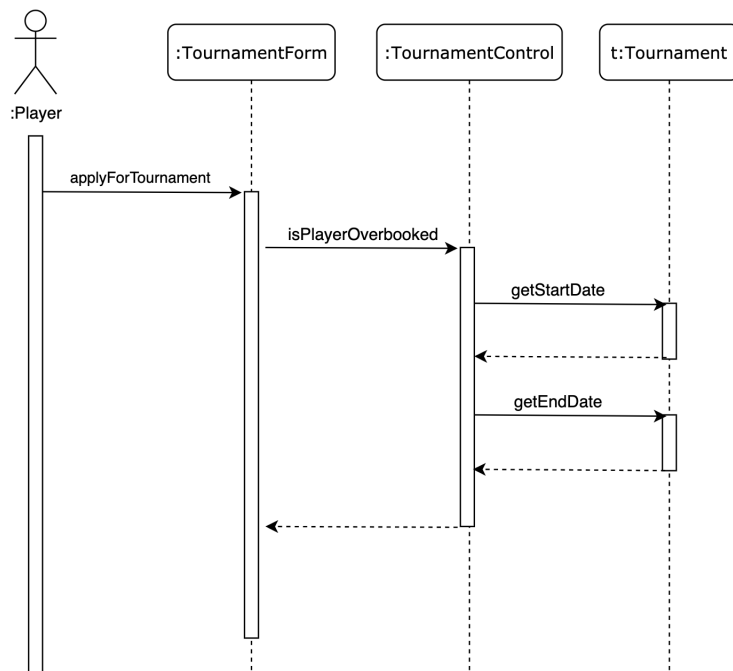
– Akcje

Wykonywalne obliczenie atomowe. Akcje mogą obejmować wywołania operacji (do obiektu będącego właścicielem maszyny stanu, a także do innych widocznych obiektów), utworzenie lub zniszczenie innego obiektu lub wysłanie sygnału do obiektu. Istnieje specjalny zapis do wysyłania sygnału - nazwa sygnału jest poprzedzona słowem kluczowym `send` jako wskazówką wizualną.



74.2.2 Diagram sekwencji

Diagram ten opisuje interakcje pomiędzy częściami systemu w postaci sekwencji komunikatów wymienianych między nimi. Ukazuje dynamiczne aspekty realizacji scenariuszy, w których dochodzi do złożonych oddziaływań pomiędzy obiektami



Rodzaje komunikatów:

- Wywołanie funkcji



- Powrót z wywołania



- Wywołanie asynchroniczne



74.3 Encja-Brzeg-Sterowanie

Model ten dzieli obiekty wchodzące w ramy systemu na trzy typy:

- **Obiekty encji**

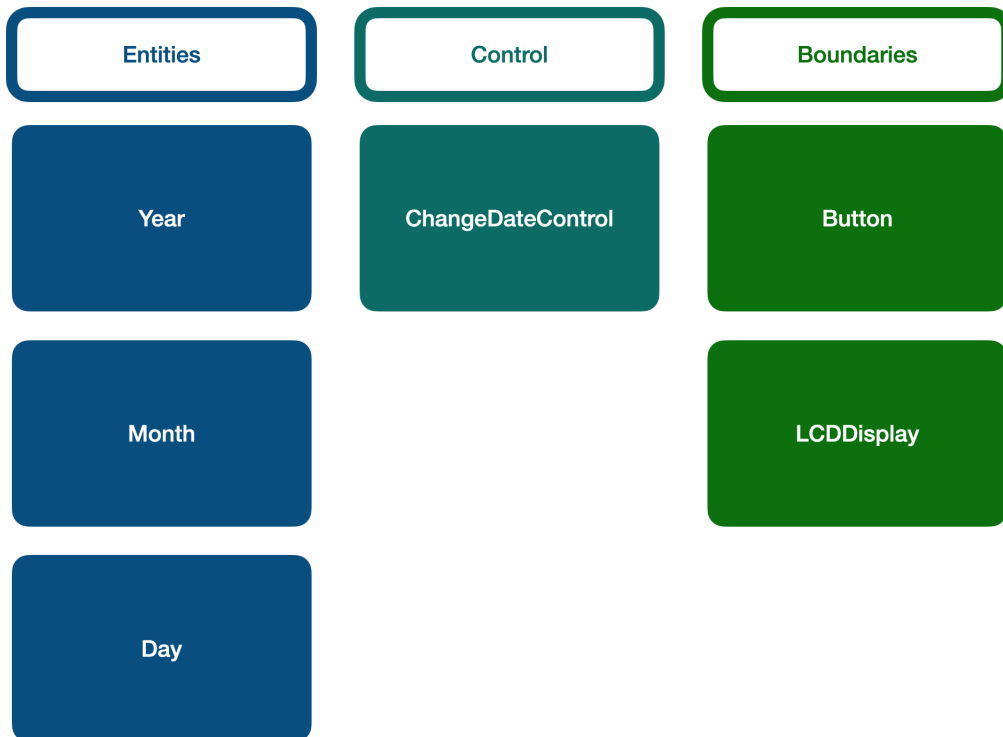
Reprezentują trwałą informację przetwarzaną przez system

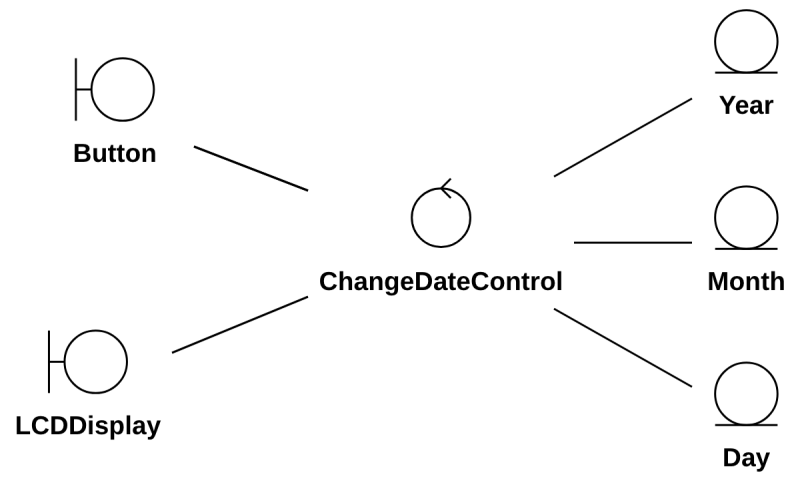
- **Obiekty brzegowe**

Odzwierciedlają interakcje między aktorami a systemem

- **Obiekty sterujące**

Odpowiadają za realizację przypadków użycia





75 Wzorce architektury systemów.

75.1 Architektura warstwowa

Efekt dekompozycji hierarchicznej jest uporządkowany zbiór warstw. Warstwa stanowi zgrupowanie podsystemów oferujących powiązane usługi.

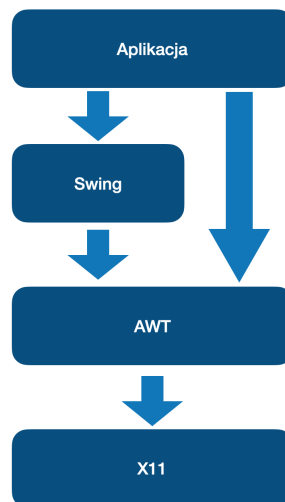
Wyróżniamy dwa rodzaje architektury warstwowej: otwartą i zamkniętą.

Zalety:

- modyfikowalność
- prostota wykorzystania warstwy
- spójność i przejrzystość
- wielokrotne wykorzystanie
- naturalna ewolucja z tradycyjnego modelu aplikacji

75.1.1 Architektura otwarta

W otwartej architekturze warstwowej warstwa może wywołać dowolną z warstw poniżej niej.

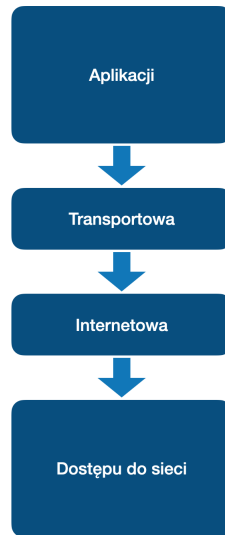


Przykłady:

- Serverless
- Swing

75.1.2 Architektura zamknięta

W zamkniętej architekturze warstwa może wywołać tylko warstwę bezpośrednio pod nią



Przykłady:

- ISO/OSI
- TCP/IP

75.1.3 Architektura trójwarstwowa

Architektura trójwarstwowa to architektura typu klient-serwer, często stosowana przy tworzeniu aplikacji internetowych.

Warstwy:

1. Warstwa prezentacji

Interfejs użytkownika. Tłumaczy żądania i wyniki do i z języka zrozumiałego dla użytkownika.

2. Warstwa logiki biznesowej

Koordynuje pracę aplikacji, przetwarza żądania warstwy prezentacji, dokonuje obliczeń.

3. Warstwa danych

Zawiera w sobie mechanizmy persystencji danych (jak na przykład bazy danych) oraz dostępu do nich.



75.2 Model View Controller

Wzorzec architektoniczny, dzielący system na trzy główne części:

1. Model

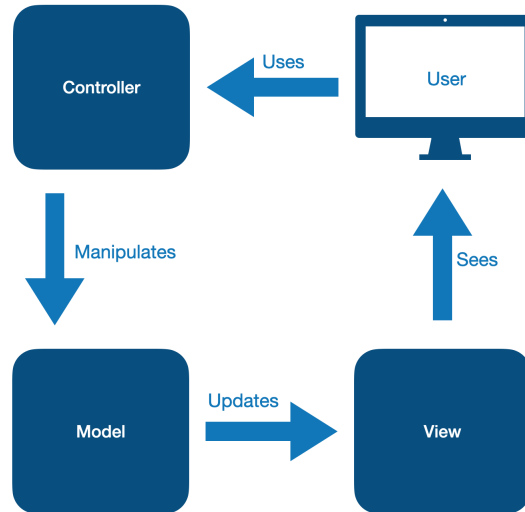
Reprezentuje podstawową część funkcjonalności systemu

2. Widok

Wyświetla dane dostarczone przez model użytkownikowi

3. Kontroler

Przyjmuje dane wejściowe od użytkownika oraz przetwarza jego żądania

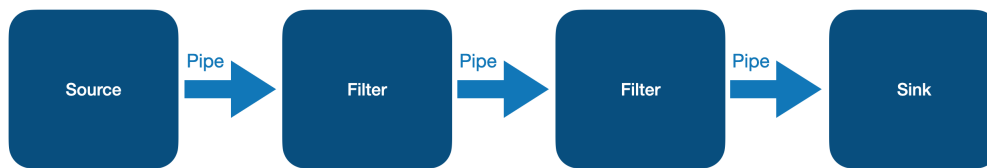


Przykłady:

- Smalltalk
- Swing

75.3 Pipeline (filtry i potoki)

Wzorzec Pipeline pozwala na uporządkowanie systemu, która przetwarza strumienie danych. Każdy krok przetwarzania jest zamknięty w filtrze. Dane przesłane są pomiędzy elementami za pomocą potoków (pipes).

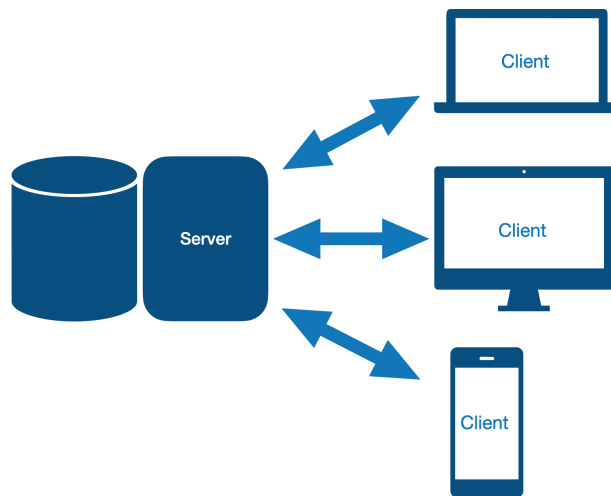


Przykłady:

- Unix pipeline
- Multimedia (GStreamer, Membrane)

75.4 Client - Server

Architektura ta polega na podziale systemu na dostawcę usług (server) oraz ich odbiorców (clients). Serwer oczekuje na żądania (request) klientów, przetwarza je, a następnie wysyła odpowiedzi (response). Pracuje zatem, w przeciwieństwie do klientów, w trybie pasywnym.

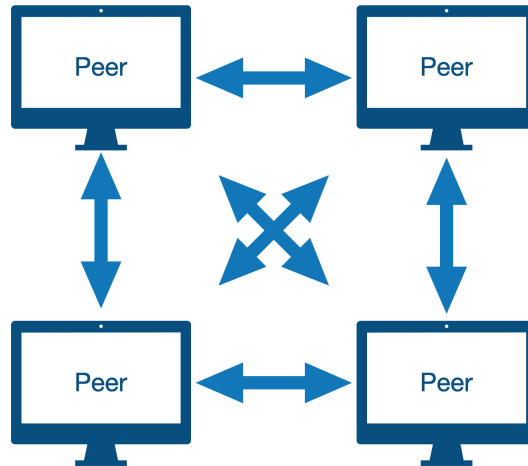


Przykłady:

- Serwer WWW i przeglądarka internetowa
- System bankowy

75.5 Peer-to-Peer

Każdy z podsystemów (hostów, peerów) może pełnić zarówno funkcję serwera, jak i klienta. Architektura systemu jest rozproszona, każdy z hostów ma te same uprawnienia.



Przykłady:

- Torrenty
- Videochaty (nie zawsze)

76 Relacyjny model danych, normalizacja relacji (w szczególności algorytm doprowadzenia relacji do postaci Boyce’a-Codda), przykłady.

LINK :

http://www.informatyka.orawskie.pl/?pl_relacyjny-model-bazy-danych,112 <https://drive.google.com/drive/folders/1vsBsmaooGSXhDF0WGDRy8CEIA7667xy9>

76.1 Relacyjny model danych

W systemach baz danych opartych o model relacyjny zachodzą własności

- Dane przechowywane są w strukturach nazywanych **tabelami**. Tabelę można traktować jako strukturę reprezentującą (i przedstawiającą) tzw. relację bazodanową
- Wiersz tabeli zawiera dane reprezentujące pewien **byt** (obiekt, czynność, zdarzenie, zjawisko) lub **zależność między bytami** (związek)
- Cechy „bytu”, zwane **atrybutami** są reprezentowane w **kolumnach**. Kolumny mają **nazwę** i określony **typ** danych związany z dziedziną atrybutu. Dane wpisywane do kolumny są elementami dziedziny i są logicznie niepodzielne (elementarne, atomowe)
- Każdy wiersz w tabeli musi (powinien) być **unikalny**

Definicja 76.1 Klucz główny

Minimalna kombinacja pól identyfikująca każdy rekord w tabeli w sposób jednoznaczny (pole, które dla każdego rekordu będzie przyjmowało inną, niepowtarzalną wartość)

76.2 Relacja bazodanowa

Projektując bazę danych, dzielimy dane na wiele tabel tematycznych, tak aby każda informacja została zapisana tylko raz. Aby zestawień razem dane zapisane w różnych tabelach, tworzy się między nimi połączenia zwiane relacjami.

Relacja jest to zdefiniowanie logicznego połączenia między tabelami bazy danych. W efekcie zmiana wiersza bieżącego w tabeli głównej powoduje automatyczną zmianę wiersza bieżącego w tabeli przyłączonej.

76.3 Typy relacji bazodanowej

W rzeczywistości występują trzy typy relacji:

- **Relacja jeden do jednego**

W relacji „jeden do jednego” każdemu rekordowi z pierwszej tabeli może odpowiadać tylko jeden rekord z drugiej tabeli i każdemu rekordowi z drugiej tabeli może odpowiadać tylko jeden rekord z pierwszej tabeli. Jest to nietypowy rodzaj relacji, ponieważ najczęściej informacje powiązane w ten sposób są przechowywane w jednej tabeli. Opisywanej relacji używa się w celu odizolowania części tabeli ze względu na bezpieczeństwo danych lub w celu podzielenia danych na podzbiory.

- **Relacja wiele do jednego**

W relacji „wiele do jednego” każdemu rekordowi z pierwszej tabeli może odpowiadać najwyżej jeden rekord z drugiej tabeli, a każdemu rekordowi z drugiej tabeli może odpowiadać wiele rekordów z pierwszej tabeli. Jest to typ relacji najczęściej występujący w relacyjnych bazach danych.

- **Relacja wiele do wielu**

W relacji „wiele do wielu” każdemu rekordowi z pierwszej tabeli może odpowiadać wiele rekordów z drugiej tabeli i każdemu rekordowi z drugiej tabeli może odpowiadać wiele rekordów z pierwszej tabeli.

76.4 Własności relacji bazodanowej

- Nie ma powtarzających się krotek
- Krotki i atrybuty **nie są uporządkowane**
- Wartości atrybutów są **skalarne (atomowe, niepodzielne)**

76.5 Postaci normalne

- **Pierwsza postać normalna 1NF**

Relacja jest w pierwszej postaci normalnej jeżeli każdy jej atrybut jest ska-

larny (atomowy, niepodzielny). Atrybut nie może być strukturą danych (listą, macierzą itp.). Relacja musi mieć zdefiniowany klucz.

- **Druga postać normalna 2NF**

Relacja jest w drugiej postaci normalnej jeżeli jest w 1NF i żaden atrybut nie będący kluczem nie jest częściowo funkcyjnie zależny od jakiegokolwiek klucza potencjalnego.

- **Trzecia postać normalna 3NF**

Relacja jest trzeciej postaci normalnej jeżeli jest w 2NF i żadna kolumna niekluczowa nie jest zależna funkcyjnie od innych atrybutów niekluczowych.

- **Postać normalna Boyce’a-Codda**

W tej postaci zależności funkcyjne muszą mieć następującą postać: jeżeli $X \rightarrow A$ i atrybut A nie jest zawarty w X, to X jest kluczem lub zawiera klucz.

76.6 Normalizacja

Definicja 76.2 Normalizacja

Proces organizowania danych w bazie danych. Obejmuje to tworzenie tabel i ustanawianie relacji między tymi tabelami zgodnie z regułami zaprojektowanymi w celu zarówno ochrony danych, jak i zapewnienia większej elastyczności bazy danych przez wyeliminowanie nadmiarowości i niespójnych zależności.

Cel normalizacji

- Uniknięcie redundancji
- Wyeliminowanie niewygodnych relacji wieloznacznych
- Uniknięcie anomalii przy aktualizacji (CRUD)
- Uniknięcie niespójności

Normalizacja

- **Pierwsza postać normalna 1NF**

Wyeliminowanie powtarzających się grup

- **Druga postać normalna 2NF**

Wyeliminowanie nadmiarowych danych

- **Trzecia postać normalna 3NF**

Wyeliminowanie danych nie zależnych od klucza

PRZYKŁADY

1. 1NF

Tabela nieznormalizowana

Naruszenie 1NF (atrybut imię nie jest skalarny, jest strukturą danych - listą).

Relacja nie ma też zdefiniowanego klucza.

Płeć	Imię
Męska	Jan, Piotr, Zenon
Żeńska	Anna, Maria, Zofia

Tabela znormalizowana

Rozdział zagnieżdżonego atrybutu na pojedyncze krotki. Relacja może mieć teraz klucz główny składający się z dwój atrybutów: Płeć + Imię

Płeć	Imię
Męska	Jan
Męska	Piotr
Męska	Zenon
Żeńska	Anna
Żeńska	Maria
Żeńska	Zofia

2. 2NF

Tabela nieznormalizowana

Klucz potencjalny składa się z dwóch atrybutów Imię + Nazwisko, atrybut Płeć zależy tylko od jednego z atrybutów klucza (Imię) zatem atrybut Płeć zależny jest od podzbioru właściwego klucza.

Imię	Nazwisko	Płeć
Edward	Szczypka	Męska
Rafał	Kawa	Męska

Tabela znormalizowana

Podział tabeli na dwie tabele.

Imię	Nazwisko
Edward	Szczypka
Rafał	Kawa

Imię	Płeć
Edward	Męska
Rafał	Męska

3. 3NF

Tabela nieznormalizowana

Atrybut Stawka jest zależny od atrybutu Stanowisko, który nie jest atrybutem kluczowym.

Imię	Nazwisko	Stanowisko	Stawka za godzinę
Edward	Szczypka	Żołnierz	15
Rafał	Kawa	Pracownik dydaktyczny	10

Tabela znormalizowana

Podział tabeli na dwie

Imię	Nazwisko	Stanowisko
Edward	Szczypka	Żołnierz
Rafał	Kawa	Pracownik dydaktyczny

Stanowisko	Stawka za godzinę
Żołnierz	15
Pracownik dydaktyczny	10

4. Boyce-Codd

Tabela nieznormalizowana

Kluczem jest Student + Przedmiot. Zachodzi zależność Przedmiot $\Rightarrow$ Profesor ale Przedmiot nie jest nadkluczem (czyli zbiorem atrybutów zawierających klucz)

Student	Przedmiot	Profesor
X	WDI	A
Y	ASD	B

Tabela znormalizowana
Podział tabeli na dwie

student	przedmiot
X	WDI
Y	ASD

Przedmiot	Profesor
WDI	WDI
ASD	ASD

77 Indeksowanie w bazach danych: drzewa B+, tablice o organizacji indeksowej, indeksy haszowe, mapy binarne.

Definicja 77.1 Indeks

Jest to **struktura danych** na dysku umożliwiająca szybkie wyszukiwanie danych w bazie danych na podstawie wartości klucza.

W najprostszej postaci wyszukiwanie polega na tym, że mając wartość poszukiwanych rekordów, w których ta wartość występuje w danym polu.

Zalety stosowania indeksów

- Zmniejszenie czasu odszukania potrzebnych danych (minimalizowanie liczby odczytów bloków danych z dysku)

Wady stosowania indeksów

- Zwiększenie czasu zmiany danych, na których założony jest indeks (zmiana takich danych oznacza zmianę w indeksie)

77.1 Typy indeksów (dla plików sekwencyjnych)

Indeksy dzieli się na

1. Główne (Primary)
2. Drugorzędne (Secondary)
 1. Gęste (Dense)
Każdy wiersz danych ma swój wpis w indeksie
 2. Rzadkie (Sparse)
Każdy blok danych ma jeden wpis w indeksie. Dane muszą być posortowane według klucza indeksu.

77.2 Indeksy dla standardowych typów danych

- Drzewa B+
- Tablice haszujące
- Indeksy binarne (mapy binarne)

77.3 Drzewa B+

Definicja 77.2 Drzewa B+

B-drzewa, które nie przechowują danych w węzłach wewnętrznych, a jedynie w swoich liściach. W B-drzewie każdy węzeł wewnętrzny posiada od N do $2N$ wartości.

Rząd B+-drzewa — maksymalna liczba wskaźników p , którą można zapisać w jednym węźle

Struktura węzła wewnętrznego

$\langle P_1, K_1, \dots, P_{n-1}, K_{n-1}, P_n \rangle$

gdzie $n \leq p$

K_i jest wartością klucza indeksu

P_i jest wskaźnikiem do poddrzewa, przechowuje wartości kluczy z przedziału $[K_{i-1}, K_i)$, z uwzględnieniem $K_0 = -\infty$, $K_n = +\infty$ oraz $K_1 < K_2 < \dots < K_n$

Struktura liścia

$\ll K_1, Pr_1 \rangle, \dots, \langle K_n, Pr_n \rangle, P_{next} \gg$

gdzie $n \leq p$

K_i jest wartością klucza indeksu

Pr_i jest wskaźnikiem do rekordów na dysku lub bloków danych P_{next} wskazuje następnego liścia (na tym samym poziomie) oraz $K_1 < K_2 < \dots < K_n$

78 Podstawowe cechy transakcji (ACID). Metody sterowania współbieżnością transakcji, poziomy izolacji transakcji, przykłady.

78.1 Transakcja

Transakcja jest wykonywanym programem, który tworzy logiczną jednostkę przetwarzania w bazie danych. Transakcja składa się z jednej lub wielu operacji dostępu do bazy danych i może obejmować operacje wstawiania, usuwania, modyfikowania i pobierania danych.

W celu wyjaśnienia pojęć dotyczących przetwarzania transakcji przyjmiemy pewien uproszczony model bazy danych. Baza danych będzie reprezentowana przez zbiór nazwanych elementów danych. Podstawowymi operacjami wykonywanymi na elementach bazy danych będą $\text{read}(A)$ i $\text{write}(A)$.

Operacja $\text{read}(A)$ oznacza odczyt z twardego dysku elementu danych A i zapisanie do pamięci RAM do zmiennej lokalnej (dla uproszczenia też oznaczonej przez A). Operacja $\text{write}(A)$ oznacza zapis zmiennej A na twardy dysk (w odpowiednie miejsce, tj. do elementu danych A).

W rzeczywistych systemach operacje odczytu i zapisu są realizowane całymi blokami (stronami). Zwykle blok (strona) ma kilka (2,4,8,16) kilobajtów. W systemie Microsoft SQL Server wielkość strony jest stała i wynosi 8kB. W systemie Oracle wielkość bloku można definiować, ponadto w jednej bazie danych można korzystać z bloków o różnych rozmiarach.

W systemie z blokowymi odczytami i zapisami operacja $\text{read}(A)$ może być realizowana następująco:

1. Znalezienie adresu bloku dyskowego, zawierającego element A .
2. Sprawdzenie czy blok ten znajduje się już w pamięci operacyjnej, jeśli nie to skopiowanie znalezionej bloku do bufora pamięci.
3. Skopiowanie elementu A z bufora do zmiennej lokalnej o nazwie A .

Operacja $\text{write}(A)$ może być wykonana następująco:

1. Znalezienie adresu bloku dyskowego, zawierającego element A .

2. Sprawdzenie czy blok ten znajduje się już w pamięci operacyjnej, jeśli nie to skopiowanie znalezionego bloku do bufora pamięci.
3. Skopiowanie elementu A ze zmiennej lokalnej A do odpowiedniego miejsca w buforze.
4. Skopiowanie bloku z bufora na dysk (ta operacja może być wykonana od razu lub z pewnym opóźnieniem).

78.2 Przykład transakcji:

Przelew pieniędzy: należy przelać 1000 zł z jednego konta na drugie. W poniższej tabeli numer stanu określa stan po wykonaniu operacji w linii, np. stan numer 1 oznacza stan po wykonaniu operacji $\text{read}(A)$.

	Konto 1	Konto 2
0	5000.00	300.00
1	$\text{read}(A)$	
2	$A := A - 1000$	
3	$\text{write}(A)$ ($A=4000$)	
4		$\text{read}(B)$
5		$B := B + 1000$
6		$\text{write}(B)$ ($B=1300$)

Problem w przypadku awarii systemu w momencie oznaczonym jako 4.

Mówimy, że w chwilach 0 i 7 baza danych jest w stanie spójnym (lub stan 0 i 7 są spójne), tzn. wszystkie ograniczenia narzucone na dane (jawne i niejawne) są spełnione.

78.3 ACID

- **Atomicity** (atomowość, niepodzielność) – transakcja jest niepodzielną jednostką przetwarzania; albo jest wykonywana w całości, albo wcale.
- **Consistency** (spójność) – po zakończeniu transakcji baza musi być w stanie spójnym, tj. muszą być zachowane wszystkie więzy narzucone na dane.
- **Isolation** (separacja, odseparowanie, izolacja) Transakcja powinna wyglądać tak, jakby była wykonywana w izolacji od innych transakcji.

- **Durability** (trwałość) – po zatwierdzeniu (tj. pomyślnym zakończeniu) transakcji jej efekty muszą być trwałe w systemie (nawet jeśli nastąpi uszkodzenie systemu natychmiast po zatwierdzeniu transakcji).

78.3.1 ACID dla przykładowej transakcji

Co oznaczałyby powyższe warunki w stosunku do wcześniej przedstawionej transakcji (nazwijmy ją T_i)?

Atomowość: Efekt musi być taki, jakby wykonały się wszystkie kroki 1–6, albo żaden.

Spójność: Suma ($A + B$) musi być taka sama przed i po wykonaniu transakcji T_i .

Odseparowanie: Załóżmy, że inna transakcja T_k sumuje zawartość kont A i B . Rezultat współbieżnego działania obydwu transakcji powinien być poprawny, tj. żadne pieniądze nie powinny zaginać lub pojawić się znikąd.

Trwałość: Nowe wartości A oraz B powinny znaleźć się na dysku bez względu na to, co stanie się po zatwierdzeniu transakcji. Jeśli natychmiast po zatwierdzeniu transakcji nastąpi awaria, może być wymagane wykonanie specjalnej operacji odtworzenia bazy danych (w zależności od architektury systemu i sposobu działania menadżera odtwarzania), efekty transakcji muszą być odtworzone.

78.4 Stany transakcji:

1. **Aktywna** (active) – transakcja jest w tym stanie podczas wykonywania operacji. Transakcja przechodzi do tego stanu zaraz po rozpoczęciu wykonywania.
2. **Częściowo zatwierdzona** (partially committed) – ostatnia operacja transakcji została wykonana. Teraz protokół zatwierdzania musi zapewnić trwałość zmian danych wykonanych przez transakcję (również w przypadku awarii). Dokonuje się to na przykład przez zapis w tzw. dzienniku transakcji (rejestrowane są tu wszystkie operacje rozpoczęcia transakcji, zmiany danych, zakończenia transakcji, czasem rejestrowane są też operacje odczytu). Jednak zanim zostanie podjęta decyzja o zatwierdzeniu, w zależności od typu systemu (przyjętych rozwiązań) zarządca odtwarzania może też sprawdzać pewne dodatkowe warunki określające, czy transakcja może być zatwierdzona (np. ze względu na sterowanie wieloma współbieżnymi transakcjami). Jeśli sprawdzenie tych warunków dało wynik pozytywny, czyli transakcja może zostać zatwierdzona mówi się, że transakcja osiągnęła swój punkt zatwierdzenia.

3. **Nieudana** (failed) – normalne wykonywanie transakcji nie może być kontynuowane. Może to wynikać z przebiegu operacji transakcji (np. niespełniony warunek CHECK, wycofanie jawne ROLLBACK, zakleszczenie). Transakcja może się również znaleźć w tym stanie wskutek działania zarządcy odtwarzania (omówionego powyżej, w punkcie 2).
4. **Przerwana, wycofana** (aborted) – baza jest odtworzona do stanu sprzed rozpoczęcia transakcji.
5. **Zatwierdzona, wypełniona** (committed) – wszelkie zmiany wykonane przez transakcje muszą być trwałe. Fizyczny zapis może być elementów danych może być później, jednak muszą działać mechanizmy zapewniające trwałość tych zmian (również w obliczu możliwych awarii).

Wypełnionej (zatwierdzonej) transakcji nie można już wycofać. Jeśli wykonanie transakcji A było błędem, to należy wykonać odpowiednią **transakcję kompensującą B**, która odwróci skutki działania transakcji A. Operacja zatwierdzenia (wypełnienia) transakcji w języku SQL to **COMMIT**. Operacja wycofania transakcji (wycofania wszystkich wprowadzonych przez nią zmian) w języku SQL to **ROLLBACK**.

78.5 Techniki sterowania współbieżnością transakcji

Twórca transakcji (programista) na ogół ma możliwość wyboru tego, w jaki sposób jego system będzie zarządzał (sterował) współbieżnością transakcji. Podstawowy sposób to stosowanie blokad (omówione niżej).

78.5.1 Typy blokad

Blokada (lock) jest zmienną związaną z elementem danych, która opisuje stan tego elementu pod względem możliwości działań, jakie mogą być na nim w danej chwili wykonywane. **Blokowanie binarne** – element wolny lub zablokowany. Wada: mała efektywność operacji współbieżnych. Propozycja dwóch typów blokad:

- **Blokady wyłączne** (typu X, exclusive, blokady do zapisu)
- **Blokady wspólne** (typu S, shared, blokady do odczytu)

Zasady:

1. Jeśli transakcja A założy blokadę wyłączną (X) na wiersz p, to próba założenia jakiegokolwiek blokady przez inną transakcję B na tym samym wierszu zostanie oddalona.
2. Jeśli transakcja A założy blokadę wspólną (S) na wiersz p, to:
 - (a) próba założenia blokady wyłączonej (X) przez transakcję B na tym samym wierszu zostanie oddalona,
 - (b) próba założenia blokady wspólnej (S) przez transakcję B na tym samym wierszu zostanie zaakceptowana - transakcja B też będzie utrzymywać blokadę na p.

78.5.2 Blokada na poziomie tabeli

IS (intent shared) – wspólna blokada zapobiegająca – transakcja zamierza założyć blokadę S na poszczególnych wierszach tabeli (ogólnie: węzłach potomnych)

IX (intent exclusive) – wyłączna blokada zapobiegająca – zamiar założenia blokady X na wierszach (ogólnie: węzłach potomnych)

SIX (shared intent exclusive) – wspólna wyłączna blokada zapobiegająca – na tabeli (węzle macierzystym) jest już blokada S, zamierza się założyć blokadę X na wiersze (połączenie S oraz IX).

78.5.3 Sterowanie współbieżnością transakcji w systemie Microsoft SQL Server.

SQL Server realizuje poziomy izolacji transakcji w wersji blokowania:

- locking READ UNCOMMITTED,
- locking READ COMMITTED,
- locking REPEATABLE READ,
- locking SERIALIZABLE.

Można włączyć tzw. wielowersyjne techniki sterowania współbieżnością (system może przechowywać wiele wersji tego samego elementu danych). Dostępne są wówczas dwa dodatkowe poziomy izolacji transakcji:

- READ COMMITTED SNAPSHOT (READ CONSISTENCY) oraz
- SNAPSHOT (anomalny SERIALIZABLE).

78.5.4 Implementacja poziomów izolacji transakcji:

Poziom izolacji	P0 Dirty Write	P1 Dirty Read	P2 Non- repeatable Read	P3 Phantoms	Blokady X	Blokady S
Locking READ UNCOMMITTED	NIE	TAK	TAK	TAK	T, długie	Nie ma
Locking READ COMMITTED	NIE	NIE	TAK	TAK	T, długie	T, krótkie
Locking REPEATABLE READ	NIE	NIE	NIE	TAK	T, długie	T, długie
Locking SERIALIZABLE	NIE	NIE	NIE	NIE	T, długie	T, długie predykatowe

A1 (Dirty Read): Transaction T1 modifies a data item. Another transaction T2 then reads that data item before T1 performs a COMMIT or ROLLBACK. If T1 then performs a ROLLBACK, T2 has read a data item that was never committed and so never really existed.

A2 (Non-repeatable or Fuzzy Read): Transaction T1 reads a data item. Another transaction T2 then modifies or deletes that data item and commits. If T1 then attempts to reread the data item, it receives a modified value or discovers that the data item has been deleted.

A3 (Phantom): Transaction T1 reads a set of data items satisfying some <search condition>. Transaction T2 then creates data items that satisfy T1's <search condition> and commits. If T1 then repeats its read with the same <search condition>, it gets a set of data items different from the first read.

78.5.5 Poziom izolacji Cursor Stability

Poziom izolacji Cursor Stability może być zdefiniowany poprzez pewne rozszerzenie sposobu blokowania w poziomie Locking READ COMMITTED. Dodaje się operację rc (czytaj kursor, pobierz wiersz) dla instrukcji FETCH, blokada (S lub nowy typ blokady do odczytu scroll lock) będzie utrzymywana do chwili przejścia do innego

wiersza lub do zamknięcia kursora. Aktualizacja wiersza przez kursor – operacja w której powoduje założenie na ten wiersz blokady X utrzymywanej do końca transakcji.

78.5.6 Poziom izolacji Snapshot

W poziomie izolacji Snapshot transakcja czyta dane (zatwierdzone) z chwili swojego początku, Start-Timestamp.

Protokoły odczytu spójnych wersji czasowych.

- Propozycja 1 (Snapshot isolation): Transakcja odczytuje dane zatwierdzone przed początkiem transakcji (czyta wartości aktualne w momencie rozpoczęcia transakcji, wartości z pewnej migawki - snapshot). Wszelkie zmiany są wykonywane na lokalnych kopiach i zapisywane na końcu transakcji. Aktualizacje wykonywane przez inne transakcje nie są odczytywane.
- Propozycja 2 (Snapshot isolation): Podobna do propozycji 1, ale są stosowane blokady do zapisu, ponadto przy każdym zapisie transakcja wykonuje podobne sprawdzenie jak wykonywane w propozycji 1 na końcu transakcji.
- Propozycja 3: Read Committed Snapshot (Read Consistency): Podobna do propozycji 2, ale operacja odczytu czyta ostatnią zatwierdzoną wartość elementu danych (niekoniecznie sprzed początku transakcji).

79 Złączenia, grupowanie, podzapytania w języku SQL.

79.1 JOIN

Łączenie tabel umożliwia odczytywanie danych z wielu tabel jednocześnie. Tabele zostają połączone pomiędzy kluczem podstawowym jednej tabeli, a kluczem obcym drugiej (klucz obcy to kolumna, która stanowi klucz podstawowy w innej tabeli). Połączenia tabel dzielą się na wewnętrzne i zewnętrzne.

Połączenia wewnętrzne są realizowane przy pomocy instrukcji INNER JOIN, która wybiera rekordy posiadające takie same wartości w obu tabelach. Jej składnia wygląda następująco:

```
tabela1 INNER JOIN tabela2
ON tabela1.kolumna1=tabela2.kolumna2
```

Przykładowo, wypisanie idZamowienia z tabeli zamowienia i nazwa klienta z tabeli klienci:

```
SELECT zamowienia.idZamowienia, klienci.nazwa_klienta
FROM zamowienia
INNER JOIN klienci ON zamowienia.idKlienta = klienci.IdKlienta;
```

Połączenia zewnętrzne definiuje się za pomocą poleceń:

LEFT OUTER JOIN – zwraca wszystkie wiersze z pierwszej tabeli i pasujące wiersze z drugiej RIGHT OUTER JOIN – zwraca wszystkie wiersze z drugiej tabeli i pasujące wiersze z pierwszej FULL OUTER JOIN – zwraca wszystkie pasujące i niepasujące wiersze z obu tabel

Przykładowo:

Wypisanie nazwy klientów z tabeli klienci i identyfikatorów zamówień z tabeli zamówienia

```
SELECT zamowienia.IdZamowienia, pracownicy.nazwisko, pracownicy.imie
FROM zamowienia
RIGHT OUTER JOIN pracownicy ON zamowienia.IdPracownika =
pracownicy.IdPracownika
ORDER BY zamowienia.IdZamowienia;
```

Wypisanie nazwy klientów z tabeli klienci i identyfikatorów zamówień z tabeli zamówienia

```

SELECT klienci.nazwa_klienta, zamowienia.IdZamowienia
FROM Klienci
FULL OUTER JOIN zamowienia ON klienci.IdKlienta=zamowienia.IdKlienta
ORDER BY Klienci.nazwa_klienta;

```

79.2 GROUP BY

Grupowanie danych polega na tworzeniu grup rekordów w oparciu o definicję grupowania (klauszę GROUP BY). Krok ten, wykonywany jest jako kolejny po filtrowaniu rekordów, zgodnie z warunkami określonymi w WHERE (o ile w ogóle cokolwiek filtrujemy), lub bezpośrednio po FROM jeśli nie korzystamy z selekcji wierszy. W przypadku stosowania GROUP BY, w trakcie przetwarzania zapytania, tworzone są dwie sekcje danych.

- Sekcja grupująca składa się z atrybutów tworzących definicję grupowania (są to kolumny wyszczególnione w GROUP BY). Grupy tworzone są przez wiersze, które mają takie same wartości w ramach tej sekcji. Grupa może być tworzona przez jeden (jeśli jest unikalny w ramach definicji grupy) lub wiele wierszy, w przypadku gdy posiadają te same wartości w kolumnach po których grupujemy.
- Sekcja danych surowych (raw data section) zawierają wszystkie pozostałe dane, które mogą być już unikalne dla każdego wiersza w ramach sekcji grupującej. Sekcja danych surowych jest zawsze rozpatrywana w kontekście grupy.

	Sekcja grupująca		Sekcja RAW data			
	Title	Country	LastName	FirstName	BirthDate	HireDate
1	Sales Manager	UK	Buchanan	Steven	1955-03-04	1993-10-17
2	Sales Representative	UK	Suyama	Michael	1963-07-02	1993-10-17
3	Sales Representative	UK	King	Robert	1960-05-29	1994-01-02
4	Sales Representative	UK	Dodsworth	Anne	1966-01-27	1994-11-15
5	Inside Sales Coordinator	USA	Callahan	Laura	1958-01-09	1994-03-05
6	Sales Representative	USA	Davolio	Nancy	1948-12-08	1992-05-01
7	Sales Representative	USA	Leverling	Janet	1963-08-30	1992-04-01
8	Sales Representative	USA	Peacock	Margaret	1937-09-19	1993-05-03
9	Vice President, Sales	USA	Fuller	Andrew	1952-02-19	1992-08-14

```

select Title, Country, COUNT(*) as EmQty
from dbo.Employees
GROUP BY Country, Title

```

Results		Messages	
	Title	Country	EmQty
1	Inside Sales Coordinator	USA	1
2	Sales Manager	UK	1
3	Sales Representative	UK	3
4	Sales Representative	USA	3
5	Vice President, Sales	USA	1

79.3 Podzapytania

Podzapytanie polega na umieszczeniu instrukcji SELECT wewnątrz innej instrukcji SELECT. Serwer bazodanowy w pierwszej kolejności wykona zapytania wewnętrzne. Wynik zapytania wewnętrznego zwracany jest do zapytania zewnętrznego.

Podział podzapytań ze względu na korelację z zapytaniem nadrzędnym:

- podzapytania niepowiązane (nieskorelowane)
- podzapytania powiązane (skorelowane)

79.3.1 Podzapytania niepowiązane

W podzapytaniach niepowiązanych wewnętrzne zapytanie jest wykonywane tylko raz. Zwraca jeden wynik. Zapytanie wewnętrzne jest niezależne od zewnętrznego i może być wykonane samodzielnie.

Za pomocą poniższego zapytania możemy odczytać nazwę podkategorii, do której należy produkt posiadający ID o numerze 748.

```
SELECT b.Name
FROM Production.ProductSubcategory b
WHERE b.ProductSubcategoryID =
(SELECT a.ProductSubcategoryID
FROM Production.Product a
WHERE a.ProductID = 748);
```

Z tabeli Production.ProductSubcategory odczytujemy nazwy wszystkich podkategorii (kolumna Name).

Następnie sprawdzamy warunek w zewnętrznej klauzuli WHERE.

W celu sprawdzenia warunku należy wykonać wewnętrzne zapytanie. W zapytaniu wewnętrznym możemy wybrać tylko jedną wartość (wartość skalarna).

79.3.2 Podzapytania powiązane

Podzapytania powiązane (skorelowane) są bezpośrednio powiązane z zapytaniem nadrzędnym. Łącznikiem jest jeden lub więcej atrybutów, przekazywanych z zapytania nadrzędnego. Zapytanie wewnętrzne wykonywane jest osobno dla każdego wiersza zwróconego przez zapytanie zewnętrzne. Zwraca ono tyle wyników, ile jest rekordów w wyniku zapytania zewnętrznego.

Ze względu na powiązanie z zapytaniem nadrzędnym, zapytanie wewnętrzne nie może być wykonane samodzielnie.

```
SELECT b.BusinessEntityID,  
       (SELECT COUNT(a.AddressTypeID)  
        FROM Person.BusinessEntityAddress as a  
        WHERE a.BusinessEntityID = b.BusinessEntityID) as licz_zam  
FROM Person.Person as b  
ORDER BY licz_zam DESC;
```

W powyższym przykładzie w zapytaniu wewnętrznym zliczana jest liczba rekordów w tabeli Person.BusinessEntityAddress. Zapytanie wewnętrzne uruchomione jest dla każdego rekordu z tabeli Person.Person, w przypadku którego BusinessEntityID jest takie samo jak BusinessEntityID w tabeli Person.BusinessEntityAddress.

80 Szeregowalność harmonogramów w bazach danych.

Definicja 80.1 Harmonogram - inaczej historia S zbioru n transakcji $T_1, \dots, T_n$ jest takim ciągiem wszystkich operacji transakcji, że dla każdej transakcji $T_i \in S$ operacje tej transakcji w S muszą występować w takiej samej kolejności, w jakiej występują w T_i . Operacje z różnych transakcji mogą się przeplatać.

Rodzaje harmonogramów:

- **Harmonogram szeregowy** - operacje każdej transakcji są wykonywane kolejno, **bez przeplatania** operacji z różnych transakcji. Są nieefektywne.
- **Harmonogram szeregowalny** - jego wpływ na stan bazy danych jest taki sam jak pewnego harmonogramu szeregowego.
- **Harmonogram nieszeregowy** - harmonogram, który nie jest sekwencyjny.

Definicja 80.2 Konflikt operacji. Dwie operacje są w stanie konfliktu, jeśli spełnione są warunki:

- Należą do **różnych transakcji**,
- Uzyskują dostęp do **tego samego elementu**,
- **Przynajmniej jedna z operacji jest operacją zapisu elementu.**

Definicja 80.3 Harmonogram pełny - harmonogram zawierający wszystkie operacje z transakcji składowych, gdzie dla dowolnych dwóch **operacji**, które są w **konflikcie**, jedna z nich musi **poprzedzać** drugą w harmonogramie.

Definicja 80.4 Równoważność konfliktowa. Mówimy, że dwa harmonogramy są równoważne konfliktowo, jeżeli **kolejność** wszystkich **operacji konfliktowych** jest w nich **taka sama**. Wyniki harmonogramów równoważnych konfliktowo są takie same.

Definicja 80.5 Szeregowalność konfliktowa. Harmonogram S jest szeregowalny konfliktowo, jeżeli jest on **konfliktowo równoważny** pewnemu harmonogramowi **szeregowemu** S' . Możemy zmieniać kolejność niekonfliktowych operacji w S .

Szeregowalność konfliktowa stanowi warunek wystarczający zachowania spójności danych.

Definicja 80.6 Równoważność perspektywiczna harmonogramów. Harmonogram S i S' zawierają te same instrukcje i dla każdego elementu danych Q :

- Jeśli w S T_k jest transakcją, która w harmonogramie czyta Q jako pierwsza, to w S' T_k musi być transakcją, która czyta Q jako pierwsza.
- Jeśli w S T_i czyta Q zapisany przez T_j , to w S' T_i czyta Q zapisany przez T_j .
- Jeśli w S T_m jest ostatnią transakcją, która zapisuje Q , to w S' T_m jest ostatnią transakcją, która zapisuje Q .

Definicja 80.7 Szeregowalność perspektywiczna. Harmonogram szeregowalny perspektywicznie jest harmonogramem **równoważnym perspektywicznie** jakiemuś harmonogramowi szeregowemu.

Szeregowalność perspektywiczna to szeregowalność konfliktowa z dodatkowymi ograniczeniami operacji zapisów:

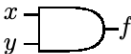
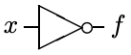
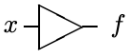
- każda operacja $w(x)$ jest poprzedzona $r(x)$,
- wartość zapisana przez operację $w(x)$ jest **nie-stałą funkcją** zależną tylko od wartości $r(x)$.

81 Definicja cyfrowego układu kombinacyjnego - przykłady układów kombinacyjnych i ich implementacje.

Definicja 81.1 *Układ kombinacyjny* to taki układ logiczny, w którym każda kombinacja wartości zmiennych wejściowych jednoznacznie określa kombinację wartości zmiennych wyjściowych.

Układy kombinacyjne opisujemy za pomocą bramek logicznych. Można też za pomocą tablicy prawdy.

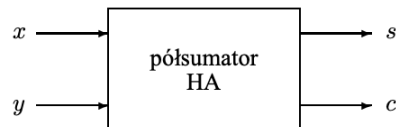
Definicja 81.2 *Bramka logiczna* to układ elektroniczny realizujący funkcję logiczną jednej lub wielu zmiennych (w podstawowej wersji dwóch).

Nazwa	Symbol graficzny	Funkcja logiczna	Tablica prawdy
AND		$f = x \cdot y = xy$	x y f
			0 0 0
			0 1 0
			1 0 0
			1 1 1
OR		$f = x + y$	x y f
			0 0 0
			0 1 1
			1 0 1
			1 1 1
NOT (Inverter)		$f = \overline{x}$	x f
			0 1
			1 0
Bufor		$f = x$	x f
			0 0
			1 1
NAND (Not AND)		$f = \overline{xy}$	x y f
			0 0 1
			0 1 1
			1 0 1
			1 1 0
NOR (Not OR)		$f = \overline{x + y}$	x y f
			0 0 1
			0 1 0
			1 0 0
			1 1 0
XOR (Exclusive-OR)		$f = x \oplus y$	x y f
			0 0 0
			0 1 1
			1 0 1
			1 1 0
XNOR (Exclusive-NOR)		$f = \overline{x \oplus y}$	x y f
			0 0 1
			0 1 0
			1 0 0
			1 1 1

81.1 Układy arytmetyczne.

81.1.1 Półsumator (Half Adder).

Zmiennymi wejściowymi półsumatora są bity składników sumy (x , y). Zmiennymi wyjściowymi są bity sumy s i przeniesienia c .



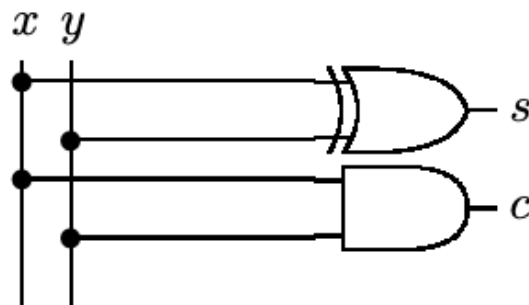
Wejścia		Wyjścia	
x	y	c	s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Z tablicy prawdy półsumatora otrzymujemy następujące funkcje logiczne:

$$s(x, y) = \sum(1, 2) = \bar{x}y + x\bar{y} = x \oplus y$$

$$c(x, y) = \sum(3) = xy$$

na podstawie których możemy zbudować schemat logiczny półsumatora:



81.1.2 Sumator (Full Adder).

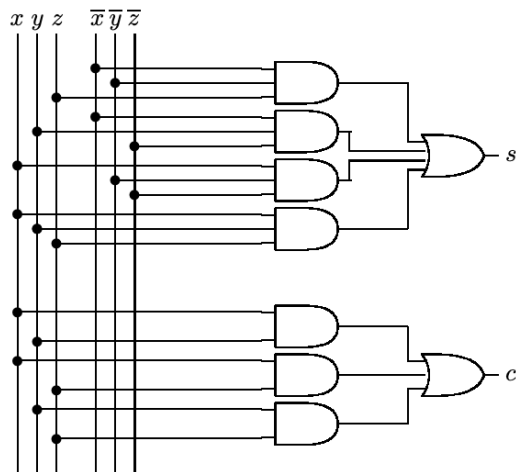
Układ kombinacyjny dodający trzy cyfry dwójkowe. Ma trzy wejścia. Dwie ze zmiennych wejściowych (x , y) reprezentują bity składników sumy. Trzecie wejście (z) reprezentuje przeniesienie z poprzedniej, mniej znaczącej pozycji. Zmiennymi wyjściowymi są bity sumy s i przeniesienia c .



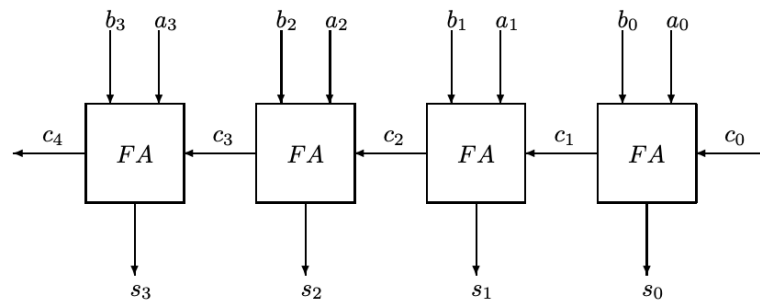
Wejścia			Wyjścia	
x	y	z	c	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$s(x, y) = \sum(1, 2, 4, 7) = \bar{x}\bar{y}z + \bar{x}y\bar{z} + xyz$$

$$c(x, y) = \sum(3, 5, 6, 7) = xz + yz + xy$$

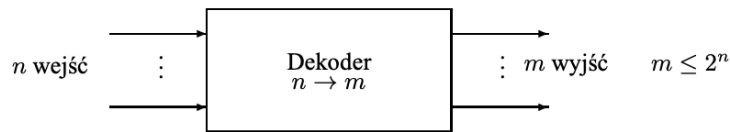


Schemat blokowy 4-bitowego sumatora równoległego (sumatora dwóch liczb czterobitowych a i b):

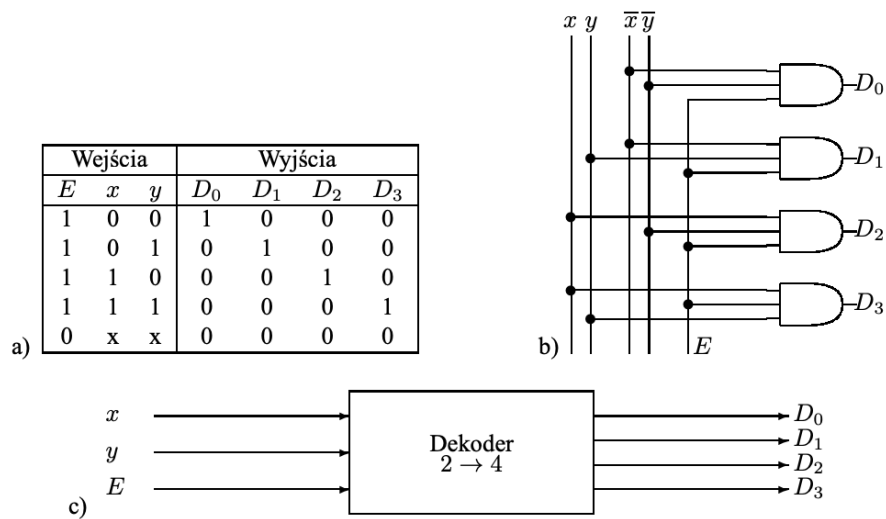


81.1.3 Dekoder.

Układ kombinacyjny mający n wejść i $m \leq 2^n$ wyjść, w którym każdej kombinacji zmiennych wejściowych odpowiada pojawienie się jedynki logicznej tylko na jednym wyjściu, na pozostałych wyjściach zera.



Rys. 3.23. Schemat blokowy dekodera $n \rightarrow m$

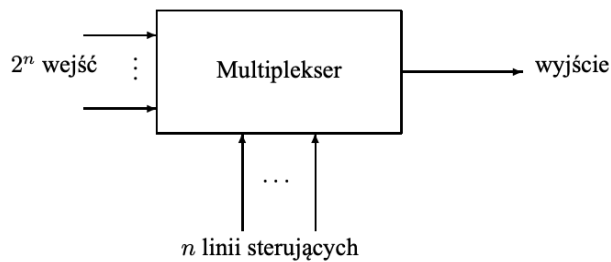


Rys. 3.24. Dekoder $2 \rightarrow 4$ z wejściem E (*enable*): tablica prawdy (a)), schemat logiczny (b)) i schemat blokowy (c))

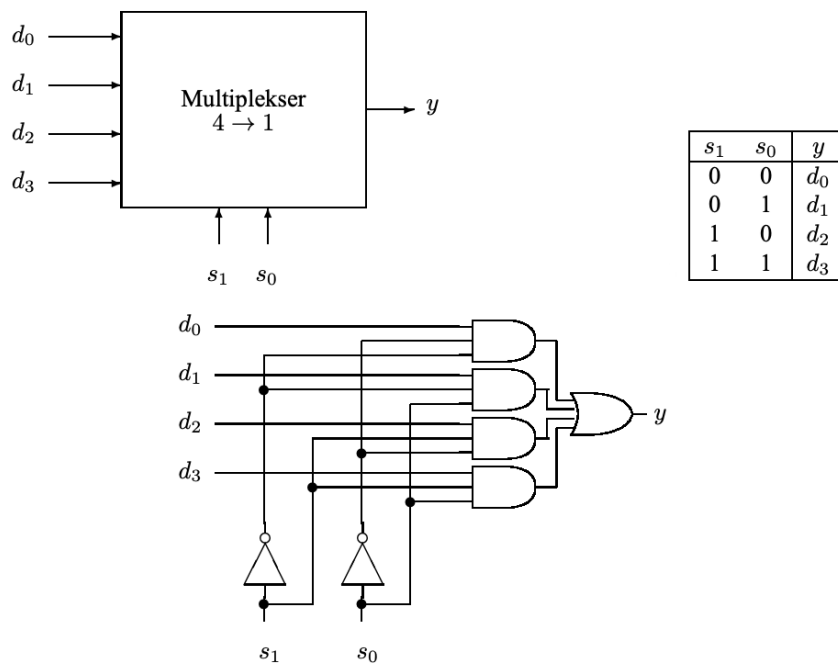
Koder to układ kombinacyjny działający odwrotnie do dekodera.

81.1.4 Multiplekser.

Układ kombinacyjny wybierający informację dwójkową na jednej z linii wejściowych i kierujący ją na jedną linię wyjściową. Wybór linii wejściowej jest określany przez linie sterujące.



Linie wejściowe (d_0, d_1, d_2, d_3) są doprowadzone do bramek AND, których wyjścia są doprowadzone do bramki OR. W danej chwili tylko jedna linia wejściowa jest połączona z wyjściem y . O wyborze wejścia połączanego decydują linie sterujące s_0, s_1 .



82 Definicja cyfrowego układu sekwencyjnego - przykłady układów sekwencyjnych i ich implementacje.

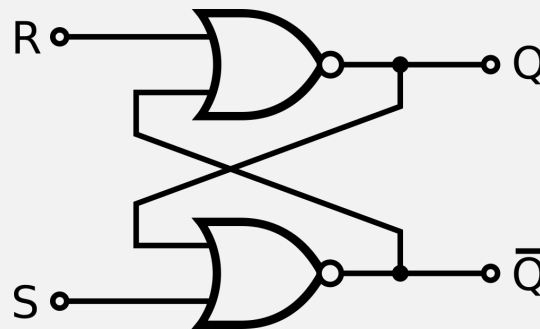
Definicja 82.1 *Układem sekwencyjnym (ang. sequential logic circuits) nazywamy układ, w którym poziomy sygnałów wyjściowych zależą nie tylko od aktualnego stanu poziomów sygnałów na jego wejściu, ale również od stanu poziomów, które występowały uprzednio. Oznacza to, że układy te zawierają elementy pamiętające i nazywane są czasem układami kombinacyjnymi z pamięcią.*

Definicja 82.2 Przerzutnik RS (Reset-Set) - jest najprostszym rodzajem przerzutnika zbudowanego z 2 bramek NOR (w innej wersji $NAND$). Posiada dwa wejścia: S i R ; oraz dwa wyjścia Q i $\bar{Q}$. Zasadę jego działania przedstawia poniższa tabelka:

R	S	Q_{n-1}	Q_n
0	0	0	0
0	0	1	1
0	1	X	1
1	0	X	0
1	1	X	N

Jak widać, po podaniu 1 na wejście S , wyjście Q niezależnie od swojego poprzedniego stanu jest ustawiane na 1, natomiast podanie 1 na wejście R , powoduje ustawienie wyjścia Q na stan 0 niezależnie od jego poprzedniego stanu. Gdy ani na wejście S , ani na wejście R nie podamy 1, wtedy przerzutnik “pamięta” poprzedni stan (stan wyjścia Q się nie zmienia), natomiast podanie na oba wejścia 1 jest niedozwolone, ponieważ powoduje ustawienie stanu 0 na obu wyjściach, co jest niezgodne z założeniami (wyjście $\bar{Q}$ ma być zawsze negacją wyjścia Q).

Przykładowy schemat blokowy przerzutnika (na bramkach NOR):



Definicja 82.3 Przerzutnik D (Delay) - jest pewną modyfikacją przerzutnika RS . Ma on wejście D oraz C (Clock). Jego działanie polega na przekazaniu stanu wejścia D na wyjście Q w momencie “tyknięcia” zegara (podania stanu 1) na wejściu C . Zasadę jego działania przedstawia poniższa tabelka:

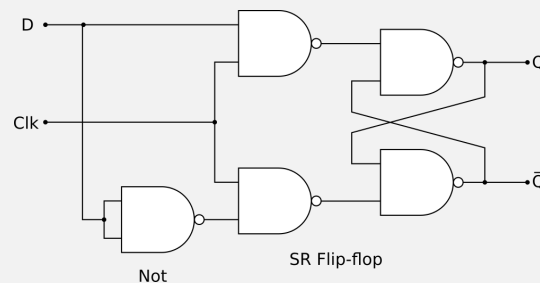
D	Q_{n-1}	Q_n
0	0	0
0	1	0
1	0	1
1	1	1

lub równoważnie:

D	Q_{n-1}	Q_n
0	X	0
1	X	1

Jak widać w przeciwieństwie do samego przerzutnika RS , który jest asynchroniczny, tutaj mamy wejście zegarowe, które synchronizuje jego pracę.

Przykładowy schemat blokowy przerzutnika (w tym przypadku część odpowiedzialna za przerzutnik RS została zbudowana na bramkach $NAND$ - ta wersja nazywa się SR , ponieważ w przypadku podania na oba wejścia 1, na obu wyjściach pojawi się stan 1 - Set ma priorytet):

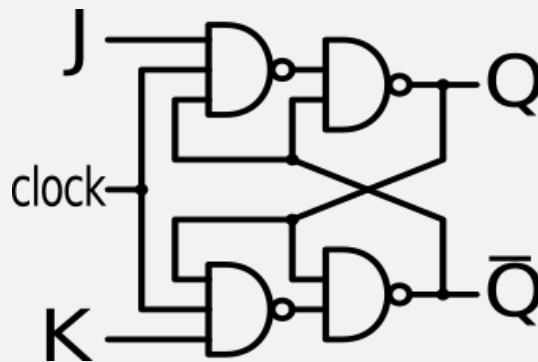


Definicja 82.4 Przerzutnik JK - podobnie jak przerzutnik D jest synchroniczny oraz powstał poprzez modyfikację przerzutnika RS (SR). W tej wersji pozbyto się kłopotliwego stanu, gdy na oba wejścia przerzutnika RS podana zostanie 1. Przerzutnik JK posiada 3 wejścia: J, K oraz Clock. Porównując RS do JK: odpowiednikiem wejścia S jest wejście J, a odpowiednikiem wejścia R jest wejście K. Zasadę działania przerzutnika JK przedstawia poniższa tabelka:

J	K	Q_{n-1}	Q_n
0	0	0	0
0	0	1	1
1	0	X	1
0	1	X	0
1	1	0	1
1	1	1	0

Jak widać w przypadku podania na oba wejścia (J i K) 1, przerzutnik zmienia stan wyjścia na odwrotny w stosunku do tego, który był przed podaniem sygnału zegarowego.

Przykładowy schemat blokowy przerzutnika (na bramkach NAND):



83 Minimalizacja funkcji logicznych.

83.1 Minimalizacja z wykorzystaniem algebry Boole'a

Definicja 83.1 *Minimalizacja z wykorzystaniem algebry Boole'a polega na zapisaniu funkcji w postaci sumy stanów wejść, dla których funkcja przyjmuje wartość 1, a następnie wykorzystując prawa algebry Boole'a uproszczenie jej.*

83.2 Minimalizacja z wykorzystaniem tablic Karnaugh

Definicja 83.2 *Tablica Karnaugh* składa się z pól reprezentujących wszystkie kombinacje zmiennych naszej funkcji, którą chcemy zminimalizować jednak są one ułożone w specyficzny sposób: każde sąsiednie dwa pola (pól po ukośniku nie uznajemy za sąsiednie) muszą się różnić wartością tylko jednej zmiennej. Za sąsiednie pola uważamy również pola na krańcach danego wiersza i danej kolumny - możemy więc traktować tę tablicę jako tablicę cykliczną.

Przykłady gotowych tablic Karnaugh:

Dla dwóch zmiennych

		y	
		00	01
x	0	10	11
	1		

		y	
		0	1
x	0	2	3
	1		

		y	
		$\bar{x}\bar{y}$	$\bar{x}y$
x	0	$x\bar{y}$	xy
	1		

Dla trzech zmiennych

				y			
				000	001	011	010
x	{	100	101	111	110		
				z			

		y			
		0	1	3	2
x	{	4	5	7	6
		z			

				y	
				$\bar{x}\bar{y}\bar{z}$	$\bar{x}\bar{y}z$
x	0	$x\bar{y}\bar{z}$	$x\bar{y}z$	xyz	$xy\bar{z}$
	1				
				z	

Dla czterech zmiennych

				y	
		0000	0001	0011	0010
		0100	0101	0111	0110
w	{	1100	1101	1111	1110
		1000	1001	1011	1010
				z	

				y			
0		1		3		2	
4		5		7		6	
w	{	12	13	15	14	}	x
		8	9	11	10		
				z			

Przy pomocy tablic Karnaugh możemy minimalizować funkcję o maksymalnie 6 zmiennych.

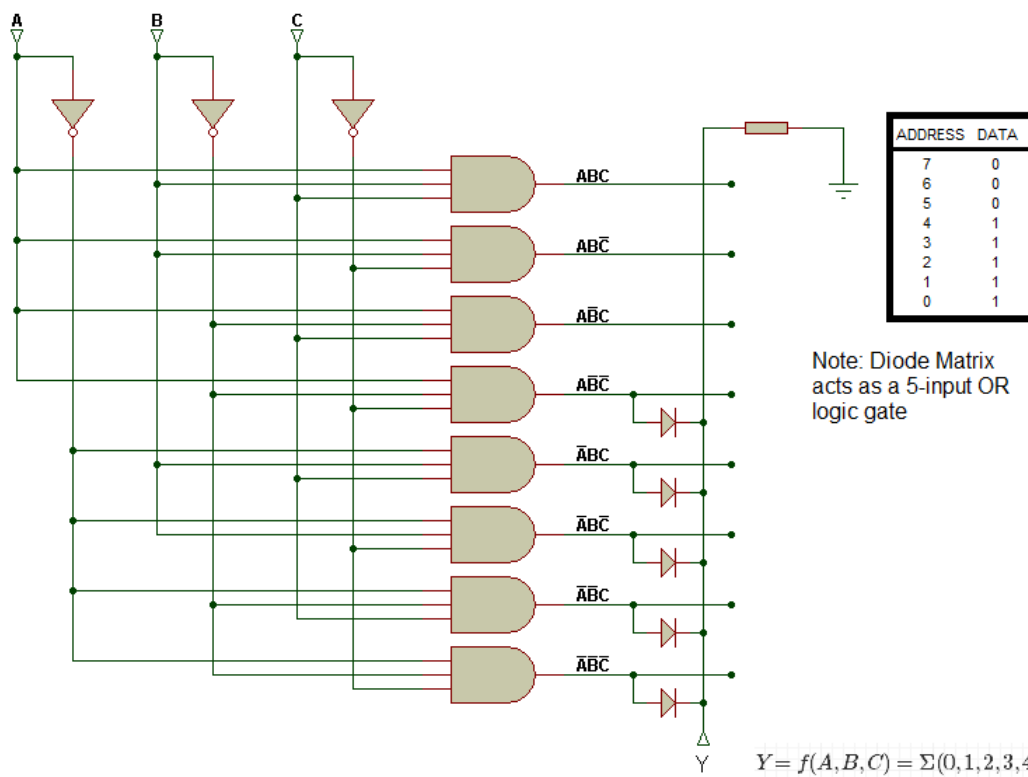
84 Programowalne układy logiczne PLD (ROM, PAL, PLA).

Programowalne układy logiczne (Programmable Logic Devices / PLD) dzielą się na:

- Read Only Memory (ROM)
- Programmable Array Logic (PAL)
- Programmable Logic Array (PLA)

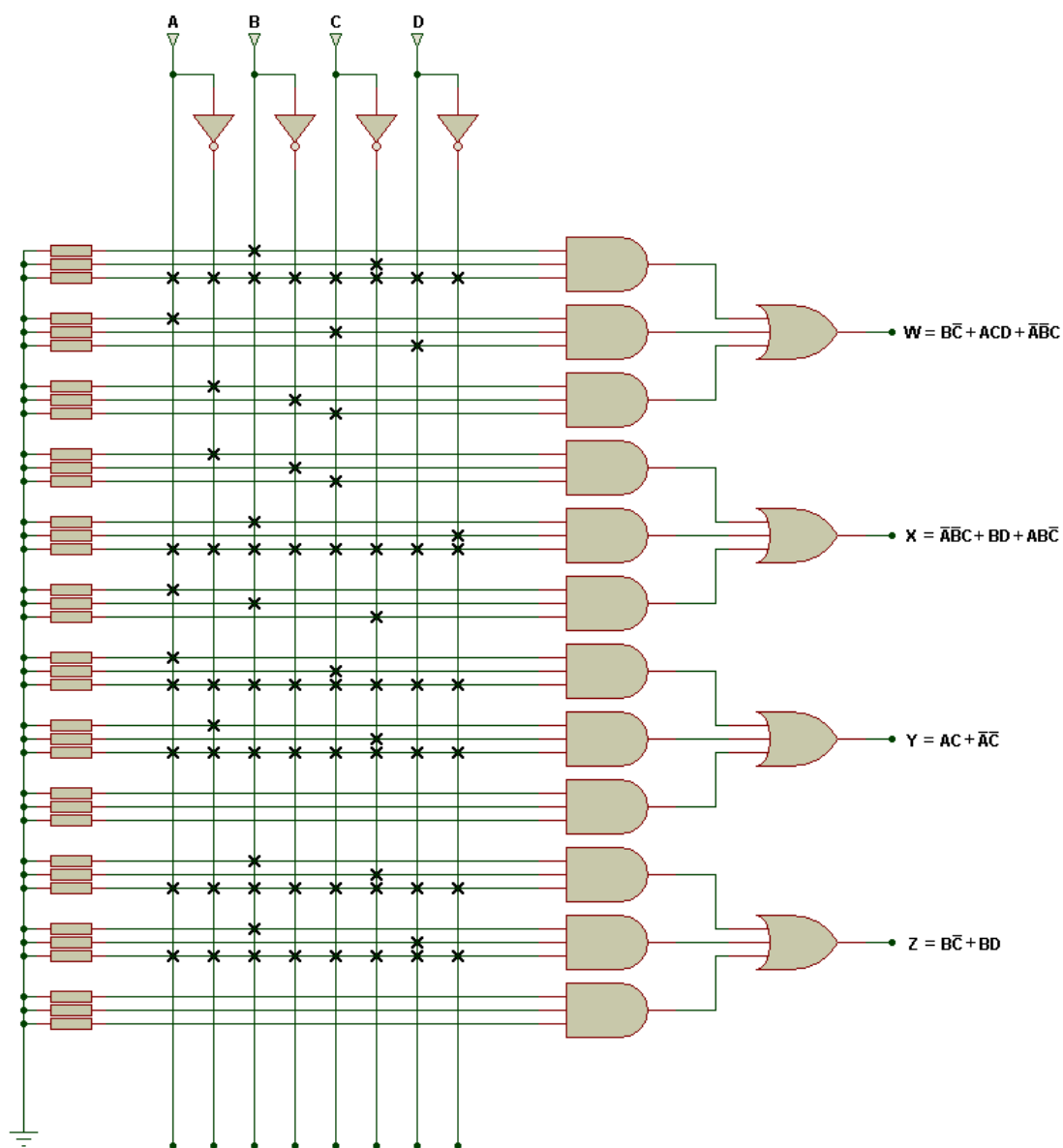
84.1 ROM

Wejście jest 'hard-wired' i zawiera wszystkie możliwe mintermy, wyjście jest programowalne.



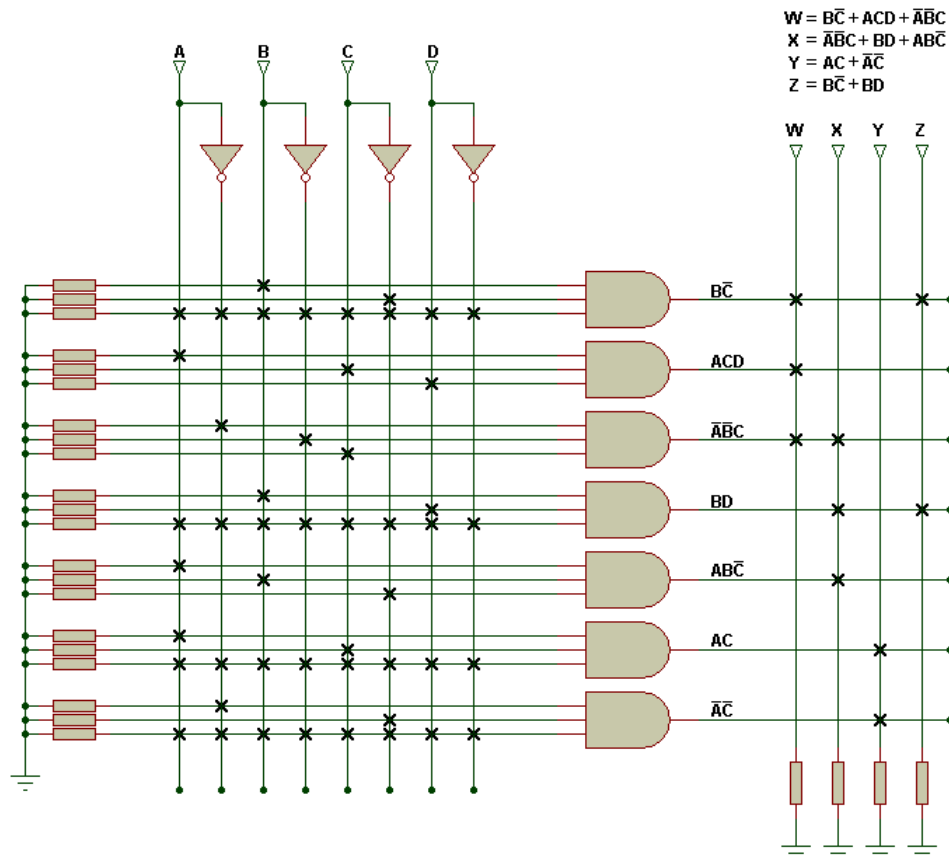
84.2 PAL

Układy PAL zawierają część nieprogramowalną (wbudowane bramki OR) oraz programowalną matrycę bramek AND. Wejście jest programowalne, wyjście jest *'hard-wired'*.



84.3 PLA

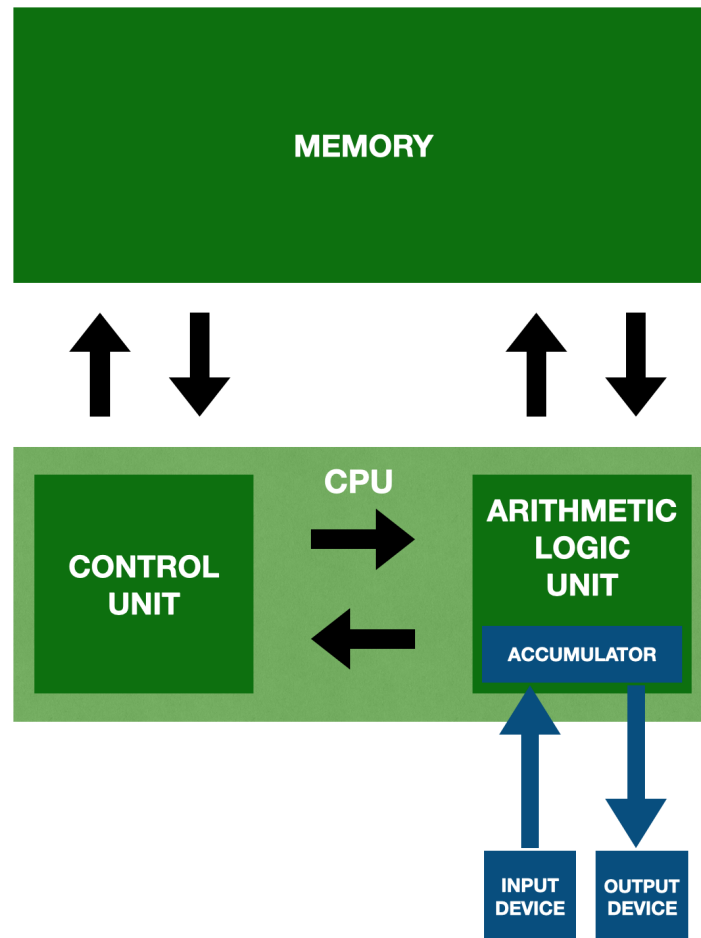
Układy PLA zawierają część programowalną matrycę bramek OR oraz programowalną matrycę bramek AND. Wejście jest programowalne, wyjście jest programowalne.



PLA vs PAL:

- PLA konstruuje się bramkami AND i OR. PAL tak samo, ale liczba OR jest ustalona.
- PLA są droższe w produkcji
- PAL są szybsze
- Liczba funkcji zaimplementowanych w PAL jest ograniczona, układy PAL są mniej elastyczne i programowalne niż PLA

85 Schemat blokowy komputera (maszyna von Neumanna).



Architektura von Neumanna to rodzaj architektury cyfrowego komputera opracowana w 1945 przez Johna von Neumanna. Składa się z następujących części:

- Pamięci - zawiera zarówno dane, jak i instrukcji
- CPU, Central Processing Unit, Procesora. CPU składa się z:
 - Jednostki kontrolnej (nadzorującej)
 - Jednostki arytmetyczno-logicznej wykonującej podstawowe operacje przetwarzania informacji.

I/O:

- Akumulator, wchodzący w skład jednostki arytmetyczno-logicznej
- Urządzenia wejścia
- Urządzenia wyjścia

Cechy architektury von Neumanna:

- Jednolitość pamięci
 - Możliwość uruchomienia samomodyfikującego się programu
 - Dzielenie pamięci instrukcji z pamięcią danych może prowadzić do bottle-necków (w tym celu wprowadza się cache)
- Skończona liczba rozkazów
- Informacje są przetwarzane poprzez sekwencyjny odczyt kolejnych instrukcji z pamięci i ich wykonanie przez procesor

Warto zapamiętać:

- Niemalże wszystkie cyfrowe, elektroniczne współczesne komputery są oparte na architekturze von Neumanna
- Architektura von Neumanna pozwoliła na oddzielenie programu od procesora
- Istnieje architektura Harvardzka, oddzielająca pamięć z rozkazami od pamięci z danymi.

86 Zarządzanie procesami: stany procesu, algorytmy szeregowania z wywłaszczaniem.

86.1 Proces

Proces służy do **organizowania wykonywania programu** w ten sposób, że stanowi on powiązanie **niezbędnych zasobów systemu komputerowego** i umożliwia **kontrolę stanu** tych zasobów związaną z wykonywaniem programu.

86.2 Proces a program

Program

- Zbiór instrukcji
- Element procesu znajdujący się w segmencie kodu
- Stan nie ulega modyfikacji w czasie wykonywania
- Wykorzystuje dodatkowe zasoby (procesor, pamięć itp.)

Stan procesu ulega zmianie: zmienia się stan wykonywania programu podobnie jak stan większości zasobów z tym związanych. Zmianie w wyniku wykonywania procesu ulega np. segment danych, segment stosu, stan rejestrów procesora itp. Procesem jest więc cały ten kontekst niezbędny do wykonania programu. Wyodrębnienie procesu wiąże się z współbieżnością przetwarzania. W systemie może istnieć wiele procesów (wiele niezależnych przetwarzania), stąd ważne jest utrzymanie informacji o tym, które zasoby przydzielone na potrzeby każdego przetwarzania.

86.3 Zarządzanie procesami

Fragmenty kodu jądra związane z obsługą procesów i zasobów nazywa się zarządcami. Wyróżnia się:

- **zarządców procesów** (grupują i wykonują funkcje obsługi procesów)
- **zarządców zasobów** (kontrolują stan zajętości zasobów i realizację żądań procesów)

86.4 Stany procesu

W zależności od stanu wykonywania programu i dostępności zasobów można wyróżnić następujące, ogólne stany procesu:

- **Nowy** — formowanie procesu, czyli gromadzenie zasobów niezbędnych do rozpoczęcia wykonywania procesu, z wyjątkiem procesora (kwantu czasu procesora), a po zakończeniu formowania oczekiwanie na przyjęcie do kolejki procesów gotowych.
- **Wykonywany** — wykonywanie instrukcji programu danego procesu i wynikająca z ich wykonywania zmiana stanu odpowiednich zasobów systemu.
- **Oczekujący** — zatrzymanie wykonywania instrukcji programu danego procesu ze względu na potrzebę przydziału dodatkowych zasobów, konieczność otrzymania danych lub osiągnięcia odpowiedniego stanu przez otoczenie procesu (np. urządzenia zewnętrzne lub inne procesy).
- **Gotowy** — oczekiwanie na przydział kwantu czasu procesora (dostępność wszystkich niezbędnych zasobów z wyjątkiem procesora).
- **Zakończony** — zakończenie wykonywania programu, zwolnienie większości zasobów i oczekiwanie na możliwość przekazania informacji o zakończeniu innym procesom lub jądro systemu operacyjnego. Pozostawianie procesu z stanie zakończony (w systemach uniksopodobnych zwany zombi) spowodowane jest przetrzymywaniem pewnych informacji o procesie po jego zakończeniu (np. statusu zakończenia). Całkowite usunięcie procesu mogłoby oznaczać zwolnienie pamięci i utratę tych informacji.

86.5 Cykl zmiany stanów procesu

- Gotowy → Wykonywany
Decyzja modułu szeregującego procesy (planista przydziału procesora), która oparta jest na priorytetach procesów. Jeżeli w systemie jest jedna jednostka przetwarzająca (procesor), to w stanie wykonywany może być tylko jeden proces, podczas gdy pozostałe procesy znajdują się w innych stanach (w szczególności w stanie gotowy).
- Wykonywany → Gotowy
Oznacza wywłaszczenie procesu z procesora. Wywłaszczenie może być następstwem:

1. Upływu kwantu czasu.

W systemach z **podziałem czasu** proces otrzymuje tylko kwant czasu na wykonanie kolejnych instrukcji. Upływ kwantu czasu odmierzany jest przez przerwanie zegarowe, a po stwierdzeniu wyczerpania kwantu czasu następuje przełączenie kontekstu i kolejny kwant czasu otrzymuje inny proces (rotacyjny algorytm planowania przydziału procesora).

2. Pojawienia się procesu gotowego z wyższym priorytetem.

W systemie z **dynamicznymi priorytetami** przerwanie zegarowe lub inne zdarzenie obsługiwane przez jądro wyznacza momenty czasu, w którym przeliczane są priorytety procesów. Jeśli stosowane jest wyłączenieowe podejście do planowania przydziału procesora, oparte na priorytecie, proces o najwyższym priorytecie otrzymuje procesor. Więcej szczegółów zostanie omówionych w następnym module.

86.6 Szeregowanie procesów

Szeregowanie (planowanie) sprowadza się do wyboru jednego z procesów (lub wątków) gotowych i przekazaniu mu procesora.

Planowanie opiera się na trzech elementach, z których dwa zasadnicze to tryb decyzji oraz funkcja priorytetu.

- **Funkcja priorytetu** — funkcja wyznaczająca aktualny priorytet procesu na podstawie parametrów procesu i stanu systemu.
- **Tryb decyzji** — określa okoliczności, w których oceniane i porównywane są priorytety procesów oraz dokonywany jest wybór procesu do wykonania.
- **Reguła arbitrażu** — reguła rozstrzygania konfliktów w dostępie do procesora w przypadku procesów o tym samym priorytecie.

86.6.1 Funkcja priorytetu

Funkcja priorytetu jest zbiorem wytycznych dla planisty. Zadaniem planisty jest po prostu realizacja funkcji priorytetu, ewentualnie rozwiązywanie problemów, wynikających z takiej samej wartości priorytetów dla więcej niż jednego procesu.

Wartość funkcji priorytetu jest jakąś liczbą, przy czym w niektórych systemach większa wartość tej liczby oznacza wyższy priorytet (np. w Windows), w innych wyższy priorytet to mniejsza wartość funkcji (np. w tradycyjnym systemie UNIX).

Funkcja priorytetu uwzględnia przede wszystkim stan procesu. Może też uwzględniać stan systemu, np. stan zasobów pamięci. Ubocznym skutkiem wyznaczania wartości priorytetu jest wskazanie następnego procesu do wykonania, dlatego czasami używa się określenia funkcja wyboru.

- Argumentami funkcji priorytetu są wybrane składowe stanu procesu oraz stanu systemu (czas oczekiwania, czas obsługi, czas przebywania w systemie, priorytet zewnętrzny)
- Priorytet procesu w danej chwili jest wartością wynikową funkcji priorytetu dla bieżących wartości parametrów stanu danego procesu i aktualnego stanu systemu.

86.6.2 Tryb decyzji

Tryb decyzji jest z zbiorem wytycznych odnośnie uruchamiania ekspedytora. Szczegółowe wytyczne mogą obejmować również wartość funkcji priorytetu, gdyż jedną z okoliczności zmiany przydziału procesora jest wzrost lub spadek priorytetu procesów gotowych lub wykonywanego.

Tryb decyzji można sklasyfikować jako wywłaszczeniowy lub niewywłaszczeniowy. W schemacie niewywłaszczeniowym procesor traktowany jest jako zasób niewywłaszczalny. Nie można go odebrać procesowi, ale proces może się go zrzec dobrowolnie (służy do tego np. funkcja `yield`, dostępna w niektórych systemach) lub się zakończyć. Rezygnacja z procesora jest też uboczną konsekwencją wejścia w stan oczekiwania (np. w wyniku zażądania operacji wejścia-wyjścia).

W schemacie wywłaszczeniowym, w ramach kontrolnego przekazania sterowania do jądra systemu operacyjnego może zostać podjęta decyzja o przełączeniu kontekstu pomimo, że wykonywany proces nie zażądał żadnej usługi, oznaczającej rezygnację z procesora. Przechodzi on wówczas do stanu gotowy, a rozpoczyna się wykonywanie innego procesu.

86.6.3 Reguła arbitrażu

Arbitraż losowy przy małej zmienności priorytetów mógłby prowadzić do głodzenia procesów.

Arbitraż cykliczny jest trudny w realizacji przy zmiennych priorytetach. Można go z powodzeniem realizować przy stałych priorytetach przy odpowiednim wsparciu ze strony struktur danych.

Arbitraż chronologiczny wydaje się być najbardziej sprawiedliwym, ale wymaga utrzymania odpowiednich atrybutów procesów lub użycia pewnych struktur danych do powiązania procesów w celu ustalenia kolejności przyjmowania ich do systemu.

Losowo — możliwe w przypadku, gdy liczba procesów o tym samym priorytecie jest niewielka
Cyklicznie — cykliczny przydział procesora kolejnym procesom
Chronologicznie — w kolejności przyjmowania procesów do systemu (w kolejności FIFO)

86.7 Algorytmy planowania przydziału procesora

Wyróżnia się między innymi następujące algorytmy planowania przydziału procesora:

- FCFS (First Come First Served) - Pierwszy zgłoszony, pierwszy obsługony
- LCFS (Last Come First Served) - Ostatni zgłoszony, pierwszy obsługony
- SJF (Shortest Job First) - Najpierw najkrótsze zadanie

86.7.1 FCFS

Algorytm FCFS jest naturalnym algorytmem w systemach obsługi masowej, takich jak kasy sklepowe, kasy biletowe, banki, urzędy itp. Procesy otrzymują procesor w kolejności, w jakiej zgłosiły się do systemu. Specyficzną cechą w systemach komputerowych, nie zawsze mającą odpowiednik w systemach masowej obsługi, jest możliwość oddania procesora innemu procesowi na czas oczekiwania na przydział dodatkowego zasobu, np. wykonania operacji wejścia-wyjścia.

86.7.2 LCFS

Algorytm LCFS obsługuje procesy w kolejności odwrotnej do kolejności zgłoszeń. Algorytm nie wywłaszcza procesów, więc nowo przychodzący proces jest pierwszy w kolejce i czeka na zwolnienie procesora przez bieżąco wykonywany proces. Algorytm ten wymieniany jest dla porządku, nie ma natomiast praktycznego zastosowania we współczesnych koncepcjach planowania przydziału procesora.

86.7.3 SJF

Algorytm SJF preferuje procesy, które mają najmniejsze wymagania odnośnie czasu procesora, potrzebnego na realizację przetwarzania. W kontekście systemów komputerowych preferencje te należałoby raczej określić jako najpierw zadanie z najkrótszą

następną fazą procesora, gdyż po odwołaniu do jądra w celu przydziału dodatkowych zasobów nastąpi zwolnienie procesora. Algorytm ten ma sens również w systemach masowej obsługi (często w kolejce do kasy w sklepie przepuszczamy kogoś, kto ma niewiele produktów w koszyku). Problemem praktycznej stosowalności jest określenie przyszłego zapotrzebowania na procesor.

Zakładając, że procesy kolejgowane są zgodnie z kolejnością zgłoszeń, w algorytmie FCFS wybierany jest proces z czoła kolejki, w algorytmie LCFS wybierany jest proces z ogona (końca) kolejki, a w algorytmie SJF kolejkę należy przejrzeć w celu znalezienia procesu, który najmniej zaabsorbują procesor.

87 Muteks, semafor, monitor jako narzędzia synchronizacji procesów.

87.1 Sekcja krytyczna

Wielozadaniowość systemu oprócz oczywistych korzyści stwarza również pewne problemy. W środowisku, w którym wiele wątków działa równolegle i niezależnie od siebie, potrzebne są mechanizmy, które pozwolą w pewien sposób zdeterminować ich kolejność, oraz dostęp do tzw. sekcji krytycznych. Sekcje te są to fragmenty kodu, które nie mogą zostać wykonane „w tym samym czasie” przez kilka wątków.

87.2 Semafor

Zmienna całkowita, która z logicznego punktu widzenia przyjmuje wartości nieujemne (semafor ogólny) lub wartości logiczne (semafor binarny). Zmienna semaforowa musi mieć nadaną początkową wartość (oczywiście nieujemną).

- Semafor binarny — zmienna semaforowa przyjmuje tylko wartości true (stan podniesienia, otwarcia) lub false (stan opuszczenia, zamknięcia).
- Semafor ogólny (zliczający) — zmienna semaforowa przyjmuje wartości całkowite nieujemne, a jej bieżąca wartość jest zmniejszana lub zwiększana o 1 w wyniku wykonania odpowiednio operacji opuszczenia lub podniesienia semafora.

Po nadaniu początkowej wartości zmiennej semaforowej można na niej wykonywać tylko dwa rodzaje operacji:

- P — opuszczanie semafora (zmniejszenie wartości)
- V - podnoszenie semafora (zwiększenie wartości)

Wykonując operację semaforową, proces może zastać zablokowany (przejsć w stan oczekiwania). Typowym przypadkiem jest blokowanie w operacji opuszczania semafora. Operacja opuszczania nie zakończy się do czasu, aż wartość zmiennej semaforowej będzie na tyle duża (być może zostanie zwiększona w międzyczasie), że zmniejszenie jej wartości w wyniku tej operacji nie spowoduje przyjęcia wartości ujemnej. W przypadku semaforów dwustronnie ograniczonych blokowanie może wystąpić również w przypadku podnoszenia semafora.

87.2.1 Implementacja semafora ogólnego

```
procedure P(var s: Semaphore)
begin
  while s=0 do nic;
  s := s - 1;
end;

procedure V(var s: Semaphore)
begin
  s := s+1;
end;
```

87.3 Muteks (Zamek)

Muteksy (ang. mutual exclusion semaphores), stanowią szczególny rodzaj semaforów binarnych. Muteks może być zablokowany (ma wartość 1) lub odblokowany (ma wartość 0 - nie zablokowany). Muteksy korzystają z operacji atomowych, tylko jeden wątek może być w posiadaniu muteksa i tylko on może go zwolnić.

Operacje:

- lock (mutex_lock)
- zajęcie muteksu jeśli jest wolny
- unlock (mutex_unlock)
- ustawienie zamka na wolny, jeśli kolejka oczekujących wątków jest pusta
- wybranie wątku z niepustej kolejki wątków oczekujących i ustawienie jego stanu na gotowy
- trylock (mutex_trylock)
- zajęcie zamka lub kontynuacja przetwarzania

Cechy

- Cecha posiadania - jeżeli zadanie zablokuje muteks (nadanie wartości 1) to tylko ono może ten muteks odblokować (nadać wartość 0)
- Rekurencyjne blokowanie - określa ile razy zadanie, które posiada dany muteks wykonało na nim operacje blokowania (zapobiega to zakleszczeniu)

87.4 Monitor (Zmienna warunkowa)

Są to zmienne i działające na nich procedury zebrane w jednym module. Monitory definiowane są jako klasy. Dostęp do zmiennych monitora jest możliwy tylko za pomocą procedur monitora. W danej chwili tylko jeden proces może wykonywać procedury monitora. Gdy inny proces wywoła procedurę monitora proces ten będzie zablokowany do chwili opuszczenia monitora przez znajdujący się tam już proces. Istnieje możliwość wstrzymania i wznowienia procedur monitora za pomocą zmiennych warunkowych. Procesy oczekujące na wejście do monitora są zorganizowane w kolejkę FIFO.

- wait
 - powoduje wstrzymanie procesu i wstawienie go na koniec kolejki
- signal
 - powoduje wznowienie pierwszego procesu z kolejki, o ile kolejka nie jest pusta
- signal
 - powoduje wznowienie pierwszego procesu z kolejki, o ile kolejka nie jest pusta

87.4.1 Ogólny schemat definicji monitora

```
type nazwa_monitora = monitor
-> deklaracje zmiennych
procedure entry proc_1(...);
begin ... end;
...
procedure entry proc_n(...);
begin ... end;
begin
kod inicjalizujący
end
```

88 Pamięć wirtualna i mechanizm stronicowania.

88.1 Stronicowanie

- Arbitralny podział pamięci fizycznej na ramki, w które ładowane są odpowiednie strony obrazu procesu.
- Podział logicznej przestrzeni adresowej na strony o takim samym rozmiarze, jak ramki w pamięci fizycznej.
- Zalety:
 - brak problemu fragmentacji zewnętrznej
 - wspomaganie dla współdzielenia i ochrony pamięci
- Wady:
 - narzut czasowy przy transformacji adresu
 - narzut pamięciowy (na potrzeby tablicy stron)
 - fragmentacja wewnętrzna (niewielka)

Pamięć wirtualna jest organizacją zasobów pamięci, zrealizowaną w oparciu o tzw. przestrzeń wymiany w pamięci drugiego rzędu (na dysku). Pamięć operacyjna (fizyczna) jest dla tych zasobów tylko pewnym oknem, przechowującym część zawartości na potrzeby bieżącego przetwarzania. Stosowanie pamięci wirtualnej umożliwia bardziej racjonalne wykorzystanie pamięci operacyjnej, gdyż programy tworzone są często z nadmiarem w stosunku do typowych potrzeb.

Podstawą funkcjonowania pamięci wirtualnej jest mechanizm stronicowania na żądanie, od omówienia którego rozpoczyna się wykład. Działanie mechanizmu oparte jest na stronicowaniu i polega na sprowadzaniu do pamięci operacyjnej stron adresowanych przez procesor.

Realizacja pamięci wirtualnej oprócz wsparcia sprzętowego wymaga rozwiązania dwóch zasadniczych problemów na poziomie systemu operacyjnego:

- problemu wyboru ramki ofiary do wymiany strony, jeśli zajdzie potrzeba wymiany
- problemu wznawiania rozkazów, którego rozwiązanie sprowadza się do zapewnienia dostępności odpowiednio dużej liczby ramek dla procesu

Rozwiązanie problem wyboru ofiary bazuje na przesłankach o charakterze losowym i związane jest ściśle z algorytmem wymiany.

88.2 Mechanizm stronicowania na żądanie

Działanie mechanizmu: strony są sprowadzane do pamięci tylko wówczas, gdy jest to konieczne, czyli wówczas gdy następuje odniesienie do komórki o adresie, znajdującym się na tej stronie.

Stronicowanie na żądanie związane jest przede wszystkim z wymianą stron pomiędzy pamięcią pierwszego rzędu (operacyjną, fizyczną) i drugiego rzędu (masową, dyskową).

88.2.1 Problemy zastępowania stron

- Problem wyboru ofiary — niewłaściwy wybór ramki-ofiary powoduje wzrost kosztu wymiany. W skrajnym przypadku może dojść do zjawiska migotania, w przypadku którego często dochodzi do wystąpienia odniesienia do właśnie usuniętej strony.
- Problem wznowiania rozkazów — w przypadku wielokrotnego odniesienia do pamięci w jednym cyklu rozkazowym należy zapewnić, że wszystkie adresowane strony są jednocześnie dostępne w ramach w pamięci fizycznej.

88.2.2 Problem wyboru ofiary

- Zakładając, że przyszły ciąg odniesień do pamięci nie jest znany, na podstawie historii odniesień należy wybrać taką ramkę, do której prawdopodobieństwo odniesienia w przyszłości jest małe.
- Podstawowa własność programów, na podstawie której można szacować takie prawdopodobieństwo, nazywana jest lokalnością.

88.2.3 Klasyfikacja algorytmów wymiany z względu na okoliczności sprowadzania i usuwania stron

- Algorytmy wymiany na żądanie
 - sprowadzenia odbywa się na żądanie — strona sprowadzana jest dopiero wówczas, gdy następuje odniesienie do niej i nie ma jej w pamięci
 - usuwanie odbywa się na żądanie — strona usuwana jest wówczas, gdy konieczne jest sprowadzenie innej strony w wyniku odniesienia i nie ma wolnej ramki

- Algorytmy wymiany ze sprowadzaniem na żądanie - tylko sprowadzanie dobywa się na żądanie
- Algorytmy wstępnego sprowadzania - sprowadzana jest strona żądana, a wraz z nią inne strony

88.2.4 Klasyfikacja algorytmów wymiany ze względu na sposób zastępowania stron

- Zastępowanie lokalne (ang. local replacement) — algorytm wymiany zastępuje tylko strony w ramach przydzielonych procesowi, który spowodował błąd strony.
- Zastępowanie globalne (ang. global replacement) — algorytm wymiany zastępuje strony znajdujące się w dostępnej puli ramek w całym systemie (w szczególności zatem usuwa strony innych procesów).

88.2.5 Klasyfikacja algorytmów wymiany ze względu na przydział ramek dla procesów

- Przydział statyczny — liczba ramek przydzielonych procesowi jest ustalona i nie ulega zmianie w trakcie przetwarzania.
- Przydział dynamiczny — liczba ramek przydzielonych procesowi może się zmienić w trakcie przetwarzania.

88.2.6 Dobór liczby ramek

- Minimalna liczba ramek — zdefiniowana przez architekturę komputera (zależna od maksymalnej liczby komórek adresowanych przez jeden rozkaz).
- Liczba ramek przydzielona dla procesu
 - podział równomierny (ang. equal allocation)
 - podział proporcjonalny (ang. Proportional allocation)
 - przydział zależny od priorytetu procesu

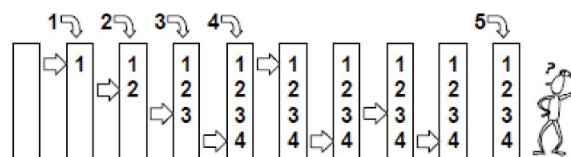
88.2.7 Algorytmy wymiany na żądanie

- MIN — zastępowana jest strona, która najdłużej nie będzie używana (optymalny w tej klasie)
- FIFO (ang. First In First Out) — zastępowana jest strona najstarsza (najwcześniej sprowadzona)
- LIFO (ang. Last In First Out) — zastępowana jest strona najmłodsza (najpóźniej sprowadzona)
- LRU (ang. Least Recently Used) — zastępowana jest najdawniej użyta strona (najdłużej nie używana)
- LFU (ang. Least Frequently Used) — zastępowana jest najrzadziej używana strona
- MFU (ang. Most Frequently Used) — zastępowana jest najczęściej używana strona

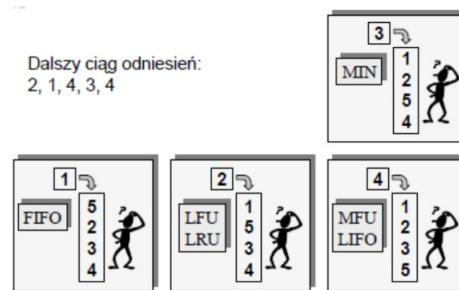
88.2.8 Przykład działania algorytmów wymiany na żądanie

- W systemie pamięci wirtualnej są 4 ramki.
- Wszystkie ramki są początkowo puste
- W systemie pojawiają się następujące odniesień (odwołań) do stron: 1, 2, 3, 4, 1, 4, 3, 4, 5, 2, 1, 4, 3, 4

Przedstawiony przykład pokazuje przebieg odniesień do stron o numerach od 1 do 5 w obrębie 4 początkowo pustych ramek. Pierwsze 4 odniesienia powodują błąd strony, ale nie występuje problem zastępowania, gdyż dostępne są wolne ramki. Dopiero odniesienie do strony nr 5 wymaga zwolnienia jakiejś ramki. Wybór strony do usunięcia zależy od algorytmu wymiany.



Na kolejnych rysunkach pokazany jest stan poszczególnych ramek, zależnie od zastosowanego algorytmu. Przy okazji pokazane jest odniesienie do pamięci, które spowoduje kolejny błąd strony i tym samym problem zastępowania. Usunięcie tej samej strony w przypadku algorytmów LFU i LRU oraz MFU i LIFO jest przypadkowe i kolejna wymiana w przypadku tych algorytmów może wykazać różnice. Warto też przy tej okazji zwrócić uwagę, że dla algorytmów licznikowych wybór może być niejednoznaczny, gdyż liczba odniesień do różnych stron może być taka sama. Jako kryterium rozstrzygnięcia można przyjąć czas sprowadzenia, czyli strony z taką samą wartością licznika usuwane są w kolejności FIFO. Przy założeniu, że strony sprowadzane są pojedynczo nie ma tego problemu w algorytmach opartych na kolejności zdarzeń w czasie, czyli MIN, FIFO, LIFO i LRU.



89 Systemy plikowe - organizacja fizyczna i logiczna (na przykładzie wybranego systemu uniksopodobnego).

89.1 Organizacja fizyczna

- Przydział miejsca na dysku
 - Przydział ciągły (ang. contiguous allocation) — cały plik zajmuje ciąg kolejnych bloków
 - Przydział listowy (łańcuchowy, ang. linked allocation, chained allocation) — bloki pliku tworzą listę powiązaną
 - Przydział indeksowy (ang. indexed allocation) — bloki z danymi wskazywane są przez bloki indeksowe, które mogą być zorganizowane w:
 - * schemat listowy
 - * schemat wielopoziomowy
 - * schemat kombinowany
- Zarządzanie wolną przestrzenią
 - Wektor bitowy — każdy bit odpowiada jednemu blokowi, wartość 1 oznacza wolny blok.
 - Lista powiązana — każdy wolny blok zawiera indeks następnego wolnego bloku.
 - Grupowanie — niektóre wolne bloki wypełnione są w całości indeksami innych wolnych bloków, ostatni indeks wskazuje na kolejny blok wypełniony w całości indeksami.
 - Zliczanie — wykaz wolnych bloków obejmuje indeks pierwszego wolnego bloku oraz liczbę wolnych bloków znajdujących się za nim, stanowiących ciągły obszar.
- Implementacja katalogu – lista liniowa, tablica haszowa, struktura indeksowa

89.2 organizacja logiczna

89.2.1 Zadania systemu operacyjnego

Zadaniem systemu operacyjnego w odniesieniu do plików jest zapewnienie odwzorowania pomiędzy abstrakcyjnym obrazem informacji a jego reprezentacją na urządzeniu fizycznym.

- identyfikacja pliku (hierarchiczna struktura katalogów)
- udostępnienie interfejsu operacji plikowych (API)
- realizacja operacji dostępu do plików i katalogów z zapewnieniem bezpieczeństwa (synchronizacja i autoryzacja dostępu), spójności i efektywności

89.2.2 Atrybuty pliku

- Nazwa — ciąg znaków służących użytkownikowi do identyfikacji pliku
- Typ — informacja służąca do rozpoznania rodzaju zawartości pliku i tym samym sposobu interpretacji
- Lokalizacja — informacja służąca do odnalezienia pliku w systemie komputerowym (urządzenie i położenie pliku w tym urządzeniu)
- Rozmiar — bieżący rozmiar pliku w ustalonych jednostkach (bajtach, słowach, blokach itp.)
- Ochrona — informacje umożliwiające kontrolę dostępu
- Czasy dostępu — daty i czasy wykonywania pewnych operacji na pliku, typu odczyt, modyfikacja, utworzenie

89.2.3 Struktura pliku

Logiczna struktura pliku określa powiązanie informacji wewnątrz pliku (właściwym byłoby zatem określenie struktury informacji). Jako przykład można sobie wyobrazić plik z tabelą bazy danych, w którym:

- pierwsze 4 bajty określają liczbę rekordów (krotek),
- następne 2 — długość rekordu w bajtach,
- kolejny bajt — długość nagłówka z definicją atrybutów,

- reszta przeznaczona jest na dane, przy czym każdy rekord ma dodatkowo 2 bajty kontrolne.

Z długości nagłówka wynika, gdzie rozpoczynają się dane, z wielkości rekordu oraz informacji o strukturze, mówiącej o 2 dodatkowych bajtach dla każdego rekordu, można wyliczyć początek rekordu o podanym numerze itd.

89.3 UNIX

89.3.1 Informacje ogólne

- Z każdym plikiem związany jest i-węzeł, który przechowuje wszystkie atrybuty pliku z wyjątkiem nazwy.
- Nazwa znajduje się w katalogu obok numeru i-węzła danego pliku.
- Katalogi tworzą strukturę wielopoziomową (katalog zawiera wpis specyfikujący inny katalog).
- Dane (zawartość pliku) znajdują się w blokach (jednostkach alokacji) o ustalonym rozmiarze.
- Bloki danych identyfikowane są za pośrednictwem indeksu kombinowanego.
- Wolne bloki zorganizowane są zgodnie z zasadą grupowania.

Podstawą implementacji jest i-węzeł, grupujący wszystkie atrybuty pliku z wyjątkiem nazwy. I-węzły tworzą tablicę, której rozmiar limituje liczbę plików w systemie. I-węzeł pośrednio lub bezpośrednio identyfikuje bloki danych pliku. Wolna przestrzeń jest identyfikowana przez grupowanie, przy czym pierwszy węzeł (blok) z indeksami wolnych bloków zlokalizowany jest w całości w bloku nadrzędnym. W pewnych odmianach stosowany jest wektor bitowy. Wektor bitowy w tych odmianach używany jest również do identyfikacji wolnych i-węzłów.

89.3.2 Format partycji

Dwa główne elementy partycji stanowią: tablica i-węzłów oraz bloki danych. Liczba i-węzłów jest ustalona, podobnie jak liczba bloków danych. Ograniczenia na rozmiar katalogu wynikają wyłącznie z dostępności bloków danych, gdyż katalog zajmuje przestrzeń dyskową na takich samych zasadach, jak plik. Jego zawartość jest jednak interpretowana przez system zgodnie z zasadami budowy katalogów w danej implementacji. Jeden, ustalony i-węzeł opisuje korzeń drzewa katalogów.

89.3.3 Fizyczna struktura pliku

Z każdym plikiem związany jest 1 i-węzeł. Zawiera on podstawowe atrybuty pliku w modelu uniksowym, czyli:

- identyfikator właściciela i grupy,
- typ pliku — plik zwykły, katalog, dowiązanie symboliczne, łącze nazwane, urządzenie znakowe, urządzenie blokowe, gniazdo,
- prawa dostępu — tradycyjne rwx dla właściciela, grupy i pozostałych,
- czasy dostępu — czas modyfikacji pliku, czas modyfikacji i-węzła, czas dostępu,
- licznik dowiązań — liczba różnych nazw, pod jakimi występuje plik w systemie,
- rozmiar w bajtach.

Pozostała część i-węzła wypełniona jest indeksami bloków z danymi. Część indeksów w i-węźle (10 – 12, zależnie od odmiany) wskazuje bezpośrednio bloki z danymi. Ponadto są jeszcze 3 indeksy, z których jeden wskazuje blok indeksowy 1- poziomowy, jeden — blok indeksu 2-poziomowego, a jeden — blok indeksu 3-poziomowego.

89.3.4 Struktura wpisu katalogowego

Katalog składa się z wpisów, kojarzących nazwę z i-węzłem. W tradycyjnym podejściu nazwa ograniczona była do 14 znaków, ale oczywiście w toku rozwoju systemów uniksowych wielkość tę zwiększano i współcześnie dopuszcza się nawet 256 znaków. W katalogu istnieją też wpisy specjalne o nazwie . (kropka) i .. (dwie kropki), skojarzone z numerem i-węzła odpowiednio katalogu bieżącego i nadrzędnego. Wyjątkiem w tym zakresie jest korzeń drzewa katalogów, który nie ma nadkatalogu.

90 Model ISO OSI. Przykłady protokołów w poszczególnych warstwach.

Model OSI (Open Systems Interconnection) utworzony przez Międzynarodową Organizację Normalizacyjną ISO stanowi **model referencyjny**.

Nr warstwy OSI	Nazwa warstwy OSI	Przykładowe protokoły
7	Aplikacji	HTTP, HTTPS, DNS, Telnet
6	Prezentacji	SSL, TLS
5	Sesji	NetBios, SAP
4	Transportu	TCP, UDP
3	Sieci	IPv4, IPv6, IGMP
2	Łączy danych	ARP
1	Fizyczna	Ethernet

Warstwa aplikacji - interfejs między aplikacjami a usługami sieci.

- **HTTP/HTTPS** (Hypertext Transfer Protocol (Secure)) - protokół przesyłania dokumentów hipertekstowych.
- **DNS** (Domain Name System) - hierarchiczny rozproszony system nazw sieciowych, który odpowiada na zapytania o nazwy domen.
- **Telnet** - protokół do obsługi odległego terminala w architekturze klient-serwer.

Warstwa prezentacji - kompresja, kodowanie i translacja między niezgodnymi schematami kodowania oraz szyfrowanie.

- **SSL** (Secure Socket Layer) - protokół szyfrowania zapewniający poufność i integralność danych. Otwarty protokół umożliwiający autoryzację serwerów i klientów, wykorzystuje szyfrowanie symetryczne z kluczem publicznym, stosuje sumy kontrolne dla zapewnienia integralności danych.
- **TSL** (Transport Layer Security) - jak wyżej. Ustalenie klucza symetrycznego szyfrowaniem niesymetrycznym, symetryczne szyfrowanie danych.

Warstwa sesji - zarządzanie przebiegiem komunikacji podczas połączenia między komputerami.

- **NetBios** (Network Basic Input/Output System) - protokół sieciowy zapewniający podstawowy interfejs łączenia aplikacji z innymi aplikacjami w sieci lokalnej.
- **SAP** (Session Announcement Protocol) - protokół wykorzystywany przez usługi powodujące tzw. “burze rozgłoszeń” (radio, telewizja internetowa).

Warstwa transportu - kontrola błędów i przepływu danych poza lokalnymi segmentami LAN, protokoły zapewniające komunikację procesów uruchomionych na odległych komputerach, protokoły TCP, UDP.

- **TCP** (Transmission Control Protocol) - połączeniowy, niezawodny, strumieniowy protokół komunikacyjny stosowany do przesyłania danych pomiędzy procesami na różnych maszynach.
- **UDP** (User Datagram Protocol) - prosty bezpołączeniowy protokół komunikacyjny, nie zapewnia niezawodności.

Warstwa sieci - określenie trasy przesyłania danych między komputerami poza lokalnym segmentem sieci LAN, protokoły trasowane takie jak IP (ze stosu protokołów TCP/IP).

- **IP** (Internet Protocol) - protokół internetowy będący zbiorem kroków postępowania wykonywanych automatycznie przez urządzenie w celu nawiązania łączności.
- **IGMP** (Internet Group Management Protocol) - protokół internetowy służący do zarządzania grupami multicastowymi w sieciach opartych na protokole IP.

Warstwa łącza danych – grupowanie danych wejściowych (z warstwy fizycznej) w bloki zwane **ramkami** danych („jednostki danych usług warstwy fizycznej”), mechanizmy kontroli poprawności transmisji (FCS).

- **ARP** (Address Resolution Protocol) - protokół sieciowy umożliwiający mapowanie logicznych adresów warstwy sieciowej na fizyczne adresy warstwy łącza danych (np. mapowanie IP na MAC).

Warstwa fizyczna - standard połączenia fizycznego, charakterystyki wydajnościowe nośników. Same media transmisyjne pozostają poza dziedziną jej zainteresowania (czasem określane są terminem warstwa zerowa).

- **Ethernet** - technologia zawierająca specyfikację przewodów, sygnałów, ramek.
- **DSL** (Digital Subscriber Line) - technologia cyfrowego szerokopasmowego dostępu do Internetu.

91 Adresowanie w protokołach IPv4 i IPv6.

91.1 Adresowanie w IPv4.

Adres IP jest przypisywany do karty sieciowej, nie do komputera. Składają się z czterech części po 8 bajtów

Istnieją **trzy typy adresów IPv4**:

- **Adresy jednostkowe** (unicast) – pojedynczy interfejs sieciowy (one-to-one).
- **Adresy rozgłoszeniowe** (broadcast) – wszystkie węzły w tym samym segmencie sieci (one-to-everyone).
- **Adresy grupowe** (multicast) – jeden lub wiele komputerów w jednej lub w różnych segmentach sieci (one-to-many).

W **adresie IP** zapisanym binarnie można wyróżnić **dwie części**:

- **Identyfikator sieci** (Network ID) - pewna liczba bitów z lewej strony adresu.
- **Identyfikator hosta** (Host ID) - pozostałe bity.

Granica między identyfikatorem sieci a identyfikatorem hosta może być wyznaczona przez tzw. **maskę sieci**.

Adres IP, który zawiera **same zera** w części hosta jest traktowany jako **adres sieci**. **Adresy rozgłoszenia do sieci lub podsieci mają jedynki tylko w części hosta**.

Adres ograniczonego rozgłoszenia - 255.255.255.255- adres rozgłoszenia w danym segmencie sieci ograniczonym routerami.

91.1.1 Adresowanie oparte na klasach

Pierwszy bajt adresu determinuje do jakiej klasy należy sieć.

Klasa	Adres sieci	Adresy	Zakres 1-go bajtu	Najstarsze bity
A	w.0.0.0	1.0.0.0 - 126.0.0.0	1 – 126	0
B	w.x.0.0	128.0.0.0 - 191.255.0.0	128 – 191	10
C	w.x.y.0	192.0.0.0 - 223.255.255.0	192 – 223	110
D	nie dotyczy	nie dotyczy	224 – 239	1110
E	nie dotyczy	nie dotyczy	240 – 255	11110

Klasa	Ilość sieci	# Hostów	ID sieci	ID hosta	pierwszy	ostatni
A	126	$2^{24} - 2$	1 bajt	3 bajty	w.0.0.1	w.255.255.254
B	$(191 - 128 + 1) * 256$	$2^{16} - 2$	2 bajty	2 bajty	w.x.0.1	w.x.255.254
C	$(192 - 223 + 1) * 256^2$	$2^8 - 2$	3 bajty	1 bajt	w.x.z.1	w.x.z.254

- **Adresy klasy D** - przeznaczone są do transmisji grupowych.
- **Adresy klasy E** - zarezerwowane (nie wykorzystywane normalnie do transmisji pakietów).
- **Adresy pętli zwrotnej** (loopback) - postaci 127.x.y.z (na ogół 127.0.0.1). Cały ruch przesyłany na ten adres nie wychodzi z komputera.

91.1.2 Adresowanie bezklasowe

Dzielenie na podsieci z **użyciem dowolnej liczby jedynek**. Do określenia sieci należy podać adres sieci oraz maskę. Obecnie w Internecie powszechnie jest wykorzystywane adresowanie bezklasowe.

- Protokół **warstwy trzeciej** modelu ISO OSI.
- Oprogramowanie implementujące protokół IP jest odpowiedzialne za:
 - **adresowanie IP**,
 - **tworzenie datagramów IP** (pakietów)
 - uczestniczenie w **kierowaniu ich** w sieci z punktu początkowego do punktu docelowego.
- Realizuje usługę **zawodną**. Jeśli komunikacja powinna zawierać mechanizmy niezawodności, to muszą one być dostarczone przez protokoły warstwy wyższej.
- Datagram IP składa się z nagłówka (header) i bloku danych (payload).
 - **Nagłówek** dzięki informacjom w nim zawartym umożliwia obsługę routowania, identyfikację bloku danych, określenie rozmiaru nagłówka i datagramu oraz obsługę fragmentacji. W nagłówku mogą się znaleźć również tzw. opcje rozszerzające.
 - **Blok danych**.

91.2 Adresowanie w IPv6.

- **osiem 16 bitowych sekcji w zapisie szesnastkowym**, oddzielonych dwukropkami,
np.: fe80:0000:0000:0000:0202:b3ff:fe1e:8329
- długi ciąg zer może być **raz** połączony i przedstawiony jako dwa dwukropki "::
"
- w środowisku z węzłami IPv4 oraz IPv6 często **w adresie IPv6 umieszcza się również IPv4**. Są dwa typy takich adresów:
 - **Typ 1: IPv4-Compatible IPv6 Address** - 96 bitów zero + adres IPv4
 - **Typ 2: IPv4-Mapped IPv6 Address** (zalecany) - 0000.....0000
ffff + IPv4 address
- Zamiast identyfikatora sieci (jak było w IPv4), w IPv6 używa się tzw. **prefiksu**. Prefiks to **pewna liczba bitów** adresu licząc z lewej strony. Prefiks **określa** zatem **sieć**. Prefiks podaje się standardowo jako pewną liczbę sekcji adresu.

91.2.1 Typy adresów

- **Adres niewyspecyfikowany** - ::/128
- **Pętla zwrotna (Loopback)** - ::1/128
- **Multicast** - ff00::/8 (1111 1111)
- **Link-local unicast** - fe80::/10 (1111 1110 10)
- **Global unicast**

91.2.2 Klasyfikacja adresów.

1. **adresy jednostkowe (unicast)** - może być przypisany do wielu interfejsów w węźle lub wielu komputerów; podzielony na prefiks i identyfikator interfejsu, Typowy prefix - 48 bitów od dostawcy, 16 od podsieci.
2. **adresy pobliskie (anycast)** - z zakresu adresów jednostkowych organizacji, przydzielane do grupy węzłów (np. routerów).

3. **adresy grupowe (multicast)** - zastępują adresy rozgłoszeniowe i grupowe w IPv4, przydzielane do grupy węzłów lecz w przeciwieństwie do anycasta wszystkie węzły odbierają pakiety wysłane do danego adresu - 11111111 + 4 bity flagi + 4 bity zakresu + 112 bitów ID grupy.

91.2.3 Skąd wziąć adres MAC?

- Unicastowy adres IP na multicastowy (Solicited Node Multicast address) - ff02:0:0:0:1:ff__:__, z trzema najmłodszymi bajtami adresu IP
- Multicastowy MAC - 33:33:__:__:__ z czterema najmłodszymi bajtami adresu multicastowego
- Ramka skonstruowana z adresem MAC, z pakietem IP z multicastowym IP w środku, z unicastowym w komunikacie ICMP
- Switche się znają na tyle, że ramka w miarę dochodzi tylko tam gdzie trzeba
- Komputer otrzymujący ramkę sprawdza czy o niego chodzi w komunikacie ICMP, i unicastowo odpowiada

92 Najważniejsze procesy zachodzące w sieci komputerowej od momentu wpisania adresu strony WWW do wyświetlenia strony w przeglądarce (komunikat HTTP, segment TCP, system DNS, pakiet IP, ARP, ramka).

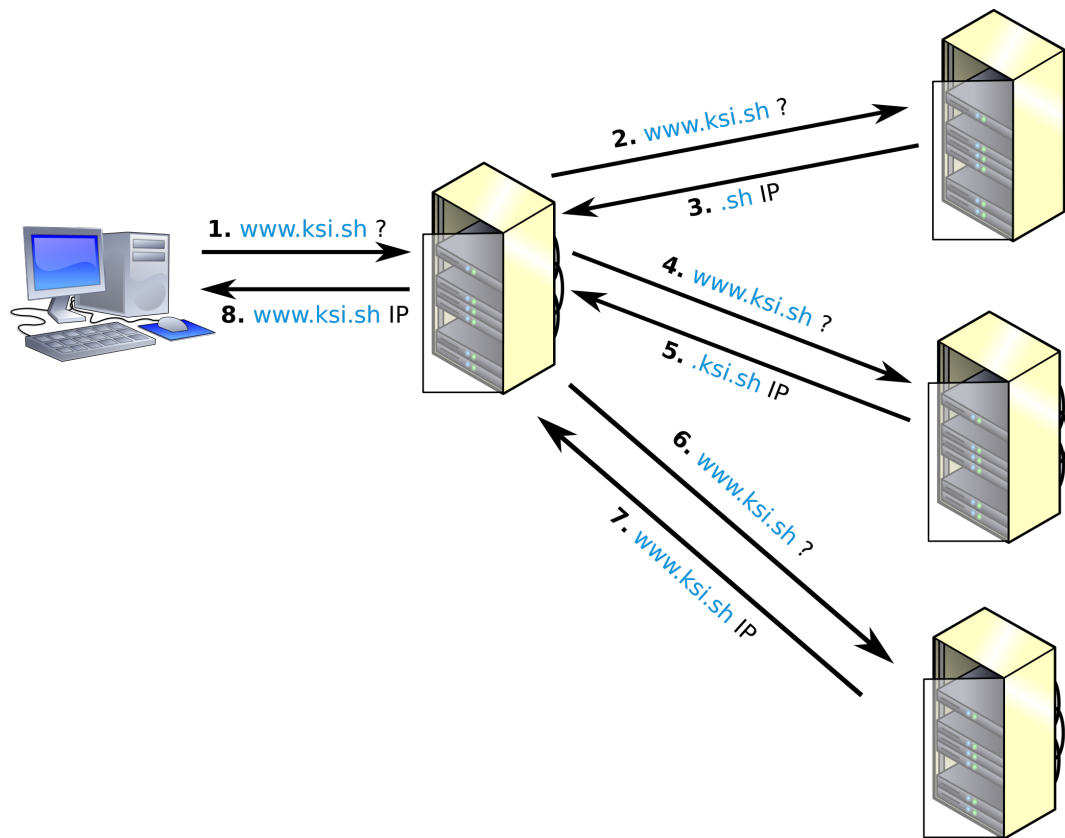
W poniższych rozważaniach zakładamy, że klient wpisał w przeglądarkę internetową adres `www.ksi.sh`. Oto kolejne procesy, które dzieją się od wpisania tego adresu do wyświetlenia strony.

1. Zapytanie DNS

- (a) Przeglądarka internetowa uruchamia odpowiednie wywołanie systemowe odpowiedzialne za rozwiązanie nazwy domenowej na adres IP
- (b) System operacyjny sprawdza, czy żądany adres nie jest zapisany w cache'u
- (c) Jeżeli wpisu nie ma w cache'u wysyła on zapytanie do skonfigurowanego w systemie serwera DNS (najczęściej serwera naszego ISP - Internet Service Provider'a) o adres IP związany z domeną `www.ksi.sh`. Zapytanie to nazywamy **zapytaniem rekurencyjnym** - klient DNS przerzuca odpowiedzialność za znalezienie adresu IP na serwer DNS.
- (d) Powyższy serwer DNS sprawdza, czy sam nie obsługuje żądanej domeny. Jeżeli tak, zwraca on adres IP szukanej domeny bezpośrednio ze swojej bazy.
- (e) Powyższy serwer DNS sprawdza, czy zapytania o tę domenę również nie ma w cache'u
- (f) Jeżeli serwer ten nie znajdzie odpowiedniego wpisu w swoim cache'u, rozpoczyna on serię **zapytań iteracyjnych** do **autorytatywnych** serwerów poszczególnych domen. **Serwer autorytatywny** to taki, który nie obsługuje rekurencyjnych zapytań od resolver'ów DNS (czyli np. zwykłych komputerów), a jedynie zapytania iteracyjne od innych serwerów o konkretne domeny.
- (g) Przykładowy proces wyszukiwania iteracyjnego może wyglądać następująco:
 - i. Serwer DNS naszego ISP pobiera ze swojej konfiguracji adresy IP głównych serwerów DNS. Tych adresów IP jest 13, chociaż tak na

prawdę, fizycznie tych serwerów jest oczywiście więcej - dzięki technice **anycast** wiele fizycznych serwerów jest widocznych pod tym samym adresem IP, a zapytanie na dany adres jest kierowane do najbliższego z nich względem pytającego.

- ii. Serwer DNS naszego ISP wysyła zapytanie o domenę **www.ksi.sh** do jednego z głównych serwerów DNS
 - iii. Główny serwer DNS odpowiada mu, że nie wie jaki jest adres IP należący do **www.ksi.sh**, ale wie jaki serwer obsługuje domenę **sh** (jaki serwer jest serwerem autorytatywnym dla tej domeny) i zwraca on adres tego serwera
 - iv. Serwer DNS naszego ISP wysyła więc zapytanie do serwera autorytatywnego domeny **sh** o adres IP dla domeny **www.ksi.sh**
 - v. Serwer obsługujący domenę **sh** odpowiada, że nie wie jaki jest adres IP dla **www.ksi.sh**, ale wie, jaki serwer obsługuje domenę **ksi.sh** i zwraca jego adres
 - vi. Serwer naszego ISP wysyła więc zapytanie o domenę **www.ksi.sh** do serwera autorytatywnego dla domeny **ksi.sh**
 - vii. Ten serwer z kolei okazuje się, że zna adres IP dla domeny **www.ksi.sh**, więc sprawdza go w swojej bazie danych i odsyła odpowiedź do serwera DNS naszego ISP
- (h) Ostatecznie serwer DNS naszego ISP zapisuje znaleziony adres w swoim cache'u i odsyła go do naszego komputera z uruchomioną przeglądarką internetową



2. Tworzenie pakietów zgodnie ze stosem TCP/IP

- (a) Komputer z przeglądarką tworzy zapytanie HTTP o stronę `www.ksi.sh`
- (b) Komputer tworzy **pakiet IP**:
 - Adres źródłowy: adres IP komputera (jego adres w sieci LAN)
 - Adres docelowy: adres serwera WWW otrzymany z zapytania DNS
 - Port źródłowy: losowo wybrany spośród wolnych portów na lokalnym komputerze
 - Port docelowy: specyficzny dla protokołu (w tym przypadku 80 lub 443 - chyba że w adresie URL podano inaczej)
 i opakowuje w ten pakiet zapytanie HTTP
- (c) Komputer na podstawie swojego lokalnego adresu IP i ustawionej maski sprawdza, czy docelowy komputer znajduje się w jego sieci LAN. Załóżmy tutaj, że się nie znajduje.

- (d) Komputer odczytuje więc ze swojej konfiguracji adres IP swojej bramy domyślnej
- (e) Komputer musi dowiedzieć się teraz, jaki jest adres MAC odpowiadający adresowi IP bramy. W tym celu komputer sprawdza, czy nie ma go zapisanego w pamięci cache u siebie. Jeżeli nie, wysyła on na rozgłoszeniowy adres MAC (FF:FF:FF:FF:FF:FF) **ARP Request** - jest to zapytanie o to jaki adres MAC ma komputer o podanym adresie IP
- (f) Komputer będący bramą odpowiada mu **unicastem** (na adres MAC naszego komputera) **ARP Response** - w ten sposób nasz komputer zna adres MAC bramy
- (g) Komputer tworzy **ramkę**:
 - Adres MAC źródłowy: adres MAC naszego komputera
 - Adres MAC docelowy: adres MAC naszej bramyi opakowuje w tę ramkę pakiet IP
- (h) Ramka następnie jest wysyłana (po uprzednim przetworzeniu na sygnały zgodne z medium, którym jest wysyłana - np. sygnały elektryczne) do sieci

3. Przekazywanie pakietów do serwera WWW i z powrotem

- (a) Switch, do którego podłączony jest nasz komputer odbiera wysłane przez niego dane i zagląda do ramki
- (b) Sprawdza on docelowy adres MAC i zgodnie ze swoją tablicą przełączania przekazuje tę ramkę na właściwy port lub w przypadku, gdy nie znajdzie odpowiedniego wpisu w swojej tablicy, rozsyła tę ramkę do wszystkich portów
- (c) Po przejściu być może przez kolejne switch'e, które zachowują się tak samo, ramka trafia wreszcie do naszej bramy
- (d) Brama jako router zagląda do pakietu IP i sprawdza zgodnie ze swoją tablicą routing'u (do której należą również bezpośrednio przyłączone do tego routera sieci) do jakiego kolejnego routera lub sieci skierować ten pakiet
- (e) Gdy już znajdzie adres następnego skoku (kolejny router lub już komputer docelowy), router konstruuje kolejną ramkę do tego właśnie kolejnego routera lub komputera docelowego (wszystko dzieje się analogicznie, jak

w przypadku konstruowania ramki przez komputer źródłowy), opakowuje w nią pakiet IP i wysyła

- (f) Tak dzieje się do momentu aż pakiet nie dotrze do serwera WWW. Ten odczytuje żądanie HTTP, generuje odpowiedź HTTP i opakowuje ją w pakiet IP, w którym port źródłowy i docelowy oraz adres źródłowy i docelowy są zamienione ze sobą względem pakietu, który dostał - nadawca staje się odbiorcą
- (g) Następnie serwer WWW tworzy ramkę (znów analogicznie jak to miało miejsce u klienta), w którą opakowuje wcześniej wspomniany pakiet i wysyła
- (h) Wysłane przez serwer dane znów są przekazywane przez switch'e i router'y - tak jak to miało miejsce w przypadku wysłanego żądania od klienta do serwera, aż dojdą do naszego komputera (klienta)
- (i) Ten z kolei odczytuje odpowiedź HTTP od serwera WWW i wyświetla zawartość strony

93 Działanie przełączników Ethernet, sieci VLAN, protokół STP.

93.1 Działanie przełączników Ethernet

Przełącznik Ethernet'owy (Switch) w przeciwieństwie do Hub'ów nie rozsyła ramek, które do niego przyszły na wszystkie porty - co umożliwia łatwe podsłuchiwanie komunikacji, której nie jesteśmy odbiorcą oraz niepotrzebnie zwiększa ruch w sieci; lecz w zdecydowanej większości przypadków jest w stanie wysłać odebraną ramkę na jeden port, pod który jest podłączony jej odbiorca. Kieruje się on przy tym tablicą CAM (Content Addressable Memory – rodzaj tablicy asocjacyjnej), która jest swego rodzaju powiązaniem adresów MAC z portem (gniazdem) Switch'a, a którą uzupełnia na podstawie odbieranych ramek. Zasada działania Switch'a jest następująca:

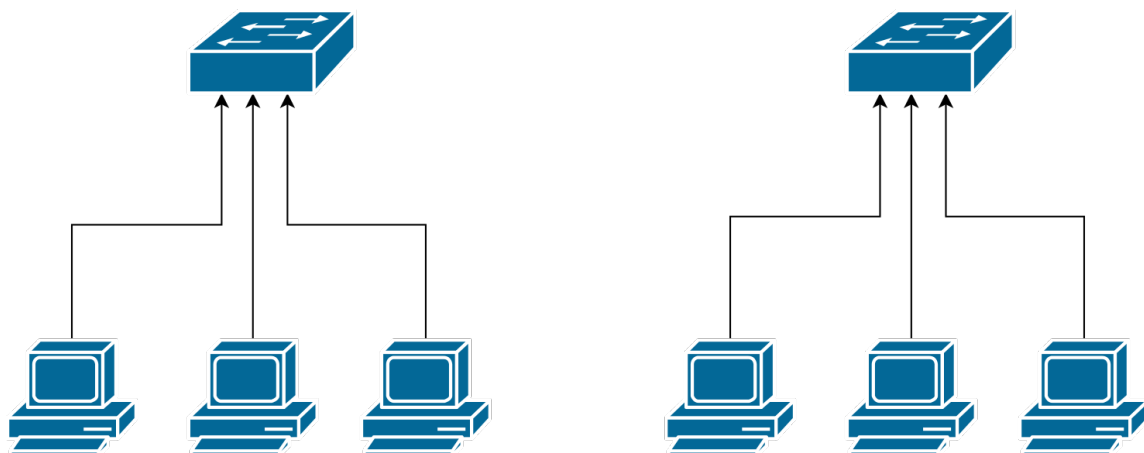
1. Gdy switch odbiera na jakimś porcie ramkę, sprawdza on czy adresat (odbiorca), a właściwie jego adres MAC znajduje się w tablicy CAM:
 - (a) Jeżeli tak - ramka wysyłana jest tylko na port powiązany z tym adresem MAC
 - (b) Jeżeli nie - Switch postąpi podobnie jak Hub - roześle tę ramkę do wszystkich portów. Podobnie postąpi, jeżeli adres MAC odbiorcy jest adresem broadcastowym
2. Switch odczytuje z ramki adres MAC nadawcy i wiąże go z portem, na który przyszła ta ramka, oraz zapisuje tę informację w tablicy CAM

Standardowe przełączniki warstwy drugiej mogą pracować w dwóch trybach pracy:

1. **Store-and-Forward** – przełącznik pobiera całą ramkę, analizuje czy nie jest błędna i przesyła ją dalej. Ten tryb wprowadza największe opóźnienie.
2. **Cut-through** (dwa podtypy):
 - (a) **Fast-forward switching** - po otrzymaniu adresu MAC miejsca docelowego ramki, już rozpoczyna się jej nadawanie. Jest to najszybsza metoda, ale nie sprawdza czy nie nastąpiła kolizja i ramka jest niepoprawna.
 - (b) **Fragment-free switching** - ramka jest przesyłana do portu docelowego dopiero po otrzymaniu 64 bajtów (rozmiar najmniejszej poprawnej ramki w Ethernetie). Zapewnia to, że ramka nie koliduje z inną

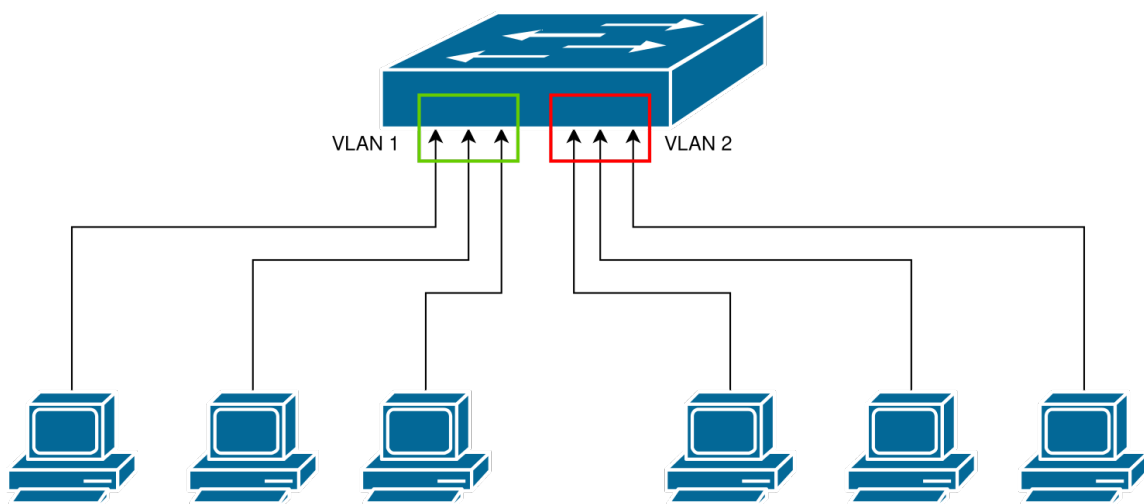
93.2 Sieci VLAN

W sieciach komputerowych często zachodzi potrzeba oddzielenia od siebie pewnej grupy komputerów tak, aby mogły one porozumiewać się między sobą w obrębie jednej grupy, ale jednocześnie, aby nie mogły się komunikować między grupami. Naturalnym rozwiązaniem wydaje się podłączenie grup do osobnych switch'y:



Nie jest to jednak idealne rozwiązanie: dla każdej kolejnej grupy musimy kupować kolejne urządzenie, co przy większej ilości grup znacznie zwiększa koszty. Poza tym co jeżeli komputery z każdej z grup są np. na różnych piętrach? Musimy wtedy kupować kolejne urządzenia do obsługi komputerów na kolejnych piętrach oraz rozciągać między piętrami tak wiele kabli do połączeń między switch'ami, jak wiele mamy tych grup.

Odpowiedzią na te potrzeby są **VLAN'y** (Virtual LAN). Dzięki nim na jednym switch'u jesteśmy w stanie wydzielić wiele osobnych "podswitch'y": każdy port (gniazdo) w switch'u możemy przypisać do konkretnego VLAN'u - te, które są w tym samym VLAN'ie mogą się ze sobą komunikować (zachowują się jak podpięte do tego samego switch'a), natomiast niemożliwa jest bezpośrednia komunikacja między dwoma różnymi VLAN'ami (tak jak komunikacja między dwoma niepołączonymi ze sobą switch'ami):



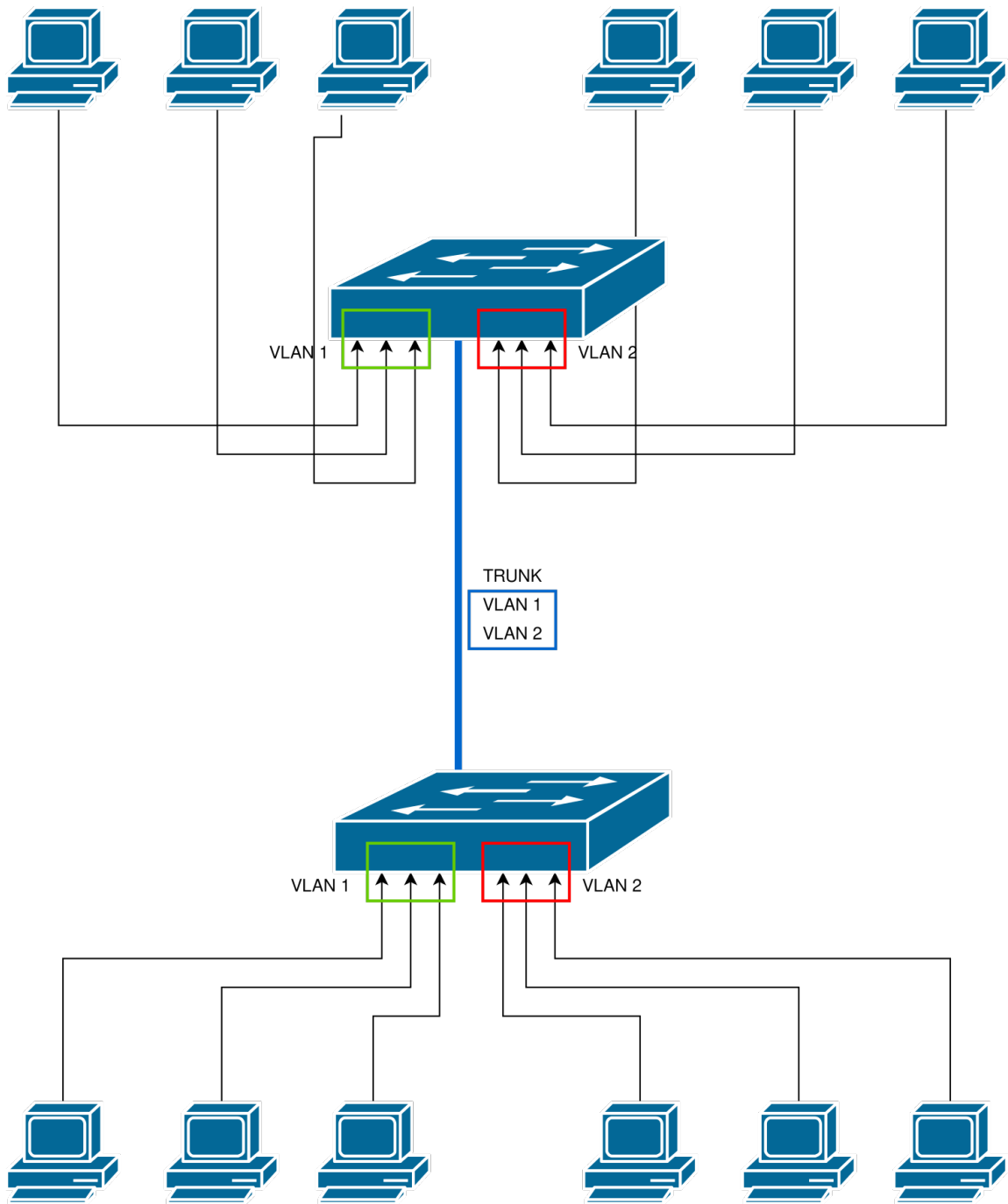
Daje to nam dużą swobodę:

- Nie musimy fizycznie przełączać kabli, gdy chcemy zmienić grupę dla danego komputera
- Utworzenie kolejnej grupy nic nas nie kosztuje - po prostu tworzymy kolejny VLAN i przypisujemy do niego odpowiednie porty
- Jednym kablem możemy przysyłać wiele VLAN'ów - nie potrzebujemy rozciągać wielu kabli między switch'ami na różnych piętrach (patrz dalej)

Ramki w przypadku technologii VLAN mogą być **tagowane** lub **nietagowane**. Te drugie to po prostu zwykłe ramki używane do tej pory w sieciach Ethernet. Ramki tagowane natomiast mają w sobie dodatkową informację do którego VLAN'u należą.

Porty i połączenia, na których wysyłane/odbierane są ramki nietagowane nazywamy **dostępowymi**, natomiast te, gdzie komunikacja odbywa się przy pomocy ramek tagowanych nazywamy **trunk'owymi**. Standardowo komputery rozumieją tylko ramki nietagowane.

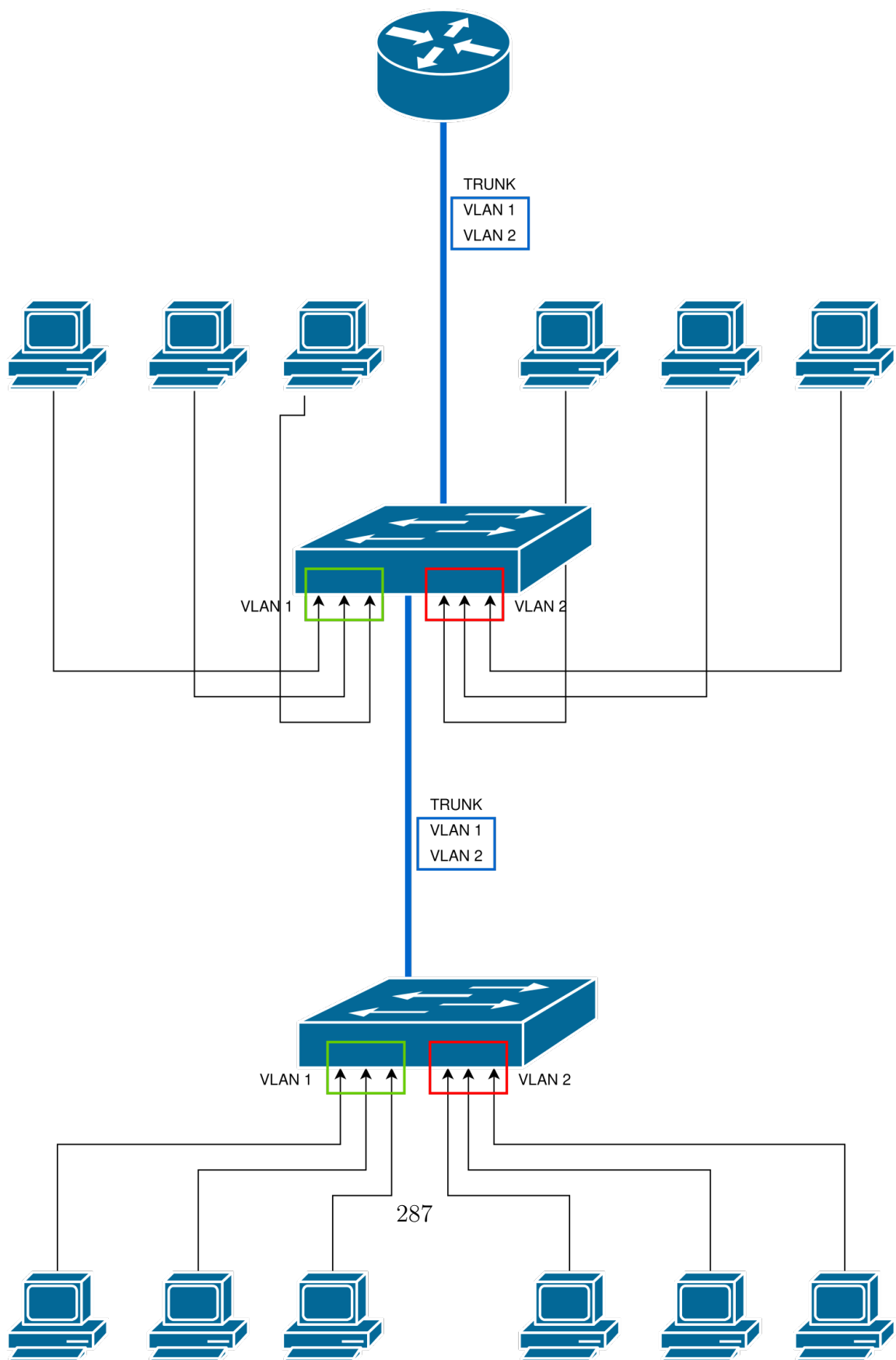
Ramki tagowane otwierają nam nowe możliwości: możemy przy pomocy jednego kabla przysyłać ramki z wielu sieci VLAN, co pozwala np. połączyć za pomocą tylko jednego kabla switch'e z różnych pięter w rozważanym na początku przypadku, gdy komputery z jednej grupy znajdują się na różnych piętrach. Switch'e połączone są wtedy ze sobą łączem **trunk'owym**, dzięki czemu wysyłają one między sobą ramki tagowane, więc wiedzą, do którego VLAN'u (do której grupy komputerów) skierować przychodzącą ramkę:



Czasami zachodzi jednak potrzeba, aby komputery mogły się ze sobą komunikować nawet między grupami (ale przeważnie chodzi nam o kontrolę takiej komunikacji - zezwolenie tylko na określony ruch). Ponieważ każdy VLAN traktowany jest jako osobna podsieć (w każdym z nich powinniśmy używać adresów IP z innej podsieci), aby umożliwić komunikację między VLAN'ami potrzebny nam jest **router**. Będzie on po prostu robił prosty routing między tymi sieciami, a dzięki zastosowaniu zasad na firewall'u będziemy mogli ograniczyć ten ruch tak, jak chcemy.

Ponieważ router potrzebuje dostępu do wszystkich sieci VLAN, najprościej jest to osiągnąć poprzez podłączenie go do portu switch'a skonfigurowanego jako trunk. Takie podłączenie routera nazywamy **routerem na kiju** (router on a stick). Dzięki temu - podobnie jak w przypadku komunikacji między switch'ami będziemy mogli przy pomocy jednego kabla podłączyć do routera wiele sieci. Pomimo fizycznego podłączenia tylko do jednego interface'u routera, w praktyce dla każdego VLAN'u zostaną na nim utworzone **wirtualne interface'y**, które będą reprezentować poszczególne VLAN'y i to między nimi będzie wykonywany routing.

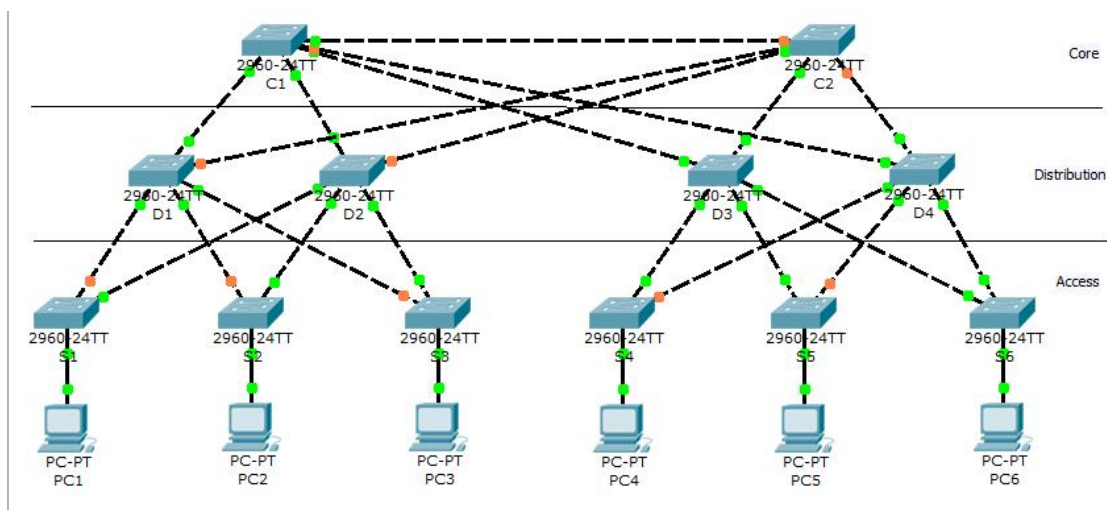
Zauważmy, że w takiej konfiguracji, każdy ruch, który odbywa się między VLAN'ami musi przejść przez router. Może to znacznie zwiększyć obciążenie sieci: np. aby dwa komputery w różnych VLAN'ach jednak podłączone do tego samego switch'a mogły się ze sobą komunikować, cały ruch musi przejść niekiedy być może długą drogę przez całą sieć, aż do router'a i z powrotem. Aby temu zaradzić można użyć **Switch'y (przełączników) warstwy 3**. Są one mini routerami, które są w stanie wykonywać routing między VLAN'ami, które obsługują (z możliwością ustawienia zasad komunikacji między nimi - podobnie jak zasady firewall na routerze). Dzięki temu we wspomnianym wcześniej przykładzie ruch między tymi dwoma komputerami może odbywać się praktycznie tylko przy pomocy switch'a, do którego są podłączone - nie ma konieczności angażowania w to routera i innych switch'y.



93.3 Protokół STP

STP - Spanning-Tree Protocol - jest protokołem umożliwiającym uniknięcie pętli w sieci LAN oraz mogącym zapewnić redundantne ścieżki komunikacji, które uaktywnią się w momencie awarii dotychczasowych.

W nieco bardziej rozbudowanej sieci łączy się kilka lub więcej przełączników. Korzystnie jest, gdy przełączniki w sieci są połączone tak, by umożliwić zduplikowane ścieżki między nimi, ale żeby nie występowały pętle i niekorzystne zjawiska z nimi związane (burze rozgłoszeń w warstwie drugiej, dublowanie ramek, niestabilność zawartości tablicy CAM). Do realizacji tego celu służy protokół STP:



STP umożliwia tworzenie redundantnych ścieżek (połączeń) między przełącznikami, ale w danej chwili tylko jedno ze zduplikowanych połączeń jest aktywne. Aktywne połączenia tworzą strukturę drzewiastą z jednym przełącznikiem wybranym jako korzeń (root, root-switch lub root-bridge). STP powstał jak mosty były jeszcze powszechnie wykorzystywane i stąd w terminologii STP często zamiast switch mówi się bridge.

STP wykorzystuje specjalne ramki BPDU (Bridge Protocol Data Units), które są regularnie przesyłane między przełącznikami (np. co dwie sekundy). Ramki BPDU mają specjalny adres MAC docelowy typu multicast (01:80:C2:00:00:00) i są przesyłane według innych zasad jak zwykłe ramki: otrzymane ramki BPDU nie są wprost przekazywane do innych portów, przełącznik buduje swoje ramki BPDU uwzględniające otrzymane od innych informacje, takie jak np. koszt dotarcia do root-switch'a - powiększa go przed przesłaniem, podobnie jak w protokole routowania OSPF.

Działanie STP:

1. **Wybór przełącznika-korzenia** (root-bridge) - jest realizowany na podstawie priorytetu przełącznika, który jest konfigurowalny. Jako korzeń wybierany jest przełącznik o najniższym priorytecie. Jeśli priorytety są jednakowe, to wybierany jest przełącznik o najniższym adresie MAC. Wybór korzenia drzewa rozpinającego jest realizowany z wykorzystaniem specjalnych ramek BPDU.
2. **Wybór portów do korzeni** (root ports). Są to porty, którymi będą przekazywane ramki do przełącznika-korzenia. Port staje się zawsze „root-portem” jeśli jest bezpośrednio połączony do przełącznika – korzenia. Jeśli nie ma portów bezpośrednio dołączonych do korzenia, to jest analizowany koszt połączenia. Koszt ten jest zależny od szerokości pasma np. dla 10Mbps koszt wynosi 100, dla 100Mbps koszt wynosi 10, itd. Koszty są przekazywane w ramach BPDU i przełączniki wyliczają całkowite koszty połączenia z przełącznikiem korzeniem – jako sumy kosztów poszczególnych połączeń. W każdym przełączniku port o najniższym koszcie trasy do przełącznika korzenia staje się root-portem (oczywiście tylko w przypadku, gdy w danym przełączniku nie ma bezpośredniego połączenia z przełącznikiem-korzeniem).
3. **Wybór portów wyznaczonych** (designated ports). Są to porty, które uczestniczą w wymianie informacji w drzewie rozpinającym (spanning tree). Dla każdego segmentu sieci LAN jest określany jeden przełącznik wyznaczony (designated switch), przez który wiedzie normalna droga z danego segmentu do przełącznika-korzenia. Przykładowy segment sieci LAN: koncentrator połączony z dwoma przełącznikami oraz z pewną liczbą komputerów – hostów.
4. **Blokada niepotrzebnych portów**. Po ustaleniu stanu połączeń wszystkie porty umożliwiające połączenia redundantne, a nie będące root-portami ani portami wyznaczonymi, są blokowane.

Stan połączeń jest analizowany na podstawie ramek BPDU, które są przesyłane cyklicznie, standardowo co 2s (ten czas można zmienić). W przypadku uszkodzeń istniejących połączeń automatycznie odblokowywane są nowe porty, które przejmują przekazywanie ramek, ramki są zatem przesyłane nową trasą.

94 Rola routerów i podstawowe protokoły routingu (RIP, OSPF).

Sposób obsługi routowania przez warstwę IP nazywa się mechanizmem routowania. Określenie to dotyczy przeglądania przez system operacyjny tablicy routowania i podejmowania określonych decyzji, co do przesyłania datagramów IP. Przez pojęcie polityka routowania określa się działania procesu routowania podejmowane w celu ustanowienia i bieżącej modyfikacji tablicy routowania. Polityka routowania realizowana jest z wykorzystaniem protokołów routowania.

Pożądane cechy protokołów routowania:

1. Wyznaczenie najlepszej trasy do punktu docelowego
2. Odporność (robustness) na nieprzewidziane, nieporządkanie zdarzenia: "kopara", niewłaściwa konfiguracja, obciążenia sieci
3. Szybkie osiągnięcie zbieżności, tj. osiągnięcie stanu w którym routery "widzą" jednakowo topologię sieci.
4. Dopasowanie do zmian, np.:
 - Odłączenie danej sieci
 - Podłączenie nowej sieci
 - Zmiana szerokości pasma
 - Zmiana opóźnienia

Ze względu na sposób działania protokoły routowania wewnętrzne dzielimy na

- protokoły wektora odległości (DV, distance-vector)
- protokoły stanu łącza (link-state)

94.1 Protokoły wektora odległości

Protokoły wektora odległości opierają się na algorytmie Bellmana Forda, gdzie węzły to router, krawędzie - połączenia pomiędzy nimi.

Protokołami wektora odległości są:

- RIP
- RIP2
- IGRP
- EIGRP (wg Cisco hybrydowy)

94.1.1 RIP

Metryką w RIP jest liczba skoków / routerów po drodze do celu - 0 to połączenie bezpośrednie, 1 to jeden skok do następnego router, itd. Co ważne, metryka 16 oznacza umownie nieskończoność. Trasa o metryce 16 jest traktowana jako trasa niedostępna, przez co RIP nie sprawdza się w przypadku bardzo dużych sieci.

Liczniki RIP:

- Update timer (30s) – co 30s wysyłany jest wektor odległości (routing update)
- Invalid timer (180s) – za każdym razem, gdy router dostaje uaktualnienie pewnej trasy, zegar dla tej trasy jest zerowany. Po upływie 180s trasa zaznaczana jest jako nieprawidłowa.
- Hold-down timer (180s) – po osiągnięciu wartości progowej przez invalid timer trasa jest ustawiana w stan hold-down.
- Flush timer (240s) – zegar ten jest zerowany po otrzymaniu informacji o danej trasie. Po upływie trasa jest usuwana (nawet, jeśli znajduje się w stanie hold-down).

Zalety:

- prostota
- łatwa konfiguracja

Wady:

- Wolno rozprzestrzenianie się informacji o zmianach w topologii

- Wolne osiągnięcie zbieżności
- Częste przesyłanie dużych porcji informacji (wektory odległości)

94.1.2 RIP2

- Przekazuje maski podsieci
- Umożliwia stosowanie sieci bezklasowych CIDR
- Umożliwia stosowania podsieci o zmiennym rozmiarze VLSM
- Umożliwia prostą autentykację (hasła)
- Przekazuje adresy następnego skoku w komunikatach
- Metryka 16 dalej oznacza nieskończoność
- Dalej brak alternatywnych tras

94.2 Protokoły stanu łącza

Łącze reprezentuje połączenie z sąsiednim routerem (routerami, w przypadku technologii wielodostępnych, np. Ethernet), komputerem lub siecią. Stan łącza składa się z wielu atrybutów, np. prędkość transmisji, opóźnienie, adres IP, sąsiedzi

Protokołami stanu łącza są:

- OSPF
- OSPF2
- IS-IS
- NLSP

94.3 OSPF

Zasada działania:

- Każdy router posiada (albo buduje) informację o połączeniach w całej sieci
- Za pomocą algorytmu Dijkstry obliczane są najkrótsze ścieżki od danego routera do innych węzłów
- Przesyłane są informacje o zmianach w stanie łączy - nie całe tablice routowania
- Na początku pracy sieć jest zalewana informacją o stanie łączy (flooding), późniejsze updaty są szybkie
- Ruch w wielodostępnych fragmentach sieci jest ograniczany przez wybranie DR (designated router) i BDR (backup designated router)
- Obliczanie najkrótszych ścieżek Dijkstrą jest zasobożerne, przez co tworzy się obszary OSPF:
 - Każdy obszar ma przypisany numer
 - Jeśli jest jeden obszar ma on numer 0
 - Routery na granicy obszarów OSPF nazywamy ABR (Area Border Routers)
 - Routerem granicznym wprowadzającym do OSPF trasy zewnętrzne (z innych protokołów) nazywamy ASBR (Autonomous System Border Router)

- Rodzaje obszarów OSPF:
 - * Stub Area - nie ma tras zewnętrznych, sumy tras z innych obszarów są wprowadzane
 - * Totally Stubby Area - nie ma ani tras zewnętrznych, ani sum tras z innych obszarów. Wyjście z takiego obszaru jest przez trasę domyślną
 - * Not So Stubby Area - Stub Area, do którego wprowadzane są pewne, nieliczne trasy zewnętrzne. Trasy te wprowadzane są do innych obszarów tak jak sumy tras
 - * Not So Stubby Totally Stubby Area - obszar połączenia NSSA i TSA
- W przypadku istnienia kilku tras o jednakowym koszcie realizowane jest równoważenie obciążeń (load balancing).

Metryki:

- Standardowa metryka = $\frac{\text{wartość odniesienia}}{\text{szerokość pasma}}$
- Standardowa wartość odniesienia to 100Mbps
- Ethernet 100Mbps ma metrykę 1
- Łącze 56kps ma metrykę 1768

95 Szyfrowanie z kluczem publicznym, podpis cyfrowy, certyfikaty.

95.1 Szyfrowanie z kluczem publicznym

Szyfrowanie i odszyfrowanie jest realizowane przy pomocy pary kluczy - **prywatnego** i **publicznego**. Jeśli coś zostanie zaszyfrowane przy pomocy klucza prywatnego, to może być odszyfrowane tylko przy pomocy odpowiadającego mu klucza publicznego i na odwrót.

Jeśli chcemy coś przekazać innej osobie w wersji zaszyfrowanej, to powinniśmy zaszyfrować to kluczem publicznym tej osoby. Ponieważ nikt poza tą osobą nie zna odpowiadającego klucza prywatnego, nikt przesyłki nie odszyfruje.

95.2 Podpis cyfrowy

Zaszyfrowanie kluczem prywatnym daje gwarancję, że zaszyfrowana wiadomość pochodzi z odpowiedniego źródła. Odgadnięcie klucza prywatnego, jeśli znany jest publiczny jest „praktycznie niemożliwe” w sensownym czasie (przy użyciu współczesnych komputerów).

95.2.1 Skrót(Hash)

Szyfrowanie dużych porcji danych może być kosztowne i czasem niepożądane. Skrót ma zwykle 160 bitów (SHA-1) lub 224–512 (rodzina SHA-2). Jeśli w oryginalnej wiadomości zmieniony zostanie chociaż jeden bit, to skrót będzie zupełnie inny niż ten, który został utworzony przed zmianą. Jednak na podstawie skrótu odtworzenie oryginalnej wiadomości jest prawie niemożliwe. Algorytmy haszujące są deterministyczne.

95.2.2 Uzyskanie podpisu cyfrowego

1. Generacja skrótu wiadomości
2. Szyfrowanie go kluczem prywatnym

Zaszyfrowany skrót stanowi podpis cyfrowy. Niezaszyfrowana wiadomość może być przesłana jawnie razem z podpisem cyfrowym.

95.2.3 Sprawdzenie podpisu cyfrowego

1. Odszyfrowanie skrótu kluczem publicznym nadawcy
2. Tworzenie skrótu wiadomości używając tej samej funkcji haszującej

Jeśli wyniki obu operacji są identyczne, to znaczy, że wiadomość na pewno podpisał określony nadawca, a ponadto nikt po tej wiadomości nie zmienił już po podpisaniu.

95.3 Certyfikaty

Klucze mogą być generowane na komputerze lokalnym przy pomocy odpowiedniego oprogramowania i powinny być podpisane przez jakieś centrum certyfikacyjne.

Centrum certyfikacyjne (ang. CA – Certification Authority) wydaje tzw. certyfikaty cyfrowe.

Certyfikat zawiera:

- Identyfikator osoby/firmy/obiektu
- Identyfikator Centrum certyfikacyjnego (CA), który wydał certyfikat
- Numer identyfikacyjny certyfikatu
- Cel stosowania (np. podpisywanie bezpiecznych stron WWW albo podpisywanie listów elektronicznych)
- Wartość klucza publicznego
- Okres ważności
- Podpis cyfrowy wydawcy (CA)

Jeśli ufamy danemu wydawcy (CA), to ufamy, że zawarty w certyfikacie klucz publiczny jest prawdziwy. W systemach operacyjnych oraz różnych programach (np. przeglądarkach internetowych) jest wpisana lista zaufanych tzw. głównych urzędów certyfikacji oraz lista pośrednich urzędów certyfikacji, których certyfikaty są podpisywane przez główne urzędy. Poprzez panel sterowania (w opcjach internetowych) można przeglądać i zarządzać listą zaufanych wydawców i dodawać do tej listy certyfikaty swoich zaufanych urzędów certyfikacji.

96 Wirtualne sieci prywatne, protokół IPsec.

Wirtualna Sieć Prywatna, zwaną potocznie „siecią VPN” (z ang. Virtual Private Network) jest siecią przekazu danych korzystającą z publicznej infrastruktury telekomunikacyjnej, która dzięki stosowaniu protokołów tunelowania i procedur bezpieczeństwa zachowuje poufność danych. Infrastrukturą może być sieć szkieletowa operatora telekomunikacyjnego (np. Frame Relay lub ATM), a także globalna sieć Internet. Sieci te nazwano „wirtualnymi”, gdyż opierają się one na dynamicznych, wirtualnych połączeniach - wirtualnych obwodach (Virtual Circuit – wirtualny obwód – jest to połączenie zestawiane w sieci pomiędzy nadawcą a odbiorcą, w którym trasy przesyłu danych oraz przepływność pasma dla połączenia są ustawiane dynamicznie) czy też tunelach – nie mających fizycznego bytu i istniejącymi tylko wtedy, gdy jest nimi przenoszony ruch. Połączenia takie mogą być zestawiane między dwoma osobnymi urządzeniami, urządzeniem a siecią oraz między dwoma sieciami.

Sieć VPN w założeniu powinna zapewniać:

- poufność
- integralność
- autentyczność

Ogólnie wyróżniamy trzy powszechnie stosowane rodzaje sieci VPN:

- zaufany VPN
- bezpieczny VPN
- hybrydowy VPN

96.1 Zaufany VPN (z ang. Trusted VPN)

To łącza uzyskane od operatora telekomunikacyjnego z zapewnieniem, że dane są odpowiednio chronione. Z technicznego punktu widzenia nie ma gwarancji, iż urządzenia w sieci operatora, przez które przesyłane są dane nie zostaną przejęte przez osoby nieupoważnione. Usługa tego typu świadczona przez operatora może opierać się o łącza dzierżawione zestawione w technologii ATM, Frame Relay czy MPLS. Operator może zapewnić odpowiednio wysoką dostępność usługi, odpowiedni poziom jakości usług QoS (Quality-of-Service) i wymagane pasmo.

96.2 Bezpieczny VPN (Secure VPN)

Sieć VPN bazująca na urządzeniach i oprogramowaniu klienta) wg IETF, gdzie transmisja odbywa się przez Internet, a do jej ochrony stosuje się techniki kryptograficzne. Użytkownik lub firma sama decyduje, jakie mechanizmy ochrony mają być zastosowane. Zaszyfrowany ruch jest widoczny jako tunel między dwoma sieciami i nawet, jeżeli napastnik widzi ruch sieciowy, to i tak nie może go odczytać, jak również zmodyfikować danych niezauważalnie dla strony odbiorczej. Dużą zaletą tego rozwiązania jest koszt, ponieważ VPN wykorzystuje zwykłe łącze internetowe posiadane przez firmę, którego koszt utrzymania jest o wiele niższy niż dedykowanego łącza dzierżawionego. Należy jednak zauważyć, że bezpieczny VPN realizowany jest przez łącza publiczne, gdzie operatorzy nie mogą zapewnić takich samych parametrów sieci na całej drodze transmisji.

96.3 Hybrydowy VPN

Istnieje jeszcze rodzaj sieci VPN integrujący dwa powyższe. Są to hybrydowe sieci VPN (z ang. Hybrid VPN), które stosuje się do przesyłania poufnych informacji wrażliwych na opóźnienia w sieci, np. VoIP. Transmisja w tym wypadku realizowana jest przez łącza dzierżawione, odpowiednio zabezpieczone za pomocą technik kryptograficznych.

96.4 Zalety VPN

- niezależność od fizycznej realizacji połączeń - lokalizacja z przyłączem w postaci linii dzierżawionej bez problemów kontaktuje się z lokalizacją dołączoną do dostawcy Internetu przez Ethernet czy ATM
- niskie koszty ponoszone za dzierżawę łączy - opłaty ponoszone są głównie za łącze dostępne
- dostępność - Internet jest największą siecią na świecie, poza ewentualnymi problemami z szybkością transmisji nie ma problemów w łączności między lokalizacjami dołączonymi do różnych dostawców Internetu
- szybkość zestawiania nowych bezpiecznych połączeń - z uwagi na fakt, że Internet oparty jest o bezpołączeniowy protokół IP, umożliwiający komunikację między dowolnymi punktami, zestawienie nowych "połączeń" konfigurowane jest lokalnie, bez udziału dostawców usługi

[Źródło \(click me\)](#)

96.5 Protokół IPsec

(ang. Internet Protocol Security, IP Security) – zbiór protokołów służących implementacji bezpiecznych połączeń oraz wymiany kluczy szyfrowania pomiędzy komputerami. Protokoły tej grupy mogą być wykorzystywane do tworzenia Wirtualnej Sieci Prywatnej (ang. VPN).

VPN oparta na IPsec składa się z dwóch kanałów komunikacyjnych pomiędzy połączonymi komputerami: kanał wymiany kluczy, za pośrednictwem którego przekazywane są dane związane z uwierzytelnianiem i szyfrowaniem (klucze), oraz kanału (jednego lub więcej), który niesie pakiety transmitowane poprzez sieć prywatną. Kanał wymiany kluczy jest standardowym protokołem UDP (port 500).

96.5.1 Usługi zapewniane przez IPsec

- Poufność (szyfrowanie) transmisji
- Zapewnienie integralności danych w pakiecie
- Uwierzytelnianie pochodzenia danych
- Zabezpieczenie przed atakami polegającymi na powtórny przesłaniu pakietu
- Ograniczone zabezpieczenie przed analizą ruchu
- Kontrola dostępu
- IPsec jest jednym ze sposobów na budowanie VPN

96.5.2 Składowe standardu IPsec

IPSec opiera się na trzech protokołach: AH (Authentication Header), który zapewnia zarówno identyfikację oryginalności danych, jak i ich integralność, ESP (Encapsulating Security Payload), dostarczający poufności danych, identyfikację oryginalności oraz integralności danych, i ISAKMP (Internet Security Association and Key Management Protocol), który jest odpowiedzialny za automatyczne ustawianie SA oraz zarządzanie kluczami kryptograficznymi tych SA.

- AH (Authentication Header) - nagłówek dodawany do pakietu IP, zawierający dane służące do uwierzytelnienia (i kontroli integralności)
AH zapewnia uwierzytelnienie integralności i pochodzenia danych. Integralność

danych jest zapewniona poprzez sumę kontrolną generowaną przez kod identyfikacyjny wiadomości. Natomiast sprawdzenie oryginalności danych dokonuje się poprzez zamieszczenie tajnego, współdzielonego klucza w danych przeznaczonych do identyfikowania.

AH może być aplikowany zarówno w trybie tunelowym, jak i w trybie transportowym. AH może być stosowany samodzielnie lub w połączeniu z ESP. W takiej kombinacji identyfikacja może być zapewniona między parą komputerów, zapór ogniowych lub między komputerem a zaporą.

- ESP (Encapsulating Security Payload) - szyfrowanie oryginalnego pakietu IP i obduowanie w pakiet ESP
 - Tryb transportowy - podmiana zawartości pakietu IP na zaszyfrowaną
 - Tryb tunelowy - zanurzenie pakietu ESP w nowy pakiet IP

IPSec (IP Security), który ma dwa tryby szyfrowania: tunelowy i transportowy. W pierwszym z nich szyfruje się nagłówek i ładunek każdego z pakietów, natomiast w drugim tylko ładunek. Wszystkie urządzenia muszą używać wspólnego klucza, a zapory ogniowe muszą mieć ustanowione podobne polityki bezpieczeństwa. IPSec może szyfrować dane między różnymi urządzeniami, takimi jak np. dwa routery, router i zaporę ogniową czy komputer osobisty i router.

ESP zapewnia poufność danych dzięki szyfrowaniu. Może także opcjonalnie zapewnić uwierzytelnienie pochodzenia danych, sprawdzanie integralności i replayprotection. Porównując ESP do AH, widać, że tylko ten pierwszy dostarcza szyfrowania, natomiast obydwa zapewniają identyfikację, i sprawdzenie integralności i replayprotection. Szyfrowanie ESP stosuje symetryczny współdzielony klucz, tzn. jest on współużytkowany przy szyfrowaniu i deszyfrowaniu danych.

- IKE (Internet Key Exchange) - uniwersalny algorytm wymiany klucza

96.5.3 Security Association

Relacja zabezpieczeń SA określa zależności bezpieczeństwa pomiędzy dwoma lub więcej jednostkami. Wyjaśnia, w jaki sposób jednostki używać będą systemów zabezpieczeń do bezpiecznej komunikacji. Relacja zabezpieczeń jest jednokierunkowa. Zatem dla każdej pary komunikujących się urządzeń zdefiniować należy przynajmniej dwa

bezpieczne połączenia - z A do B i z B do A. Relacja zabezpieczeń jest jednoznacznie definiowana przez losowo wybrany jednoznaczny numer tzw. Security Parameters Index (SPI), docelowy adres IP odbiorcy oraz identyfikator protokołu zabezpieczeń (AH lub ESP). Kiedy system wysyła pakiet, który wymaga ochrony IPsec, przegląda relacje zabezpieczeń w swojej bazie i następnie umieszcza SPI w nagłówku IPsec. Parametry relacji zabezpieczeń są negocjowane w trakcie zestawiania bezpiecznego połączenia i obowiązują przez cały czas funkcjonowania tej relacji.

Abstrakcyjny kanał komunikacji w IPsec

- jednokierunkowy
- dla ustalonych nadawcy i odbiorcy
- z określonymi aspektami szyfrowania i aktualnym kontekstem

Identyfikowany przez

- adres IP odbiorcy (może być unicastowy, broadcastowy lub multicastowy)
- liczbę nazywaną Security Parameter Index (SPI)
- Identyfikator protokołu bezpieczeństwa (ESP lub AH)